



Appunti del Corso di Computational Intelligence and Deep Learning

Università di Pisa
Dipartimento di Ingegneria dell'Informazione
Artificial Intelligence and Data Engineering

Professore: Marco Cococcioni

Assistente: Federico Galatolo

Studente: Francesco Panattoni

Questi appunti sono stati redatti come supporto allo studio in preparazione all'esame scritto/orale del corso di **Computational Intelligence and Deep Learning**. Il contenuto è il risultato di una revisione degli argomenti trattati a lezione e di un'analisi delle domande più frequenti negli esami orali. Richiede una conoscenza di base del Machine Learning, una discreta familiarità con Python e in particolare con la libreria `numpy`.

Pur cercando di sintetizzare i concetti più rilevanti del corso, non si garantiscono completezza o accuratezza assoluta delle informazioni. Inoltre, **declino ogni responsabilità** riguardo alla qualità del contenuto o a eventuali risultati derivanti dall'uso di questo documento come risorsa di studio.

Questi appunti sono suddivisi in tre parti: una parte teorica (che raccoglie gli argomenti previsti per la prova orale e alcuni approfondimenti), una parte pratica (che sintetizza le lezioni svolte dall'assistente senza concentrarsi sugli esercizi) e una sezione dedicata alla prova scritta. In quest'ultima, le domande sottolineate sono state effettivamente proposte in appelli precedenti, mentre le restanti sono state formulate per coprire argomenti potenzialmente oggetto d'esame, pur non essendo mai state assegnate.

Questi appunti sono piuttosto verbosi e approfondiscono alcuni argomenti in modo esteso per chiarirne le dinamiche di funzionamento, spesso più di quanto richiesto dal corso. Si consiglia comunque agli studenti di utilizzarli solo dopo aver completato la revisione delle slide ufficiali del corso di **Computational Intelligence and Deep Learning**.

Indice

I	Teoria	7
1	Introduzione al Deep Learning	8
1.1	Computational Intelligence e Machine Learning	8
1.2	Linear Least Squares	10
1.3	Discesa del Gradiente	12
1.4	Dalla Regressione alla Classificazione	13
1.5	Perceptron	14
1.6	Loss Functions	16
1.7	Logistic Regression	22
1.8	Funzioni di Attivazione	23
1.9	Backpropagation	25
2	Multilayer Perceptron	28
2.1	Input Layer, Hidden Layers e Output Layer	28
2.2	Teorema dell'Approssimazione Universale	28
2.3	MLP_MSELIN	29
2.4	MLP_BCESIG	34
2.5	MLP_CCESM	39
3	Data Preprocessing	46
3.1	Introduzione al Preprocessing	46
3.2	Data Splitting	46
3.3	Feature Scaling	47
3.4	Weight Initialization	48
3.5	Gestione dei Dati Mancanti	50
3.6	Data Cleaning	50
4	Addestramento e Ottimizzazione	51
4.1	Ciclo di Addestramento di una Neural Network	51
4.2	Overfitting e Underfitting	52
4.3	Batch Normalization	53
4.4	Dropout	54
4.5	Regularisation	55
4.6	Scelta del Learning Rate	57
4.7	Adaptive Methods	58
4.8	Vincoli	60
5	Radial Basis Function Networks	62
5.1	Teorema di Cover	62
5.2	Radial Basis Function	62
5.3	Architettura della Rete	63
5.4	Forward Step	64
5.5	Kernel Density Estimation	64
5.6	Quando usare gli MLP e quando le RBFN?	65
5.7	Codice	66

6	Evolutionary Algorithms	69
6.1	Hyperparameter Tuning	69
6.2	Simulated Annealing	71
6.3	Evolution Strategies	74
6.4	Genetic Algorithms	79
6.5	Teorema No Free Lunch	89
6.6	Genetic Programming	89
6.7	Neuroevolution	91
6.8	Applicazioni	92
7	Swarm Intelligence	93
7.1	Intelligenza Collettiva e Auto-organizzazione	93
7.2	Particle Swarm Optimization	94
7.3	Ant Colony Optimization	98
8	Advanced Deep Learning	105
8.1	La necessità di ottimizzare il Deep Learning	105
8.2	Low-Rank Adaptation per il Fine-Tuning	106
8.3	Neural Architecture Search	106
8.4	Knowledge Distillation	109
8.5	Forward Automatic Differentiation	111
8.6	Multi-Task Learning	113
II	Laboratorio	121
9	PyTorch	122
9.1	Autograd	122
9.2	Tensori	124
9.3	Operazioni sui Tensori	127
9.4	Dati	131
9.5	Moduli	133
9.6	Loss Function e Optimizer	135
9.7	Maschere Booleane	139
10	Convolutional Neural Network	140
10.1	Matematica della Convoluzione	140
10.2	Convoluzione 2D	141
10.3	Convolutional Layer	142
10.4	Pooling Layer	143
10.5	Anatomia di una CNN	144
10.6	Residual Networks	145
10.7	Codice	147
11	Recurrent Neural Network	149
11.1	Meccanismo della Memoria	149
11.2	Famiglie di Applicazione delle RNN	150
11.3	Backpropagation Through Time	150
11.4	Problema del Vanishing Gradient	152
11.5	Long Short-Term Memory	153
11.6	Gated Recurrent Unit	154
11.7	Codice	155
12	Variational Autoencoder	158
12.1	Autoencoder	158

12.2	Denoising Autoencoder	159
12.3	VAE come Modelli Generativi	160
12.4	Face Swap	162
12.5	Vector Quantized Variational Autoencoders	163
13	Advanced Models	167
13.1	Generative Adversarial Networks	167
13.2	You Only Look Once	170
13.3	Faster R-CNN	172
13.4	U-Net	173
13.5	CLIP	174
III	Esame	176
14	Domanda 1	177
14.1	MLP_MSELIN	177
14.1.1	<u>Domanda Standard</u>	177
14.2	MLP_BCESIG	180
14.2.1	<u>Domanda Standard</u>	180
14.2.2	<u>Domanda Tanh</u>	183
14.3	MLP_CCESM	187
14.3.1	<u>Domanda Standard</u>	187
14.3.2	Domanda Tanh e Layer in più	190
15	Domanda 2	194
15.1	Radial Basis Function Network	194
15.1.1	<u>Domanda Standard</u>	194
15.2	Evolutionary Algorithms	195
15.2.1	Differenza tra Grid Search, Random Search e Genetic Algorithms	195
15.2.2	<u>Evolutionary Strategies</u>	197
15.3	Genetic Algorithms	198
15.3.1	<u>TSP</u>	198
15.3.2	<u>Genetic Algorithm al posto della Backpropagation</u>	199
15.3.3	<u>Genetic Programming</u>	201
15.4	Simulated Annealing	202
15.4.1	Domanda Standard	202
15.5	Particle Swarm Optimization	204
15.5.1	<u>Domanda Standard</u>	204
15.6	Ant Colony Optimization	206
15.6.1	Domanda Standard	206
15.7	Advanced Deep Learning	208
15.7.1	NAS	208
15.7.2	Knowledge Distillation	209
16	Domanda 3	211
16.1	Autograd	211
16.1.1	Computational Graph	211
16.2	PyTorch	213
16.2.1	<u>Dataset, DataLoader e Optimizer</u>	213
16.2.2	<u>Tensori</u>	215
16.2.3	SGD	216
16.3	CNN	217
16.3.1	<u>Domanda Standard</u>	217
16.3.2	<u>ResNet e Skip Connection</u>	219

16.4	Variational AutoEncoder	220
16.4.1	<u>Differenza fra AE e VAE</u>	220
17	Domanda 4	223
17.1	RNN	223
17.1.1	<u>Domanda su RNN</u>	223
17.2	AutoEncoder	225
17.2.1	<u>Domanda Standard</u>	225
17.3	VQ-VAE	226
17.3.1	<u>Quantization Trick e .detach</u>	226
17.4	GAN	227
17.4.1	<u>Training GAN</u>	227
17.4.2	<u>Domanda su ACGAN</u>	229

Parte I

Teoria

Capitolo 1: Introduzione al Deep Learning

1.1 Computational Intelligence e Machine Learning

La **Computational Intelligence** è un insieme di metodologie e approcci computazionali ispirati alla natura per affrontare problemi complessi del mondo reale. L'obiettivo è risolvere problemi complessi, stocastici o non lineari, per i quali gli algoritmi matematici tradizionali risultano inefficaci o troppo lenti. I pilastri principali di questa disciplina sono le **Neural Network** (ispirate al cervello), l'**Evolutionary Computing** (ispirato all'evoluzione darwiniana) e la **Fuzzy Logic** (che gestisce l'incertezza e il ragionamento sfumato).

Il **Machine Learning** è la capacità di un computer di imparare dall'esperienza. È un processo che produce un modello utilizzabile per fare previsioni o prendere decisioni a partire dai dati.

Il **Deep Learning** è un sottoinsieme del *Machine Learning* e della *Computational Intelligence*. Si basa sull'utilizzo di Reti Neurali Artificiali (Neural Network) "profonde", ovvero composte da molti strati (**layers**) di elaborazione consecutivi. Questa struttura a strati permette al modello di apprendere rappresentazioni dei dati via via più astratte e complesse, passando da concetti di base (come i contorni in un'immagine) a concetti semantici di alto livello (come riconoscere un volto o un oggetto specifico).

La creazione di un modello si articola in due fasi: **addestramento** e **inferenza**. Nell'addestramento il modello apprende dai dati del **training set**, mentre nell'inferenza viene utilizzato per assegnare etichette o valori a nuovi dati. Le prestazioni vengono valutate su un **test set** indipendente, mentre il **validation set** serve a guidare l'ottimizzazione durante l'addestramento.

I modelli possono imparare in modi diversi a seconda della natura dei dati a disposizione e dell'obiettivo da raggiungere. Si distinguono quattro paradigmi principali:

- **Supervised Learning:** Il modello impara da un dataset etichettato. Ad ogni input (esempio: l'immagine di un cane) è associato l'output corretto (l'etichetta "cane"). Il sistema calcola l'errore tra la sua predizione e l'etichetta reale e aggiusta i propri parametri per minimizzarlo. Ne fanno parte i problemi di *Classificazione*.
- **Unsupervised Learning:** Il modello riceve dati privi di etichette. Il suo scopo è trovare autonomamente una struttura nascosta, raggruppando dati simili o riducendo la dimensionalità delle informazioni. Ne fanno parte i problemi di *Clustering*.
- **Reinforcement Learning:** Un "agente" impara a comportarsi in un determinato ambiente andando per tentativi ed errori. Riceve delle "ricompense" se compie azioni positive o delle "penalità" per azioni negative, con lo scopo di massimizzare la ricompensa totale nel tempo (tipico nei giochi o nella robotica).
- **Self-Supervised Learning:** Un approccio in cui il modello genera le proprie etichette a partire dai dati stessi. Ad esempio, si nasconde una parte dell'input (come una parola in una frase) e si chiede al modello di predirla usando il contesto circostante.

Come si stabilisce se un sistema è davvero "intelligente"?

Storicamente, il punto di riferimento è stato il **Test di Turing**, un esperimento proposto nel 1950 dal matematico e informatico *Alan Turing* per valutare se una macchina possa essere considerata intelligente. L'idea è semplice: un esaminatore umano conversa, tramite testo, con due interlocutori nascosti, uno umano e uno artificiale. Se, dopo la conversazione, l'esaminatore non riesce a distinguere in modo affidabile quale dei due sia la macchina, allora la macchina ha superato il test. Qualcuno ritiene che il modello *ChatGPT-4* abbia già superato il test di Turing nel 2024.

Il filosofo *John Searle* propose nel 1980 il celebre esperimento mentale della **Stanza Cinese** per smentire il Test di Turing. Immagina una persona che non conosce il cinese chiusa in una stanza. Seguendo un enorme manuale di istruzioni, riesce a rispondere correttamente a messaggi scritti in cinese. Dall'esterno sembra comprendere la lingua, ma in realtà sta solo manipolando dei simboli secondo delle regole. Secondo Searle, un computer potrebbe fare qualcosa di analogo: produrre risposte corrette senza capire il significato. Per questi motivi, oggi il Test di Turing è visto soprattutto come una pietra miliare storica dell'Intelligenza Artificiale, più che come una prova definitiva di intelligenza.

Oggi l'attenzione si sta spostando verso i test per la **Artificial Capable Intelligence (ACI)**. Questi test non si limitano a valutare l'inganno linguistico, ma misurano la capacità pratica dell'**Intelligenza Artificiale (IA)** di portare a termine in autonomia compiti multi-fase complessi nel mondo reale (come gestire un progetto o risolvere problemi logistico/operativi).

A questo punto ci si potrebbe domandare perché sia necessario faticare per costruire un modello a partire da un dataset di coppie input-output, invece di inserire semplicemente tali coppie all'interno di una *Look-Up-Table* e utilizzare poi un approccio basato sul singolo vicino più prossimo (*1-Nearest Neighbor*)?

Un sistema basato su Look-Up-Table e 1-Nearest Neighbor (1-NN) non è in grado di effettuare un'interpolazione sui dati. L'interpolazione consiste nel cercare una funzione matematica complessa che passi esattamente attraverso tutti i punti del dataset. Inoltre un sistema basato su una tabella e 1-NN non è in grado di rimuovere il rumore dai dati. Possiamo pensare di usare k -NN con $k > 1$, ma ciò porta solo a scaricare l'intera complessità computazionale sulla fase di inferenza (invece che sull'addestramento). Tra l'altro, essendo un metodo privo di modello, questo sistema non ha alcuna capacità di comprimere l'informazione, di semplificarla o di sfruttare attivamente le regolarità che si trovano all'interno del training set.

Tuttavia, si preferisce l'**approssimazione** all'interpolazione perché si accetta un lieve errore sui dati noti del training set pur di ottenere un modello in grado di generalizzare. L'obiettivo è infatti costruire un sistema che, anche a costo di non adattarsi perfettamente al training set, sia capace di effettuare previsioni affidabili e robuste su dati nuovi e mai osservati prima.

Prima di andare avanti soffermiamoci un attimo su un argomento basilare, ma fondamentale. Un **Dataset** è una raccolta organizzata di dati, generalmente strutturata in righe e colonne, utilizzata per analisi, ricerca, Machine Learning o gestione delle informazioni.

I dataset possono contenere molti tipi di dati:

- Numerici (età, temperatura, prezzi);
- Testuali (recensioni, articoli, messaggi);
- Immagini;
- Audio;
- Video;
- Dati provenienti da sensori o dispositivi IoT (Internet of Things).

Nel Deep Learning i dati si dividono in:

- **training set**: addestra il modello aggiornando i pesi. È la porzione più grande del dataset, poiché la sua dimensione influisce sulle prestazioni del modello. Se è troppo piccolo può causare underfitting, mentre se è eccessivamente ampio aumenta tempi di addestramento e costi computazionali;
- **validation set**: utilizzato per ottimizzare gli iperparametri e monitorare l'overfitting. Non viene impiegato nell'addestramento del modello e consente di valutarne la capacità di generalizzare su dati non osservati;
- **test set**: valuta le prestazioni finali su dati mai visti. Sono dati completamente indipendenti dal train e dal validation set.

Questa separazione evita stime ottimistiche e garantisce una valutazione affidabile del modello.

1.2 Linear Least Squares

L'obiettivo fondamentale dell'*approssimazione lineare* (o **Regressione Lineare**) consiste nel trovare una relazione matematica semplice e gestibile tra un pattern di input p e il relativo output reale o . Infatti, quando si dispone di un insieme discreto di coppie di dati sperimentali (o osservazioni), il modo più immediato per approssimare la funzione sottostante è ipotizzare una relazione di tipo lineare. Il modello matematico per un singolo output si esprime attraverso l'equazione:

$$o = f(p) = v \cdot p + b$$

Dove i parametri liberi da stimare sono:

- $v \in \mathbb{R}^D$: un vettore colonna contenente i coefficienti lineari, ossia i **pesi**, associati alle D dimensioni dell'input p ;
- $b \in \mathbb{R}$: il **bias** (o termine noto), che geometricamente rappresenta lo spostamento della retta rispetto all'origine.

Per calcolare quanto il modello si discosti dai dati reali, si introduce una funzione di costo basata sul **Mean Squared Error (MSE)** istantaneo:

$$E = \frac{1}{2}(f(p) - o)^2$$

Non vogliamo minimizzare l'errore di un singolo punto o , ma di tutti i dati insieme. Per questo bisogna trovare i valori di v e b che rendono l'errore totale il più piccolo possibile. Questo problema prende il nome di **Linear Least Squares fitting** (*Metodo Lineare dei Minimi Quadrati*).

Invece di lavorare punto per punto, si mettono tutti i dati in una struttura unica. Su un intero dataset composto da N campioni $((p_1; o_1), (p_2; o_2), \dots, (p_N; o_N))$, è necessario organizzare le informazioni in strutture matriciali.

La **Matrice dei Dati Estesa** X ha dimensioni $(1+D) \times N$ ed è definita come una "*matrice piatta*" (una rappresentazione lineare di una matrice bidimensionale o multidimensionale, ottenuta convertendo tutti gli elementi in un array monodimensionale) poiché tipicamente il numero di campioni è molto superiore alle dimensioni dello spazio di input ($N \gg D$). D rappresenta la dimensionalità dell'input. All'interno di X si inserisce una prima riga interamente composta da valori 1 per accorpare il calcolo del bias b :

$$X = \begin{bmatrix} 1 & 1 & \dots & 1 \\ p_1^1 & p_1^2 & \dots & p_1^N \\ \vdots & \vdots & \dots & \vdots \\ p_D^1 & p_D^2 & \dots & p_D^N \end{bmatrix}$$

Il **Vettore dei Target** $y \in \mathbb{R}^{1 \times N}$ è un vettore riga che coincide con la trasposta del vettore colonna degli output reali o ($y = o^T$).

Tutti i parametri incogniti da mappare vengono racchiusi in un unico vettore riga globale $w \in \mathbb{R}^{1 \times (1+D)}$:

$$w = [b, v^T] = [b, v_1, v_2, \dots, v_D]$$

Il vantaggio straordinario dei Linear Least Squares è l'esistenza di una soluzione analitica esatta (forma chiusa) che non richiede cicli iterativi o la scelta manuale di un tasso di apprendimento, come con la *Gradient Descent*. La formula sfrutta la **tecnica della pseudo-inversa di Moore-Penrose**:

$$w = yX^T(XX^T)^{-1}$$

La complessità computazionale per calcolare questa espressione è legata principalmente all'inversione della matrice dei coefficienti ed è pari a $\mathcal{O}(N(D+1)^2)$.

In alcuni scenari, potremmo voler dare maggiore importanza ad alcuni campioni del dataset rispetto ad altri. Per fare ciò, assegniamo un peso non negativo δ_n a ciascuna coppia di input-output e costruiamo una matrice diagonale Δ contenente questi pesi sulla diagonale principale. La soluzione esatta (il vettore dei pesi w) si ottiene tramite una leggera variazione della formula della pseudo-inversa:

$$w = y\Delta X^T(X\Delta X^T)^{-1}$$

Dimostrazione Matematica della Formula

La minimizzazione dell'errore quadratico totale per il vettore di parametri w si imposta formalmente definendo la funzione obiettivo globale:

$$\min_w f(w) = \frac{1}{2}(wX - y)(wX - y)^T$$

Sviluppando il prodotto e sfruttando le proprietà della trasposta $(wX - y)^T = X^T w^T - y^T$, l'espressione quadratica si espande nella forma:

$$f(w) = \frac{1}{2} [w(XX^T)w^T - 2wXy^T + yy^T]$$

Per trovare il punto di minimo globale, si calcola la derivata parziale (il gradiente) rispetto al vettore dei pesi w e la si impone pari a zero. La derivazione procede termine a termine:

1. **Termine Quadratico:** sapendo che per una matrice simmetrica la derivata della forma quadratica $\frac{\partial}{\partial w}(wQw^T) = 2wQ$, ponendo $Q = XX^T$ si ottiene:

$$\frac{\partial}{\partial w} \left(\frac{1}{2} w(XX^T)w^T \right) = \frac{1}{2} \cdot 2w(XX^T) = wXX^T$$

2. **Termine Lineare:** la derivata rispetto a w di $-wXy^T$ è semplicemente il coefficiente trasposto:

$$\frac{\partial}{\partial w} (-wXy^T) = -(Xy^T)^T = -yX^T$$

3. **Termine Costante:** la derivata della componente yy^T non dipendendo da w si annulla:

$$\frac{\partial}{\partial w} \left(\frac{1}{2} yy^T \right) = 0$$

Combinando i tre passaggi, il gradiente finale risulta:

$$\nabla_w f(w) = wXX^T - yX^T = (wX - y)X^T$$

Uguagliando il gradiente al vettore nullo per determinare la condizione di ottimo:

$$(wX - y)X^T = 0 \implies wXX^T = yX^T$$

Isolando il vettore dei parametri w mediante la moltiplicazione a destra per l'inversa di (XX^T) , si ricava esattamente la formula finale della pseudo-inversa:

$$w = yX^T(XX^T)^{-1}$$

Per visualizzare e comprendere l'impatto pratico della formula dei Linear Least Squares, puoi interagire con questa applicazione grafica. Consente di posizionare punti arbitrari su un piano bidimensionale e analizzare in tempo reale la costruzione della matrice estesa X , l'inversione della matrice di covarianza $(XX^T)^{-1}$ e i parametri ottimali del modello (b e v).

1.3 Discesa del Gradiente

Sebbene il problema dei minimi quadrati lineari (Linear Least Squares) ammetta una soluzione analitica chiusa, i parametri del modello possono essere stimati anche mediante un approccio iterativo, noto come **Discesa del Gradiente (Gradient Descent)**.

Per ciascun campione (pattern) n , si definisce l'errore istantaneo come la perdita quadratica tra il valore target y_n e la predizione del modello $\hat{y}_n = p_n \cdot v + b$. Al fine di determinare la direzione di aggiornamento dei parametri, si calcolano le derivate parziali di tale errore rispetto al vettore dei pesi v e al termine di bias b .

Sviluppando il calcolo in forma vettoriale, si osserva che il gradiente risulta proporzionale alla differenza tra valore predetto e valore reale, moltiplicata per l'input. Tale formulazione consente un'efficiente valutazione computazionale, con complessità pari a $\mathcal{O}(D)$.

La funzione di perdita per un singolo campione n è definita come:

$$E_n = \frac{1}{2} (y_n - \hat{y}_n)^2, \quad \text{con} \quad \hat{y}_n = p_n \cdot v + b$$

Il gradiente rispetto ai parametri (applichi la Chain Rule) del modello è dato da:

$$\frac{\partial E_n}{\partial v} = -(y_n - \hat{y}_n) p_n \qquad \frac{\partial E_n}{\partial b} = -(y_n - \hat{y}_n)$$

In forma vettoriale, indicando con $p_n \in \mathbb{R}^D$, si ottiene:

$$\nabla_v E_n = (\hat{y}_n - y_n) p_n \qquad \nabla_b E_n = (\hat{y}_n - y_n)$$

Per l'intero dataset di N campioni, la funzione di costo è:

$$E = \frac{1}{N} \sum_{n=1}^N \frac{1}{2} (y_n - \hat{y}_n)^2$$

Gli aggiornamenti del Gradient Descent sono:

$$v \leftarrow v - \eta \frac{1}{N} \sum_{n=1}^N (\hat{y}_n - y_n) p_n$$

$$b \leftarrow b - \eta \frac{1}{N} \sum_{n=1}^N (\hat{y}_n - y_n)$$

L'algoritmo è il seguente:

1. **Inizializzazione:** Si parte assegnando valori casuali (o anche zeri) ai pesi v e b ;
2. **Learning Rate:** Si fissa un parametro critico chiamato *learning rate* (η), che decide la "lunghezza" dei passi da fare in ogni aggiornamento;
3. **Previsione:** Si calcola l'output predetto dal modello per i dati correnti;
4. **Aggiornamento:** Si modificano i pesi sottraendo il gradiente moltiplicato per η , muovendosi così nella direzione di massima discesa dell'errore. Questi passaggi vengono ripetuti ciclicamente fino a quando i parametri si stabilizzano (convergenza) o si raggiunge un numero massimo di iterazioni prefissato.

C'è da considerare che un'iterazione completa sul dataset presenta una complessità computazionale pari a $\mathcal{O}(N(1 + D))$, dove N è il numero di campioni e D la dimensionalità dell'input. Il learning rate influenza in modo determinante la convergenza: valori troppo bassi rallentano l'ottimizzazione, mentre valori eccessivi possono causare instabilità numerica, oscillazioni dei parametri e possibile divergenza del processo. Per favorire una convergenza stabile ed efficiente, è spesso necessario applicare tecniche di normalizzazione o scalatura dei dati prima dell'addestramento del modello.

Perché dovremmo usare un metodo iterativo quando esiste una formula esatta per i minimi quadrati?

In scenari su larga scala, il calcolo della pseudo-inversa richiede l'inversione di matrici, operazione computazionalmente onerosa, mentre il Gradient Descent si basa su aggiornamenti iterativi tramite semplici prodotti matrice-vettore.

Dal punto di vista della memoria, la soluzione in forma chiusa richiede la gestione dell'intera matrice dei dati simultaneamente, con possibile saturazione delle risorse, mentre il Gradient Descent opera su rappresentazioni parziali, riducendo significativamente l'occupazione di memoria.

Inoltre, la natura iterativa del Gradient Descent ne facilita la parallelizzazione su architetture moderne come GPU e sistemi multi-core, consentendo una distribuzione efficiente del carico computazionale.

1.4 Dalla Regressione alla Classificazione

Dopo aver trattato i Linear Least Squares (che sono un problema di **Regressione**), vediamo invece come affrontare i problemi di **Classificazione**.

Un problema di Classificazione è un compito di Machine Learning in cui l'obiettivo è assegnare un'osservazione a una categoria discreta tra un insieme finito di classi. Dato un input (ad esempio un'immagine, un testo o un vettore di caratteristiche), il modello apprende una funzione che associa tale input a un'etichetta, come "spam/non spam" o "malato/sano".

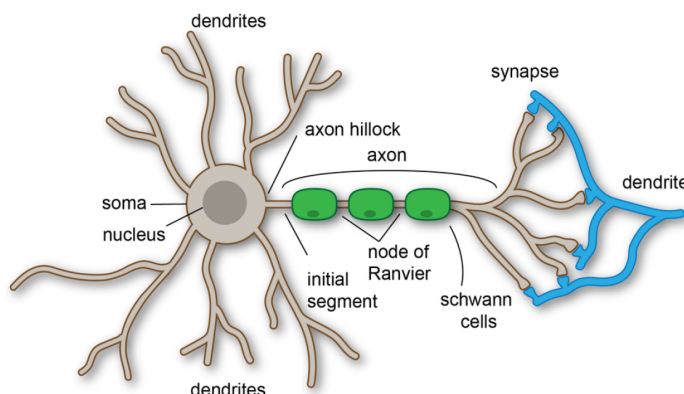
Ci sono tre tipi di Classificazione:

- **Classificazione Binaria:** prevede due sole classi possibili e ogni istanza appartiene a una sola delle due;
- **Classificazione Multi-classe mutuamente esclusiva (MEMC):** estende il caso binario a più di due classi, ma ogni istanza è assegnata a una sola classe tra le possibili alternative, ad esempio il riconoscimento di una cifra tra 0 e 9;
- **Classificazione Multi-label:** l'input può appartenere a più classi contemporaneamente, tipico della Computer Vision moderna.

Nella prossima sezione vediamo come risolvere un problema di Classificazione tramite lo stesso meccanismo delle Linear Least Squares.

1.5 Perceptron

Le **Artificial Neural Network** sono modelli matematici ispirati, in modo semplificato, alla struttura dei **neuroni biologici** del sistema nervoso. In ambito biologico, un neurone è costituito da un soma, da cui si diramano i dendriti, responsabili della ricezione dei segnali in ingresso, e l'assone, che trasmette l'output verso le sinapsi.

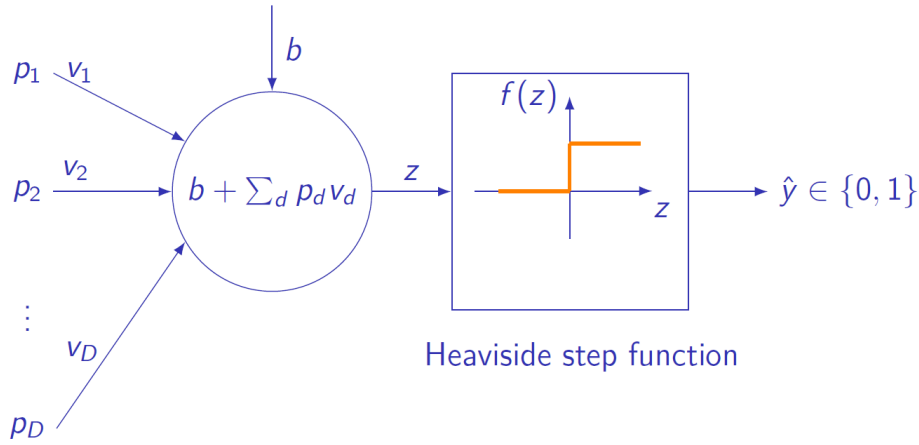


Non è tuttavia necessario che un **neurone artificiale** riproduca fedelmente tale complessità, non ancora completamente compresa. Il neurone biologico è una cellula reale caratterizzata da dinamiche elettrochimiche complesse e comunicazione tramite impulsi sinaptici. Il neurone artificiale è invece un modello matematico statico (non dinamico) semplificato che combina input numerici tramite pesi e applica una funzione di attivazione per produrre un'uscita. Entrambi condividono un'analogia funzionale di base, ma l'approccio ingegneristico si limita a cogliere i principi essenziali, organizzando l'informazione in reti stratificate in cui essa viene progressivamente trasformata e resa più astratta.

Storicamente, la prima Artificial Neural Network dotata di un vero algoritmo di addestramento (la *Delta Rule*) è stata il **Perceptron** di *Frank Rosenblatt* (sviluppato nel 1958). Il modello matematico del Perceptron si basa sulla **funzione di attivazione di Heaviside** (una funzione a gradino):

$$\hat{y} = \text{Heaviside}(p \cdot v + b)$$

Se la combinazione lineare degli input supera lo zero, l'output $\hat{y}$ è pari a 1, altrimenti è 0. In questo modo possiamo anche risolvere problemi di classificazione binaria utilizzando questa funzione di attivazione dopo l'output delle Linear Least Squares.



La *Delta Rule* procede per step, valutando un pattern di input p_n (un vettore riga) alla volta. Inizialmente, i pesi v e b vengono impostati su valori casuali. Successivamente, per ogni input, il modello calcola la propria previsione $\hat{y}_n$ applicando la funzione a gradino di Heaviside alla combinazione lineare degli ingressi:

$$\hat{y}_n = \text{Heaviside}(p_n \cdot v + b)$$

A questo punto si calcola l'errore commesso, dato dalla differenza tra la previsione e l'etichetta reale: $(\hat{y}_n - y_n)$. Poiché sia l'output della funzione Heaviside che le etichette target possono valere solo 0 o 1, questo errore può assumere esclusivamente tre valori discreti: 0, 1, oppure -1. La Delta Rule aggiorna i parametri del modello secondo queste due equazioni (quelle già viste nella Gradient Descent):

$$\begin{aligned} b &= b - \eta(\hat{y}_n - y_n) \\ v &= v - \eta(\hat{y}_n - y_n)p_n^T \end{aligned}$$

Se la previsione è corretta ($\hat{y}_n = y_n$), il termine di errore si annulla e i parametri v e b rimangono invariati. Se invece il modello commette un errore di classificazione, la regola "sposta" i parametri in una direzione geometrica tale da rendere più probabile una risposta corretta se lo stesso input venisse ripresentato in futuro.

Per quanto rivoluzionario, il Perceptron singolo mostra un limite invalicabile: non è in grado di risolvere problemi di Classificazione in cui le classi non sono **linearmente separabili**, come la funzione logica XOR.

Geometricamente, un Perceptron traccia una linea (in due dimensioni), un piano (in tre dimensioni) o, più in generale, un iperpiano nello spazio dei dati. Questa linea divide lo spazio in due regioni: da una parte la classe 0, dall'altra la classe 1. Tuttavia non esiste alcuna retta che separi perfettamente i punti $(0; 1), (1; 0)$ da $(0; 0), (1; 1)$. Quindi il problema non è l'algoritmo di apprendimento. Il problema è proprio il modello. Un singolo Perceptron può creare solo una frontiera lineare. Per ricreare lo XOR si richiede una frontiera più complessa.

È importante notare che il Perceptron applica una trasformazione non lineare dell'input tramite la funzione di Heaviside, che però è non differenziabile (derivata nulla quasi ovunque e non definita in zero). Per questo motivo non è possibile applicare direttamente il Gradient Descent. La Delta Rule fu inizialmente introdotta in modo empirico, tramite tentativi ed errori (*trial and error*), ma successivamente è stata interpretata come equivalente allo **Stochastic Gradient Descent** sotto una specifica funzione di perdita, la **negative log-likelihood**. Per poter utilizzare il Gradient Descent in modo diretto, è quindi necessario sostituire la funzione di Heaviside con una funzione differenziabile.

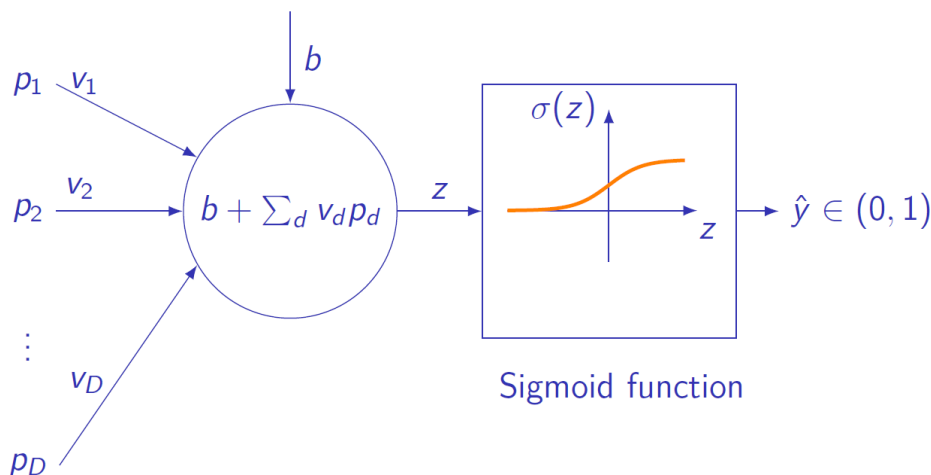
Questi due problemi hanno causato il primo "inverno dell'IA", un periodo di calo di interesse per l'intelligenza artificiale da parte di ricercatori, accademici e professionisti.

1.6 Loss Functions

Cosa è una Loss Function?

Ricapitolando abbiamo un limite strutturale, legato alla "capacità espressiva" del modello, e un limite legato all'algoritmo di ottimizzazione. Il primo problema si risolse usando **Perceptron multipli interconnessi tra di loro**. Il secondo problema si risolse **sostituendo la funzione di Heaviside con la funzione Sigmoidea**:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$



Prima il campione veniva classificato correttamente oppure erroneamente: non esisteva una misura di quanto la predizione fosse vicina al target. Con output continuo $\hat{y} \in (0, 1)$, l'errore binario del Perceptron non è più adeguato.

Se ho due campioni:

1. $y = 1, \hat{y} = 0.51$
2. $y = 1, \hat{y} = 0.99$

Entrambe le predizioni sarebbero classificate come corrette usando una soglia di 0.5, ma intuitivamente la seconda è una predizione qualitativamente molto migliore della prima.

Per questo serve una funzione di costo continua che trasformi un problema di Classificazione logica in un problema di ottimizzazione geometrica continua: una Loss Function.

In pratica, una **Loss Function** agisce come una funzione di costo che misura quantitativamente la distanza matematica tra la probabilità predetta $\hat{y}$ e l'etichetta reale y . Questo ci permette di creare una vera e propria superficie dell'errore (un panorama di valli e colline), dove possiamo calcolare il gradiente in ogni punto e guidare i parametri v e b verso il punto di minimo globale.

Differenza tra Loss Function ed Error Function

Nel Deep Learning, i termini **Error Function** (*funzione di errore*) e **Loss Function** (*funzione di costo*) sono spesso usati in modo intercambiabile, ma possono avere interpretazioni leggermente diverse a seconda del contesto.

Il termine "*Error Function*" si riferisce in genere a un concetto più ampio che comprende qualsiasi misura delle prestazioni o dell'accuratezza (*Accuracy*) del modello. Può rappresentare la discrepanza complessiva tra gli output previsti e i valori target reali. In questo senso, la funzione di errore è un termine più generale che può essere utilizzato per descrivere diverse metriche di valutazione, come il Mean Squared Error (MSE), il Mean Absolute Error (MAE), o il tasso di errore di Classificazione.

Il termine "*Loss Function*" viene spesso utilizzato specificamente per riferirsi alla funzione obiettivo che viene ottimizzata durante l'addestramento di una Neural Network. La funzione di perdita quantifica la discrepanza tra gli output previsti e i valori target reali, e l'obiettivo è minimizzare tale valore.

In pratica, la Error Function e la Loss Function possono riferirsi alla stessa quantità, soprattutto quando la Loss viene utilizzata come misura principale delle prestazioni durante l'addestramento. Tuttavia, è importante notare che in alcuni casi la Error Function può essere una metrica di valutazione diversa utilizzata per riportare le prestazioni del modello dopo l'addestramento, mentre la funzione di perdita è la funzione specifica che viene minimizzata durante il processo di addestramento.

Mean Squared Error

Verrebbe naturale riciclare il Mean Squared Error, già usato nei Linear Least Squares. La formula matematica dell'MSE è la seguente:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Se il modello sbaglia di 2, l'errore quadratico è 4. Se sbaglia di 10, l'errore quadratico diventa 100. L'MSE è severissimo con i grandi errori. Questo è un bene se vuoi un modello che non prenda mai "cantonate" colossali. Dal punto di vista del calcolo (esempio: la Gradient Descent), una funzione quadratica è continua e differenziabile ovunque, il che rende molto più facile per gli algoritmi trovare il punto di errore minimo.

Tuttavia, accoppiare l'MSE con un neurone sigmoideo crea un grave problema noto come **Vanishing Gradient**.

Supponiamo di avere un campione di questo tipo:

- Target: $y = 1$
- Predizione: $\hat{y} = 0.0001$

Il neurone è praticamente certo che la risposta sia 0, mentre la risposta corretta è 1. Intuitivamente, sappiamo che l'errore qua è molto alto e **vorremmo che di conseguenza il Perceptron si correggesse di molto**. Invece **succede il contrario**. Se calcoliamo il gradiente dell'MSE rispetto a un peso utilizzando la sigmoide, la Chain Rule porta a includere la derivata della sigmoide stessa, che sappiamo essere $\sigma'(z) = \hat{y}(1 - \hat{y})$.

La *Chain Rule* è una regola del calcolo differenziale che si usa quando una funzione è "composta" da più funzioni. Se abbiamo $y = f(g(x))$ la derivata diventa: $\frac{dy}{dx} = \frac{df}{dg} \cdot \frac{dg}{dx}$. Nel neurone hai una catena di operazioni:

1. **Somma Pesata:** $z = v \cdot p + b$;
2. **Attivazione Sigmoidale:** $\hat{y} = \sigma(z)$;
3. **MSE come Loss Function:** $\mathcal{L} = (\hat{y} - y)^2$

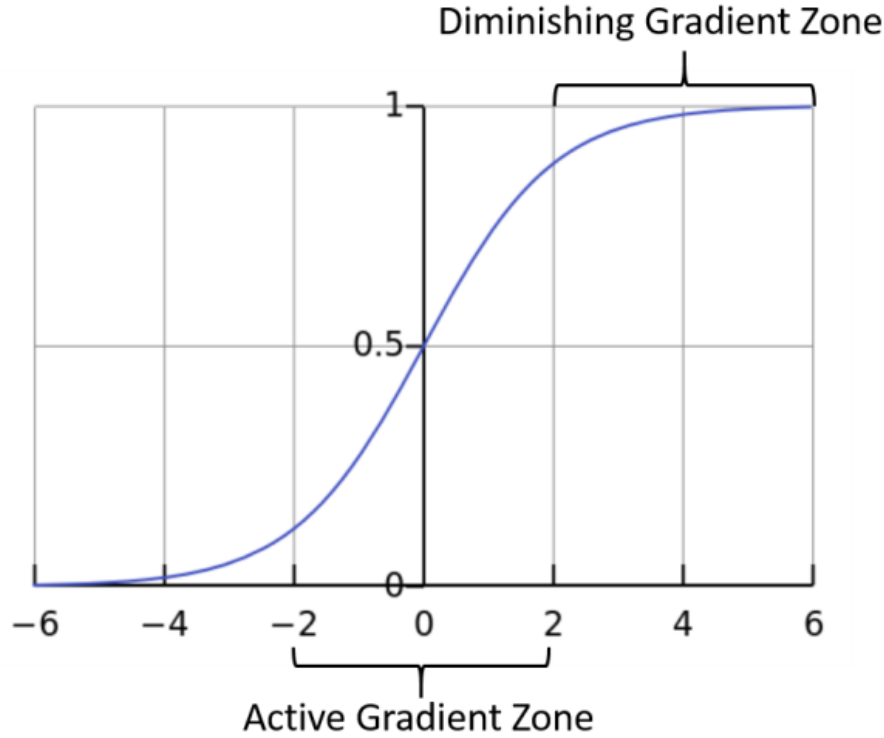
Quindi il peso v non influenza direttamente la loss, ma passa attraverso due passaggi. Quando vuoi capire come modificare v , devi calcolare:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Il termine centrale è $\frac{\partial y}{\partial z} = \sigma'(z) = \hat{y}(1 - \hat{y})$. Ricordandoci che $\hat{y} = 0.0001$, verrà:

$$\hat{y}(1 - \hat{y}) \approx 0.0001 \cdot 0.9999 \approx 0$$

Di conseguenza tutto il gradiente si annulla completamente. Per capire intuitivamente perché succede, pensa alla sigmoide come a una curva a "S".



Al centro ($z \approx 0$) la curva è ripida: piccoli cambiamenti nei pesi modificano molto l'output. Agli estremi ($z \gg 0$ oppure $z \ll 0$) la curva è quasi piatta ed è schiacciata. Quando la curva è piatta, la derivata è quasi zero. Il gradiente quindi "si spegne".

Questo è un problema che si manifesta solo nei problemi di Classificazione, infatti nella Regressione si continua ad usare l'MSE come metrica.

Binary Cross-Entropy

Per superare questo problema nella Classificazione, si introduce la **Binary Cross-Entropy (BCE)**:

$$\mathcal{L}^{\text{BCE}} = -y \cdot \log(\hat{y}) - (1 - y) \cdot \log(1 - \hat{y})$$

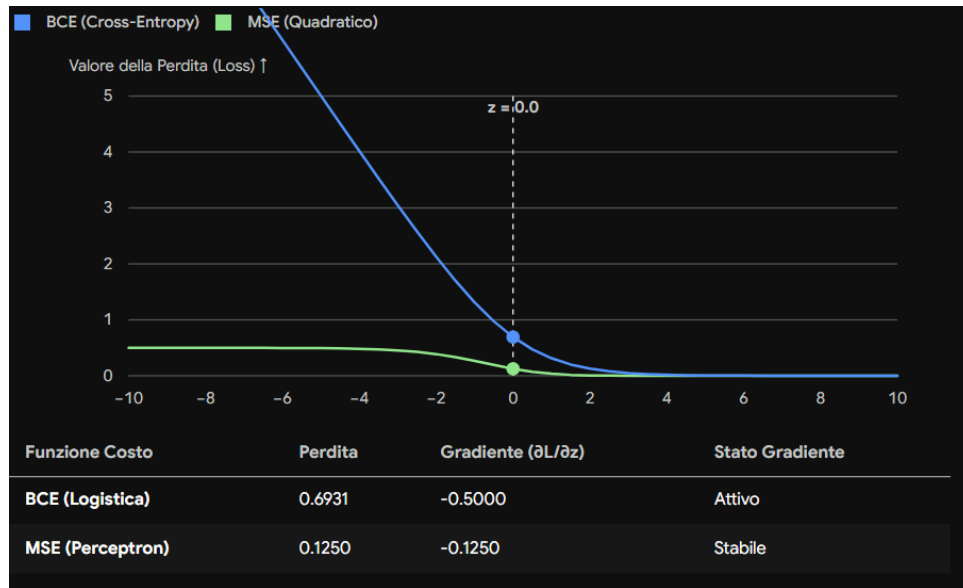
Questa specifica formulazione risolve il problema del Vanishing Gradient grazie a due proprietà:

- **Punizione asintotica degli errori gravi:** Se il target è $y = 1$ e il modello predice erroneamente $\hat{y} \rightarrow 0^+$, il termine $-\log(\hat{y})$ tende a $+\infty$. La loss punisce l'errore con un costo infinito, deformando la superficie per creare una forte pendenza che spinge via i parametri da quella zona tossica;

- **Saturazione della Derivata:** Quando deriviamo matematicamente la BCE rispetto ai pesi, la derivata parziale della loss rispetto all'output è $\frac{\partial \mathcal{L}^{\text{BCE}}}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})}$. Moltiplicando questo termine per la derivata della sigmoide (tramite Chain Rule), il denominatore della BCE cancella perfettamente il termine di appiattimento della sigmoide:

$$\frac{\partial \mathcal{L}^{\text{BCE}}}{\partial w_j} = \frac{\hat{y} - y}{\hat{y}(1 - \hat{y})} \cdot \hat{y}(1 - \hat{y}) \cdot x_j = (\hat{y} - y)x_j$$

Così il gradiente finale risultante è pulito, lineare e direttamente proporzionale all'errore commesso $(\hat{y} - y)$.



Quando si lavora su un intero batch o dataset composto da B campioni, la loss complessiva diventa la media aritmetica delle singole loss:

$$\mathcal{L}_{\text{batch}}^{\text{BCE}} = -\frac{1}{B} \sum_{b=1}^B [y_b \cdot \log(\hat{y}_b) + (1 - y_b) \cdot \log(1 - \hat{y}_b)]$$

Se si calcolasse il gradiente sull'intero dataset, adottando il metodo esatto noto come Full Batch Gradient Descent, si otterrebbe una stima estremamente accurata della direzione di massima riduzione della funzione di errore. Tuttavia, il paesaggio di ottimizzazione delle reti neurali è caratterizzato da una notevole complessità e presenta numerosi minimi locali. In tali condizioni, un gradiente calcolato con elevata precisione potrebbe condurre il processo di apprendimento a convergere prematuramente verso uno di questi minimi, limitando la capacità del modello di individuare soluzioni migliori.

L'impiego di mini-batch (ad esempio, gruppi di 32 immagini) consente invece di calcolare una stima approssimata e intrinsecamente rumorosa del gradiente. Questa componente stocastica introduce fluttuazioni nella traiettoria di aggiornamento dei parametri, che non segue più un percorso deterministico ma presenta variazioni continue. Tale comportamento risulta particolarmente vantaggioso, poiché può favorire l'uscita da regioni associate a minimi locali e agevolare l'esplorazione dello spazio delle soluzioni, aumentando la probabilità di convergere verso minimi più robusti e caratterizzati da una migliore capacità di generalizzazione.

Mean Absolute Error

Il **Mean Absolute Error (MAE)** è una Loss Function utilizzata per i problemi di Regressione. A differenza dell'MSE (che eleva l'errore al quadrato), il MAE calcola semplicemente la media dei valori assoluti degli errori commessi:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

È estremamente robusta agli outlier. Poiché l'MSE eleva gli errori al quadrato, un singolo campione completamente fuori scala genererà un errore gigantesco, "tirando" i pesi della rete verso di sé e distorcendo il modello. Il MAE, crescendo linearmente, non dà un peso sproporzionato agli errori estremi.

C'è però un problema: il valore assoluto rende la funzione non differenziabile nell'origine (quando l'errore è zero). Inoltre, il suo gradiente rimane costante (pari a +1 o -1) anche quando ci si avvicina al minimo, rischiando di far "rimbalzare" i parametri attorno alla soluzione ottimale se il learning rate non viene ridotto adeguatamente.

Huber Loss

La **Huber Loss** è una funzione di perdita ibrida, concepita per combinare i principali vantaggi della MSE e della MAE. Introduce un **iperparametro di soglia**, chiamato δ (delta), che definisce la transizione tra due comportamenti matematici distinti:

$$L_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{se } |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2 & \text{se } |y - \hat{y}| > \delta \end{cases}$$

Se l'errore è piccolo (inferiore alla soglia δ), la funzione si comporta come un MSE: diventa una curva quadratica dolce e perfettamente differenziabile che aiuta la rete a convergere dolcemente verso il minimo. Se l'errore è grande (superiore a δ), la funzione diventa lineare come il MAE, garantendo la robustezza contro gli outlier e impedendo che i gradienti esplodano.

L'MSE si usa quando gli outlier sono rari (ma importanti) e si vogliono evitare grossi errori. Minimizzare l'MSE equivale a stimare la media condizionata ed è ottimale se gli errori seguono una distribuzione gaussiana.

Il MAE si usa se i dati contengono outlier o rumore significativo e si vuole una loss più robusta. Minimizzare il MAE equivale a stimare la mediana condizionata ed è più robusta quando gli errori hanno code pesanti o outlier.

Si usa la Huber Loss se si vuole un compromesso tra le due, che sia convesso e differenziabile (C^1).

Hinge Loss

La **Hinge Loss** è storicamente associata alle **Support Vector Machines (SVM)**, ma può essere utilizzata anche come funzione di output nelle reti neurali per la classificazione binaria (spesso definita come Maximum-Margin Classification).

Richiede che le etichette delle classi non siano codificate come 0 e 1, ma come -1 e +1. La formula è:

$$L = \max(0; 1 - y \cdot \hat{y})$$

Questa funzione introduce il concetto di *margin*: non è sufficiente classificare correttamente un esempio, ma è necessario farlo con un adeguato livello di confidenza:

- Se y e $\hat{y}$ hanno lo stesso segno e $|\hat{y}| \geq 1$, allora $L = 0$ e non viene applicata alcuna penalità;
- Se la classificazione è corretta ma $|\hat{y}| < 1$, la perdita è positiva e il modello riceve una penalità proporzionale alla distanza dal margine;
- Se la classificazione è errata (y e $\hat{y}$ hanno segno opposto), la perdita aumenta linearmente all'aumentare dell'errore.

Kullback-Leibler Divergence

La **Kullback-Leibler Divergence (KLD)** è una misura proveniente dalla teoria dell'informazione che serve a quantificare quanto una distribuzione di probabilità predetta (P) si discosti da una distribuzione di probabilità di riferimento o reale (Q):

$$D_{KL}(P \parallel Q) = \sum_x P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

Viene usata nella Classificazione.

La KLD misura, quindi, quanta informazione perdiamo utilizzando Q al posto di P . In altre parole: quanto è "sbagliata" Q rispetto a P . Può essere interpretata come il numero medio di bit aggiuntivi necessari per codificare dati generati da P usando un codice ottimizzato per Q .

Considerando il termine $\log \left(\frac{P(x)}{Q(x)} \right)$, abbiamo che se $P(x) = Q(x) \implies \log(1) = 0$ e quindi non c'è alcuna penalità. Se $P(x) > Q(x)$ significa che il modello sottostima un evento importante e quindi la penalità aumenta. Se invece abbiamo che $P(x) < Q(x)$ la penalità diventa negativa, ma il peso $P(x)$ nella somma fa sì che il risultato complessivo resti sempre non negativo. Un singolo termine della KLD può essere negativo, ma la somma pesata secondo $P(x)$ non può mai diventare negativa perché la KLD è una misura di inefficienza informativa, e nessuna codifica basata su una distribuzione sbagliata può risultare mediamente migliore della codifica corretta.

Non è una distanza perché $D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P)$.

L'**Entropia** misura l'incertezza intrinseca dei dati reali. Poiché il nostro dataset è fisso e le etichette non cambiano durante l'addestramento, questo termine è una costante fissa. La **Cross-Entropy** si usa per quantificare quanto una distribuzione di probabilità stimata si discosta da una distribuzione "vera" o di riferimento. Poiché l'Entropia di P è una costante ineliminabile, **minimizzare la Cross-Entropy equivale matematicamente a minimizzare la KLD**. Quando la Neural Network usa la Cross-Entropy come funzione di Loss, sta di fatto cercando di azzerare la divergenza KLD tra le sue predizioni e le etichette reali:

$$\text{Cross-Entropy}(P; Q) = \text{Entropy}(P) + \text{KLD}(P \parallel Q)$$

Se minimizzare la Cross-Entropy è uguale a minimizzare la KLD, perché le librerie di Deep Learning offrono la KLD come funzione separata?

Quando usiamo il One-Hot Encoding (dove la probabilità della classe vera è 1 e le altre 0), l'Entropia di P è esattamente zero. La Cross-Entropy diventa identica alla KLD, ed è computazionalmente più veloce da calcolare. Tuttavia, la KLD viene chiamata esplicitamente quando la distribuzione target P NON è One-Hot (ovvero ha un'entropia maggiore di zero). Si ha questo caso nella Knowledge Distillation, nei Variational Autoencoders e nel Reinforcement Learning.

Wing Loss

La **Wing Loss** è una funzione altamente specializzata, originariamente sviluppata per i compiti di Computer Vision legati al Facial Landmark Detection (il rilevamento dei punti chiave del viso, come i contorni degli occhi o della bocca). In questi compiti, è cruciale che il modello sia estremamente preciso sui piccoli dettagli. La funzione è definita a tratti:

$$L_{wing}(y, \hat{y}) = \begin{cases} w \log(1 + \frac{|y - \hat{y}|}{\varepsilon}) & \text{se } |y - \hat{y}| < w \\ |y - \hat{y}| - C & \text{altrimenti} \end{cases}$$

dove w e ε sono parametri costanti che ne definiscono la forma, e C serve a raccordare i due tratti matematici. w è tipicamente scelta tra 5 e 15, mentre ε è tipicamente scelta tra 1 e 5.

La Wing Loss ribalta la logica della Huber Loss. Invece di essere quadratica per gli errori piccoli e lineare per i grandi, utilizza una curva logaritmica per gli errori piccoli e una lineare per quelli grandi. La componente logaritmica amplifica artificialmente l'importanza dei piccoli errori. Questo forza la rete neurale a continuare a ottimizzare e rifinire le sue predizioni anche quando si trova già molto vicina al target, garantendo un posizionamento dei punti chiave (**landmark**) di precisione millimetrica.

1.7 Logistic Regression

La **Logistic Regression** è una tecnica di modellazione statistica utilizzata per risolvere problemi di Classificazione binaria. A dispetto del nome "*Regressione*", l'obiettivo finale non è prevedere un valore continuo, ma determinare a quale classe appartenga un determinato input.

Convenzionalmente, le due classi vengono codificate matematicamente associando il valore 1 alla classe target (ω_T) e il valore 0 alla classe degli outlier (ω_O). La struttura di una regressione logistica ricalca da vicino quella di un singolo neurone artificiale, ma introduce una modifica fondamentale rispetto al Perceptron classico:

1. **Combinazione Lineare (Logit):** Per prima cosa, il modello calcola una combinazione lineare delle feature di input p pesate tramite un vettore di pesi v a cui viene sommato un termine noto o bias b :

$$z = b + \sum_{d=1}^D v_d p_d = w \cdot x$$

2. **Funzione di Attivazione (Sigmoide):** Il valore continuo z (chiamato pre-attivazione) può variare tra $-\infty$ e $+\infty$. Per trasformarlo in un valore interpretabile come probabilità, viene passato attraverso la funzione sigmoide (o logistica):

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

3. **Frontiera di Decisione:** Per prendere la decisione finale sul posizionamento della classe, si applica una soglia fissa a 0.5:
 - Se $\hat{y} \geq 0.5$ (che equivale a dire $z \geq 0$), il pattern viene classificato come appartenente alla classe target (1);
 - Se $\hat{y} < 0.5$ (ovvero $z < 0$), il pattern viene rifiutato e classificato come outlier (0).

Per misurare l'accuratezza delle predizioni durante la fase di calcolo e calibrare i pesi, non si usa l'errore quadratico medio, bensì la **Binary Cross-Entropy**. L'obiettivo è minimizzare questa loss function per ridurre al minimo gli errori di Classificazione.

A differenza dei Linear Least Squares, la Logistic Regression **non possiede una soluzione analitica in forma chiusa** calcolabile direttamente tramite operazioni matriciali dirette (escludendo il caso eccezionale monodimensionale dopo l'ordinamento dei dati). Bisogna obbligatoriamente ricorrere a un **algoritmo iterativo** come la Gradient Descent.

Tuttavia, l'accoppiamento della sigmoide con la loss BCE genera una superficie di errore non lineare ma strettamente convessa. Pertanto un problema di Logistic Regression è **non lineare, ma convesso** e ciò significa che esiste un unico ottimo globale. Di conseguenza la Gradient Descent non rischia di rimanere bloccata in minimi locali e guiderà sempre i parametri verso un **ottimo globale**, a prescindere dai valori casuali scelti all'inizializzazione.

Ad ogni iterazione del ciclo di addestramento, il vettore dei pesi viene aggiornato muovendosi nella direzione opposta al gradiente, scalato dal parametro del learning rate η :

$$w = w - \frac{\eta}{B}(\hat{y} - y)X^T$$

Possiamo concludere che i tre "ingredienti" fondamentali per l'addestramento di un neurone o di una rete di neuroni sono:

- **Ingrediente 1:** Il modello del singolo neurone;
- **Ingrediente 2:** Una loss function, definita in relazione al problema che si intende risolvere;
- **Ingrediente 3:** Un algoritmo in grado di aggiornare i parametri liberi del neurone (o dei neuroni in una rete), al fine di ottimizzare la Loss Function (ad esempio: Gradient Descent, metodo di Newton-Raphson, eccetera ...).

I **logits** sono gli output grezzi (*raw outputs*) non normalizzati generati dall'ultimo strato lineare di una Neural Network, calcolati un attimo prima di essere passati a una Funzione di Attivazione finale. In statistica, il "logit" è la funzione inversa della sigmoide logistica, usata per mappare probabilità $[0; 1]$ in numeri reali $[-\infty, +\infty]$. Nel Deep Learning, il termine è stato "rubato" e invertito per indicare i numeri reali prima che vengano trasformati in probabilità).

L'ultimo strato di una Neural Network è solitamente un **layer Fully Connected** (quindi un livello lineare). L'operazione matematica che compie è una semplice moltiplicazione di matrici più un bias:

$$z = W \cdot x + b$$

Quel vettore risultante z è il vettore dei logits.

I logits possono assumere qualsiasi valore reale, ma non sono probabilità, infatti la loro somma non è 1. In pratica, il logit rappresenta la pura "confidenza" interna della rete neurale verso una specifica classe, senza badare alle proporzioni matematiche.

Utilizzando più Perceptron completamente connessi (*fully connected*), la rete trasforma progressivamente i dati in uno **spazio latente** in cui risultano linearmente separabili. L'aggiunta di soli layer lineari non è sufficiente, poiché la loro composizione rimane una trasformazione lineare. Le *funzioni di attivazione non lineari* superano questo limite, consentendo la costruzione di frontiere decisionali complesse.

1.8 Funzioni di Attivazione

Una **Funzione di Attivazione** è una funzione applicata all'output di un neurone artificiale per introdurre non linearità nel modello. Senza funzione di attivazione, una rete di neuroni sarebbe equivalente a un singolo modello lineare, indipendentemente dal numero di layer. Con l'attivazione, invece, la rete può apprendere relazioni complesse e non lineari.

Le funzioni di attivazione possono essere prive di parametri (fisse) oppure contenere parametri configurabili. In quest'ultimo caso si distinguono due approcci:

- Il parametro viene scelto a priori dall'utente attraverso un approccio trial-and-error;
- Il parametro viene appreso direttamente durante la fase di addestramento insieme ai pesi e ai bias. Questo secondo approccio estende la flessibilità del modello, ma tipicamente rallenta la velocità di convergenza dell'algoritmo di ottimizzazione.

Sigmoide

La **Sigmoide** l'abbiamo già vista:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Schiaccia l'intero asse reale nell'intervallo aperto $(0; 1)$. Risulta particolarmente utile nello strato di output per problemi di Classificazione binaria o multi-label.

Tangente Iperbolica

La **Tangente Iperbolica** è una funzione simmetrica rispetto all'origine (zero-centered), il che spesso facilita l'ottimizzazione rispetto alla sigmoide. Mappa l'input nell'intervallo $(-1; 1)$ ed è definita come il rapporto tra seno iperbolico e coseno iperbolico:

$$\tau(z) = \tanh(z) = \frac{\sinh(z)}{\cosh(z)} = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Rectified Linear Unit

La Rectified Linear Unit rappresenta lo standard *de facto* per gli hidden layer delle Deep Neural Network grazie alla sua semplicità computazionale e alla capacità di mitigare il problema del Vanishing Gradient per input positivi. La formula è ad attivazione netta:

$$\text{ReLU}(z) = \max(0; z)$$

L'output è nullo per tutte le pre-attivazioni negative, mentre coincide con l'input stesso se questo è positivo.

Vediamo alcune varianti della ReLU:

- **Leaky ReLU**: introduce una piccola pendenza per $z < 0$ per evitare la "morte della ReLU" (neuroni con gradiente nullo).

$$\text{LeakyReLU}(z) = \begin{cases} z & z \geq 0 \\ \alpha z & \text{altrimenti} \end{cases}$$

dove α è una costante fissata prima del training.

- **PReLU**: variante della Leaky ReLU in cui α non è fissato ma viene appreso durante l'addestramento.

$$\text{PReLU}(z) = \begin{cases} z & z \geq 0 \\ \alpha z & \text{altrimenti} \end{cases}$$

Linear Unit

La **Linear Unit** non applica alcuna distorsione non lineare e si limita a propagare il valore di pre-attivazione inalterato:

$$\text{LU}(z) = z$$

Trova la sua applicazione naturale nei neuroni dell'output layer per problemi di Regressione numerica [dove l'output può variare liberamente nell'intervallo $(-\infty; +\infty)$].

Funzione Segno

La **Sign** rappresenta la versione rigida della tangente iperbolica:

$$\text{Sign}(z) = \begin{cases} +1 & \text{se } z \geq 0 \\ -1 & \text{se } z < 0 \end{cases}$$

È una funzione differenziabile quasi ovunque ed utilizzata principalmente a tempo di inferenza all'interno di architetture hardware specifiche, come le reti neurali binarie (Binary Neural Networks) e le reti spiking (Spiking Neural Networks).

SoftMax

A differenza di tutte le funzioni esaminate in precedenza, la **SoftMax** non opera sul singolo nodo in modo isolato, ma accetta un vettore di pre-attivazioni e restituisce un vettore di attivazioni della medesima dimensionalità. Per questo motivo è considerata una funzione di attivazione per-layer (di livello):

$$\text{SoftMax}(z_c) = \frac{e^{z_c}}{\sum_{k=1}^C e^{z_k}}$$

C rappresenta il numero complessivo di classi del problema. La SoftMax normalizza le esponenziali dei componenti in modo che la loro somma globale sia esattamente pari a 1, generando una distribuzione di probabilità mutuamente esclusiva ideale per i problemi di Classificazione multi-classe.

1.9 Backpropagation

La **Forward Pass** è la fase che abbiamo visto fino ad ora. In questa fase la Neural Network elabora un input e produce una previsione tramite passaggi successivi tra i suoi strati. I dati vengono inseriti nel livello di input e trasformati nei livelli nascosti, dove ogni neurone calcola una somma pesata degli input più un bias e applica una funzione di attivazione per ottenere l'output dello strato. Questo processo si ripete fino allo strato finale, che restituisce la previsione $\hat{y}$. In questa fase pesi e bias non vengono modificati: la rete esegue solo calcoli. La previsione viene poi confrontata con il valore reale y tramite una Loss Function, che misura l'errore che servirà come base per la fase successiva.

La **Backpropagation** è un algoritmo estremamente efficiente per calcolare il gradiente della funzione di Loss rispetto a tutti i pesi e i bias della rete. Poiché una Neural Network è essenzialmente una gigantesca funzione composta (una funzione dentro una funzione, dentro un'altra funzione), la *Chain Rule* ci permette di calcolare la derivata di questa struttura complessa moltiplicando tra loro le derivate dei singoli passaggi più semplici.

Si calcolano questi gradienti partendo dalla fine (l'output) e andando a ritroso (verso l'input):

1. **Errore all'output:** Si parte dallo strato finale e si calcola la derivata della loss rispetto all'output del modello:

$$\frac{\partial \mathcal{L}}{\partial \hat{y}}$$

2. **Definizione del delta di output:** Si introduce il termine di errore locale dell'ultimo strato:

$$\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial z^{(L)}} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \cdot f'(z^{(L)})$$

dove $z^{(L)}$ è la pre-attivazione, $a^{(L)}$ è il vettore delle attivazioni e f la funzione di attivazione;

3. **Propagazione all'indietro dell'errore:** Per ogni strato nascosto l'errore viene propagato usando i pesi dello strato successivo:

$$\delta^{(l)} = \left(V^{(l+1)} \right)^T \delta^{(l+1)} f'(z^{(l)})$$

In questo modo ogni neurone riceve una quota dell'errore proporzionale al contributo delle connessioni successive;

4. **Calcolo dei gradienti dei parametri:** Una volta ottenuti i δ , si calcolano i gradienti rispetto a pesi e bias:

$$\frac{\partial \mathcal{L}}{\partial V^{(l)}} = \delta^{(l)} (a^{(l-1)})^T \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$$

$(a^{(l-1)})^T$ è la trasposta del vettore delle attivazioni dello strato precedente;

5. **Interpretazione finale:** Ogni peso viene aggiornato in base al suo contributo all'errore complessivo, permettendo alla rete di ridurre progressivamente la Loss.

$$V^{(l)} \leftarrow V^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial V^{(l)}}$$

$$b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial b^{(l)}}$$

```

1 def heaviside(x):
2     """Heaviside step function (activation function for perceptron).
3
4     Args:
5         x: Input value (can be scalar or numpy array)
6
7     Returns:
8         1 if x > 0, 0 otherwise"""
9     return (x > 0).astype(np.float32)
10
11 def perceptron_train(X, y, w_init, eta, max_iter=10000, tol=1e-4):
12     """Train a perceptron using the delta rule.
13
14     Args:
15         X: Input features matrix (including bias term)
16         y: Target labels (0 or 1)
17         w_init: Initial weight vector
18         eta: Learning rate
19         max_iter: Maximum number of iterations
20         tol: Convergence tolerance
21
22     Returns:
23         w: Trained weight vector"""
24     w = w_init.copy()

```

```
25     norm_diff = np.inf
26     iteration = 0
27
28     while norm_diff > tol and iteration < max_iter:
29         w_old = w.copy()
30
31         # Online learning - update weights for each sample
32         for i in range(X.shape[0]):
33             hat_y = heaviside(np.dot(X[i,:], w))
34             w -= eta * (hat_y - y[i]) * X[i,:]
35
36         # Check convergence
37         norm_diff = np.linalg.norm(w_old - w)
38         iteration += 1
39
40     return w
```

Capitolo 2: Multilayer Perceptron

2.1 Input Layer, Hidden Layers e Output Layer

Il **Multilayer Perceptron (MLP)** è una Artificial Neural Network composta da più strati (layer) di Perceptron, progettata per apprendere relazioni non lineari tra input e output. In pratica, è una catena di Perceptron collegati tra loro su più livelli. A differenza di un singolo Perceptron, non prende una decisione direttamente dai dati, ma li trasforma gradualmente attraverso più passaggi. Un MLP è composto da:

- **Input Layer:** riceve i dati;
- **Hidden Layers:** uno o più layer adibiti ad elaborare i dati;
- **Output Layer:** produce il risultato finale.

Ogni neurone di un livello è tipicamente connesso a tutti i neuroni del livello successivo (architettura fully connected o densa). Ogni connessione è caratterizzata da un "peso" numerico che determina l'importanza di quel segnale, e ogni neurone possiede un "bias" per traslare la propria funzione di attivazione.

Nella profondità si contano solo gli Hidden Layer e l'Output Layer. Quindi una rete profonda 2 ha un Hidden Layer e un Output Layer, una rete profonda 3 ha due Hidden Layer e un Output Layer.

2.2 Teorema dell'Approssimazione Universale

Teorema (Approssimazione Universale). *Una rete neurale sufficientemente grande può approssimare qualsiasi funzione matematica continua con un errore arbitrariamente piccolo.*

Se un solo livello basta, perché costruiamo reti "profonde" (Deep Neural Networks) con decine o centinaia di livelli nascosti? Ci sono due motivi:

- **Apprendimento Gerarchico:** I dati del mondo reale (immagini, suoni, testi) hanno una struttura intrinsecamente gerarchica. In una rete profonda, il primo hidden layer impara a riconoscere concetti base (esempio: linee, bordi, contrasti di colore). Il secondo livello combina questi bordi per riconoscere forme geometriche semplici. Il terzo livello combina le forme per riconoscere oggetti complessi (esempio: un occhio, una ruota). Questa scomposizione del problema rende l'apprendimento molto più naturale ed efficace.
- **Efficienza dei Parametri:** Per risolvere un problema molto complesso, una rete "piatta" (shallow) con un solo livello nascosto avrebbe bisogno di un numero spropositato di neuroni, diventando un modello enorme, lentissimo da addestrare e pronò all'overfitting (imparare a memoria i dati senza generalizzare). Aggiungere profondità permette di risolvere lo stesso problema usando esponenzialmente meno parametri, poiché i livelli successivi riutilizzano e ricombinano in modo intelligente i concetti appresi dai livelli precedenti.

2.3 MLP_MSELIN

MLP_MSELIN fa riferimento a una specifica configurazione di una Neural Network Multi-Layer Perceptron (MLP). L'acronimo descrive le due scelte architetturali chiave che definiscono il modello:

- **MSE (Mean Squared Error):** Indica l'utilizzo dell'Errore Quadratico Medio come Loss Function.
- **LIN (Linear):** Indica l'utilizzo di una funzione di attivazione Lineare nell'ultimo strato (quello di output).

Questa combinazione è il vero e proprio "standard de facto" quando si affrontano problemi di Regressione.

A differenza della Classificazione (in cui vogliamo prevedere etichette discrete come 0 o 1 usando una Sigmoid e la Binary Cross-Entropy), nella Regressione l'obiettivo è prevedere un valore continuo che può trovarsi ovunque sull'asse dei numeri reali $(-\infty; +\infty)$, come la stima di una temperatura o l'approssimazione di una complessa superficie matematica (come la funzione bidimensionale *sinc*).

L'architettura si sviluppa attraverso i classici strati del MLP:

- **Input Layer:** Riceve i dati in ingresso. La sua dimensionalità dipende dal problema (ad esempio, un singolo valore per la Regressione lineare semplice, o due valori se stiamo cercando di mappare una superficie 2D);
- **Hidden Layer:** Elabora le informazioni ricevute applicando una trasformazione non lineare. Qui i neuroni utilizzano funzioni di attivazione classiche (come la Sigmoid o la ReLU). È grazie a questa non linearità interna che la rete acquisisce la capacità di modellare funzioni complesse e curve arbitrarie, superando i limiti dei modelli lineari semplici;
- **Output Layer:** L'ultimo neurone riceve i segnali dal livello nascosto e non applica alcuna distorsione. Usa una semplice *LU*. Rimuovere la funzione di attivazione (o usare una funzione identità) è fondamentale: se usassimo una sigmoide in output, la nostra rete potrebbe fare previsioni solo tra 0 e 1, paralizzando la Regressione.

La fase di addestramento di questo modello si basa sul processo iterativo di Gradient Descent (spesso implementata in modalità mini-batch per equilibrare velocità computazionale e stabilità dell'aggiornamento dei pesi). Le fasi principali sono:

- **Inizializzazione:** I due set di parametri della rete (la matrice dei pesi W_1 per lo strato nascosto e W_2 per lo strato di output) vengono inizializzati con piccoli valori casuali per rompere la simmetria;
- **Forward Propagation:** I dati di input vengono moltiplicati per W_1 e passati attraverso la funzione di attivazione non lineare per produrre le uscite dello strato nascosto. Queste uscite vengono poi moltiplicate per W_2 . L'output finale è calcolato in modo puramente lineare;
- **Calcolo della Loss:** Viene calcolato il Mean Squared Error confrontando la predizione reale e lineare del modello con il target desiderato;
- **Backpropagation:** Si calcolano i gradienti dell'errore. Grazie all'utilizzo dell'attivazione lineare in output, il gradiente dell'MSE rispetto all'ultimo strato assume una forma matematica estremamente pulita e diretta: è semplicemente la differenza tra la predizione e il valore reale $(\hat{y} - y)$. Questo errore viene propagato all'indietro attraverso la rete, moltiplicandolo per le derivate delle funzioni di attivazione del hidden layer;
- **Aggiornamento:** I gradienti calcolati vengono usati per aggiornare i pesi W_1 e W_2 sottraendo una frazione determinata dal learning rate.

Monitorando il valore della Loss Function (l'MSE) epoca dopo epoca o batch dopo batch, è possibile tracciare la curva di apprendimento: si osserverà l'errore scendere rapidamente nelle prime iterazioni per poi stabilizzarsi asintoticamente quando i pesi convergono verso una soluzione ottimale.

Una volta completato l'addestramento, le matrici dei pesi ottimali (W_1 e W_2) vengono "congelate". Quando si invoca la rete per effettuare previsioni su dati mai visti prima, il modello esegue esclusivamente la Forward Pass.

La rete, avendo generalizzato la relazione tra l'input e l'output, sarà in grado di produrre stime continue accurate. Ad esempio, nel caso di un problema 2D, le previsioni generate su una griglia fitta di punti di input permettono di ricostruire e visualizzare l'intera superficie tridimensionale appresa dal modello.

Codice

```

1  def MLP_sigmoid(z):
2      """
3      Computes the sigmoid activation function.
4
5      Args:
6      z (np.array): Input to the sigmoid function (scalar, vector, or matrix).
7
8      Returns:
9      np.array: Output of the sigmoid function, element-wise.
10     """
11     return 1.0 / (1.0 + np.exp(-z))
12
13 def MLP_sigmoid_derivative(rZ):
14     """
15     Computes the derivative of the sigmoid function.
16
17     Args:
18     rZ (np.array): Input to the sigmoid function (pre-activation values).
19
20     Returns:
21     np.array: Derivative of the sigmoid, element-wise, at rZ.
22     """
23     sigma_of_rZ = MLP_sigmoid(rZ)
24     return sigma_of_rZ * (1.0 - sigma_of_rZ)
25
26 def MLP_MSE_cost(y_true, y_pred, model, W1, W2):
27     """
28     Computes the Mean Squared Error (MSE) cost function.
29
30     Args:
31     y_true (np.array): True target values (1xBatchSize or BatchSize x 1).
32     y_pred (np.array): Predicted output values (1xBatchSize).
33     model (dict): Dictionary containing model parameters (e.g., regularization lambdas).
34     W1 (np.array): Weights matrix for the hidden layer.
35     W2 (np.array): Weights matrix for the output layer.
36
37     Returns:
38     float: The scalar MSE cost.
39     """
40     # Ensure y_true has the same shape as y_pred for element-wise operations
41     if y_true.ndim == 1:
42         y_true = y_true.reshape(1, -1)
43
44     cost = np.sum((y_true - y_pred)**2) / (2 * y_true.shape[1])
45
46     # Optional: Add L1 and L2 regularization terms (commented out by default)
47     # l1 = model.get('l1', 0)

```

```

48     # l2 = model.get('l2', 0)
49     # L1_term = L1_reg(l1, W1, W2)
50     # L2_term = L2_reg(l2, W1, W2)
51     # cost = cost + L1_term + L2_term
52
53     return cost
54
55 def MLP_initialize_weights(model):
56     """
57     Initializes weights for a two-layer MLP with values uniformly distributed in [-1, 1].
58     Bias terms are included in the weight matrices as the first column.
59
60     Args:
61     model (dict): Dictionary containing network dimensions (n_hidden, n_features, n_output).
62
63     Returns:
64     tuple: W1 (np.array) - Weights for hidden layer, W2 (np.array) - Weights for output layer.
65     """
66     n_hidden = model['n_hidden']
67     n_features = model['n_features']
68     n_output = model['n_output']
69
70     # W1: (n_hidden x (n_features + 1)) matrix, including bias for hidden layer
71     # The +1 accounts for the bias term which will be multiplied by a constant 1 in the input A0
72     W1 = 2 * np.random.rand(n_hidden, n_features + 1) - 1
73
74     # W2: (n_output x (n_hidden + 1)) matrix, including bias for output layer
75     # The +1 accounts for the bias term which will be multiplied by a constant 1 in the activated
76     # hidden layer A1
77     W2 = 2 * np.random.rand(n_output, n_hidden + 1) - 1
78
79     return W1, W2
80
81 def MLP_MSELIN_forward(P_input, W1, W2):
82     """
83     Performs the forward pass through the MLP.
84
85     Args:
86     P_input (np.array): Input patterns, where each row is an observation (BatchSize x n_features).
87     W1 (np.array): Weights matrix for the hidden layer (n_hidden x (n_features + 1)).
88     W2 (np.array): Weights matrix for the output layer (n_output x (n_hidden + 1)).
89
90     Returns:
91     tuple: rA2 (np.array) - Output layer activations (predictions),
92     A1 (np.array) - Hidden layer activations with bias,
93     A0 (np.array) - Input patterns with bias,
94     rZ1 (np.array) - Hidden layer pre-activations.
95     """
96     batch_size = P_input.shape[0]
97
98     # A0: Input patterns with bias term (column of ones) appended
99     # Shape: (n_features + 1) x BatchSize
100     A0 = np.vstack((np.ones(batch_size), P_input.T))
101
102     # rZ1: Pre-activation values for the hidden layer
103     # W1: n_hidden x (n_features + 1)
104     # A0: (n_features + 1) x BatchSize
105     # rZ1: n_hidden x BatchSize
106     rZ1 = W1 @ A0
107
108     # rA1: Activated values of the hidden layer
109     # Shape: n_hidden x BatchSize
110     rA1 = MLP_sigmoid(rZ1)
111
112     # A1: Activated hidden layer values with bias term appended
113     # Shape: (n_hidden + 1) x BatchSize

```

```

113     A1 = np.vstack((np.ones(batch_size), rA1))
114
115     # rA2: Output layer activations (predictions)
116     # W2: n_output x (n_hidden + 1)
117     # A1: (n_hidden + 1) x BatchSize
118     # rA2: n_output x BatchSize
119     rA2 = W2 @ A1
120
121     return rA2, A1, A0, rZ1
122
123 def MLP_MSELIN_backprop(rA2, A1, A0, rZ1, Y_true, W1, W2):
124     """
125     Computes the partial derivatives of the loss with respect to the weight matrices W2 and W1
126     using the backpropagation algorithm.
127
128     Args:
129     rA2 (np.array): Output layer activations (predictions) (n_output x BatchSize).
130     A1 (np.array): Hidden layer activations with bias ( (n_hidden + 1) x BatchSize).
131     A0 (np.array): Input patterns with bias ( (n_features + 1) x BatchSize).
132     rZ1 (np.array): Hidden layer pre-activations (n_hidden x BatchSize).
133     Y_true (np.array): True target values (n_output x BatchSize).
134     W1 (np.array): Weights matrix for the hidden layer (n_hidden x (n_features + 1)).
135     W2 (np.array): Weights matrix for the output layer (n_output x (n_hidden + 1)).
136
137     Returns:
138     tuple: delta_W1_unscaled (np.array) - Gradient for W1,
139           delta_W2_unscaled (np.array) - Gradient for W2.
140     """
141     batch_size = A0.shape[1]
142
143     # Step 1: Compute dL_dZ2 (error signal at the output layer pre-activation)
144     # This is the derivative of the MSE loss with respect to the output layer's pre-activation Z2.
145     # For MSE and linear output, dL/dZ2 = (rA2 - Y_true)
146     # Shape: n_output x BatchSize
147     dL_dZ2 = rA2 - Y_true
148
149     # Step 2: Compute dL_dW2 (gradient for W2)
150     # dL/dW2 = dL/dZ2 * A1.T (matrix multiplication)
151     # W2: n_output x (n_hidden + 1)
152     # dL_dZ2: n_output x BatchSize
153     # A1.T: BatchSize x (n_hidden + 1)
154     # dL_dW2: n_output x (n_hidden + 1)
155     dL_dW2 = dL_dZ2 @ A1.T
156
157     # Step 3: Compute dL_dA1 (error signal propagated back to hidden layer activations)
158     # dL/dA1 = W2.T * dL/dZ2 (matrix multiplication)
159     # W2.T: (n_hidden + 1) x n_output
160     # dL_dZ2: n_output x BatchSize
161     # dL_dA1: (n_hidden + 1) x BatchSize
162     dL_dA1 = W2.T @ dL_dZ2
163
164     # Step 4: Compute dL_drZ1 (error signal at hidden layer pre-activation)
165     # This involves the derivative of the sigmoid activation function at the hidden layer.
166     # dL/drZ1 = (dL/dA1 excluding bias) * sigmoid_derivative(rZ1) (element-wise multiplication)
167     # dL_dA1[1:, :] extracts error signal for actual hidden units (excluding bias row)
168     # sigma_prime_of_rZ1: n_hidden x BatchSize
169     # dL_drZ1: n_hidden x BatchSize
170     sigma_prime_of_rZ1 = MLP_sigmoid_derivative(rZ1)
171     dL_drZ1 = dL_dA1[1:, :] * sigma_prime_of_rZ1
172
173     # Step 5: Compute dL_dW1 (gradient for W1)
174     # dL/dW1 = dL/drZ1 * A0.T (matrix multiplication)
175     # W1: n_hidden x (n_features + 1)
176     # dL_drZ1: n_hidden x BatchSize
177     # A0.T: BatchSize x (n_features + 1)
178     # dL_dW1: n_hidden x (n_features + 1)

```



```

179     dL_dW1 = dL_drZ1 @ A0.T
180
181     # Step 6: Apply regularization (if enabled - currently commented out)
182     # These are the unscaled gradients before applying learning rate and momentum.
183     delta_W1_unscaled = dL_dW1
184     delta_W2_unscaled = dL_dW2
185
186     # Example of regularization addition (if l1, l2 were active in model):
187     # l1 = model.get('l1', 0)
188     # l2 = model.get('l2', 0)
189     # delta_W1_unscaled[:, 1:] += W1[:, 1:] * (l1 + l2) # For L1/L2 on non-bias weights
190     # delta_W2_unscaled[:, 1:] += W2[:, 1:] * (l1 + l2) # For L1/L2 on non-bias weights
191
192     return delta_W1_unscaled, delta_W2_unscaled
193
194 def MLP_MSELIN_predict(P_input, W1, W2):
195     """
196     Predicts the outputs for given input patterns using the trained MLP.
197
198     Args:
199     P_input (np.array): Input patterns, where each row is a distinct observation.
200     W1 (np.array): Trained weights matrix for the hidden layer.
201     W2 (np.array): Trained weights matrix for the output layer.
202
203     Returns:
204     np.array: Predicted outputs (n_output x BatchSize).
205     """
206     rA2, _, _, _ = MLP_MSELIN_forward(P_input, W1, W2)
207     return rA2
208
209 def MLP_MSELIN_train(P_train, y_train, model):
210     """
211     Trains the Multi-Layer Perceptron model using mini-batch Gradient Descent.
212
213     Args:
214     P_train (np.array): Training input patterns (N_observations x n_features).
215     y_train (np.array): True target values for training (N_observations x n_output).
216     model (dict): Dictionary containing model configuration (eta, epochs, minibatches, etc.).
217
218     Returns:
219     tuple: model (dict) - Updated model dictionary with trained weights and cost history,
220     W1 (np.array) - Trained weights for the hidden layer,
221     W2 (np.array) - Trained weights for the output layer.
222     """
223     # Initialize weights
224     W1, W2 = MLP_initialize_weights(model)
225
226     eta = model['eta']
227     epochs = model['epochs']
228     minibatches = model['minibatches']
229
230     # Initialize cost history
231     model['cost_history'] = []
232
233     # Momentum parameters (commented out by default)
234     # delta_W1_prev = np.zeros_like(W1)
235     # delta_W2_prev = np.zeros_like(W2)
236     # alpha = model.get('alpha', 0) # Momentum coefficient
237
238     num_observations = P_train.shape[0]
239
240     for e in range(1, epochs + 1):
241         # Optional: Adaptive learning rate (commented out)
242         # decrease_const = model.get('decrease_const', 0)
243         # current_eta = eta / (1 + decrease_const * e)
244         current_eta = eta # Use fixed learning rate if adaptive is off

```

```

245     # Shuffle data for each epoch
246     shuffled_indices = np.random.permutation(num_observations)
247     P_shuffled = P_train[shuffled_indices, :]
248     y_shuffled = y_train[shuffled_indices, :]
249
250
251     # Reshape indices for mini-batching
252     # np.array_split handles cases where num_observations is not perfectly divisible by minibatches
253     mini_batch_indices = np.array_split(np.arange(num_observations), minibatches)
254
255     for m_idx, idx in enumerate(mini_batch_indices):
256         # Extract mini-batch data
257         P_mini_batch = P_shuffled[idx, :]
258         y_mini_batch = y_shuffled[idx, :].T # Transpose y_mini_batch to match expected 1xBatchSize or
n_output x BatchSize
259
260         # Forward pass
261         rA2, A1, A0, rZ1 = MLP_MSELIN_forward(P_mini_batch, W1, W2)
262
263         # Compute cost
264         cost = MLP_MSE_cost(y_mini_batch, rA2, model, W1, W2)
265         model['cost_history'].append(cost)
266
267         # Print progress
268         print(f'Epoch {e}/{epochs}, Minibatch {m_idx+1}/{minibatches}, Loss (MSE) {cost:.6f}')
269
270         # Compute gradient via backpropagation
271         delta_W1_unscaled, delta_W2_unscaled = MLP_MSELIN_backprop(rA2, A1, A0, rZ1, y_mini_batch, W1,
W2)
272
273         # Update parameters using Gradient Descent
274         delta_W1 = current_eta * delta_W1_unscaled
275         delta_W2 = current_eta * delta_W2_unscaled
276
277         W1 = W1 - delta_W1
278         W2 = W2 - delta_W2
279
280         # Optional: Apply momentum (commented out)
281         # W1 = W1 - (delta_W1 + (alpha * delta_W1_prev))
282         # W2 = W2 - (delta_W2 + (alpha * delta_W2_prev))
283         # delta_W1_prev = delta_W1
284         # delta_W2_prev = delta_W2
285
286     model['W1'] = W1
287     model['W2'] = W2
288     return model, W1, W2

```

2.4 MLP_BCESIG

MLP_BCESIG descrive una specifica configurazione di rete MLP strutturata per risolvere problemi di Classificazione **multi-label**. L'acronimo descrive le due scelte architetturali chiave che definiscono il modello:

- **BCE (Binary Cross-Entropy)**: È la Loss Function utilizzata per misurare la discrepanza tra la previsione e la realtà.
- **SIG (Sigmoid)**: È la funzione di attivazione impiegata, in particolare nello strato di output.

Ci si potrebbe domandare come mai si usa la BCE e la Sigmoid per un problema multi-label, quando queste sono più adatte ad un problema di Classificazione binaria. La risposta è che in questo caso, il

numero di neuroni nell'ultimo strato deve coincidere con il numero di classi del problema, indicato con C . In questo modo possiamo individuare le label, guardando quali neuroni hanno output pari a 1 e ignorando quelli con output pari a 0.

Il flusso delle informazioni dai dati grezzi fino alla predizione finale avviene attraverso una sequenza di operazioni matriciali precise:

- **Input Layer:** Il vettore o la matrice dei dati di ingresso X viene espanso ("augmented") inserendo una colonna interamente composta da uni. Questo artificio matematico assorbe il calcolo del termine di bias direttamente nel prodotto matriciale. Indicheremo questa matrice iniziale con A_0 ;
- **Hidden Layer:** Si calcolano i valori di pre-attivazione lineare moltiplicando gli input per la matrice dei pesi trasposta: $Z_1 = A_0 W_1^T$. Il risultato attraversa la funzione di attivazione non lineare: $A_1 = \sigma(Z_1)$. Esattamente come avvenuto per l'input, anche la matrice risultante A_1 viene estesa con una colonna di uni per propagare il nuovo bias verso lo strato successivo (A_1^{ext});
- **Output Layer:** Si calcola la pre-attivazione finale: $Z_2 = A_1^{\text{ext}} W_2^T$. Si applica nuovamente la sigmoide per ottenere la predizione probabilistica del modello: $\hat{Y} = \sigma(Z_2)$.

L'errore commesso dalla rete su un singolo campione p per una specifica classe c viene calcolato mediante la formula della Binary Cross-Entropy:

$$L(y_{\text{true},c}; \hat{y}_c) = -y_{\text{true},c} \log(\hat{y}_c) - (1 - y_{\text{true},c}) \log(1 - \hat{y}_c)$$

Il costo globale su cui si allena la rete corrisponde alla somma di questi errori calcolata sull'intero batch di campioni e su tutte le classi di output.

Una volta calcolata la Loss, l'errore viene propagato a ritroso per determinare come aggiornare le matrici dei pesi. Nel caso della Binary Cross-Entropy combinata con una funzione di attivazione sigmoide, il termine di errore dello strato di output assume la forma particolarmente semplice:

$$\delta_2 = \hat{Y} - Y_{\text{true}}$$

Da questo si ricava il gradiente dei pesi di output:

$$\nabla_{W_2} J = \delta_2^T A_1^{\text{ext}}$$

L'errore viene quindi propagato allo strato nascosto, escludendo il contributo del bias e moltiplicando per la derivata della sigmoide:

$$\delta_1 = (\delta_2 W_2)_{\text{escluso bias}} \odot \sigma'(Z_1)$$

da cui si ottiene il gradiente dei pesi nascosti:

$$\nabla_{W_1} J = \delta_1^T A_0.$$

Infine, ai gradienti di W_1 e W_2 vengono sommati i contributi derivanti dagli eventuali termini di regolarizzazione L1 e L2.

La dinamica di apprendimento procede mescolando casualmente i dati ad ogni epoca e dividendoli in frazioni dette Mini-batch. I pesi vengono corretti iterativamente secondo questa equazione:

$$W \leftarrow W - \eta \nabla J + \alpha \Delta W_{\text{prev}}$$

η lo abbiamo già visto. È il learning rate. Il termine $\alpha \Delta W_{\text{prev}}$ reintroduce nella formula una frazione dell'aggiornamento calcolato nello step precedente. Questo "*slancio d'inerzia*" (**momentum**) ammortizza le variazioni caotiche e conferisce alla rete una velocità direzionale utile a fuggire dai minimi locali stagnanti.

Codice

```

1  def MLP_sigmoid(z):
2      """
3      Computes the sigmoid activation function.
4      \(\sigma(z) = 1 / (1 + e^{-z})\)
5      Args:
6      z (np.array): Input array (pre-activation values).
7      Returns:
8      np.array: Output of the sigmoid function.
9      """
10     return 1.0 / (1.0 + np.exp(-z))
11
12 def MLP_sigmoid_gradient(Z):
13     """
14     Computes the derivative of the sigmoid function with respect to its input Z.
15     \(\sigma'(z) = \sigma(z)(1 - \sigma(z))\)
16     Args:
17     Z (np.array): Input array (pre-activation values).
18     Returns:
19     np.array: Derivative of the sigmoid function.
20     """
21     sig = MLP_sigmoid(Z)
22     return sig * (1 - sig)
23
24 def MLP_initialize_weights(model):
25     """
26     Initializes the weights for a two-layer MLP (input-to-hidden W1, hidden-to-output W2).
27     Weights are initialized uniformly in [-1, 1]. Bias terms are included in the weight matrices.
28     Args:
29     model (dict): Dictionary containing network architecture parameters (n_hidden, n_features,
30     n_output).
31     Returns:
32     tuple: W1 (np.array) - Weights for the hidden layer (n_hidden, n_features + 1),
33     W2 (np.array) - Weights for the output layer (n_output, n_hidden + 1).
34     """
35     n_hidden = model['n_hidden']
36     n_features = model['n_features']
37     n_output = model['n_output']
38
39     # W1 includes weights for n_features inputs + 1 bias term
40     W1 = 2 * np.random.rand(n_hidden, n_features + 1) - 1
41     # W2 includes weights for n_hidden activations + 1 bias term
42     W2 = 2 * np.random.rand(n_output, n_hidden + 1) - 1
43     return W1, W2
44
45 def encode_labels(y, k):
46     """
47     Performs one-hot encoding on a vector of class labels.
48     Args:
49     y (np.array): Original class labels (1D array).
50     k (int): Total number of unique classes.
51     Returns:
52     np.array: One-hot encoded labels (k, len(y)).
53     """
54     onehot = np.zeros((k, len(y)))
55     for i, val in enumerate(y):
56         onehot[val, i] = 1 # Assuming y are 0-indexed classes
57     return onehot
58
59 def L1_reg(lambda_reg, W1, W2):
60     """
61     Computes the L1 regularization cost for the weights (excluding bias terms).
62     Args:
63     lambda_reg (float): Regularization strength.

```

```

63     W1 (np.array): Weights of the hidden layer.
64     W2 (np.array): Weights of the output layer.
65     Returns:
66     float: L1 regularization cost.
67     """
68     return (lambda_reg / 2) * (np.sum(np.abs(W1[:, 1:])) + np.sum(np.abs(W2[:, 1:])))
69
70 def L2_reg(lambda_reg, W1, W2):
71     """
72     Computes the L2 regularization cost for the weights (excluding bias terms).
73     Args:
74     lambda_reg (float): Regularization strength.
75     W1 (np.array): Weights of the hidden layer.
76     W2 (np.array): Weights of the output layer.
77     Returns:
78     float: L2 regularization cost.
79     """
80     return (lambda_reg / 2) * (np.sum(W1[:, 1:]**2) + np.sum(W2[:, 1:]**2))
81
82 def MLP_BCESIG_forward(X, W1, W2):
83     """
84     Performs the forward pass through the MLP with sigmoid activation in both hidden and output layers
85     .
86     Args:
87     X (np.array): Input data (batch_size, n_features).
88     W1 (np.array): Weights for the hidden layer (n_hidden, n_features + 1).
89     W2 (np.array): Weights for the output layer (n_output, n_hidden + 1).
90     Returns:
91     tuple: rA2 (output activations), A1 (hidden activations + bias),
92     A0 (input + bias), rZ1 (hidden pre-activations), rZ2 (output pre-activations).
93     """
94     batch_size = X.shape[0]
95     # A0: Input layer activations, augmented with a bias term (column of ones)
96     A0 = np.hstack((np.ones((batch_size, 1)), X)) # Shape: (batch_size, n_features + 1)
97
98     # rZ1: Pre-activation values for the hidden layer
99     rZ1 = A0 @ W1.T # Shape: (batch_size, n_hidden)
100    # A1: Activated hidden layer values
101    A1 = MLP_sigmoid(rZ1) # Shape: (batch_size, n_hidden)
102    A1 = np.hstack((np.ones((batch_size, 1)), A1)) # Shape: (batch_size, n_hidden + 1)
103
104    # rZ2: Pre-activation values for the output layer
105    rZ2 = A1 @ W2.T # Shape: (batch_size, n_output)
106    # rA2: Final output layer activations (predictions)
107    rA2 = MLP_sigmoid(rZ2) # Shape: (batch_size, n_output)
108    return rA2, A1, A0, rZ1, rZ2
109
110 def get_BCE_cost(y_enc, y_pred, model, W1, W2):
111     """
112     Computes the Binary Cross-Entropy cost with L1 and L2 regularization.
113     Args:
114     y_enc (np.array): One-hot encoded true labels (n_output, batch_size).
115     y_pred (np.array): Predicted probabilities (batch_size, n_output).
116     model (dict): Dictionary containing model parameters (l1, l2).
117     W1 (np.array): Weights of the hidden layer.
118     W2 (np.array): Weights of the output layer.
119     Returns:
120     float: Total cost.
121     """
122     # Clip predictions to avoid log(0) which leads to NaN/inf
123     y_pred = np.clip(y_pred, 1e-12, 1 - 1e-12);
124
125     # Calculate Binary Cross-Entropy term
126     # Note: y_enc is (n_output, batch_size), y_pred.T is (n_output, batch_size)
127     term1 = -y_enc * np.log(y_pred.T)
128     term2 = (1 - y_enc) * np.log(1 - y_pred.T)

```

```

128     cost = np.sum(term1 - term2) # Sum across all classes and all patterns in the batch
129
130     # Add regularization terms
131     L1_term = L1_reg(model['l1'], W1, W2)
132     L2_term = L2_reg(model['l2'], W1, W2)
133     cost = cost + L1_term + L2_term
134     return cost
135
136 def get_BCESIG_gradient(rA2, A1, A0, rZ1, rZ2, Y_enc, W1, W2, l1, l2):
137     """
138     Computes gradients for weights via backpropagation for BCE loss and sigmoid output.
139     Args:
140     rA2 (np.array): Output activations (predictions) from forward pass (batch_size, n_output).
141     A1 (np.array): Hidden layer activations (including bias) from forward pass (batch_size, n_hidden +
142     1).
143     A0 (np.array): Input layer activations (including bias) from forward pass (batch_size, n_features
144     + 1).
145     rZ1 (np.array): Hidden layer pre-activations from forward pass (batch_size, n_hidden).
146     rZ2 (np.array): Output layer pre-activations from forward pass (batch_size, n_output).
147     Y_enc (np.array): One-hot encoded true labels (n_output, batch_size).
148     W1 (np.array): Current weights of the hidden layer.
149     W2 (np.array): Current weights of the output layer.
150     l1 (float): L1 regularization strength.
151     l2 (float): L2 regularization strength.
152     Returns:
153     tuple: delta_W1_unscaled (gradient for W1), delta_W2_unscaled (gradient for W2).
154     """
155     # Step 1: Compute dL_dZ2 (error signal at output layer pre-activation)
156     # For BCE with sigmoid output, the error signal is simply (predicted_output - true_output)
157     dL_dZ2 = rA2 - Y_enc.T # Shape: (batch_size, n_output)
158
159     # Step 2: Compute dL_dW2 (gradient for W2)
160     # Gradient is outer product of output error and hidden layer activations
161     dL_dW2 = dL_dZ2.T @ A1 # Shape: (n_output, n_hidden + 1)
162
163     # Step 3: Compute dL_dA1 (error signal propagated back to hidden layer activations)
164     # This is the error from the output layer multiplied by the output weights
165     dL_dA1 = dL_dZ2 @ W2 # Shape: (batch_size, n_hidden + 1)
166
167     # Step 4: Compute dL_drZ1 (error signal at hidden layer pre-activation)
168     # Exclude the bias term from dL_dA1 (first column) and multiply by the sigmoid derivative of
169     # hidden layer
170     sigma_prime_of_rZ1 = MLP_sigmoid_gradient(rZ1)
171     dL_drZ1 = dL_dA1[:, 1:] * sigma_prime_of_rZ1 # Shape: (batch_size, n_hidden)
172
173     # Step 5: Compute dL_dW1 (gradient for W1)
174     # Gradient is outer product of hidden error and input layer activations
175     dL_dW1 = dL_drZ1.T @ A0 # Shape: (n_hidden, n_features + 1)
176
177     # Step 6: Add regularization terms to gradients (for non-bias weights)
178     # L1 and L2 regularization derivatives are added to the gradients
179     delta_W1_unscaled = dL_dW1 + (l1 * np.sign(W1) + 2 * l2 * W1) # Add regularization derivatives
180     delta_W2_unscaled = dL_dW2 + (l1 * np.sign(W2) + 2 * l2 * W2) # Add regularization derivatives
181
182     # Note: Regularization is typically applied only to the non-bias weights.
183     # The current implementation applies it to all, including bias. For strict adherence
184     # to non-bias regularization, one would zero out the first column of the regularization
185     # terms before adding them to delta_W1_unscaled and delta_W2_unscaled.
186     # e.g., delta_W1_unscaled[:, 1:] += (l1 * np.sign(W1[:, 1:]) + 2 * l2 * W1[:, 1:])
187     # and delta_W1_unscaled[:, 0] = dL_dW1[:, 0] (no regularization on bias)
188
189     return delta_W1_unscaled, delta_W2_unscaled
190
191 def MLP_BCESIG_predict(X, W1, W2):
192     """

```

```

191 Predicts class labels for input X using the trained MLP.
192 Args:
193 X (np.array): Input data for prediction (batch_size, n_features).
194 W1 (np.array): Trained weights of the hidden layer.
195 W2 (np.array): Trained weights of the output layer.
196 Returns:
197 np.array: Predicted 0-indexed class labels (batch_size,).
198 """
199 # Perform forward pass to get output probabilities
200 rA2, _, _, _ = MLP_BCESIG_forward(X, W1, W2)
201 # Predict class by selecting the index with the highest probability
202 y_pred = np.argmax(rA2, axis=1) # Get the class with highest probability
203 return y_pred

```

2.5 MLP_CCESM

MLP_CCESM descrive una MLP progettata per risolvere problemi di Classificazione Multi-Classe Mutuamente Esclusiva. A differenza dei problemi multi-label (dove un campione può appartenere a più classi contemporaneamente), qui ogni input appartiene a una e una sola classe.

Il nome del modello deriva dalle due scelte architetturali fondamentali adottate nello strato di output:

- **CCE (Categorical Cross-Entropy):** È la Loss Function utilizzata per calcolare l'errore.
- **SM (SoftMax):** È la funzione di attivazione vettoriale applicata ai neuroni finali.

Nei problemi multi-classe, le etichette vengono rappresentate tramite **One-Hot Encoding**. Ogni neurone di output stima la probabilità di appartenenza a una classe specifica, trasformando la pre-attivazione nell'intervallo (0; 1) mediante una funzione sigmoide.

Il *One-Hot Encoding* è una tecnica che rappresenta una variabile categorica trasformandola in un vettore binario. Ogni categoria è associata a un vettore in cui un solo elemento vale 1 (quello della classe corretta) e tutti gli altri valgono 0. Esempio con 3 classi:

- classe A $\rightarrow (1; 0; 0)$
- classe B $\rightarrow (0; 1; 0)$
- classe C $\rightarrow (0; 0; 1)$

In questo scenario, il numero di neuroni nell'ultimo strato deve essere esattamente uguale al numero totale di classi (C). Di conseguenza, le etichette reali (ground truth) devono essere fornite in formato One-Hot Encoding.

A differenza della Sigmoide (che opera su ogni neurone in modo indipendente), la SoftMax è una funzione vettoriale per-layer: prende in ingresso l'intero vettore delle pre-attivazioni (dette logits) dell'ultimo strato e restituisce un nuovo vettore.

La sua formula per l'output della classe c è:

$$a_c = \text{SoftMax}(z_c) = \frac{e^{z_c}}{\sum_{k=1}^C e^{z_k}}$$

La forza della SoftMax risiede nella normalizzazione al denominatore: trasforma i valori grezzi in una vera e propria distribuzione di probabilità. Ogni output sarà un numero compreso tra 0 e 1, e la somma di tutti gli output sarà sempre forzatamente uguale a 1.

Poiché l'output di un neurone c dipende dai valori di tutti gli altri neuroni k (a causa del denominatore condiviso), la sua derivata è frammentata in due casi:

- Se deriviamo rispetto al suo stesso input ($k = c$): $\frac{\partial a_c}{\partial z_c} = a_c(1 - a_c)$
- Se deriviamo rispetto all'input di un altro neurone ($k \neq c$): $\frac{\partial a_c}{\partial z_k} = -a_c \cdot a_k$

Per valutare l'errore tra le probabilità previste dalla SoftMax e l'etichetta One-Hot Encoding reale, si utilizza la Loss CCE. Per un singolo campione, la formula è:

$$L_{\text{CCE}} = - \sum_{c=1}^C y_c \log(\hat{y}_c)$$

Essendo le etichette y_c quasi tutte nulle tranne una, questa sommatoria si riduce di fatto a valutare solo il logaritmo della probabilità predetta per la classe corretta. Se la rete è insicura (probabilità bassa per la classe vera), il logaritmo restituisce un numero molto negativo che, invertito di segno, genera un costo altissimo.

Il calcolo in avanti procede per moltiplicazioni matriciali (considerando un mini-batch di B campioni):

- **Input Layer:** Si limita a prendere gli input nella matrice A_0 ;
- **Hidden Layer:** L'input A_0 (esteso con il termine di bias) viene moltiplicato per i pesi W_1 ottenendo la pre-attivazione rZ_1 . Si applica un'attivazione non lineare (esempio: la Sigmoidale) per ottenere rA_1 ;
- **Output Layer:** L'output nascosto A_1 (nuovamente esteso col bias) viene moltiplicato per i pesi finali W_2 calcolando le pre-attivazioni finali rZ_2 . Infine, si applica la SoftMax per ottenere la previsione probabilistica rA_2 (o $\hat{Y}$).

Per calcolare come aggiornare le matrici dei pesi W_1 e W_2 , dobbiamo applicare la *Chain Rule* andando a ritroso. Quando combiniamo la funzione di costo CCE con l'attivazione SoftMax, i termini complessi (come le derivate incrociate della SoftMax) si elidono e si cancellano a vicenda. L'errore locale dell'ultimo strato rispetto alle pre-attivazioni (il differenziale $\frac{\partial L}{\partial z_{2c}}$) si riduce a una banale sottrazione:

$$\delta_2 = a_{2c} - y_c = \hat{Y} - Y_{\text{true}}$$

Una volta noto δ_2 , i gradienti per l'aggiornamento dei pesi seguono le classiche proiezioni matriciali:

- **Gradiente dei pesi finali (W_2):** Si ottiene moltiplicando l'errore δ_2 per le attivazioni dello strato nascosto A_1^T ;
- **Errore dello strato nascosto (δ_1):** L'errore viene spinto indietro moltiplicando i pesi W_2^T per δ_2 , e poi filtrato dalla derivata della funzione di attivazione nascosta (esempio: la derivata della Sigmoidale);
- **Gradiente dei pesi iniziali (W_1):** Si incrocia l'errore nascosto δ_1 con i dati di input originali A_0^T .

Codice

```

1  def MLP_sigmoid(z):
2      """
3      Computes the sigmoid activation function element-wise.
4      The sigmoid function squashes any real-valued number into the range (0, 1).
5      Mathematically:  $\sigma(z) = 1 / (1 + e^{-z})$ 
6      Args:
7      z (np.array): Input array (pre-activation values).
8      Returns:
9      np.array: Output of the sigmoid function, with the same shape as 'z'.
10     """
11     return 1.0 / (1.0 + np.exp(-z))
12
13 def MLP_sigmoid_gradient(Z):
14     """
15     Computes the derivative of the sigmoid function with respect to its input Z.
16     This derivative is utilized during the backpropagation process.
17     Mathematically:  $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ 
18     Args:
19     Z (np.array): Input array (pre-activation values), for which the sigmoid and its derivative are
20     calculated.
21     Returns:
22     np.array: Derivative of the sigmoid function, with the same shape as 'Z'.
23     """
24     sig = MLP_sigmoid(Z)
25     return sig * (1 - sig)
26
27 def MLP_softmax(rZ):
28     """
29     Computes the softmax activation function. This function is typically used in the output layer
30     of a multi-class classification neural network to convert raw scores (logits) into probabilities.
31     It ensures that the output values are between 0 and 1 and sum to 1 across classes for each sample.
32     Mathematically:  $\text{softmax}(z_i) = e^{z_i} / \sum_{j=1}^K e^{z_j}$ 
33
34     Args:
35     rZ (np.array): Input array of pre-activation values (logits). Expected shape (batch_size,
36     n_output_classes).
37     Returns:
38     np.array: Output probabilities (batch_size, n_output_classes), where each row sums to 1.
39     """
40     # Subtract max for numerical stability to prevent overflow with large exponents
41     # Axis=1 means calculate max for each row (each sample in the batch)
42     exp_rZ = np.exp(rZ - np.max(rZ, axis=1, keepdims=True))
43     # Sum of exponents for normalization (sum across classes for each sample)
44     total = np.sum(exp_rZ, axis=1, keepdims=True)
45     rA = exp_rZ / total
46     return rA
47
48 def MLP_initialize_weights(model):
49     """
50     Initializes the weight matrices for the MLP. Weights are initialized uniformly
51     between -1 and 1. Bias terms are incorporated as the first column in each weight matrix.
52     This initialization helps in breaking symmetry and allows the network to learn diverse features.
53     Args:
54     model (dict): A dictionary containing network architecture parameters:
55     'n_hidden' (int): Number of neurons in the hidden layer.
56     'n_features' (int): Number of input features.
57     'n_output' (int): Number of neurons in the output layer (number of classes).
58     Returns:
59     tuple: W1 (np.array) - Weight matrix for the hidden layer, shape (n_hidden, n_features + 1).
60     W2 (np.array) - Weight matrix for the output layer, shape (n_output, n_hidden + 1).
61     """
62     n_hidden = model['n_hidden']
63     n_features = model['n_features']

```

```

62     n_output = model['n_output']
63
64     # W1: Weights connecting input layer (n_features + 1 for bias) to hidden layer (n_hidden).
65     # Dimensions are (n_hidden, n_features + 1) because W1 will be multiplied by A0 @ W1.T.
66     # Initialized in range [-1, 1].
67     W1 = 2 * np.random.rand(n_hidden, n_features + 1) - 1
68
69     # W2: Weights connecting hidden layer (n_hidden + 1 for bias) to output layer (n_output).
70     # Dimensions are (n_output, n_hidden + 1).
71     W2 = 2 * np.random.rand(n_output, n_hidden + 1) - 1
72     return W1, W2
73
74 def encode_labels(y, k):
75     """
76     Performs one-hot encoding on a vector of integer class labels.
77     One-hot encoding converts categorical labels into a binary vector representation,
78     which is suitable for training with categorical cross-entropy loss.
79     Args:
80     y (np.array): A 1D NumPy array of original class labels (e.g., [0, 1, 2, 1, ...]).
81     k (int): The total number of unique classes (e.g., 10 for MNIST digits 0-9).
82     Returns:
83     np.array: A 2D NumPy array of one-hot encoded labels, with shape (N_samples, k).
84     Each row corresponds to a sample, and each column to a class.
85     """
86     # Create a zero array with shape (number of samples, number of classes)
87     onehot = np.zeros((len(y), k), dtype=int)
88     # For each sample, set the column corresponding to its class label to 1
89     # np.arange(len(y)) creates indices for rows, y for columns
90     onehot[np.arange(len(y)), y] = 1 # Assuming y are 0-indexed classes
91     return onehot
92
93 def L1_reg(lambda_reg, W1, W2):
94     """
95     Computes the L1 regularization cost. This penalty term is added to the loss function
96     and is proportional to the sum of the absolute values of the weights (excluding bias terms).
97     It promotes sparsity in the weight matrices.
98     Args:
99     lambda_reg (float): The regularization strength (lambda).
100    W1 (np.array): Weight matrix for the hidden layer.
101    W2 (np.array): Weight matrix for the output layer.
102    Returns:
103    float: The calculated L1 regularization cost.
104    """
105    # Sum the absolute values of all weights, excluding the bias column (index 0)
106    return (lambda_reg / 2) * (np.sum(np.abs(W1[:, 1:])) + np.sum(np.abs(W2[:, 1:])))
107
108 def L2_reg(lambda_reg, W1, W2):
109     """
110     Computes the L2 regularization cost. This penalty term is added to the loss function
111     and is proportional to the sum of the squared values of the weights (excluding bias terms).
112     It discourages large weights and helps prevent overfitting.
113     Args:
114     lambda_reg (float): The regularization strength (lambda).
115     W1 (np.array): Weight matrix for the hidden layer.
116     W2 (np.array): Weight matrix for the output layer.
117     Returns:
118     float: The calculated L2 regularization cost.
119     """
120     # Sum the squared values of all weights, excluding the bias column (index 0)
121     return (lambda_reg / 2) * (np.sum(W1[:, 1:]**2) + np.sum(W2[:, 1:]**2))
122
123 def MLP_CCESM_forward(X, W1, W2):
124     """
125     Performs the forward pass through the MLP with sigmoid activation in the hidden layer
126     and softmax activation in the output layer.
127

```

```

128     Args:
129     X (np.array): Input data for the current batch, shape (batch_size, n_features).
130     W1 (np.array): Weight matrix for the hidden layer, shape (n_hidden, n_features + 1).
131     W2 (np.array): Weight matrix for the output layer, shape (n_output, n_hidden + 1).
132
133     Returns:
134     tuple: rA2 (np.array): Output activations (probabilities) from the softmax layer,
135           shape (batch_size, n_output).
136     A1_ext (np.array): Hidden layer activations, including the bias term,
137           shape (batch_size, n_hidden + 1).
138     A0 (np.array): Input activations, including the bias term,
139           shape (batch_size, n_features + 1).
140     rZ1 (np.array): Pre-activation values of the hidden layer,
141           shape (batch_size, n_hidden).
142     rZ2 (np.array): Pre-activation values of the output layer,
143           shape (batch_size, n_output).
144     """
145     batch_size = X.shape[0]
146
147     # A0: Input layer activations augmented with a bias term (column of ones).
148     # This allows the bias to be treated as a weight connected to a constant input of 1.
149     A0 = np.hstack((np.ones((batch_size, 1)), X)) # Shape: (batch_size, n_features + 1)
150
151     # rZ1: Pre-activation values for the hidden layer.
152     # Computed by multiplying input activations (A0) with the transpose of hidden layer weights (W1.T)
153     .
154     rZ1 = A0 @ W1.T # Shape: (batch_size, n_hidden)
155     # A1: Activated hidden layer values using the sigmoid function.
156     A1 = MLP_sigmoid(rZ1) # Shape: (batch_size, n_hidden)
157     # A1_ext: Hidden layer activations augmented with a bias term for the output layer.
158     A1_ext = np.hstack((np.ones((batch_size, 1)), A1)) # Shape: (batch_size, n_hidden + 1)
159
160     # rZ2: Pre-activation values for the output layer.
161     # Computed by multiplying extended hidden activations (A1_ext) with the transpose of output layer
162     weights (W2.T).
163     rZ2 = A1_ext @ W2.T # Shape: (batch_size, n_output)
164     # rA2: Final output layer activations (predictions) using the softmax function.
165     # Softmax converts logits into a probability distribution over classes.
166     rA2 = MLP_softmax(rZ2) # Shape: (batch_size, n_output)
167
168     return rA2, A1_ext, A0, rZ1, rZ2
169
170 def get_CCE_cost(y_enc, y_pred, model, W1, W2):
171     """
172     Computes the Categorical Cross-Entropy (CCE) cost, including L1 and L2 regularization terms.
173     This function measures the performance of a classification model whose output is a probability
174     value
175     between 0 and 1. It's suitable for multi-class classification when true labels are one-hot encoded
176     .
177     Args:
178     y_enc (np.array): One-hot encoded true labels, shape (batch_size, n_output_classes).
179     y_pred (np.array): Predicted probabilities from the softmax layer, shape (batch_size,
180           n_output_classes).
181     model (dict): Dictionary containing model parameters 'l1' (L1 regularization weight) and 'l2' (L2
182           regularization weight).
183     W1 (np.array): Weight matrix for the hidden layer.
184     W2 (np.array): Weight matrix for the output layer.
185     Returns:
186     float: The total cost, which is the sum of CCE loss and regularization terms.
187     """
188     # Clip predictions to a small positive value to prevent log(0) which results in NaN or infinite
189     loss.
190     # Clipping also prevents values from reaching 1.0 which would make log(1-p) problematic in BCE (
191     not CCE here).
192     y_pred = np.clip(y_pred, 1e-12, 1.0 - 1e-12)

```

```

186 # Calculate the Categorical Cross-Entropy (CCE) loss.
187 # The formula is -sum(y_true * log(y_pred)) over all classes and all samples in the batch.
188 # np.sum() without specifying axis will sum all elements of the resulting matrix.
189 cce_loss = -np.sum(y_enc * np.log(y_pred))
190
191 # Add L1 and L2 regularization terms to the cost.
192 l1 = model['l1']
193 l2 = model['l2']
194 L1_term = L1_reg(l1, W1, W2)
195 L2_term = L2_reg(l2, W1, W2)
196
197 cost = cce_loss + L1_term + L2_term
198 return cost
199
200 def get_CCESM_gradient(rA2, A1_ext, A0, rZ1, Y_enc, W1, W2, l1, l2):
201     """
202     Computes the gradients for the weights of the MLP using the backpropagation algorithm.
203     This specific implementation is for Categorical Cross-Entropy loss with Softmax output activation.
204     Args:
205     rA2 (np.array): Output activations (predicted probabilities) from the softmax layer,
206     shape (batch_size, n_output_classes).
207     A1_ext (np.array): Hidden layer activations, including the bias term,
208     shape (batch_size, n_hidden + 1).
209     A0 (np.array): Input activations, including the bias term,
210     shape (batch_size, n_features + 1).
211     rZ1 (np.array): Pre-activation values of the hidden layer, shape (batch_size, n_hidden).
212     Y_enc (np.array): One-hot encoded true labels, shape (batch_size, n_output_classes).
213     W1 (np.array): Current weight matrix for the hidden layer.
214     W2 (np.array): Current weight matrix for the output layer.
215     l1 (float): L1 regularization strength.
216     l2 (float): L2 regularization strength.
217     Returns:
218     tuple: delta_W1_unscaled (np.array): Gradient for W1 (excluding regularization).
219     delta_W2_unscaled (np.array): Gradient for W2 (excluding regularization).
220     """
221     # Step 1: Compute dL_dZ2 (Error signal at the output layer's pre-activation).
222     # For CCE loss with Softmax, this derivative simplifies to (predicted_probabilities -
223     # true_one_hot_labels).
224     dL_dZ2 = rA2 - Y_enc # Shape: (batch_size, n_output)
225
226     # Step 2: Compute dL_dW2 (Gradient for the output layer weights W2).
227     # This is calculated as the dot product of the transpose of the output error (dL_dZ2.T)
228     # and the activations of the previous layer (A1_ext).
229     dL_dW2 = dL_dZ2.T @ A1_ext # Shape: (n_output, n_hidden + 1)
230
231     # Step 3: Compute dL_dA1 (Error propagated back to the hidden layer's activations).
232     # This is done by multiplying the output error (dL_dZ2) with the output weights (W2).
233     dL_dA1 = dL_dZ2 @ W2 # Shape: (batch_size, n_hidden + 1)
234
235     # Step 4: Compute dL_drZ1 (Error signal at the hidden layer's pre-activation).
236     # This involves taking the portion of dL_dA1 corresponding to actual neurons (excluding bias),
237     # and element-wise multiplying it by the derivative of the hidden layer's activation function (
238     # sigmoid_gradient(rZ1)).
239     sigma_prime_of_rZ1 = MLP_sigmoid_gradient(rZ1) # Shape: (batch_size, n_hidden)
240     dL_drZ1 = dL_dA1[:, 1:] * sigma_prime_of_rZ1 # Shape: (batch_size, n_hidden)
241
242     # Step 5: Compute dL_dW1 (Gradient for the hidden layer weights W1).
243     # This is calculated as the dot product of the transpose of the hidden error (dL_drZ1.T)
244     # and the input activations (A0).
245     dL_dW1 = dL_drZ1.T @ A0 # Shape: (n_hidden, n_features + 1)
246
247     # Step 6: Add regularization terms to the gradients (for non-bias weights).
248     # The regularization terms' derivatives are added to the computed gradients.
249     # Note: Regularization is applied only to the actual weights (columns from index 1 onwards),
250     # not to the bias terms (column at index 0).
251     delta_W1_unscaled = dL_dW1.copy()

```

```

250     delta_W2_unscaled = dL_dW2.copy()
251
252     # L1 and L2 derivatives are added to the non-bias parts of the gradients
253     delta_W1_unscaled[:, 1:] += (l1 * np.sign(W1[:, 1:]) + 2 * l2 * W1[:, 1:])
254     delta_W2_unscaled[:, 1:] += (l1 * np.sign(W2[:, 1:]) + 2 * l2 * W2[:, 1:])
255
256     return delta_W1_unscaled, delta_W2_unscaled
257
258 def MLP_CCESM_predict(X, W1, W2):
259     """
260     Predicts class labels for a given input dataset X using the trained MLP.
261     The prediction is made by performing a forward pass and then selecting the class
262     with the highest probability from the softmax output.
263     Args:
264     X (np.array): Input data for prediction, shape (N_samples, n_features).
265     W1 (np.array): Trained weight matrix for the hidden layer.
266     W2 (np.array): Trained weight matrix for the output layer.
267     Returns:
268     np.array: Predicted 0-indexed class labels, shape (N_samples,).
269     """
270     # Perform forward pass to get output probabilities from the softmax layer
271     rA2, _, _, _ = MLP_CCESM_forward(X, W1, W2)
272     # Select the class with the highest probability for each sample
273     y_pred = np.argmax(rA2, axis=1)
274     return y_pred

```

Capitolo 3: Data Preprocessing

3.1 Introduzione al Preprocessing

Il **Data Preprocessing** è un passaggio fondamentale nell'ambito del Machine Learning. I dati nel mondo reale sono spesso "sporchi", ovvero incompleti, rumorosi o incoerenti a causa di errori umani, malfunzionamenti della strumentazione, incomprensioni o problemi di trasmissione. L'obiettivo del preprocessing è migliorare la qualità dei dati, la quale viene valutata secondo diverse metriche chiave: accuratezza, completezza, consistenza, tempestività, credibilità e interpretabilità.

3.2 Data Splitting

Nel Deep Learning i dati si dividono in:

- **training set**: addestra il modello aggiornando i pesi;
- **validation set**: regola iperparametri e monitora l'overfitting;
- **test set**: valuta le prestazioni finali su dati mai visti.

Approfondiamo un attimo il validation set. Questa parte di dataset viene usata durante l'addestramento per controllare le prestazioni del modello su dati non utilizzati per l'ottimizzazione dei pesi. Serve a scegliere iperparametri (come learning rate, architettura, numero di layer) e a prendere decisioni come l'early stopping. In questo modo si monitora se il modello sta iniziando a sovradattarsi ai dati di training.

Il **Data Leakage** si verifica quando, durante il Preprocessing, le informazioni che dovrebbero essere esterne al training set vengono accidentalmente utilizzate per istruire il modello. In altre parole, è come se un modello "sbirciasse" le risposte dell'esame prima di sostenerlo. È necessario costruire una pipeline di Preprocessing, suddividere successivamente i dati nei vari set e applicare la stessa pipeline a ciascuno di essi in modo indipendente, garantendo coerenza e assenza di contaminazione tra i set.

I metodi standard per creare set di training, di validation e di test a partire da un dataset sono:

- **Simple Hold-out Partition**: è una tecnica di suddivisione del dataset in cui i dati vengono mescolati casualmente e separati in insiemi distinti. Tenzialmente si divide in uno di questi modi:
 - 70% training, 15% validation e 15% test;
 - 70% training, 10% validation e 20% test;
 - 80% training, 10% validation e 10% test;
 - 60% training, 20% validation e 20% test.

Nella realtà pratica si può usare le percentuali che si preferiscono, stando attenti a dare sempre più dati al training set. Questo approccio è semplice e computazionalmente efficiente, ma fornisce una stima delle prestazioni fortemente dipendente dalla particolare suddivisione effettuata, che potrebbe non essere rappresentativa dell'intero dataset;

- **K-Fold Cross Validation:** suddivide il dataset in K parti e ripete l'addestramento K volte, usando ogni volta un fold diverso come test set e i restanti come training set. Le prestazioni finali si ottengono mediando i risultati delle varie iterazioni. Questo approccio riduce la dipendenza dalla singola partizione e fornisce una stima più affidabile, ma richiede un costo computazionale più elevato. Spesso applicato solo ai set di addestramento/validazione per un'ottimizzazione robusta degli iperparametri, mentre il set di test rimane separato per la valutazione finale;
- **Stratified Cross Validation:** una variante della K -Fold utilizzata soprattutto in problemi di Classificazione con dataset sbilanciati, in cui la suddivisione dei dati viene effettuata in modo da preservare in ogni fold le stesse proporzioni delle classi presenti nel dataset originale. Questo evita situazioni in cui alcuni fold non rappresentano adeguatamente classi minoritarie e garantisce una valutazione più stabile e rappresentativa del modello;
- **Leave-One-Out Cross Validation (LOOCV):** è un caso limite della K -Fold in cui il numero di fold coincide con il numero di campioni del dataset, cosicché a ogni iterazione il modello viene addestrato su $N - 1$ esempi e valutato su un singolo campione, ripetendo il processo per tutti gli N dati disponibili. Questo approccio fornisce una stima molto accurata e poco distorta delle prestazioni, ma risulta computazionalmente molto oneroso e poco pratico per dataset di grandi dimensioni;
- **Time Series Splitting:** è una tecnica di validazione utilizzata per Time Series (sequenza di dati o misurazioni raccolti a intervalli di tempo regolari o sequenziali), in cui non è possibile effettuare mescolamenti casuali per evitare la rottura della sequenza cronologica e il rischio di data leakage. Il dataset viene quindi suddiviso rispettando l'ordine temporale, tipicamente con una finestra di training che si espande progressivamente e un validation set costituito dal passo temporale immediatamente successivo, così come il test set rispetto al validation, garantendo che la valutazione avvenga sempre su dati futuri rispetto all'addestramento.

Nei dataset molto grandi si usa tipicamente la suddivisione Hold-out, mentre per dataset piccoli o medi si preferisce la K-Fold Cross Validation per una stima più robusta e l'ottimizzazione degli iperparametri. In caso di dati sbilanciati si adotta la Stratified K-Fold per preservare la distribuzione delle classi, mentre per Time Series ovviamente si utilizza il Time Series Splitting per mantenere la causalità. La scelta della tecnica di split è determinante per valutare correttamente la capacità di generalizzazione del modello.

3.3 Feature Scaling

Le reti neurali calcolano i gradienti e aggiornano i pesi sulla base dell'ordine di grandezza dei dati in ingresso. Quando una variabile presenta valori nell'ordine dei milioni e un'altra valori nell'ordine dei decimali, lo spazio di ottimizzazione risulta deformato, con una geometria fortemente asimmetrica. In tali condizioni, il gradient descent incontra difficoltà significative nel convergere verso un minimo globale utilizzando un unico learning rate.

Tra le principali tecniche utilizzate per la gestione delle differenti scale delle variabili si annoverano la normalizzazione min-max e la standardizzazione Z-score. La normalizzazione min-max consiste nel comprimere o dilatare i valori di una variabile in un intervallo prefissato, tipicamente $[0, 1]$ oppure $[-1, 1]$, mediante la seguente trasformazione:

$$x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}.$$

Questa procedura risulta particolarmente efficace quando i limiti teorici del dominio dei dati sono noti e ben definiti, come nel caso dei pixel di un'immagine, i cui valori sono compresi tra 0 e 255.

La standardizzazione Z-score, invece, consiste nel trasformare la distribuzione dei dati in modo che essa presenti media nulla e deviazione standard unitaria, secondo la relazione:

$$x_{std} = \frac{x - \mu}{\sigma}.$$

Nel contesto del machine learning moderno, tale tecnica è spesso preferita alla normalizzazione min-max in quanto maggiormente robusta rispetto alla presenza di valori anomali (outliers) e in grado di mantenere i dati centrati attorno allo zero. Questa proprietà contribuisce a creare condizioni matematiche favorevoli al corretto funzionamento dei processi di ottimizzazione e delle strategie di inizializzazione dei pesi.

3.4 Weight Initialization

Prima di avviare l'addestramento di una Neural Network (la fase di Forward Pass e la successiva Backpropagation), è essenziale assegnare un valore di partenza ai parametri del modello, ovvero le matrici dei pesi V e i vettori di bias b . Questa fase è fondamentale, poiché una scorretta inizializzazione può compromettere la convergenza del modello, rallentare l'apprendimento o causare instabilità numeriche come il *vanishing* o l'*exploding gradient*.

Zero-Initialization

Impostare tutti i pesi e i bias a zero è una scelta intuitiva ma errata, poiché induce il cosiddetto **problema della simmetria**: i neuroni di uno stesso hidden layer, partendo da pesi identici, producono durante il forward pass le stesse attivazioni e ricevono durante la Backpropagation gli stessi gradienti, subendo quindi aggiornamenti identici e rimanendo indistinguibili nel tempo. Di conseguenza, la rete non riesce a specializzarsi e si comporta come se ogni strato contenesse un singolo neurone. I bias possono invece essere inizializzati a zero senza problemi, a condizione che i pesi siano inizializzati in modo tale da rompere la simmetria.

Random Initialization

L'inizializzazione casuale consiste nell'assegnare ai pesi valori estratti da una distribuzione aleatoria, spesso una normale standard $\mathcal{N}(0;1)$ scalata da un fattore piccolo, con l'obiettivo di rompere la simmetria tra i neuroni. Sebbene risolva il precedente problema, presenta limiti significativi nelle reti profonde: pesi troppo piccoli causano una progressiva riduzione della varianza del segnale e il *vanishing gradient*, impedendo l'apprendimento dei primi strati, mentre pesi troppo grandi provocano un aumento incontrollato della varianza, portando alla saturazione delle funzioni di attivazione o all'*exploding gradient*, con conseguente instabilità numerica.

Xavier Initialization

Introdotta da *Xavier Glorot*, questa tecnica affronta il problema in modo analitico. **L'obiettivo matematico è mantenere la varianza dei segnali in ingresso pari alla varianza dei segnali in uscita per ogni singolo strato.**

Un'inizializzazione con varianza troppo bassa provoca una progressiva attenuazione del segnale attraverso gli strati, fino ad attivazioni quasi nulle e gradienti estremamente piccoli, con conseguente *vanishing gradient* e mancato aggiornamento dei primi livelli. Viceversa, una varianza eccessiva

amplifica il segnale, portando le attivazioni in regioni di saturazione delle funzioni come sigmoide o tanh, dove le derivate sono prossime a zero, e determinando nuovamente il blocco dell'apprendimento.

La **varianza** è un indice statistico di dispersione. In parole povere, misura quanto i tuoi dati siano "sparpagliati" attorno alla loro media. Se la varianza è vicina a zero, significa che i dati sono tutti molto simili e concentrati vicini alla media. Se è alta, significa che i dati sono molto diversi tra loro e distanti dal valore medio. Dati N (il numero dei dati) e μ (la media dei dati), la formula è la seguente:

$$\text{Var}(x) = \sigma_x^2 = \frac{\sum_{i=1}^N (x_i - \mu)^2}{N}$$

Se x è l'input di un livello e z è la sua uscita (prima dell'attivazione), il nostro obiettivo è che:

$$\text{Var}(z) = \text{Var}(x)$$

Analizzando un singolo neurone con n_{in} connessioni in ingresso, l'uscita lineare è $z = \sum_{i=1}^{n_{in}} w_i x_i$. Assumendo che gli input e i pesi siano indipendenti e abbiano media zero, la varianza dell'output risulta essere:

$$\text{Var}(z) = n_{in} \cdot \text{Var}(w) \cdot \text{Var}(x)$$

Per soddisfare il nostro obiettivo iniziale ($\text{Var}(z) = \text{Var}(x)$), dobbiamo necessariamente imporre che:

$$\text{Var}(w) = \frac{1}{n_{in}}$$

Questa condizione garantisce un **Forward Pass corretto**, tuttavia, durante la **Backpropagation**, l'informazione deve propagarsi in senso inverso. In tale direzione, i gradienti si distribuiscono attraverso le n_{out} connessioni del livello successivo, corrispondenti al numero di neuroni dello strato di output. Affinché la varianza dei gradienti non esploda o svanisca, dobbiamo imporre:

$$\text{Var}(w) = \frac{1}{n_{out}}$$

Poiché nella quasi totalità dei casi il numero di neuroni in ingresso è diverso da quello in uscita ($n_{in} \neq n_{out}$), non è possibile soddisfare entrambe le equazioni contemporaneamente. La soluzione di Glorot consiste nel calcolare una media tra i due requisiti, imponendo una varianza di compromesso:

$$\text{Var}(W) = \frac{2}{n_{in} + n_{out}}$$

A partire da questa varianza teorica, i pesi possono essere campionati fisicamente da due tipi di distribuzioni statistiche:

- **Distribuzione Gaussiana:** I pesi vengono estratti da una curva a campana con media 0 e deviazione standard pari alla radice quadrata della varianza.

$$W \approx \mathcal{N}\left(0; \frac{2}{n_{in} + n_{out}}\right)$$

- **Distribuzione Uniforme:** I pesi vengono estratti in modo equiprobabile da un intervallo chiuso $[-a; +a]$. Sapendo che la varianza di una distribuzione uniforme è $\frac{(2a)^2}{12}$, uguagliando questo valore al nostro compromesso si ricava il limite a :

$$W \approx U\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}; +\sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$

L'inizializzazione di Xavier è adatta per funzioni di attivazione simmetriche rispetto allo zero, come la tangente iperbolica (tanh) o le funzioni di attivazione lineari. Se applichiamo l'inizializzazione di Xavier a una rete con attivazioni ReLU, la teoria crolla. La ReLU, azzerando spietatamente tutti i valori negativi, scarta esattamente il 50% delle informazioni in transito, dimezzando di fatto la varianza ad ogni strato. Questo squilibrio richiede una formula diversa, che consideri solo n_{in} e introduca un fattore moltiplicativo per compensare i valori persi (l'Inizializzazione di He).

3.5 Gestione dei Dati Mancanti

La presenza di valori mancanti (NaN o Null) all'interno di una matrice dei dati compromette il corretto svolgimento delle operazioni di algebra lineare alla base del forward pass, impedendo di fatto l'addestramento del modello. Per tale motivo, prima della fase di training, è necessario adottare strategie adeguate per la gestione delle lacune nei dati.

Una prima possibile soluzione consiste nella cancellazione delle osservazioni o delle variabili incomplete. Tale approccio risulta appropriato nei casi in cui il dataset sia sufficientemente ampio e la quantità di dati mancanti sia trascurabile, in modo da non introdurre distorsioni significative nel processo di apprendimento.

In alternativa, si ricorre all'imputazione statistica, che consiste nel sostituire i valori mancanti con stime ragionevoli derivate dalla distribuzione dei dati disponibili. Tali stime possono includere la media, la mediana o la moda della specifica feature calcolata sull'intero training set, garantendo così la continuità informativa delle variabili e la possibilità di procedere con l'addestramento del modello.

3.6 Data Cleaning

La **Data Cleaning** rappresenta una delle fasi più critiche e delicate del Data Preprocessing e consiste nel processo sistematico di identificazione, correzione o rimozione di errori e anomalie presenti nei dati grezzi. Tale fase è necessaria poiché i dataset reali, raccolti tramite sensori, input umani, sistemi informativi o processi automatizzati, contengono frequentemente imperfezioni dovute a errori di misurazione, problemi di trasmissione, incongruenze nei formati, duplicazioni, rumore o assenza parziale di informazioni.

L'obiettivo principale della Data Cleaning è garantire che i dati soddisfino proprietà fondamentali di qualità, quali accuratezza, consistenza, completezza e affidabilità, riducendo al contempo il rischio di propagare errori nelle fasi successive di modellazione. In particolare, un dataset non pulito può introdurre bias sistematici, instabilità nei processi di ottimizzazione e peggioramento significativo delle capacità di generalizzazione del modello.

Le operazioni tipiche di Data Cleaning includono la gestione dei duplicati, che consiste nell'identificazione e rimozione di osservazioni ridondanti che potrebbero alterare la distribuzione dei dati. La correzione di errori sintattici o strutturali, come formati non coerenti per variabili numeriche o categoriche, e la gestione delle inconsistenze logiche, ad esempio valori impossibili o contraddittori rispetto al dominio del problema. Un ulteriore aspetto riguarda la rilevazione e il trattamento degli **Outlier**, ovvero osservazioni significativamente distanti dalla distribuzione principale dei dati, che possono derivare da errori di misura oppure rappresentare eventi rari ma reali.

Un aspetto fondamentale della Data Cleaning riguarda inoltre la distinzione tra rumore e informazione utile (trend e rumore come abbiamo già visto). Una rimozione eccessivamente aggressiva dei dati anomali può comportare la perdita di pattern significativi, mentre una pulizia insufficiente può lasciare nel dataset elementi distorsivi che degradano le prestazioni del modello. Per questo motivo, le strategie di Data Cleaning devono essere progettate in modo dipendente dal dominio applicativo e spesso integrate con tecniche statistiche e metodi robusti di analisi esplorativa dei dati.

Capitolo 4: Addestramento e Ottimizzazione

4.1 Ciclo di Addestramento di una Neural Network

L'**addestramento di una Neural Network** consiste nel determinare in modo automatico l'insieme ottimale dei suoi parametri interni, rappresentati dalle matrici dei pesi V e dai vettori di bias b , tali da garantire una corretta mappatura tra input e target. Il processo è di natura iterativa e si basa sull'alternanza tra valutazione dell'errore e aggiornamento progressivo dei parametri.

Prima dell'avvio della fase di addestramento, i dati vengono sottoposti a opportune procedure di **Preprocessing**, quali normalizzazione o standardizzazione, e successivamente suddivisi nei sottoinsiemi di **training**, **validation** e **test**. In seguito, si procede all'istanziatura della Neural Network, definendone l'architettura in termini di numero di strati e unità per strato. I pesi vengono inizializzati mediante distribuzioni statistiche appropriate, come l'inizializzazione di Xavier per funzioni di attivazione sigmoidi o l'inizializzazione di He per unità ReLU, mentre i bias sono generalmente posti a zero. In questa fase iniziale, la rete non possiede ancora alcuna informazione appresa e le sue predizioni risultano quindi equivalenti a stime casuali.

L'addestramento non viene eseguito sull'intero dataset in un'unica operazione, poiché ciò risulterebbe inefficiente dal punto di vista computazionale e della memoria, né su singoli campioni, il che introdurrebbe eccessiva instabilità nel processo di ottimizzazione. Per tale motivo, il training set viene suddiviso in sottoinsiemi di dimensione ridotta, denominati **batch**, tipicamente composti da 32, 64 o 128 osservazioni. Un'**epoch** corrisponde al completamento di un ciclo di elaborazione di tutti i batch, ossia al passaggio della rete sull'intero dataset di addestramento una sola volta. Il processo di addestramento completo richiede generalmente un numero elevato di epoch, nell'ordine delle decine o centinaia.

Per ogni singolo batch estratto dai dati di training, la rete esegue in sequenza quattro operazioni matematiche:

1. **Forward Pass:** I dati del batch entrano nello strato di input. Vengono moltiplicati per i pesi, sommati ai bias e passati attraverso le funzioni di attivazione non lineari, livello dopo livello, fino a generare le predizioni finali ($\hat{y}$) nello strato di output;
2. **Calcolo della Loss:** Le predizioni della rete vengono confrontate con le etichette reali (y). Si applica una *Loss Function* (funzione di costo) appropriata (MSE per la Regressione, BCE o CCE per la Classificazione) per calcolare un singolo numero scalare: l'errore medio commesso su quel batch;
3. **Backpropagation:** Si applica la Chain Rule del calcolo differenziale. L'errore calcolato in output viene propagato a ritroso attraverso la rete. Per ogni singolo parametro (peso o bias) viene calcolato il suo gradiente, ovvero una misura esatta di "quanto" quel parametro abbia contribuito all'errore finale;
4. **Aggiornamento dei Pesi:** I parametri vengono fisicamente modificati per ridurre l'errore. Sfruttando la Gradient Descent (spesso arricchita da tecniche come il Momentum o learning rate adattivi), ogni peso viene corretto sottraendo una piccola frazione del suo gradiente, regolata dal learning rate.

Terminato questo step, la rete preleva il batch successivo e ripete le quattro fasi precedenti. Grazie a questi continui e minuscoli aggiustamenti, la rete "impara" e la sua capacità di predizione migliora batch dopo batch.

Al termine di ciascuna epoch, una volta completata l'elaborazione di tutti i batch, il processo di addestramento viene temporaneamente sospeso per effettuare la valutazione sul validation set, costituito da dati non utilizzati in alcuna fase di aggiornamento dei pesi. Durante tale fase viene eseguito esclusivamente il forward pass, mediante il quale si ottengono le predizioni del modello e si calcola l'errore di validazione, senza applicare alcuna procedura di backpropagation. Questa operazione è fondamentale per stimare la capacità di generalizzazione del modello.

Nel caso in cui l'errore sul training set continui a diminuire mentre quello sul validation set inizi ad aumentare, si osserva il fenomeno dell'overfitting, in cui il modello tende a memorizzare i dati di addestramento a scapito della capacità di apprendere regolarità generalizzabili.

L'addestramento si conclude quando si verifica una di queste condizioni:

- Si è raggiunto il numero massimo di epoche prefissato all'inizio.
- L'errore sul Validation Set non migliora più per un certo numero di epoche consecutive (tecnica nota come Early Stopping). Questo evita di sprecare tempo di calcolo e ferma il modello prima che l'overfitting diventi troppo grave, permettendo di recuperare la configurazione dei pesi migliore ottenuta fino a quel momento.

Al termine della fase di addestramento, il modello viene valutato in maniera definitiva sul test set e successivamente impiegato in contesti operativi per l'inferenza.

4.2 Overfitting e Underfitting

Quando addestriamo un modello, il nostro obiettivo non è fargli imparare a memoria i dati passati, ma renderlo capace di fare previsioni accurate su dati futuri e mai visti, ossia fornirgli la **capacità di generalizzare**.

Contro questa capacità di generalizzare, si possono verificare due fenomeni: l'**Overfitting** e l'**Underfitting**.

Nell'**Overfitting** il modello impara a memoria i dati di training, incluso il rumore. I dati si possono dividere in due parti: **trend** e **rumore statistico**. Il nostro obiettivo è quello di creare un modello che isoli il rumore statistico il più possibile, assorbendo completamente (o quasi) il trend che risiede nei dati. L'errore di training scende quasi a zero, ma l'errore sul set di validazione inizia a divergere e ad aumentare progressivamente iterazione dopo iterazione. Quindi il modello ha sì imparato il trend, ma ha anche assorbito completamente il rumore. Si gestisce introducendo tecniche di Regularizzazione (L1/L2, Dropout, Batch Normalization, Data Augmentation) e/o applicando l'Early Stopping.

All'opposto invece abbiamo l'**Underfitting**, quando non è in grado di catturare il trend dei dati, limitandosi ad un modello con una forma troppo approssimativa e semplicistica. In pratica il modello è troppo rigido o troppo semplice per catturare la complessità matematica dei dati. Si mitiga aumentando la capacità del modello, ovvero intervenendo sugli iperparametri strutturali: aggiungere più strati nascosti (layers), aumentare il numero di nodi per strato, o utilizzare funzioni di attivazione più appropriate.

Negli anni '90 fino al 2012, l'IA seguiva un vincolo statistico anti-overfitting: il numero di parametri doveva essere molto inferiore ai dati di training, tipicamente $p \ll N/5$, per garantire stime stabili. Con il Deep Learning tale vincolo è stato abbandonato: l'over-parametrizzazione è oggi una scelta progettuale, con reti da miliardi di parametri addestrate su dataset relativamente più piccoli, ma capaci di generalizzare efficacemente.

Questo è spiegato da fattori complementari: la **lottery ticket hypothesis**, secondo cui reti sovradimensionate contengono sotto-reti già favorevoli che l'ottimizzazione attiva, la **regolarizzazione implicita**, per cui algoritmi come la Stochastic Gradient Descent favoriscono soluzioni semplici, e i vincoli indotti da architettura e tecniche come batch normalization, data augmentation ed early stopping. L'elevata dimensionalità dello spazio dei parametri, inoltre, migliora l'ottimizzazione riducendo la probabilità di cattivi minimi.

Il paradigma classico prevede una curva a U dell'errore di test: oltre un livello ottimale di complessità, l'overfitting dovrebbe degradare la generalizzazione. Nei modelli profondi si osserva invece la **double descent**: dopo un picco critico legato all'interpolazione dei dati, un'ulteriore crescita della complessità riduce nuovamente l'errore di generalizzazione. In questo regime bias e varianza non seguono più la dinamica classica e l'early stopping non è più una regola generale, poiché il minimo iniziale della loss può precedere il regime di migliore generalizzazione.

4.3 Batch Normalization

Durante l'addestramento di una Neural Network, l'informazione attraversa molteplici livelli. L'aggiornamento continuo dei pesi negli strati iniziali induce variazioni nella distribuzione dei segnali in uscita, modificandone media e varianza nel tempo. Di conseguenza, la distribuzione statistica (la media e la varianza) dei numeri che escono dal Livello 1 cambia continuamente. Questo fenomeno, noto come **internal covariate shift**, rallenta il processo di apprendimento poiché i livelli profondi devono adattarsi continuamente a rappresentazioni variabili, rendendo spesso necessario l'uso di learning rate più bassi per garantire stabilità.

La **Batch Normalization** risolve questo problema forzando le attivazioni (tipicamente inserita prima o subito dopo la funzione di attivazione) ad avere una distribuzione stabile e prevedibile. L'operazione non viene calcolata sull'intero dataset, ma "al volo" su ogni singolo batch di dati durante l'addestramento. Viene applicata tra il neurone artificiale e la funzione di attivazione.

Data una batch di m campioni, per una specifica pre-attivazione x , l'algoritmo esegue questi tre passaggi rigorosi:

1. **Calcolo della media del batch:**

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

2. **Calcolo della varianza del batch:**

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

3. **Normalizzazione:** Si sottrae la media e si divide per la deviazione standard del batch. Viene aggiunta una microscopica costante ε al denominatore per garantire la stabilità numerica (evitando divisioni per zero se la varianza fosse nulla). Questo passaggio porta i dati ad avere esattamente media 0 e varianza 1:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

A questo punto sorge un problema teorico: normalizzare sempre e brutalmente le attivazioni a media 0 e varianza 1 potrebbe limitare la capacità della rete di esprimere concetti complessi. Ad esempio, tale vincolo può limitare la possibilità di sfruttare adeguatamente le non-linearità introdotte dalle funzioni di attivazione, come la Sigmoide, privando la rete della sua non-linearità e compromettendo la rappresentazione di relazioni complesse. Per ovviare a questo problema, la Batch Normalization

non si limita alla sola standardizzazione, ma introduce due parametri addestrabili per ciascun neurone, denotati come γ (gamma) e β (beta). Tali parametri non sono derivati direttamente dai dati, ma vengono appresi durante la fase di training tramite Backpropagation, insieme ai pesi della rete. In questo modo, la trasformazione finale delle attivazioni normalizzate $\hat{x}_i$ è espressa come::

$$y_i = \gamma \hat{x}_i + \beta$$

Dove:

- γ : Modula la scala (restituisce una nuova varianza).
- β : Modula lo spostamento (restituisce una nuova media).

Il vantaggio è che se, durante l'addestramento, la rete "scopre" che per quello specifico neurone la normalizzazione è dannosa, l'ottimizzatore può imparare a impostare $\gamma = \sqrt{\sigma_B^2 + \varepsilon}$ e $\beta = \mu_B$. Facendo così, l'operazione matematica annulla perfettamente la normalizzazione appena eseguita, ripristinando il dato originale.

Durante l'addestramento la rete usa la media e varianza del batch corrente. Tuttavia, quando la rete ha finito di imparare e viene usata nel mondo reale per fare predizioni (inferenza), spesso elabora un singolo dato alla volta, quindi non ha un "batch" su cui calcolare le statistiche. Per questo motivo, durante l'addestramento il layer calcola e salva silenziosamente una media mobile (*running average*) e una varianza mobile di tutti i batch visti. Saranno queste statistiche globali e congelate a essere usate per la normalizzazione in fase di test.

4.4 Dropout

Il **Dropout** è una delle tecniche di regolarizzazione del Deep Learning, progettata specificamente per combattere il problema dell'Overfitting.

Durante l'addestramento, ad ogni singola iterazione (ovvero per ogni passaggio di un batch), ogni neurone dello strato a cui è applicato il dropout ha una certa probabilità prefissata p di essere temporaneamente "spento" (o rimosso) dalla rete. Di solito, la probabilità di disattivazione varia tra il 20% e il 50%. Quando un neurone viene sottoposto a dropout, la sua attivazione viene annullata durante l'elaborazione del forward pass, risultando quindi nulla in ingresso allo strato successivo. Di conseguenza, tale unità non contribuisce alla propagazione dell'informazione nella specifica iterazione. Parallelamente, il neurone escluso non partecipa all'aggiornamento dei parametri durante la backpropagation, non ricevendo alcun segnale di gradiente in quell'iterazione.

Ad ogni batch, il sottoinsieme di neuroni disattivati viene determinato in modo stocastico e indipendente rispetto alle iterazioni precedenti, cosicché la rete opera, di fatto, su una diversa sottostruttura ad ogni passo di addestramento.

Il successo del dropout e il suo impatto positivo sulla capacità di generalizzazione possono essere ricondotti principalmente a due fattori:

- **Prevenzione del Co-Adattamento:** nelle Neural Network dense, le unità tendono a sviluppare dipendenze reciproche complesse, tali per cui l'attivazione di un neurone può dipendere in modo significativo dalla presenza di altri neuroni specifici. L'introduzione del dropout interrompe tali dipendenze, poiché ogni unità non può fare affidamento sulla presenza costante delle altre, che possono essere disattivate in modo stocastico. Questo meccanismo induce ciascun neurone a sviluppare rappresentazioni più indipendenti e robuste, riducendo la specializzazione eccessiva su pattern locali;

- **Ensemble Learning:** una Neural Network costituita da N neuroni può essere interpretata come l'insieme di un numero elevato di sotto-reti potenzialmente distinte, dell'ordine di 2^N configurazioni possibili. Durante il training con dropout, ciascuna iterazione corrisponde all'addestramento di una specifica sotto-rete. Il modello finale può quindi essere interpretato come una forma di media approssimata delle prestazioni di queste diverse architetture, sfruttando un principio noto dell'ensemble learning secondo cui la combinazione di molteplici modelli riduce significativamente il rischio di overfitting.

Quando l'addestramento è terminato e inseriamo la rete in produzione per fare predizioni vere e proprie su dati mai visti, il Dropout viene completamente disattivato. Tutti i neuroni sono accesi e lavorano all'unisono. Questa transizione introduce però una discrepanza nella scala delle attivazioni: durante il training, con probabilità di dropout p , ciascun neurone riceveva input mediamente ridotti, mentre in fase di test il numero di unità attive risulta pieno, alterando potenzialmente la *magnitude* dei segnali.

Per compensare tale effetto, vengono adottate due principali strategie. La prima consiste nello **scaling in fase di inferenza**, in cui le attivazioni o i pesi vengono moltiplicati per il fattore $(1 - p)$ al fine di mantenere coerente la scala dei segnali. La seconda, più comunemente utilizzata, è nota come **inverted dropout**: la normalizzazione viene applicata durante l'addestramento, dividendo le attivazioni per $(1 - p)$, rendendo così superflue modifiche in fase di test e mantenendo invariata la procedura di inferenza.

Il dropout viene generalmente applicato esclusivamente agli hidden layer, il suo utilizzo sull'input layer è raro, mentre è tipicamente evitato sull'output layer, che deve preservare integralmente l'informazione predittiva finale.

4.5 Regularisation

Quando addestriamo una Deep Neural Network (Rete Neurale molto profonda e complessa), il rischio principale in cui incorriamo è l'Overfitting, come abbiamo già visto. Invece di ridurre fisicamente il numero di neuroni o di strati (che ne ridurrebbe la potenza espressiva), possiamo costringere la rete a mantenere i suoi parametri (i pesi) il più piccoli e semplici possibile. Questo concetto prende il nome di **Regolarizzazione**.

L'idea è questa: si modifica la Loss Function originale aggiungendo un *Termine di Penalità*. La nuova funzione di costo globale da minimizzare diventa:

$$\mathcal{L}_{\text{regolarizzata}}(W) = \mathcal{L}_{\text{dati}}(W) + \lambda \cdot R(W)$$

Dove:

- $\mathcal{L}_{\text{dati}}(W)$ è l'errore classico sui dati (esempio: MSE, Cross-Entropy, ...);
- $R(W)$ è la funzione di penalità, che cresce al crescere del valore dei pesi;
- λ (Lambda) è un iperparametro cruciale impostato dall'utente, che decide "quanto" peso dare alla penalità rispetto all'errore sui dati. Se $\lambda = 0$ non c'è alcuna regolarizzazione, se λ è enorme i pesi verranno schiacciati quasi a zero (causando Underfitting).

La regolarizzazione viene applicata solo alle matrici dei pesi V , e quasi mai ai vettori di bias b . Un bias grande trasla semplicemente la funzione nello spazio senza aumentarne la "curvatura" o la complessità, quindi non causa Overfitting.

L1 Regularisation

Nella **Regolarizzazione L1**, la penalità è lineare e si calcola come la somma dei valori assoluti dei pesi:

$$R_{L1}(W) = \sum |w_i|$$

La L1 tende a guidare i pesi delle feature meno importanti esattamente a zero. Questo fenomeno si chiama **Sparsità**. In pratica, la regolarizzazione L1 esegue una vera e propria **Feature Selection automatica**: se un input non è strettamente necessario per risolvere il problema, la rete spegne la connessione azzerandone il peso, producendo un modello molto più compatto, veloce da calcolare e facile da interpretare. La derivata della funzione valore assoluto non dipende dalla grandezza del peso, ma solo dal suo segno. La regola di aggiornamento sottrarrà (o aggiungerà) sempre una quantità fissa λ ad ogni iterazione, fino a far sbattere e fermare il peso esattamente sullo zero.

L2 Regularisation

Nella **Regolarizzazione L2**, la penalità si calcola come la somma dei quadrati di tutti i pesi della rete (spesso moltiplicata per $\frac{1}{2}$ per comodità di calcolo della derivata):

$$R_{L2}(W) = \frac{1}{2} \sum w_i^2$$

È la forma di regolarizzazione più diffusa e storicamente utilizzata nel Deep Learning. Poiché la penalità è quadratica, punisce in modo estremamente severo i pesi con valori molto grandi. Questo forza la Neural Network a distribuire "l'importanza" su molti pesi piccoli, piuttosto che affidarsi a un singolo peso gigante. Il risultato è un modello molto più "morbido", liscio e stabile rispetto alle variazioni degli input.

Viene anche chiamata **Weight Decay**. Se calcoliamo il gradiente della nuova loss regolarizzata rispetto a un peso w , la derivata del termine $\frac{1}{2}w^2$ è semplicemente w . Quando applichiamo la Discesa del Gradiente, l'aggiornamento del parametro diventa:

$$w_{\text{nuovo}} = w_{\text{vecchio}} - \eta \cdot (\nabla \mathcal{L}_{\text{dati}} + \lambda \cdot w_{\text{vecchio}})$$

Riordinando i termini:

$$w_{\text{nuovo}} = (1 - \eta\lambda) \cdot w_{\text{vecchio}} - \eta \cdot \nabla \mathcal{L}_{\text{dati}}$$

Si nota subito che, prima ancora di applicare la correzione dettata dai dati, il peso originale viene moltiplicato per un fattore $(1 - \eta\lambda)$ che è strettamente minore di 1. Ad ogni iterazione, il peso "decade" e si restringe un po' verso lo zero, mantenendosi piccolo.

Differenza tra L1 e L2

Il modo migliore per capire la differenza tra le due è immaginarle geometricamente in uno spazio a due dimensioni (con due soli pesi, w_1 e w_2):

- Le curve di livello dell'errore sui dati (la Loss pura) formano delle ellissi concentriche che "scendono" verso il punto di minimo ideale;
- Il vincolo della penalità L2 disegna un cerchio centrato nell'origine. La soluzione ottimale si trova nel punto esatto in cui l'ellisse dell'errore tocca tangenzialmente il cerchio. Questo punto capita quasi sempre in mezzo ai quadranti, dove sia w_1 che w_2 hanno valori piccoli ma diversi da zero;
- Il vincolo della penalità L1 disegna invece un rombo (o diamante) centrato nell'origine, con gli spigoli taglienti posizionati esattamente sugli assi cartesiani. Quando l'ellisse dell'errore si allarga

per toccare il rombo, è enormemente probabile che il punto di contatto avvenga proprio su uno degli "spigoli". Se lo spigolo è sull'asse di w_1 , significa che w_2 in quel punto vale esattamente zero! Ecco svelato il segreto geometrico della sparsità.

In sintesi, la regolarizzazione L1 promuove la sparsità portando alcuni pesi a zero, mentre la regolarizzazione L2 incoraggia pesi piccoli ma non li impone a zero. La scelta tra regolarizzazione L1 e L2 dipende dal problema specifico e dal comportamento desiderato del modello. La regolarizzazione L1 può essere utile per la selezione delle caratteristiche e per produrre modelli più interpretabili, mentre la regolarizzazione L2 spesso porta a migliori prestazioni di generalizzazione impedendo valori di peso elevati.

4.6 Scelta del Learning Rate

Il **learning rate**, come abbiamo visto, è un iperparametro che determina quanto il modello aggiorna i suoi pesi durante l'addestramento. In pratica controlla la "dimensione del passo" con cui l'algoritmo di ottimizzazione si muove nella direzione che riduce l'errore. Se è troppo alto il modello può diventare instabile e non convergere, se è troppo basso l'apprendimento è molto lento o può fermarsi su soluzioni subottimali.

Esistono diversi metodi comuni per aiutare a determinare il valore corretto, che possono essere divisi in due approcci principali: la **ricerca di un valore statico** e l'utilizzo di **strategie dinamiche**. Partiamo da quelli statici:

- **Manual Tuning:** Si può iniziare selezionando manualmente un learning rate basandosi sulla conoscenza pregressa del problema o sull'intuizione. È pratica comune partire con un valore piccolo (ad esempio 0.1, 0.01 o persino inferiore) e osservare il comportamento della convergenza. Se l'algoritmo converge troppo lentamente, si può incrementare gradualmente il valore del parametro. Tuttavia, occorre prestare molta attenzione a non impostarlo su valori troppo alti, poiché ciò potrebbe causare la divergenza dell'algoritmo;
- **Grid Search:** Consiste nell'eseguire una ricerca sistematica su un intervallo predefinito di learning rate. Si definisce un set di valori candidati (ad esempio [0.1, 0.01, 0.001, 0.0001]) e si addestra il modello con ciascuno di essi. Valutando il comportamento della convergenza e le prestazioni per ogni tentativo, si seleziona il learning rate che offre il miglior compromesso tra velocità di convergenza e performance del modello.

L'altra famiglia di approcci abbandona l'idea di un valore fisso. Invece di usare lo stesso learning rate per tutto l'addestramento, si impiegano degli *schedules* che regolano dinamicamente il valore durante il processo di ottimizzazione. Le strategie più comuni includono:

- **Step Decay:** Il learning rate viene ridotto moltiplicandolo per un certo fattore (ad esempio 0.1) dopo aver completato un numero fisso di epoch, oppure quando si verifica una specifica condizione prestazionale;
- **Exponential Decay:** Il learning rate viene fatto diminuire in modo graduale ed esponenziale con il passare del tempo;
- **Adaptive Methods:** Si utilizzano algoritmi di ottimizzazione adattivi avanzati (come AdaGrad, RMSprop o Adam, che vediamo nella prossima sezione). Questi algoritmi sono in grado di aggiustare automaticamente il learning rate basandosi sulla storia dei gradienti calcolati nelle iterazioni precedenti.

Per capire se il learning rate in uso sia corretto o meno, è necessario monitorare l'addestramento attraverso due strumenti principali:

- **Prestazioni sul Validation Set:** È utile monitorare le performance del modello su un set di validazione separato. Se si nota che i miglioramenti si fermano o che la prestazione raggiunge un **plateau** (momento in cui una loss smette di migliorare in modo significativo e rimane quasi costante per diverse iterazioni), potrebbe indicare che il learning rate è troppo alto. In questi casi, la soluzione consiste nel ridurre il learning rate e riprendere l'addestramento.
- **Visualizzazione del Learning Rate:** Consiste nel tracciare la curva della Loss di addestramento (o di altre metriche) in funzione del numero di iterazioni o di epoche. Osservando l'andamento della curva si possono cercare segni di convergenza o instabilità:
 - Se la curva della Loss fluttua o mostra oscillazioni molto ampie, è segno che il learning rate potrebbe essere troppo alto;
 - Al contrario, se la Loss decresce in modo eccessivamente lento, significa che il learning rate impostato potrebbe essere troppo basso.

4.7 Adaptive Methods

Gli **Adaptive Methods** sono algoritmi di ottimizzazione che adattano automaticamente il learning rate per ogni parametro del modello durante l'addestramento. A differenza del Gradient Descent standard (con learning rate fisso), questi metodi modificano il passo in base alle informazioni storiche dei gradienti. In pratica, parametri che cambiano molto spesso ricevono aggiornamenti più piccoli, mentre quelli meno aggiornati possono avere passi più grandi.

Stochastic Gradient Descent

La **Stochastic Gradient Descent (SGD)** ottimizza il calcolo evitando di elaborare l'intero dataset per ogni singolo aggiornamento. Al contrario, aggiorna i parametri utilizzando i gradienti calcolati su piccoli sottoinsiemi di dati, chiamati mini-batch. La regola di aggiornamento per i parametri w in un determinato istante di tempo t è:

$$w_{t+1} = w_t - \eta \cdot \nabla L(w_t; x_i, y_i)$$

Dove $\nabla L(w_t; x_i, y_i)$ è il gradiente della loss calcolato esclusivamente sul mini-batch corrente composto dagli input x_i e dai target y_i . È semplice, efficiente e spesso generalizza bene, ma può essere instabile e richiede un learning rate ben scelto. Infatti, questo metodo non adatta il learning rate (e non è propriamente considerabile un Adaptive Method).

Adaptive Gradient

Adaptive Gradient (AdaGrad) è un algoritmo di ottimizzazione adattiva che abbandona l'idea di un learning rate unico per tutta la rete. Aggiusta invece automaticamente il learning rate in modo indipendente per ciascun parametro, basandosi sulla storia dei gradienti passati.

L'algoritmo è il seguente:

1. Si calcola il gradiente corrente: $g_t = \nabla_w L(w_{t-1})$;
2. Si accumula il quadrato dei gradienti passati: $G_t = G_{t-1} + g_t^2$;

3. Si aggiornano i parametri dividendo il learning rate globale η per la radice di questo accumulato:

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{G_t + \varepsilon}} g_t$$

ε è una minuscola costante aggiunta per scongiurare la divisione per zero

Questo meccanismo fa in modo che i parametri che subiscono aggiornamenti ampi e frequenti vedano il loro learning rate rimpicciolirsi, mentre i parametri aggiornati di rado mantengano un tasso più alto. È utile per dati sparsi, ma tende a ridurre troppo il learning rate nel tempo, fino a bloccare l'apprendimento.

Root Mean Square Propagation

Root Mean Square Propagation (RMSprop) corregge questo problema di AdaGrad introducendo una media mobile esponenziale dei gradienti al quadrato. Questo perché G_t continua a crescere indefinitamente accumulando tutti i quadrati, il learning rate effettivo finisce per diventare infinitesimale, bloccando del tutto l'apprendimento (problema del diminishing learning rate).

Le equazioni di RMSprop di aggiornamento diventano:

$$v_{t+1} = \beta \cdot v_t + (1 - \beta) \cdot (\nabla L(w_t))^2$$

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_{t+1} + \varepsilon}} \nabla L(w_t)$$

Dove β (*decay rate*) decide matematicamente quanto peso dare alla storia passata rispetto all'ultimo gradiente appena calcolato.

Adaptive Moment Estimation

Adaptive Moment Estimation (ADAM) rappresenta lo stato dell'arte dell'ottimizzazione e combina abilmente i punti di forza sia di AdaGrad che di RMSprop. Non si limita a tracciare la media mobile dei gradienti al quadrato (il secondo momento), ma mantiene attivamente anche la media mobile dei gradienti semplici (il primo momento, fungendo da sorta di slancio direzionale).

Il processo si articola in tre equazioni:

1. **Calcolo dei Momenti:**

- (a) **Primo momento (media):** $m_{t+1} = \beta_1 \cdot m_t + (1 - \beta_1) \cdot \nabla L(w_t)$;
- (b) **Secondo momento (varianza):** $v_{t+1} = \beta_2 \cdot v_t + (1 - \beta_2) \cdot (\nabla L(w_t))^2$.

2. **Correzione del Bias:** Poiché entrambi i momenti vengono inizializzati a zero, essi tendono a rimanere sbilanciati (biased) verso lo zero nelle primissime iterazioni. Adam applica quindi una correzione formale:

- $\hat{m}_{t+1} = \frac{m_{t+1}}{1 - \beta_1^{t+1}}$
- $\hat{v}_{t+1} = \frac{v_{t+1}}{1 - \beta_2^{t+1}}$

3. **Aggiornamento Finale:** I momenti corretti vengono combinati per generare l'aggiornamento definitivo dei pesi:

$$w_{t+1} = w_t - \eta \cdot \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1} + \varepsilon}}$$

Grazie al calcolo incrociato di medie e varianze, Adam adatta automaticamente un learning rate ottimale per ogni singolo parametro garantendo un percorso di convergenza incredibilmente fluido e preciso. È uno degli ottimizzatori più usati perché converge velocemente e funziona bene in molti scenari, anche se può talvolta generalizzare meno rispetto a SGD con tuning accurato.

4.8 Vincoli

Supponiamo di voler imporre dei vincoli alla funzione che la Neural Network approssima. Abbiamo due tipi di vincoli:

- **Soft Constraints:** vincoli desiderabili ma non obbligatori. Possono essere violati, ma ogni violazione comporta una penalità o una riduzione della qualità della soluzione;
- **Hard Constraints:** vincoli obbligatori che devono essere sempre soddisfatti. Se anche uno solo viene violato, la soluzione è considerata non valida.

Un Soft Constraint trasforma qualsiasi regola matematica in una "penalità" (**Penalty Term**) da sommare alla Loss principale. La formula generale diventa:

$$L_{totale} = \mathcal{L}(f(x); y) + \sum_{i=1}^K \lambda_i \cdot \text{Penalty}_i(f(x))$$

Se una regola può essere differenziata matematicamente, può diventare un Soft Constraint.

Un Hard Constraint richiede ingegneria architetturale: dobbiamo incapsulare la Neural Network grezza $NN(x)$ dentro un'equazione o una trasformazione che renda la violazione matematicamente impossibile.

- **Positività ($f(x) \geq 0$):** Invece di penalizzare la loss, forziamo l'output applicando una funzione con codominio positivo:

$$f(x) = \text{Softplus}(NN(x)) \quad \text{oppure} \quad f(x) = e^{NN(x)}$$

- **Output limitato in un range $[a; b]$:** Usiamo una Sigmoid per comprimere l'output tra 0 e 1, e poi lo scaliamo linearmente:

$$f(x) = a + (b - a) \cdot \sigma(NN(x))$$

- **Periodicità ($f(x) = f(x + T)$):** Non alteriamo l'output, ma alteriamo l'input. Se sappiamo che la funzione si ripete ogni periodo T , passiamo alla rete il seno e il coseno di x invece di x stesso:

$$f(x) = NN \left(\sin \left(\frac{2\pi}{T} x \right); \cos \left(\frac{2\pi}{T} x \right) \right)$$

Ad esempio: vogliamo addestrare una Neural Network in modo che rappresenti una funzione pari $f(x) = f(-x)$. Nell'approccio Soft, aggiungiamo quindi una penalità alla Loss totale basata sulla differenza tra queste due predizioni:

$$Loss_{totale} = \mathcal{L}(f(x); y_{vero}) + \lambda \cdot MSE(f(x); f(-x))$$

Nell'approccio hard abbiamo due possibilità:

- **Input Symmetrization:** Invece di passare x alla rete, passiamo il suo valore assoluto $|x|$ o il suo quadrato x^2 . Se definiamo l'output come $y = f(|x|)$, è matematicamente garantito che passando $-x$ otterremo $f(|-x|) = f(|x|)$. La rete è strutturalmente obbligata a essere una funzione pari;

- **Output Symmetrization:** Se vogliamo lasciare l'input inalterato (magari perché contiene relazioni complesse), possiamo avvolgere la rete in una funzione che forza la simmetria alla fine:

$$g(x) = \frac{f(x) + f(-x)}{2}$$

Perché funziona matematicamente? Se valutiamo la funzione in $-x$, otteniamo:

$$g(-x) = \frac{f(-x) + f(-(-x))}{2} = \frac{f(-x) + f(x)}{2}$$

Che è esattamente identico a $g(x)$. In questo modo, i pesi interni della rete possono essere qualsiasi cosa, ma l'output complessivo del sistema rispetterà sempre e in ogni caso la regola $f(x) = f(-x)$.

Se ad esempio, vogliamo rendere **commutativa** ($f(x_1; x_2) = f(x_2; x_1)$) la funzione approssimata da un modello di Regressione possiamo usare sempre l'*output symmetrization*:

$$g(x_1; x_2) = \frac{f(x_1, x_2) + f(x_2, x_1)}{2}$$

Se valutiamo la funzione invertita $g(x_2; x_1)$, otteniamo:

$$g(x_2; x_1) = \frac{f(x_2; x_1) + f(x_1; x_2)}{2}$$

Per la proprietà commutativa dell'addizione è esattamente identico a $g(x_1; x_2)$. I pesi interni della rete possono evolversi liberamente, ma l'architettura esterna garantirà una commutatività perfetta in ogni scenario.

Tuttavia così richiede due forward pass. Immagina lo spazio degli input come un piano cartesiano 2D, dove l'asse orizzontale è x_1 e l'asse verticale è x_2 . La proprietà commutativa $f(x_1; x_2) = f(x_2; x_1)$ significa che se prendi un punto sul piano (esempio: $[2, 5]$) e il suo simmetrico rispetto alla diagonale $x_1 = x_2$ (ovvero $[5, 2]$), la rete neurale deve restituire la stessa identica "altezza" y .

Invece di far calcolare alla rete entrambi i punti e farne la media (raddoppiando i calcoli), possiamo "*piegare*" geometricamente lo spazio degli input prima ancora che entri nella rete.

L'idea è creare un "*collo di bottiglia*" ordinato. Qualsiasi coppia di numeri ci venga fornita in ingresso, noi la ordiniamo in modo crescente prima di passarla alla rete. Imponiamo che x_2 sia sempre maggiore di x_1 e se così non è li scambiamo. Così la rete neurale non vedrà mai la permutazione $[2, 5]$. Per lei, quello specifico input non esiste. Vedrà sempre e solo vettori in cui il primo numero è minore o uguale al secondo. Così abbiamo un solo forward pass.

Capitolo 5: Radial Basis Function Networks

5.1 Teorema di Cover

Teorema (Teorema di Cover sulla separabilità dei pattern). *Un problema di Classificazione complesso (non linearmente separabile) in uno spazio a bassa dimensionalità ha un'altissima probabilità di diventare linearmente separabile se viene proiettato in modo non lineare in uno spazio a dimensionalità molto più alta.*

Quando si affrontano problemi complessi (come nelle Neural Network o nelle Support Vector Machines con kernel), i dati di partenza sono spesso sovrapposti e non possono essere divisi da un'unica linea o iperpiano. Tuttavia, aumentando il numero di dimensioni in cui sono rappresentati i dati, gli schemi geometrici dei pattern si "sparpagliano" maggiormente, facilitando l'operazione di partizione.

Il passaggio a una dimensione superiore deve avvenire attraverso funzioni non lineari. L'insieme dei dati subisce una rimappatura dove la posizione di ciascun punto viene ricalcolata sulla base di nuove variabili. In termini pratici, questo è il meccanismo che permette di trasformare una superficie complessa in uno spazio a d dimensioni in un problema facilmente risolvibile con un piano di separazione in uno spazio a N dimensioni (dove $N \gg d$).

Il teorema dimostra che la frazione di dicotomie linearmente separabili in un dato spazio cresce all'aumentare della dimensionalità dello spazio stesso. Quando si proietta un input bidimensionale in uno spazio a infinite dimensioni (ad esempio, utilizzando funzioni a base radiale o il Kernel Gaussian RBF), le classi di pattern diventano linearmente separabili con probabilità che tende a 1.

Questo teorema è alla base delle **Radial Basis Function Network (RBFN)**.

5.2 Radial Basis Function

Le **Radial Basis Function (RBF)**, o *funzioni di base radiali*, sono funzioni matematiche il cui valore dipende esclusivamente dalla distanza (solitamente la distanza euclidea) da un punto centrale fisso, detto **centro**.

Le RBF sono strumenti fondamentali per approssimare funzioni complesse, interpolare dati e trasformare problemi non lineari in problemi risolvibili. La RBF più utilizzata è la **Funzione Gaussiana**:

$$\phi_j(x) = \exp\left(-\frac{\|x - \mu_j\|^2}{2\sigma_j^2}\right) = e^{-\frac{\|x - \mu_j\|^2}{2\sigma_j^2}}$$

Immagina di posizionare queste "campane" sopra i tuoi dati nello spazio:

1. Se un punto di input è molto vicino al centro c (la distanza è vicina a zero), il valore della funzione RBF è prossimo a 1 (massima influenza);

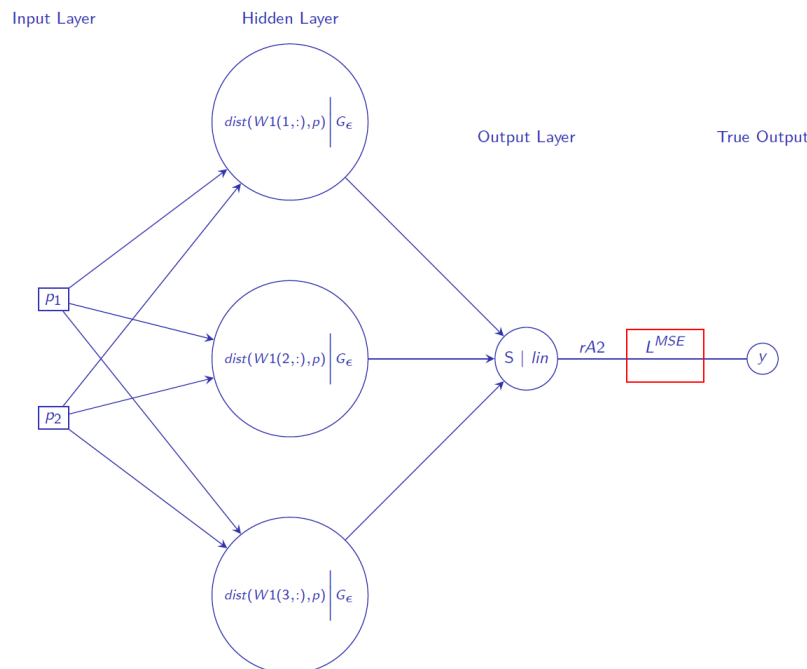
2. Man mano che il punto si allontana dal centro, il valore decade esponenzialmente fino a diventare vicino allo zero.

Le **Radial Basis Function Network** sono Neural Network a strato singolo in cui i neuroni nascosti utilizzano una funzione **RBF come funzione di attivazione**. Il loro scopo è mappare dati non lineari in spazi multidimensionali, rendendoli facilmente separabili. A differenza dei Multi-Layer Perceptron, che utilizzano il prodotto scalare (somma pesata) e funzioni di attivazione come Sigmoido o ReLU, le Radial Basis Function Network utilizzano il concetto di distanza per calcolare l'attivazione dei neuroni.

5.3 Architettura della Rete

Una RBFN ha una topologia rigidamente fissata a tre strati:

- **Input Layer:** Composto da N nodi sensoriali. Non esegue alcun calcolo, si limita a passare il vettore di input x al livello successivo;
- **Hidden Layer:** Contiene K neuroni, ognuno associato ad una Radial Basis Function. Non ci sono pesi da imparare in questo passaggio nel senso classico del termine, ma ogni neurone possiede un proprio *Centro* (μ) e una *Larghezza* (σ). Queste funzioni sono spesso gaussiane (come abbiamo visto) che misurano la distanza tra i dati di input e il Centro. Più i dati di input sono vicini al Centro, maggiore è l'attivazione del neurone;
- **Output Layer:** Questo livello è strettamente lineare e quindi esegue una semplice somma pesata delle attivazioni provenienti dall'Hidden Layer. L'output può essere una Classificazione binaria, una Classificazione Multiclasse o una Regressione, a seconda del compito specifico.



Dal **punto di vista geometrico**, l'attivazione di un neurone RBF raggiunge il valore massimo unitario quando l'input x coincide con il centro μ_j e decresce esponenzialmente verso zero all'aumentare della distanza da esso. Il parametro σ_j , detto **spread** o deviazione standard, determina l'estensione della regione di influenza del neurone: valori ridotti di σ_j producono una risposta fortemente localizzata, sensibile soltanto agli input prossimi al centro, mentre valori più elevati estendono la risposta anche a punti più distanti.

5.4 Forward Step

Poiché l'architettura di una RBFN è divisa in una parte non lineare (hidden) e una lineare (output), l'addestramento avviene tipicamente in due fasi distinte, rendendolo molto più veloce rispetto alla Backpropagation di un MLP profondo. Le fasi sono:

1. **Apprendimento Non Supervisionato (Hidden Layer):** In questa fase, senza guardare le etichette target (y), si posizionano i centri μ_j nello spazio degli input. Per trovare i centri si usa un algoritmo di Clustering (ad esempio: K-Means). Lo spread σ , invece, si calcola in base alla distanza media tra i centri appena trovati (esempio: K-Nearest Neighbors) affinché le campane gaussiane si sovrappongano dolcemente;
2. **Apprendimento Supervisionato (Output Layer):** Una volta fissati i centri e le larghezze, il livello nascosto viene "congelato". Ora la rete si comporta come un semplice modello di Regressione lineare: $Y = \Phi W$. I pesi W del livello di output possono essere trovati analiticamente e istantaneamente usando la *Matrice Pseudo-Inversa di Moore-Penrose*: $W = (\Phi^T \Phi)^{-1} \Phi^T Y$ oppure, in alternativa, si può usare la *Gradient Descent*.

Nonostante abbia un singolo strato nascosto (rete superficiale), una RBF può approssimare qualsiasi funzione al livello di precisione desiderato. L'idea è che, quando si utilizza MAE come Loss Function, si possono aggiungere in modo incrementale nuovi neuroni centrati esattamente sul pattern di input che causa l'MAE corrente, quindi riottimizzare tutti i parametri dei coefficienti e ripetere il processo.

5.5 Kernel Density Estimation

La **Kernel Density Estimation (KDE)** è una tecnica non parametrica utilizzata per stimare la funzione di densità di probabilità di una variabile casuale a partire da un insieme di dati osservati. A differenza degli approcci parametrici, non assume una forma prefissata per la distribuzione dei dati, ma costruisce una stima della densità direttamente a partire dai campioni disponibili.

Siano dati n campioni $x_1, x_2, \dots, x_n$, la stima KDE della densità nel punto x è definita come:

$$\hat{f}(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

dove $K(\cdot)$ è la funzione kernel, h è il parametro di banda (**bandwidth**) e n è il numero di osservazioni.

L'idea consiste nell'associare a ciascun campione x_i una funzione kernel centrata in quel punto e nel sommare i contributi di tutti i campioni. La densità stimata risulta quindi dalla sovrapposizione di tali funzioni.

Il kernel più comunemente utilizzato è quello gaussiano $K(u) = \frac{1}{\sqrt{2\pi}} e^{-\frac{u^2}{2}}$ da cui si ottiene:

$$\hat{f}(x) = \frac{1}{nh\sqrt{2\pi}} \sum_{i=1}^n e^{-\frac{(x-x_i)^2}{2h^2}}$$

Il parametro di banda h svolge un ruolo fondamentale nella qualità della stima. Valori troppo piccoli producono una densità molto irregolare e sensibile alle fluttuazioni dei dati (*overfitting*), mentre valori troppo elevati generano una stima eccessivamente liscia che può nascondere caratteristiche rilevanti della distribuzione (*oversmoothing*).

La KDE presenta una stretta analogia con le Radial Basis Function Network, poiché ambedue si basano sullo stesso principio geometrico: utilizzare delle funzioni gaussiane (i kernel) per costruire

una curva globale complessa. Se prendiamo diversi punti sparsi nello spazio e mettiamo una campana gaussiana sopra ciascuno di essi, la somma di tutte queste campane genererà una curva globale liscia e continua. Questa tecnica di "lisciatura" dei dati tramite l'aggiunta di kernel prende il nome di **Kernel Smoothing**. Tuttavia, mentre nelle RBFN i centri e i pesi sono utilizzati per approssimare una funzione o risolvere problemi di Classificazione e Regressione, nella KDE l'obiettivo è stimare una densità di probabilità.

5.6 Quando usare gli MLP e quando le RBFN?

I Multi-Layer Perceptron e le Radial Basis Function Networks sono **entrambi approssimatori universali**, capaci di modellare qualsiasi funzione continua.

Caratteristica	Multi-Layer Perceptron (MLP)	Radial Basis Function Network (RBFN)
Spazio di attivazione	Globale: i neuroni utilizzano il prodotto scalare (iperpiani) e funzioni di attivazione come Sigmoido o ReLU. L'output di un neurone influenza ampie regioni dello spazio, dividendo l'intero spazio in semispazi.	Locale: i neuroni utilizzano la distanza euclidea e funzioni gaussiane. L'output è significativo solo in una regione locale attorno al centro del neurone e decade rapidamente altrove.
Architettura	Profonda (Deep): può contenere molti livelli nascosti, costruendo gerarchie di rappresentazioni sempre più astratte.	Piatta (Shallow): tipicamente composta da un solo strato nascosto non lineare seguito da uno strato di output lineare.
Addestramento	Lento e iterativo: utilizza backpropagation su tutti i livelli. È sensibile a minimi locali e richiede molte epoche per la convergenza.	Veloce e ibrido: una prima fase non supervisionata (ad esempio K-means per la selezione dei centri) seguita da una fase supervisionata con soluzione analitica dei pesi di output.

Le RBFN sono particolarmente efficaci in problemi a *bassa dimensionalità, di controllo e di approssimazione di funzioni*, soprattutto quando è *richiesta un'elevata velocità di addestramento*. La loro struttura consente infatti di determinare i pesi dello strato di output in forma chiusa, riducendo significativamente il costo computazionale rispetto ai modelli iterativi. Inoltre, l'uso di funzioni locali gaussiane favorisce un'interpolazione precisa nelle regioni osservate, con risposte rapidamente nulle in aree non campionate, e contribuisce a evitare problemi legati ai minimi locali nella fase di ottimizzazione dei pesi. Le RBFN risultano tuttavia meno adatte in contesti ad alta dimensionalità o in presenza di dati complessi, dove **la distanza euclidea perde significato e diventa necessario un numero elevato di centri per una copertura efficace dello spazio**.

Per tali ragioni, nei problemi moderni ad alta complessità, come visione artificiale ed elaborazione del linguaggio naturale, si preferiscono architetture basate su Multi-Layer Perceptron profondi, capaci di apprendere rappresentazioni gerarchiche e di generalizzare meglio in spazi ad alta dimensionalità.

5.7 Codice

```

1  def RBF_extend(X):
2      """
3      Extends the input matrix by adding a row of ones (bias term)
4
5      Parameters:
6          - X: Input matrix (n_features x n_samples)
7
8      Returns:
9          - Extended matrix with bias term ((n_features+1) x n_samples)
10     """
11     return np.vstack([np.ones(X.shape[1]), X])
12
13 def RBF_forward(P, W1, W2, epsilon):
14     """
15     Forward pass of the RBF network
16
17     Parameters:
18         - P: Input patterns (n_samples x n_features)
19         - W1: Centers matrix (n_hidden x n_features)
20         - W2: Output weights (1 x (n_hidden+1))
21         - epsilon: Width parameter for Gaussian RBFs
22
23     Returns:
24         - rA2: Network output (1 x n_samples)
25         - A1: Extended hidden layer outputs ((n_hidden+1) x n_samples)
26     """
27     rA0 = P.T # Transpose input matrix (D x B)
28     # Compute Euclidean distances between each pattern and each center
29     rZ1 = np.sqrt(np.sum((W1[:, :, np.newaxis] - rA0[np.newaxis, :, :])**2, axis=1))
30     # Apply Gaussian activation function
31     rA1 = np.exp(-(epsilon**2 * (rZ1**2)))
32     # Extend hidden layer outputs with bias term
33     A1 = RBF_extend(rA1)
34     # Compute output layer pre-activation
35     rZ2 = W2 @ A1
36     # Output (linear activation)
37     rA2 = rZ2
38
39     return rA2, A1
40
41 def RBF_initialize_centres(model, Xtrain, ytrain):
42     """
43     Initializes the RBF centers based on the specified method
44
45     Parameters:
46         - model: Dictionary containing model parameters
47         - Xtrain: Training inputs (n_samples x n_features)
48         - ytrain: Training targets (n_samples x 1)
49
50     Returns:
51         - model: Updated model dictionary
52         - W1: Centers matrix (n_hidden x n_features)
53     """
54     method = model['centres_generation_method']
55     n_hidden = model['n_hidden']
56
57     if method == 'random':
58         # Randomly select training points as centers
59         idx = np.random.permutation(Xtrain.shape[0])
60         W1 = Xtrain[idx[:n_hidden], :]
61     elif method == 'clustering':
62         # K-means clustering for center initialization

```

```

63     from sklearn.cluster import KMeans
64     kmeans = KMeans(n_clusters=n_hidden, max_iter=50, tol=0.001, random_state=42) # Added
65     random_state for reproducibility
66     print('Clustering training data using K-means...')
67     kmeans.fit(Xtrain)
68     W1 = kmeans.cluster_centers_
69     print('Clustering completed - centers initialized')
70 elif method == 'debug':
71     # Fixed centers for debugging
72     if n_hidden != 3:
73         print('Warning: Forcing n_hidden=3 for debug mode')
74         model['n_hidden'] = 3
75         n_hidden = 3
76
77     if Xtrain.shape[1] == 1:
78         W1 = np.array([[ -8], [ -1], [ 7]])
79     elif Xtrain.shape[1] == 2:
80         W1 = np.array([[ -8, -8], [ -1, -1], [ 7, 7]])
81     else:
82         raise ValueError('Unsupported input dimension for debug mode')
83 else:
84     raise ValueError('Unrecognized center generation method')
85
86     model['W1'] = W1
87     return model, W1
88
89 def RBF_train_offline(X, y, model):
90     """
91     Trains the RBF network in two phases:
92     1. Center initialization
93     2. Output weight computation via least squares
94
95     Parameters:
96     - X: Training inputs (n_samples x n_features)
97     - y: Training targets (n_samples x 1)
98     - model: Model configuration dictionary
99
100     Returns:
101     - model: Updated model with trained parameters
102     - W1: Centers matrix (n_hidden x n_features)
103     - W2: Output weights (1 x (n_hidden+1))
104     """
105     # Phase 1: Center initialization
106     model, W1 = RBF_initialize_centres(model, X, y)
107
108     # Phase 2: Compute output weights via least squares
109     # Forward pass to get hidden layer outputs
110     _, A1 = RBF_forward(X, W1, np.ones((1, model['n_hidden'] + 1)), model['epsilon'])
111
112     # Reshape y to match expected dimensions (n_samples,) instead of (n_samples, 1)
113     y_flat = y.ravel()
114
115     # Solve least squares problem for output weights
116     W2 = np.linalg.lstsq(A1.T, y_flat, rcond=None)[0]
117     W2 = W2.reshape(1, -1) # Ensure W2 is (1 x (n_hidden+1))
118
119     model['W2'] = W2
120     return model, W1, W2
121
122 def RBF_predict(X, W1, W2, epsilon):
123     """
124     Makes predictions using the trained RBF network
125
126     Parameters:
127     - X: Input patterns (n_samples x n_features)
128     - W1: Centers matrix (n_hidden x n_features)

```

```
128     - W2: Output weights (1 x (n_hidden+1))
129     - epsilon: Width parameter for Gaussian RBFs
130
131     Returns:
132     - Predictions (1 x n_samples)
133     """
134     y_pred, _ = RBF_forward(X, W1, W2, epsilon)
135     return y_pred
```

Capitolo 6: Evolutionary Algorithms

6.1 Hyperparameter Tuning

Adesso il nostro obiettivo è ottimizzare gli **iperparametri** delle Neural Network. *Cosa sono gli iperparametri? E che differenza c'è con i parametri?* I parametri sono le variabili interne al modello. Sono il risultato finale del processo di addestramento e definiscono la "conoscenza" che il modello ha estratto dai dati. Nel caso delle Neural Network sono i pesi (w) e i bias (b). Gli iperparametri sono le variabili esterne o le "impostazioni di configurazione" del modello. Regolano la struttura del modello stesso e il modo in cui l'algoritmo apprende. Per le Neural Network sono: il numero di neuroni, il numero di hidden layer, il learning rate, la batch size, il numero di epoch, il dropout rate e infine anche la funzione di attivazione e l'optimizer. L'ottimizzazione degli iperparametri (**Hyperparameter Tuning**) è quasi sempre un problema fortemente non convesso.

La ricerca dell'ottimo globale di una funzione non convessa, non lineare e non differenziabile è un **problema NP-hard**. In un dominio discreto, è teoricamente possibile ricorrere a una **Exhaustive Search** (ricerca esaustiva), valutando la funzione su tutti i punti dello spazio di ricerca. Tuttavia, tale approccio risulta generalmente impraticabile nei problemi reali a causa dell'elevata complessità computazionale, anche su architetture Exascale.

Le architetture Exascale rappresentano il supercomputing di ultima generazione, capace di eseguire un miliardo di miliardi (10^{18}) di operazioni in virgola mobile al secondo (ExaFLOPS). Sfruttano enormi cluster di CPU e GPU che lavorano in parallelo e supportano la ricerca scientifica avanzata, dalla climatologia predittiva alla virologia.

La **Grid Search** è l'equivalente della Exhaustive Search quando gli iperparametri sono continui. In tal caso, tutti gli iperparametri vengono discretizzati tra un valore minimo e uno massimo, quindi viene eseguita la Exhaustive Search. Tuttavia, anche qua la sua complessità cresce rapidamente con la dimensionalità, rendendola poco scalabile.

La **Random Search** campiona casualmente lo spazio degli iperparametri tramite un metodo di ricerca stocastica che richiede solo un ottimo algoritmo per generare numeri casuali. Tuttavia questo metodo è spesso inefficace.

Andando oltre la Random Search, si collocano **metodi di ricerca stocastica ispirati alla natura**, proposti per migliorare l'efficienza della ricerca e mitigare le limitazioni dei metodi enumerativi e puramente casuali.

Vediamo un po' di terminologia. Nel Deep Learning, un "**individuo**" è una specifica configurazione di una Neural Network. Il suo "**cromosoma**" non è altro che un array o un dizionario contenente i suoi *iperparametri* (i "**geni**"). Un gene può essere continuo (esempio: il Learning Rate), discreto (esempio: il numero di livelli) o categorico (esempio: la funzione di attivazione).

Esempio di Cromosoma:

```
[Learning Rate=0.01, Livelli=3, Neuroni_per_livello=64, Dropout=0.2, Attivazione="ReLU"]
```

La **Fitness** è la metrica di successo. Per valutare la fitness di un individuo, dobbiamo costruire la Neural Network con quei geni, addestrarla per un certo numero di epoche e misurare una metrica sul Validation Set come la Loss (l'inverso per avere una metrica che più è grande meglio è), l'Accuracy o un altro mezzo di valutazione più consono al problema. Una rete che raggiunge il 95% di Accuracy avrà una fitness maggiore di una che raggiunge l'80%, e avrà più probabilità di essere selezionata per riprodursi.

Il **Crossover** è la combinazione delle caratteristiche architetturiche di due Neural Network promettenti sperando che il "*figlio*" erediti il meglio di entrambe.

Ad esempio:

Genitore A (ottima accuracy): [LR=0.01, Livelli=3, Neuroni=128, Dropout=0.5, Attivazione="ReLU"]

Genitore B (buona accuracy): [LR=0.005, Livelli=5, Neuroni=64, Dropout=0.2, Attivazione="Tanh"]

Se applichiamo un Uniform Crossover (lanciando una moneta per ogni iperparametro e quindi avendo 50 e 50 che riceva un iperparametro dal genitore A o dal genitore B), potremmo ottenere questo figlio:

Rete Figlia: [LR=0.01 (da A), Livelli=5 (da B), Neuroni=128 (da A), Dropout=0.2 (da B), Attivazione="ReLU" (da A)]

L'idea è che alcune scelte progettuali efficaci possano provenire da genitori diversi. Il crossover consente quindi di combinare queste caratteristiche in una nuova architettura che potrebbe ottenere prestazioni superiori rispetto a entrambe le reti di partenza.

La **mutazione** consiste nel prendere una Neural Network e alterare casualmente una sua piccola parte strutturale. Questo serve a esplorare nuove configurazioni che il crossover da solo non potrebbe mai generare.

Ad esempio, prendiamo la nostra "*Rete Figlia*" di prima e applichiamo due mutazioni:

- **Mutazione di un gene discreto (Random Reset):** Scegliamo di mutare il numero di neuroni. Il valore passa casualmente da 128 a 256;
- **Mutazione di un gene continuo (Gaussian Mutation):** Scegliamo di mutare il Dropout. Aggiungiamo un piccolo rumore gaussiano: $0.2 + 0.05 = 0.25$.

Avremmo che:

Rete Mutata: [LR=0.01, Livelli=5, Neuroni=256 (Mutato!), Dropout=0.25 (Mutato!), Attivazione="ReLU"]

Se, ad esempio, nell'intera popolazione iniziale nessuna rete è stata generata con 256 neuroni, il crossover non potrà mai creare una rete da 256 neuroni. La mutazione inietta nuovo materiale genetico, permettendo all'algoritmo di non bloccarsi e di scoprire che reti più larghe funzionano meglio.

6.2 Simulated Annealing

Il **Simulated Annealing** è un algoritmo di ottimizzazione probabilistica ispirato al processo della ricottura (*annealing*) nella metallurgia. È utilizzato per trovare soluzioni ottimali in problemi di ottimizzazione combinatoria, particolarmente nei casi in cui i metodi tradizionali basati sul gradiente rischierebbero di rimanere bloccati in ottimi locali.

La ricottura è un trattamento termico impiegato per migliorare la struttura interna dei metalli. Le lavorazioni meccaniche introducono difetti cristallini e aumentano l'energia interna, riducendo la stabilità del materiale. Il metallo viene riscaldato ad alta temperatura, condizione in cui gli atomi aumentano la loro mobilità e possono riorganizzarsi tra diverse configurazioni. Successivamente viene raffreddato lentamente, con una progressiva riduzione dell'energia cinetica e della mobilità atomica, fino alla stabilizzazione della struttura. Il risultato è una struttura cristallina più ordinata, con minore presenza di difetti e un'energia interna ridotta. Il sistema non raggiunge direttamente lo stato di minima energia globale, ma lo approssima attraverso la riorganizzazione progressiva durante il raffreddamento.

Analogamente, il Simulated Annealing esplora iterativamente lo spazio delle soluzioni, consentendo occasionalmente delle mosse svantaggiose in salita ("*uphill moves*") per fuggire dalle valli sub-ottimali e cercare il minimo globale. L'idea di base del Simulated Annealing è quella di partire da una soluzione iniziale e muoversi iterativamente verso soluzioni vicine, accettando sia miglioramenti che, occasionalmente, soluzioni peggiori. **L'accettazione di soluzioni peggiori è controllata da un parametro di temperatura, che viene gradualmente ridotto nel tempo.** A temperature più elevate, l'algoritmo ha una maggiore probabilità di accettare soluzioni peggiori, consentendo l'esplorazione dello spazio delle soluzioni ed evitando di rimanere intrappolato in ottimi locali. Man mano che la temperatura diminuisce, l'algoritmo diventa più selettivo e tende a **convergere verso la soluzione ottimale o quasi ottimale.**

In pratica, dopo aver inizializzato la temperatura, scelto una **cooling schedule** e aver trovato una soluzione di partenza, l'algoritmo esegue le seguenti iterazioni fino al raggiungimento del criterio di arresto:

1. Genera una soluzione vicina;
2. Valuta la funzione obiettivo per la nuova soluzione;
3. Confronta i valori della funzione obiettivo della soluzione corrente e della nuova soluzione:
 - Se la nuova soluzione è migliore, accettarla come soluzione corrente;
 - Se la nuova soluzione è peggiore, calcolare la probabilità di accettazione in base alla temperatura e a un criterio probabilistico (come il *criterio di Metropolis*). Quindi accettare la soluzione peggiore con una certa probabilità.
4. Regolare la temperatura in base alla cooling schedule.

Restituisce alla fine la soluzione migliore trovata durante le iterazioni.

Il **criterio di Metropolis** stabilisce la probabilità di accettare una soluzione peggiorativa durante l'esecuzione del Simulated Annealing. Se la nuova soluzione presenta un peggioramento pari a $\Delta E > 0$, essa viene accettata con una probabilità che segue la **Distribuzione di Boltzmann**:

$$P = e^{-\frac{\Delta E}{T}}$$

dove T è la temperatura corrente. Operativamente, si genera un numero casuale $r \in [0; 1]$ e la soluzione viene accettata se $r < P$. Questo criterio consente all'algoritmo di evitare ottimi locali nelle fasi iniziali della ricerca e di diventare progressivamente più selettivo al diminuire della temperatura.

La **cooling schedule** determina come la temperatura diminuisce nel tempo. Controlla l'equilibrio tra esplorazione (a temperature più elevate) e sfruttamento (a temperature più basse). Le cooling schedule più comuni includono il *raffreddamento lineare*, il *raffreddamento logaritmico* e il *raffreddamento esponenziale*.

Il Simulated Annealing è particolarmente utile quando si affrontano problemi di ottimizzazione complessi con spazi di ricerca ampi e ricchi di minimi locali, dove trovare l'ottimo globale è difficile.

Il **Quantum Annealing** ne rappresenta la versione quantistica, utilizzata da *Computer Adiabatici* come *D-Wave Systems*. Sfrutta fenomeni quantistici come *sovrapposizione*, *entanglement* e *quantum tunneling*, che consentono di attraversare le barriere di potenziale anziché superarle, permettendo di raggiungere il minimo globale in modo potenzialmente più rapido.

Codice

```

1  def difficult2minimafcn1D(x):
2      """
3      Function with two minima in 1D where the global minimum is difficult to find.
4
5      Mathematical definition:
6      f(x) = -exp(-(x/100)^2)          for x <= 100
7      -exp(-1) + (x-100)(x-102)      for x > 100
8
9      Parameters:
10     x : numpy array or float
11     Input value(s) at which to evaluate the function
12
13     Returns:
14     y : numpy array or float
15     Function value(s) at input x
16     """
17     x = np.asarray(x) if isinstance(x, (list, tuple, np.ndarray)) else np.array([x])
18     y = np.zeros_like(x)
19     mask_le100 = x <= 100
20     y[mask_le100] = -np.exp(-(x[mask_le100]/100)**2)
21     mask_gt100 = x > 100
22     y[mask_gt100] = -np.exp(-1) + (x[mask_gt100]-100)*(x[mask_gt100]-102)
23     return y.item() if y.size == 1 else y
24
25 def perturb(current_x, a, b):
26     """
27     Perturbs the current solution by adding a random value within bounds.
28
29     Parameters:
30     current_x : float
31     Current solution
32     a : float
33     Lower bound of search space
34     b : float
35     Upper bound of search space
36
37     Returns:
38     new_x : float
39     Perturbed solution within bounds [a, b]
40     """
41     new_x = current_x + 0.2*(b-a)*(np.random.random()-0.5)
42     # Ensure the new solution stays within bounds
43     new_x = np.clip(new_x, a, b)
44     return new_x
45
46 def SA_RE_1D(f, a, b, niter, lambda_, dispAndPlot=False):
47     time = np.arange(1, niter + 1)

```



```

48     temperature = np.exp(-lambda_ * time)
49
50     x1 = np.random.random() * (b - a) + a
51     y1 = f(x1)
52
53     current_x = x1
54     current_y = y1
55
56     best_x = x1
57     best_y = y1
58
59     if dispAndPlot:
60         # Create Plotly subplots
61         fig = make_subplots(rows=1, cols=3,
62                             subplot_titles=('Cooling Schedule (Temperature vs Iteration)',
63                                             'Objective Function Landscape',
64                                             'Simulated Annealing Progress'))
65
66         # Temperature plot
67         fig.add_trace(go.Scatter(x=time, y=temperature, mode='lines', name='Temperature'), row=1, col=1)
68         fig.update_xaxes(title_text='Iteration', row=1, col=1)
69         fig.update_yaxes(title_text='Temperature', row=1, col=1)
70
71         # Function plot
72         domain = np.linspace(a, b, 1000)
73         fig.add_trace(go.Scatter(x=domain, y=f(domain), mode='lines', name='Objective Function'), row=1,
74                                col=2)
75         fig.update_xaxes(title_text='x', row=1, col=2)
76         fig.update_yaxes(title_text='f(x)', row=1, col=2)
77
78         # SA progress plot - Objective Function
79         fig.add_trace(go.Scatter(x=domain, y=f(domain), mode='lines', name='Objective Function'), row=1,
80                                col=3)
81         # SA progress plot - Initial Solution
82         fig.add_trace(go.Scatter(x=[x1], y=[y1], mode='markers', marker=dict(color='black', size=10),
83                                name='Initial Solution'), row=1, col=3)
84         fig.update_xaxes(title_text='x', row=1, col=3)
85         fig.update_yaxes(title_text='f(x)', row=1, col=3)
86
87         colormap = plt.cm.plasma(np.linspace(0, 1, niter)) # Keep colormap for color scaling
88
89     for i in range(1, niter):
90         new_x = perturb(current_x, a, b)
91         new_y = f(new_x)
92
93         if (new_y < current_y) or (np.random.random() < temperature[i]):
94             # Add trace for accepted move in SA progress plot
95             if dispAndPlot:
96                 fig.add_trace(go.Scatter(x=[current_x, new_x], y=[current_y, new_y], mode='lines', line=dict(
97                     color=f'rgba({int(colmap[i][0]*255)}, {int(colmap[i][1]*255)}, {int(colmap[i][2]*255)}, 0.5)',
98                     width=1), showlegend=False), row=1, col=3)
99                 fig.add_trace(go.Scatter(x=[new_x], y=[new_y], mode='markers', marker=dict(color=f'rgba({int(
100                     colmap[i][0]*255)}, {int(colmap[i][1]*255)}, {int(colmap[i][2]*255)}, 0.8)', size=5), showlegend=
101                     False), row=1, col=3)
102
103             current_x = new_x
104             current_y = new_y
105
106             if new_y < best_y:
107                 best_x = new_x
108                 best_y = new_y
109
110     if dispAndPlot and i % 50 == 0:
111         print(f'Iteration {i:4d}: best_x = {best_x:.4f}, best_y = {best_y:.4f}, '
112               f'current_x = {current_x:.4f}, current_y = {current_y:.4f}')

```

```

107     if dispAndPlot:
108         # SA progress plot - Best Found Solution
109         fig.add_trace(go.Scatter(x=[best_x], y=[best_y], mode='markers', marker=dict(color='red', size
=12, symbol='star'), name='Best Found'), row=1, col=3)
110
111         # Update layout for titles and potentially legends if needed
112         fig.update_layout(title_text='Simulated Annealing Visualization', showlegend=True) # Central
title
113
114         fig.show()
115
116     return best_x, best_y

```

6.3 Evolution Strategies

Le **Evolution Strategies (ES)** sono l'algoritmo evolutivo più semplice e sono progettate principalmente per risolvere complessi problemi di ottimizzazione numerica.

(1+1)-ES

L'algoritmo più semplice di questa categoria è la **(1+1)-ES**. La notazione matematica definisce chiaramente la sua struttura:

- Il primo "1" indica che l'algoritmo mantiene in memoria una singola soluzione candidata per volta, chiamata **parent** (genitore).
- Il "+1" indica che ad ogni iterazione viene generato un singolo **offspring** (figlio) a partire da quel parent.

L'algoritmo si divide in 5 fasi:

1. **Inizializzazione:** Si genera una soluzione genitore di partenza in modo casuale o tramite euristiche;
2. **Mutazione:** Si crea il figlio applicando un operatore di mutazione al genitore. Questa mutazione introduce piccole variazioni casuali (come perturbazioni gaussiane o uniformi) per esplorare lo spazio di ricerca. Nelle ES, l'operatore di mutazione è la componente fondamentale per guidare l'esplorazione;
3. **Valutazione:** Si calcola la fitness (ovvero il valore della funzione obiettivo da ottimizzare) sia del genitore che del figlio;
4. **Selezione:** Si confrontano i punteggi di fitness. Se il figlio ottiene un punteggio migliore, sostituisce il genitore. In caso contrario, il figlio viene scartato e si riparte dal vecchio genitore;
5. **Conclusione:** Si ripetono i passaggi precedenti (da 2 a 4) fino al raggiungimento di un limite (ad esempio, un numero massimo di iterazioni, un limite di tempo o una certa soglia di fitness).

Sebbene non sia l'algoritmo più efficiente per problemi complessi, fornisce un quadro semplice per comprendere gli algoritmi evolutivi e i loro componenti di base.

```

1  # Binary Encoding
2  def logical2real(logical_vector, a, b):
3      """
4      Convert a binary-encoded vector to a real value within [a,b]
5
6      Parameters:
7      - logical_vector: binary encoded solution (numpy array of 0s and 1s)
8      - a: lower bound of search interval
9      - b: upper bound of search interval
10
11     Returns:
12     - Real-valued number in (a,b) corresponding to the binary encoding
13
14     Mathematical Formulation:
15     The conversion follows these steps:
16     1. Convert binary string to unsigned integer
17     2. Normalize to [0,1) range
18     3. Map to (a,b) interval with proper scaling
19
20     Example:
21     >>> logical2real(np.array([1, 1, 0]), 13, 17)
22     """
23     n = len(logical_vector)
24     # Convert binary vector to decimal integer
25     uint_value = np.sum(2**(n - 1 - np.where(logical_vector)[0]))
26     # Normalize to [0,1) range
27     real_0_1 = uint_value / (2**n)
28     # Map to (a,b) interval with proper scaling and centering
29     return a + real_0_1 * (b - a) + 0.5 * (b - a) / (2**n)
30
31 # Population Initialization
32 def init_population(ngenes, popsize=1):
33     """
34     Initialize random binary population
35
36     Parameters:
37     - ngenes: length of binary chromosome
38     - popsize: number of individuals in population
39
40     Returns:
41     - Binary matrix of shape (popsize, ngenes) with random 0/1 values
42
43     Implementation Note:
44     Uses numpy's random number generator for reproducibility
45     """
46     return np.random.rand(popsize, ngenes) > 0.5
47
48 # Fitness Evaluation Function
49 def evaluate_fitness(f, population, a, b):
50     """
51     Evaluate fitness for each individual in a binary-encoded population
52
53     Parameters:
54     - f: objective function to minimize
55     - population: matrix where each row is a binary-encoded individual
56     - a: lower bound of search space
57     - b: upper bound of search space
58
59     Returns:
60     - Array of fitness values for each individual
61
62     Technical Note:
63     Converts each binary individual to real value before evaluation
64     """
65     fitness = np.zeros(len(population))
66     for i in range(len(population)):

```

```

67     real_val = logical2real(population[i], a, b)
68     fitness[i] = f(real_val)
69     return fitness
70
71 # Mutation Operator
72 def uniform_mutation(logical_x, mutation_prob):
73     """
74     Uniform mutation operator for binary-encoded individuals
75
76     Parameters:
77     - logical_x: binary individual to mutate
78     - mutation_prob: probability of flipping each bit
79
80     Returns:
81     - Mutated copy of the input individual
82
83     Algorithm:
84     Each bit has independent probability mutation_prob of being flipped
85     """
86     # Create copy to avoid modifying input
87     mutated = logical_x.copy()
88     # Generate mutation mask
89     mutation_mask = np.random.rand(len(logical_x)) < mutation_prob
90     # Flip selected bits
91     mutated[mutation_mask] = ~mutated[mutation_mask]
92     return mutated
93
94 def es_1plus1_visual(f, a, b, n_iter, n_genes, mutation_prob):
95     """(1+1) ES with enhanced visualization"""
96     # Setup figure
97     fig = plt.figure(figsize=(15, 6))
98     ax1 = fig.add_subplot(1, 2, 1)
99     ax2 = fig.add_subplot(1, 2, 2)
100
101     # Plot function
102     domain = np.linspace(a, b, 1000)
103     ax1.plot(domain, f(domain), 'b-', lw=2, alpha=0.7)
104     ax2.plot(domain, f(domain), 'b-', lw=2, alpha=0.7)
105
106     # Initialize
107     current_x = init_population(n_genes, 1)[0]
108     current_real = logical2real(current_x, a, b)
109     current_fitness = f(current_real)
110
111     # Create plot elements
112     parent_line, = ax1.plot([], [], 'ko', ms=8, label='Parent')
113     child_line, = ax1.plot([], [], 'ro', ms=6, alpha=0.5, label='Child')
114     ax1.legend()
115
116     best_line, = ax2.plot([], [], 'go', ms=10, label='Best')
117     path_line, = ax2.plot([], [], 'y-', lw=1, alpha=0.5, label='Path')
118     ax2.legend()
119
120     # Store optimization history
121     history = {'x': [], 'f': [], 'best_x': [], 'best_f': []}
122
123     def update(i):
124         """Update function for animation"""
125         nonlocal current_x, current_real, current_fitness
126
127         # Generate and evaluate new candidate
128         new_x = uniform_mutation(current_x, mutation_prob)
129         new_real = logical2real(new_x, a, b)
130         new_fitness = f(new_real)

```

```

133     # Selection
134     if new_fitness < current_fitness:
135         current_x, current_real, current_fitness = new_x, new_real, new_fitness
136
137     # Update history
138     history['x'].append(new_real)
139     history['f'].append(new_fitness)
140     if not history['best_x'] or new_fitness < history['best_f'][-1]:
141         history['best_x'].append(new_real)
142         history['best_f'].append(new_fitness)
143     else:
144         history['best_x'].append(history['best_x'][-1])
145         history['best_f'].append(history['best_f'][-1])
146
147     # Update plots
148     parent_line.set_data([current_real], [current_fitness])
149     child_line.set_data([new_real], [new_fitness])
150     best_line.set_data([history['best_x'][-1]], [history['best_f'][-1]])
151     path_line.set_data(history['x'], history['f'])
152
153     ax1.set_title(f'Generation {i+1}\nCurrent Fitness: {current_fitness:.4f}')
154     ax2.set_title(f'Best Fitness: {history["best_f"][-1]:.4f}')
155
156     return parent_line, child_line, best_line, path_line
157
158     # Create animation
159     ani = FuncAnimation(fig, update, frames=n_iter, interval=200, blit=True)
160     plt.close()
161     return HTML(ani.to_jshtml())

```

Sia la $(1+1)$ -ES, sia il Simulated Annealing, si basano sulla perturbazione casuale delle soluzioni, ma differiscono nel criterio di accettazione. La $(1+1)$ -ES adotta una strategia elitaria, accettando esclusivamente soluzioni migliorative, mentre il Simulated Annealing può accettare anche soluzioni peggiorative per evitare minimi locali. Alcune varianti della $(1+1)$ -ES prevedono una riduzione progressiva dell'intensità di mutazione, avvicinandone il comportamento a quello del Simulated Annealing.

$(\mu+1)$ -ES

$(\mu + 1)$ -ES mantiene una popolazione di μ genitori, ma si genera comunque un solo figlio per iterazione e NON per genitore. Quindi abbiamo $\mu + 1$ soluzioni. Questo introduce un concetto di **memoria**: ad ogni ciclo il nuovo figlio compete con un intero bacino di soluzioni valide, permettendo all'algoritmo di attingere dalle migliori scoperte archiviate nelle generazioni precedenti. Se il figlio è migliore della più "scarsa" soluzione valida, allora viene aggiunta ai genitori.

$(\mu + \lambda)$ -ES

$(\mu + \lambda)$ -ES è una classe di algoritmi evolutivi in cui si mantiene una popolazione di μ genitori e, a ogni iterazione, si generano λ figli tramite operatori di mutazione (e talvolta ricombinazione).

Si parte da una popolazione di μ genitori e si generano λ figli contemporaneamente. In ogni iterazione viene costruita una popolazione di figli Q , inizialmente vuota. Il ciclo interno si ripete fino a generare esattamente λ individui. Ogni figlio viene prodotto selezionando un genitore da P tramite una strategia di selezione (random, torneo, eccetera) e applicando una mutazione, che modifica leggermente il genitore per esplorare lo spazio delle soluzioni. Il figlio viene poi aggiunto a Q . Una volta generata la popolazione dei figli, se ne valuta la fitness.

La fase di **selezione** avviene considerando insieme genitori e figli: dai $\mu + \lambda$ individui complessivi si selezionano i migliori μ per formare la popolazione della generazione successiva. Questo schema è

detto "+" perché i genitori competono direttamente con la discendenza, consentendo la conservazione delle soluzioni migliori già trovate.

In sintesi, l'algoritmo alterna generazione di varianti tramite mutazione e selezione elitista globale su genitori e figli, mantenendo sempre le migliori soluzioni trovate fino a quel momento.

La scelta di λ influenza fortemente il comportamento dell'algoritmo: valori elevati aumentano l'esplorazione dello spazio delle soluzioni ma riducono l'efficienza, mentre valori piccoli rendono la ricerca più conservativa.

Avere una popolazione a disposizione apre le porte anche all'eventuale utilizzo dell'operatore di crossover (ibridazione tra più genitori), avvicinando nettamente il funzionamento delle ES a quello tipico dei veri e propri Genetic Algorithms.

```

1  def ES_mu_plus_lambda(mu, lam, ngenes, evaluate_fitness, mutation, selection_method,
2      termination_condition):
3      """
4      (mu + lambda) Evolution Strategy
5
6      Parameters:
7      - mu: size of parent population
8      - lam: number of offspring generated each generation
9      - ngenes: chromosome length
10     - evaluate_fitness: objective function evaluator
11     - mutation: mutation operator
12     - selection_method: parent selection strategy
13     - termination_condition: function that returns True when stopping criterion is met
14
15     Returns:
16     - best_x: best solution found
17     - best_y: corresponding fitness value
18     """
19     # Initialize parent population P
20     P = initialize_population(mu, ngenes)
21
22     # Evaluate fitness of P
23     fit_P = evaluate_fitness(P)
24     g = 0 # generation counter
25
26     while not termination_condition(g, P, fit_P):
27         # Offspring population
28         Q = []
29         # Generate lambda offspring
30         while len(Q) < lam:
31             # Parent selection
32             parent = selection_method(P, fit_P)
33             # Mutation
34             offspring = mutation(parent)
35             # Add offspring
36             Q.append(offspring)
37
38         Q = np.array(Q)
39
40         # Evaluate offspring fitness
41         fit_Q = evaluate_fitness(Q)
42         # Combine populations
43         R = np.vstack((P, Q))
44         fit_R = np.concatenate((fit_P, fit_Q))
45         # Sort by fitness (descending)
46         indices = np.argsort(fit_R)[::-1]
47         # Select best mu individuals
48         P = R[indices[:mu]]
49         fit_P = fit_R[indices[:mu]]

```

```
50     # Increment generation
51     g += 1
52
53     # Return best individual (note: minimization assumed here)
54     best_index = np.argmin(fit_P)
55     best_x = P[best_index]
56     best_y = fit_P[best_index]
57
58     return best_x, best_y
```

6.4 Genetic Algorithms

Metaeuristiche

Una **metaeuristica** è una strategia di alto livello progettata per esplorare e navigare in modo efficiente enormi spazi di ricerca. Viene utilizzata per risolvere problemi *NP-Hard* per i quali i metodi matematici esatti tradizionali falliscono o richiederebbero tempi di calcolo infiniti.

A differenza dei metodi esatti, le metaeuristiche non garantiscono di trovare l'ottimo globale assoluto, ma forniscono soluzioni approssimate "sufficientemente buone" per scopi pratici.

Perché sono così popolari?

I **Genetic Algorithm** sono algoritmi metaeuristici ispirati alla natura, utilizzati per trovare la soluzione ottimale a un dato problema.

La diffusione di questi algoritmi è dovuta alla loro natura generale e alla limitata necessità di informazioni specifiche sul problema, non richiedendo ipotesi particolari sulla funzione obiettivo (non serve che la funzione sia derivabile o continua). Essi sono ampiamente utilizzati in problemi di ottimizzazione complessa grazie alla capacità di fornire soluzioni ottime o approssimate. La loro versatilità ne consente l'adattamento a un'ampia gamma di problemi.

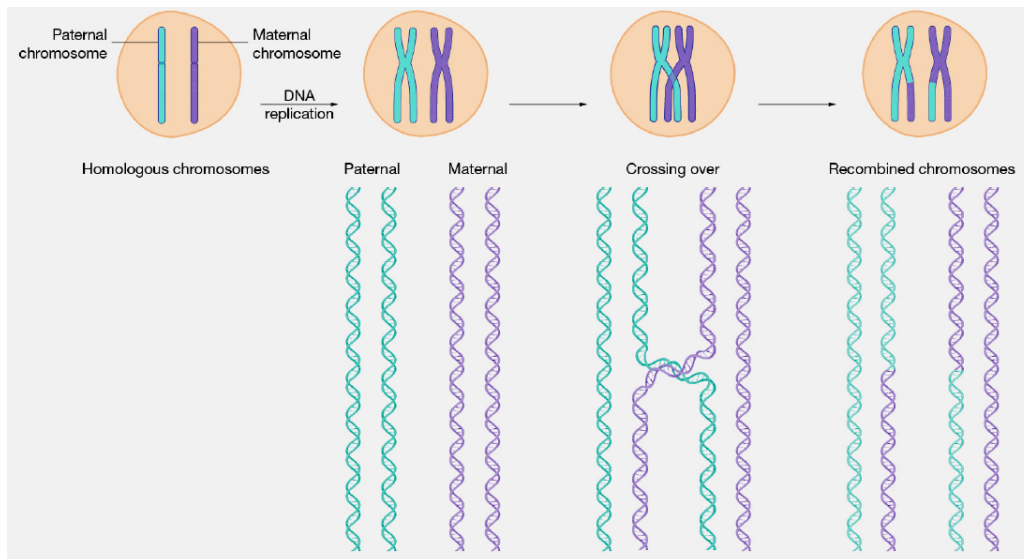
Algoritmo

L'idea di base di un Genetic Algorithm è quella di rappresentare le potenziali soluzioni a un problema come **individui all'interno di una popolazione**. Ogni individuo, spesso rappresentato come *una sequenza di bit o un cromosoma*, codifica una soluzione candidata. La popolazione si evolve nel corso delle generazioni attraverso un processo di selezione, crossover e mutazione.

L'algoritmo fa evolvere questa popolazione simulando i seguenti passaggi naturali:

1. **Inizializzazione:** Viene generata in modo casuale una popolazione iniziale di individui;
2. **Fitness Evaluation:** Ogni individuo viene valutato tramite una Funzione di Fitness che quantifica quanto sia "buona" quella specifica soluzione per il nostro obiettivo;
3. **Selezione:** Gli individui vengono scelti per diventare genitori della generazione successiva. La probabilità di essere selezionati è proporzionale alla loro fitness (principio della "sopravvivenza del più adatto");
4. **Crossover Recombination:** Le informazioni genetiche di due genitori selezionati vengono scambiate e mescolate per creare nuovi individui figli (offspring). Questo imita la ricombinazione genetica biologica;

5. **Mutazione:** Vengono applicate piccole alterazioni casuali ai geni dei figli. Questo è fondamentale per introdurre nuova varietà nella popolazione, esplorare nuove aree dello spazio di ricerca e impedire che l'algoritmo converga troppo presto su soluzioni sub-ottimali (minimi locali);
6. **Sostituzione:** La nuova generazione di figli sostituisce (totalmente o parzialmente) la vecchia generazione di genitori;
7. **Conclusioni:** Il ciclo riparte dal punto 2 e si ripete fino a quando non viene soddisfatta una condizione di arresto (ad esempio: raggiunto il numero massimo di generazioni, raggiunto un punteggio di fitness target, o l'algoritmo non migliora più).



Funzione di Fitness

La **Funzione di Fitness (Fitness Function)** è una funzione scalare associata a ciascun cromosoma e rappresenta la qualità del cromosoma stesso. Può coincidere esattamente con la funzione obiettivo matematica del nostro problema, oppure può essere una sua trasformazione necessaria per gestire vincoli o per normalizzare i dati.

Codice

```

1 def GA_BE_1D(f, a, b, generations, chrsize, popsize, pmutallgenes,
2 displayAndPlot=False):
3     """
4     Canonical Genetic Algorithm with Binary Encoding for 1D function minimization.
5
6     Parameters:
7     - f: Objective function to minimize
8     - a: Lower bound of search space
9     - b: Upper bound of search space
10    - generations: Number of generations
11    - chrsize: Chromosome length (bits)
12    - popsize: Population size (must be even)
13    - pmutallgenes: Mutation probability per gene
14    - displayAndPlot: Whether to show convergence plot
15
16    Returns:
17    - x_opt: Best solution found
18    - f_opt: Best fitness found
19    - best_fit: Array of best fitness per generation

```



```

20     """
21     if popsize % 2 == 1:
22         raise ValueError("Population size must be even.")
23
24     # Initialize population
25     P = BE_initpop(chrsize, popsize)
26     # Evaluate initial fitness
27     fit_P = BE_evaluatefitness(f, P, a, b)
28     # Track best fitness per generation
29     best_fit = np.zeros(generations)
30     best_fit[0] = np.min(fit_P)
31
32     # Evolution loop
33     for t in range(1, generations):
34         # 1. Selection - Create mating pool
35         MP = binarytournament(P, fit_P, popsize)
36
37         # 2. Shuffle mating pool to randomize pairs
38         MP = MP[np.random.permutation(popsize), :]
39
40         # 3. Crossover - Create offspring population Q
41         Q = np.zeros((popsize, chrsize), dtype=bool)
42         for i in range(0, popsize, 2):
43             offsprings, _ = BE_XOVER_singlepoint(MP[i,:], MP[i+1,:])
44             Q[i:i+2,:] = offsprings
45
46         # 4. Mutation
47         for i in range(popsize):
48             Q[i,:] = BE_MUT_unifall(Q[i,:], pmutallgenes)
49
50         # 5. Evaluate offspring fitness
51         fit_Q = BE_evaluatefitness(f, Q, a, b)
52
53         # 6. Combine parent and offspring populations
54         R = np.vstack((P, Q))
55         fit_R = np.concatenate((fit_P, fit_Q))
56
57         # 7. Elitist selection - keep best individuals
58         sorted_indices = np.argsort(fit_R)
59         new_P = R[sorted_indices[:popsize], :]
60         fit_new_P = fit_R[sorted_indices[:popsize]]
61
62         # 8. Update population
63         P = new_P
64         fit_P = fit_new_P
65
66         # 9. Track best fitness
67         best_fit[t] = np.min(fit_P)
68
69     # Find best solution
70     best_idx = np.argmin(fit_P)
71     f_opt = fit_P[best_idx]
72     x_opt = logical2real(P[best_idx,:], a, b)
73
74     # Visualization
75     if displayAndPlot:
76         plt.figure(figsize=(12, 5))
77
78         # Full convergence plot
79         plt.subplot(1, 2, 1)
80         plt.plot(range(generations), best_fit, '-b')
81         plt.xlim([0, generations-1])
82         plt.title('Best Fitness per Generation')
83         plt.xlabel('Generation')
84         plt.ylabel('Fitness')
85

```

```

86     # Early generations zoom
87     plt.subplot(1, 2, 2)
88     early_gen = min(30, generations)
89     plt.plot(range(early_gen), best_fit[:early_gen], '-b')
90     plt.xlim([0, early_gen-1])
91     plt.title(f'First {early_gen} Generations')
92     plt.xlabel('Generation')
93     plt.ylabel('Fitness')
94
95     plt.tight_layout()
96     plt.show()
97
98     return x_opt, f_opt, best_fit

```

Crossover e Mutazione per Binary Encoding

Negli Algoritmi Genetici, la rappresentazione del "*cromosoma*" (la codifica delle soluzioni) detta le regole su come gli operatori genetici possono manipolare i dati. In questa rappresentazione, ogni individuo è un array di bit (0 e 1). È la forma più classica degli Algoritmi Genetici.

Il **crossover binario** taglia e cuce le stringhe di bit dei genitori per formare i figli. Può essere di tre tipi:

- **One-point Crossover:** Si seleziona casualmente un punto di taglio (un indice k) lungo il cromosoma. Il primo figlio eredita la prima parte dal Genitore A e la seconda parte dal Genitore B. Il secondo figlio fa l'opposto;
- **Two-point Crossover:** Si selezionano due punti di taglio. Il segmento centrale dei genitori viene scambiato tra i due figli. Di conseguenza uno avrà la parte centrale del primo genitore e gli estremi del secondo, viceversa per il secondo figlio. Questo permette di preservare sequenze geniche alle estremità che altrimenti verrebbero separate dal crossover a singolo punto;
- **Uniform Crossover:** Non ci sono punti di taglio. Per ogni singolo bit, si lancia una moneta (probabilità $p = 0.5$). Se esce testa, il figlio prende il bit dal Genitore A, se croce dal Genitore B. Rompe fortemente i blocchi di geni adiacenti, favorendo un'esplorazione molto aggressiva.

La **mutazione binaria** consiste semplicemente nel "flippare" (invertire) lo stato di un bit (da 0 a 1, o da 1 a 0):

- **Single-point Hard Mutation:** Si estrae a caso un singolo bit nell'intero cromosoma e lo si inverte. È una mutazione molto conservativa;
- **Uniform All Mutation:** Si scorre l'intero cromosoma bit per bit. Ogni bit ha una piccolissima probabilità (esempio: $p_m = 0.01$) di essere invertito. È il metodo più utilizzato per garantire che ogni parte della soluzione abbia la chance di mutare.

Crossover e Mutazione per Real Encoding

Se le variabili sono numeri reali continui (vettori in $\mathbb{R}^n$), usare i bit per rappresentare numeri reali con alta precisione richiederebbe stringhe lunghissime. Pertanto, si usano array di float, e gli operatori cambiano radicalmente.

Qui il crossover non si limita a scambiare valori, ma può "fondere" matematicamente i numeri:

- **Discrete Crossover:** Funziona esattamente come l'Uniform Crossover binario, ma scambia interi numeri reali tra i genitori anziché singoli bit;
- **Arithmetic Crossover:** Genera un figlio creando una combinazione lineare (una media pesata) dei valori dei genitori. Il coefficiente $\alpha \in [0; 1]$ è generato randomicamente per ogni gene. Se Genitore A ha il gene $x = 10$ e Genitore B ha $y = 20$, il figlio avrà un gene $z = \alpha x + (1 - \alpha)y$. Se viene generato $\alpha = 0.5$, il figlio avrà 15;
- **Convex Crossover:** Simile all'aritmetico, ma $\alpha \in [0; 1]$ è fissato e non generato randomicamente;
- **Simulated Binary Crossover (SBX):** È l'operatore più avanzato. Nei Genetic Algorithms binari, il One-point Crossover ha una proprietà statistica specifica: *la distanza media tra i figli e i genitori segue una determinata distribuzione di probabilità*. L'SBX ha come scopo quello di simulare con i numeri continui lo stesso comportamento statistico che il One-point Crossover ha sui numeri binari. *Il centro di massa dei figli è uguale al centro di massa dei genitori*. Il suo comportamento spaziale è governato da un parametro definito dall'utente chiamato **Distribution Index**, indicato con η . Questo parametro regola l'ampiezza della "campana" di probabilità in cui andranno a materializzarsi i figli. Un η alto (esempio: 15 o 20) crea figli quasi identici ai genitori (sfruttamento), un η basso (esempio: 2 o 5) crea figli molto diversi (esplorazione). L'SBX non "taglia" gli array scambiandone i pezzi, ma ricalcola le coordinate esatte dei figli usando un parametro dinamico chiamato **Spread Factor** β . β definisce il rapporto tra la distanza assoluta che separerà i due nuovi figli e la distanza che separa i due genitori originali. Per calcolare β , l'algoritmo esegue due passaggi:

1. Si estrae un numero puramente casuale u distribuito uniformemente nell'intervallo $[0; 1]$;
2. Si "distorce" questo numero uniforme facendolo passare per una distribuzione esponenziale dipendente da η :

$$\beta = \begin{cases} (2u)^{\frac{1}{\eta+1}} & \text{se } u \leq 0.5 \\ \left(\frac{1}{2(1-u)}\right)^{\frac{1}{\eta+1}} & \text{se } u > 0.5 \end{cases}$$

Una volta ottenuto il valore β per uno specifico gene, le coordinate finali di quel gene per il primo figlio (c_1) e per il secondo figlio (c_2) vengono calcolate combinando i geni corrispondenti dei genitori (p_1 e p_2):

$$c_1 = \frac{1}{2} [(1 + \beta)p_1 + (1 - \beta)p_2]$$

$$c_2 = \frac{1}{2} [(1 - \beta)p_1 + (1 + \beta)p_2]$$

Osservando la formula si deduce facilmente la geometria del crossover:

- Se il destino estrae $\beta = 1$, le formule si semplificano in $c_1 = p_1$ e $c_2 = p_2$. I figli sono delle copie esatte dei genitori (nessuna modifica);
- Se $\beta < 1$, i figli atterreranno spazialmente all'interno dell'intervallo delimitato dai genitori. Si parla in questo caso di **Contracting Crossover**;
- Se $\beta > 1$, i figli verranno proiettati spazialmente all'esterno dell'intervallo delimitato dai genitori. Si parla di **Expanding Crossover**.

Questo procedimento matematico viene reiterato in modo indipendente per ogni singola variabile del cromosoma (spesso con una probabilità del 50% di innescarsi o meno per ogni gene), rendendo l'algoritmo estremamente bilanciato e imprevedibile.

Per la mutazione, invece di "flippare" bit, con i reali si aggiunge rumore ai valori esistenti:

- **Uniform Mutation:** Sostituisce il gene corrente con un numero casuale estratto uniformemente all'interno di un range predefinito $[Lb; Ub]$ (limite inferiore e superiore). Modifica drasticamente il valore;
- **Gaussian Mutation:** Si somma al gene corrente un valore estratto da una distribuzione Gaussiana con media 0 e deviazione standard σ : $x_{nuovo} = x_{vecchio} + \sigma \mathcal{N}(0; 1)$. È eccellente per fare fine-tuning: la maggior parte delle mutazioni produrrà spostamenti minuscoli, mentre spostamenti molto grandi saranno rari.

L'uso del crossover senza mutazione è sconsigliato. L'uso della mutazione insieme al crossover aiuta a esplorare la regione al di fuori dell'involuppo convesso (*convex hull*) dei genitori.

Codice Simplified Simulated Binary Crossover

```

1  def GA_RE_crossover_sbx_simplified(p1, p2, lb, ub, eta, u=None):
2      """
3      Simplified Simulated Binary Crossover (SBX) for real-valued encoding.
4
5      Parameters:
6      - p1, p2: Parent chromosomes
7      - lb, ub: Lower/upper bounds for each gene
8      - eta: Distribution index (controls spread)
9      - u: Optional random number for reproducibility
10
11     Returns:
12     - o1, o2: Offspring chromosomes
13     """
14     p1 = np.asarray(p1).flatten()
15     p2 = np.asarray(p2).flatten()
16     lb = np.asarray(lb).flatten()
17     ub = np.asarray(ub).flatten()
18
19     if u is None:
20         u = np.random.rand()
21
22     # Calculate spread factor
23     if u <= 0.5:
24         betaq = (2*u)**(1.0/(eta + 1))
25     else:
26         betaq = (1/(2*(1-u)))** (1.0/(eta + 1))
27
28     # Create offspring
29     o1 = 0.5*((1+betaq)*p1 + (1-betaq)*p2)
30     o2 = 0.5*((1-betaq)*p1 + (1+betaq)*p2)
31     # Apply bounds
32     o1 = np.clip(o1, lb, ub)
33     o2 = np.clip(o2, lb, ub)
34     # Handle identical genes
35     epsilon = 1e-14
36     identical = np.abs(p1 - p2) < epsilon
37     o1[identical] = p1[identical]
38     o2[identical] = p1[identical]
39
40     return o1, o2

```

Codice Advanced Simulated Binary Crossover

```

1  def GA_RE_crossover_sbx_advanced(p1, p2, lb, ub, eta, u=None):
2      """
3      Advanced Simulated Binary Crossover (SBX) for real-valued encoding.
4
5      Parameters:
6      - p1, p2: Parent chromosomes
7      - lb, ub: Lower/upper bounds for each gene
8      - eta: Distribution index (controls spread)
9      - u: Optional random numbers for reproducibility
10
11     Returns:
12     - o1, o2: Offspring chromosomes
13     """
14     p1 = np.asarray(p1).flatten()
15     p2 = np.asarray(p2).flatten()
16     lb = np.asarray(lb).flatten()
17     ub = np.asarray(ub).flatten()
18
19     if u is None:
20         u = np.random.rand(len(p1))
21     else:
22         u = np.asarray(u).flatten()
23
24     # Initialize offspring
25     o1 = p1.copy()
26     o2 = p2.copy()
27     epsilon = 1e-14
28
29     for i in range(len(p1)):
30         if abs(p1[i] - p2[i]) > epsilon:
31             # Order parents
32             x_min, x_max = sorted([p1[i], p2[i]])
33
34             # Calculate beta for lower bound
35             beta_low = 1.0 + (2.0 * (x_min - lb[i]) / (x_max - x_min))
36             alpha_low = 2.0 - beta_low**(-(eta + 1))
37
38             if u[i] <= 1.0 / alpha_low:
39                 betaq_low = (u[i] * alpha_low)**(1.0 / (eta + 1))
40             else:
41                 betaq_low = (1.0 / (2.0 - u[i] * alpha_low))**(1.0 / (eta + 1))
42
43             c1 = 0.5 * ((x_min + x_max) - betaq_low * (x_max - x_min))
44
45             # Calculate beta for upper bound
46             beta_up = 1.0 + (2.0 * (ub[i] - x_max) / (x_max - x_min))
47             alpha_up = 2.0 - beta_up**(-(eta + 1))
48
49             if u[i] <= 1.0 / alpha_up:
50                 betaq_up = (u[i] * alpha_up)**(1.0 / (eta + 1))
51             else:
52                 betaq_up = (1.0 / (2.0 - u[i] * alpha_up))**(1.0 / (eta + 1))
53
54             c2 = 0.5 * ((x_min + x_max) + betaq_up * (x_max - x_min))
55             # Ensure bounds and assign
56             c1 = np.clip(c1, lb[i], ub[i])
57             c2 = np.clip(c2, lb[i], ub[i])
58
59             if np.random.rand() <= 0.5:
60                 o1[i], o2[i] = c2, c1
61             else:
62                 o1[i], o2[i] = c1, c2
63     return o1, o2

```

Caratteristica	Simplified SBX	Advanced SBX
Boundaries	Calcola il fattore di spread β_q in modo "cieco", senza sapere se i genitori si trovano al centro dello spazio di ricerca o pericolosamente vicini ai limiti estremi imposti dal problema (lb e ub). Il confinamento dei figli avviene solo alla fine forzandoli con un taglio brutale ($\min(\max(o1, lb), ub)$).	Introduce due nuove variabili, β_{q_i} e α_{q_i} , che misurano esattamente quanto i genitori siano vicini ai limiti (lb e ub). Questo adatta e deforma la curva di probabilità per quel gene specifico, facendo in modo che i figli generati rispettino i confini matematicamente in modo più naturale, evitando accumuli artificiali di valori sui bordi che il taglio della versione base causerebbe.
Calcolo dei Figli	Usa un singolo numero casuale u per calcolare un unico β_q , generando le coordinate dei due figli in modo perfettamente simmetrico rispetto al centro dei genitori.	Estrae due numeri casuali distinti per ogni singolo gene ($u(1;i)$ e $u(2;i)$). Questi vengono usati per calcolare due fattori di spread completamente indipendenti (β_{q1} e β_{q2}), generando due valori potenziali ($c1$ e $c2$) che perdono la simmetria perfetta, aumentando l'imprevedibilità e la diversità genetica della generazione.
Gene Swapping	È deterministico nell'assegnazione finale: il figlio 1 ($o1$) riceverà sempre la prima combinazione e il figlio 2 ($o2$) la seconda.	Una volta calcolati i due nuovi valori $c1$ e $c2$, l'algoritmo lancia una moneta ($ifrand() \leq 0.5$) per decidere casualmente a quale figlio assegnare il valore. In base al risultato, $c1$ potrebbe andare al figlio $o1$ oppure al figlio $o2$, rimescolando ulteriormente il materiale genetico incrociando i tratti.
Vettorizzazione	È scritto per elaborare gli interi cromosomi in un unico colpo tramite veloci operazioni matriciali vettorizzate, poiché applica le stesse regole statiche a tutti i geni.	Per via della sua complessità adattiva, usa un ciclo for $i = 1:n$ scendendo ad analizzare ogni singolo gene passo-passo. Durante questo ciclo individua in modo dinamico i minimi e i massimi locali tra i due genitori ($xmin_i$, $xmax_i$). Come ottimizzazione, se rileva che i geni dei due genitori sono identici (misurati con $\text{abs}(p1(i) - p2(i)) > \epsilon$), salta i calcoli matematici pesanti mantenendo i geni intatti.

Come impostare la probabilità di mutazione?

Nel determinare la probabilità di mutazione, bisogna tenere presente che:

- una probabilità troppo bassa può portare a una convergenza prematura (ad un minimo locale);
- una probabilità troppo alta può distruggere il patrimonio genetico della prole.

Una *rule-of-thumb* (valutazione empirica) è che il numero di valutazioni di fitness deve essere compreso tra 10.000 e 50.000. Questo numero deve essere diviso per la dimensione della popolazione, per ottenere una stima del numero tipico di generazioni necessarie all'algoritmo genetico canonico. *Ad esempio, se hai una popolazione di 100 individui, avrai bisogno di far girare l'algoritmo per un numero compreso tra le 100 e le 500 generazioni (poiché $100 \times 100 = 10.000$ e $100 \times 500 = 50.000$).* Tale numero può essere proibitivo perché la valutazione tramite la Fitness Function può richiedere molto tempo.

I **Modelli Surrogati** sono modelli predittivi rapidi e approssimati addestrati a "imitare" la vera e pesante funzione di fitness. Spesso valutare la fitness reale di un individuo richiede simulazioni estremamente costose dal punto di vista computazionale (esempio: il training completo di una Neural Network per l'Hyperparameter Tuning). I modelli surrogati riducono il numero di valutazioni "reali" necessarie, garantendo una significativa riduzione dei costi computazionali pur cercando di mantenere intatta la qualità della soluzione.

I Modelli Surrogati si dividono in varie tipologie:

- **Neural Networks:** In grado di catturare relazioni non lineari molto complesse, ideali per problemi ad alta dimensionalità;
- **Kriging (Gaussian Process Regression):** Estremamente efficace per funzioni lisce e fornisce preziose stime sull'incertezza della predizione;
- **Radial Basis Functions:** Semplici da implementare e molto buone per l'interpolazione;
- **Polynomial Regression:** Modelli veloci, ma che potrebbero fallire nel catturare comportamenti non lineari complessi.

Gli step di un modello surrogato sono:

1. Si inizializza la popolazione;
2. Si valuta la fitness degli individui utilizzando la funzione reale ed esatta per un certo numero di iterazioni;
3. Si addestra il modello surrogato usando queste valutazioni reali appena ottenute (insegnandogli a prevedere la fitness);
4. Inizia il ciclo (mentre il criterio di stop non è soddisfatto): si applica selezione, crossover e mutazione. Si valuta la fitness della nuova progenie interrogando il modello surrogato (operazione rapidissima);
5. Si aggiorna periodicamente il modello surrogato effettuando nuove valutazioni reali per calibrare l'errore ed evitare che la ricerca si perda in ottimizzazioni fittizie. Questo è uno step cruciale.

```

1  def surrogate_GA(
2      initialize_population,
3      evaluate_fitness,
4      train_surrogate,
5      surrogate_predict,
6      update_surrogate,
7      selection,
8      crossover,
9      mutate,
10     not_terminated,
11     get_best_solution,
12     update_interval
13 ):

```

```

14     """Surrogate-Assisted Genetic Algorithm"""
15     # Initialize population
16     population = initialize_population()
17     # True fitness evaluation (expensive evaluation)
18     fitness = evaluate_fitness(population)
19     # Train initial surrogate model
20     surrogate = train_surrogate(population, fitness)
21
22     generation = 0
23
24     while not not_terminated(generation):
25         # Selection
26         parents = selection(population, fitness)
27
28         # Variation operators
29         offspring = crossover(parents)
30         offspring = mutate(offspring)
31
32         # Surrogate-based evaluation (cheap approximation)
33         surrogate_fitness = surrogate_predict(surrogate, offspring)
34
35         # Periodic surrogate update with true evaluations
36         if generation % update_interval == 0:
37             actual_fitness = evaluate_fitness(offspring)
38
39             surrogate = update_surrogate(
40                 surrogate,
41                 offspring,
42                 actual_fitness
43             )
44
45             # Update dataset with true evaluations
46             fitness = np.concatenate((fitness, actual_fitness))
47             population = np.concatenate((population, offspring))
48
49         else:
50             # Use surrogate predictions
51             fitness = np.concatenate((fitness, surrogate_fitness))
52             population = np.concatenate((population, offspring))
53
54         generation += 1
55
56     # Return best individual
57     return get_best_solution(population, fitness)

```

Memetic Algorithms

I **Memetic Algorithms** (*algoritmi memetici*) sono meta-euristiche ibride che combinano una strategia di ricerca globale, tipicamente basata su algoritmi evolutivi, con una procedura di ricerca locale finalizzata al miglioramento delle soluzioni candidate. L'obiettivo è coniugare la capacità esplorativa della ricerca globale con l'efficacia della ricerca locale nello sfruttare le regioni promettenti dello spazio delle soluzioni.

Nel contesto dei Genetic Algorithms, il processo evolutivo tradizionale viene esteso introducendo una fase di ottimizzazione locale dopo le operazioni di crossover e mutazione. Le nuove soluzioni generate vengono quindi raffinate prima della valutazione della fitness, ottenendo individui di qualità superiore e accelerando la convergenza verso soluzioni ottimali. Un esempio diffuso nel Machine Learning consiste nella combinazione di algoritmi genetici e metodi di Gradient Descent, dove i primi svolgono il ruolo di esplorazione globale dello spazio di ricerca, mentre la Gradient Descent realizza un affinamento locale dei parametri.

Un'applicazione classica degli algoritmi memetici riguarda il **Traveling Salesman Problem (TSP)**. Il TSP è un problema di ottimizzazione combinatoria che consiste nel determinare il percorso di lunghezza minima che consente a un commesso viaggiatore di visitare una sola volta ciascuna città di un insieme dato e di ritornare alla città di partenza. Il TSP è un problema NP-Hard ed è frequentemente utilizzato come benchmark per la valutazione di algoritmi di ottimizzazione. In questo contesto, una delle procedure di ricerca locale più utilizzate è l'euristica *2-opt*, che migliora iterativamente un percorso sostituendo coppie di archi con nuove connessioni che riducono la lunghezza complessiva del tour. Integrando tale procedura all'interno del processo evolutivo (unendo 2-opt con un algoritmo genetico per esempio), l'algoritmo memetico è in grado di perfezionare sistematicamente le soluzioni generate dalla ricerca globale, ottenendo risultati di qualità superiore rispetto all'impiego isolato delle due tecniche.

6.5 Teorema No Free Lunch

Teorema (No Free Lunch). *Nessun algoritmo di ottimizzazione supera tutti gli altri in termini di prestazioni, considerando la media dei risultati su tutti i possibili problemi. Di conseguenza, l'ottimalità universale è impossibile e le prestazioni dell'algoritmo dipendono dal problema specifico.*

Nessun algoritmo di ottimizzazione è universalmente migliore degli altri quando si considerano, in media, tutti i possibili problemi. Ne deriva che non esiste un'ottimalità generale e che le prestazioni di un algoritmo dipendono strettamente dal problema specifico. Questo implica che non sia possibile individuare un unico metodo sempre superiore, poiché ogni algoritmo può risultare più o meno efficace a seconda del contesto applicativo. Di conseguenza, diventa essenziale adattare le tecniche al dominio del problema e sfruttare conoscenze specifiche per migliorarne le prestazioni.

Il teorema ha inoltre influenzato la ricerca in ottimizzazione, favorendo l'adozione di approcci più specializzati e orientati al problema.

Potrebbe sembrare che il Deep Learning violi il teorema, ma non è così. Il punto è che il teorema parla di **tutti i possibili problemi**, ma il **Machine Learning lavora su problemi reali**, non arbitrari. **Nei problemi reali i dati hanno struttura e non sono uniformemente distribuiti.** Il teorema non considera esplicitamente le strutture del problema che possono essere sfruttate dagli algoritmi. Gli algoritmi *domain-specific*, invece, incorporano e sfruttano proprio queste informazioni strutturali per migliorare le proprie prestazioni. Inoltre il teorema si concentra sulle prestazioni medie su tutti i problemi, ma a noi tendenzialmente preoccupano di più le prestazioni sui problemi rilevanti.

Il successo del Machine Learning esiste proprio perché il mondo ha strutture sfruttabili: immagini (località spaziale), linguaggio (regolarità sintattiche e semantiche), segnali (continuità temporale), eccetera ...

Implicazioni pratiche: focalizzarsi su classi di problemi rilevanti, incorporare conoscenza del dominio e verificare le prestazioni degli algoritmi tramite validazione empirica.

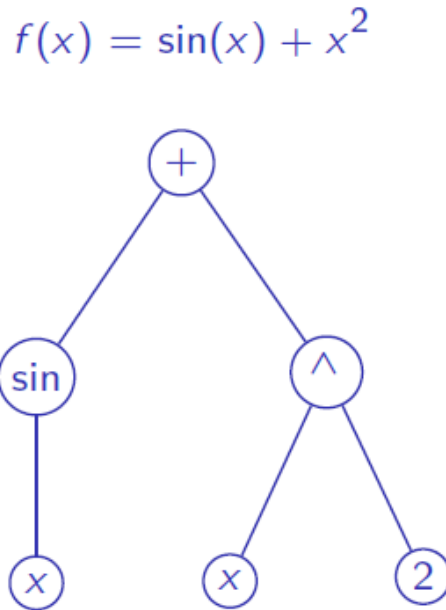
6.6 Genetic Programming

La **Genetic Programming** è un algoritmo evolutivo che fa evolvere i programmi per computer al fine di risolvere problemi. La differenza fondamentale con i Genetic Algorithms è che questi "evolvono" una soluzione, mentre **la Genetic Programming evolve un programma che produce una soluzione**. La differenza principale, quindi, sta nella rappresentazione degli individui. Nella Genetic Programming gli individui sono alberi sintattici che rappresentano programmi o espressioni matematiche, mentre nei Genetic Algorithms sono generalmente cromosomi (esempio: array di iperparametri) a lunghezza fissa.

Per costruire questi **Syntax Trees**, si definiscono due insiemi fondamentali:

- **Function Set:** Sono i nodi interni dell'albero. Possono essere operatori matematici (+, −, *, /), funzioni logiche (AND, OR), o funzioni trigonometriche (sin, cos). Ognuno ha una specifica "arietà" (il numero di figli che richiede, esempio: l'addizione richiede 2 figli);
- **Terminal Set:** Sono le foglie dell'albero, i nodi che non hanno figli. Includono le variabili di input del problema (esempio: X, Y) e costanti numeriche (esempio: 3.14, −5).

Ad esempio, l'equazione $\sin(x) + x^2$ viene rappresentata con la radice +, che ha come figlio sinistro un sotto-albero sin (con foglia x) e come figlio destro la foglia $\hat{}$ (con foglia sinistra x e foglia destra 2).



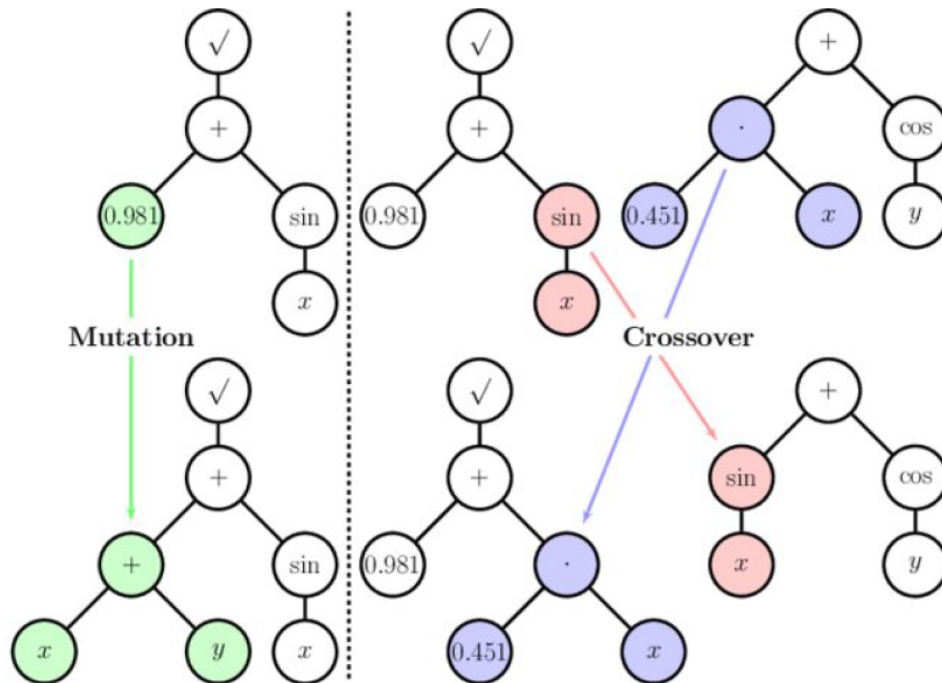
Poiché lavoriamo con strutture ad albero, le classiche operazioni di incrocio e mutazione devono preservare la sintassi del programma:

- **Subtree Crossover:** È l'operatore principale di Crossover. Si prende un Genitore A e un Genitore B:
 1. Si seleziona a caso un nodo qualsiasi all'interno dell'albero A;
 2. Si seleziona a caso un nodo qualsiasi all'interno dell'albero B;
 3. Si tagliano fisicamente i due sotto-alberi radicati in quei nodi e si scambiano. Il risultato sono due nuovi alberi (figli) che sono sintatticamente validi ma che mescolano la logica dei genitori.
- **Subtree Mutation:** Si tratta di mutazione e non di crossover. Si sceglie un nodo casuale in un individuo, si taglia l'intero sotto-albero pendente da esso e lo si sostituisce con un albero del tutto nuovo generato casualmente. Inietta materiale genetico fresco.
- **Point Mutation:** Si tratta di mutazione e non di crossover. Si sostituisce un singolo nodo con un altro nodo dello stesso tipo e della stessa arietà (esempi: si cambia un operatore + in un −, oppure la foglia x in una foglia y).

Evolvendo di generazione in generazione, gli alberi della Genetic Programming tendono a crescere a dismisura in modo patologico, accumulando "codice spazzatura" (**introni**) che non altera il risultato

ma che protegge le parti vitali dell'albero da crossover distruttivi (esempio: $x+0$, oppure $x*1$). Questo rallenta enormemente i calcoli. Per contrastarlo, si applica la **Parsimony Pressure**, una funzione di fitness che viene modificata per penalizzare gli alberi con troppi nodi, costringendo l'evoluzione a preferire soluzioni matematiche corte ed eleganti.

Esempio di Subtree Mutation e Subtree Crossover nella Symbolic Regression:



Nella Regressione classica (Machine Learning), si fornisce una forma matematica fissa (esempio: una retta $y = mx + q$) e l'algoritmo trova i coefficienti migliori. Nella **Symbolic Regression**, fornendo solo dati di Input e Output, la Genetic Programming scopre da sola l'intera equazione matematica (la forma del modello e le sue costanti) che spiega il fenomeno fisico sottostante.

6.7 Neuroevolution

Finora abbiamo parlato di usare i Genetic Algorithms per trovare gli iperparametri (lasciando l'addestramento dei pesi alla Backpropagation e agli ottimizzatori come Adam). Esiste però un approccio radicale in cui i Genetic Algorithms vengono usati per evolvere direttamente i Pesi W della Neural Network, eliminando del tutto la Backpropagation. In questo caso, il "cromosoma" è un vettore gigantesco contenente tutti i milioni di pesi della rete. La mutazione consiste nel sommare rumore gaussiano direttamente ai pesi.

La **Neuroevolution** è simile alla Genetic Programming, ma i genomi rappresentano Neural Network descrivendone la struttura e, talvolta, anche i pesi delle connessioni. La codifica del genoma può essere, quindi, diretta o indiretta.

Questo approccio è usato pesantemente nel **Reinforcement Learning**, dove calcolare le derivate esatte per la Backpropagation è impossibile.

NEAT (NeuroEvolution of Augmenting Topologies) è uno degli algoritmi più influenti della Neuroevolution.

L'idea centrale è che una buona architettura non debba essere progettata manualmente: può emergere attraverso l'evoluzione:

1. Inizializzare una popolazione di reti neurali minime (solo input e output);
2. Valutare la fitness di ciascuna rete sul problema considerato;
3. Suddividere le reti in specie in base alla loro somiglianza genetica;
4. Selezionare gli individui migliori all'interno di ogni specie;
5. Generare una nuova popolazione mediante:
 - (a) crossover tra reti compatibili;
 - (b) mutazione dei pesi;
 - (c) aggiunta di nuove connessioni;
 - (d) aggiunta di nuovi neuroni.
6. Assegnare numeri di innovazione alle nuove strutture per facilitare futuri crossover;
7. Ripetere i passi 2-6 fino al raggiungimento di un criterio di arresto;
8. Restituire la rete con la fitness migliore.

La Neuroevolution ha alcuni **vantaggi**:

- Non richiede gradienti;
- Esplora spazi di soluzioni molto ampi;
- Può trovare architetture non intuitive.

Tuttavia presenta anche degli importanti **limiti**:

- Molto costosa computazionalmente;
- Poco scalabile rispetto al backpropagation;
- Richiede molte valutazioni del modello.

6.8 Applicazioni

Abbiamo introdotto gli Evolutionary Algorithms per risolvere un problema specifico: l'Hyperparameter Tuning delle Neural Network. Ma questo tipo di algoritmi ha un enorme uso al di fuori di questo specifico problema. Essi brillano in tutti quegli scenari in cui lo spazio di ricerca è immenso, la funzione obiettivo non è derivabile (quindi niente discesa del gradiente) e non si ha il tempo fisico per calcolare tutte le combinazioni possibili. Un altro esempio può essere l'applicazione al TSP.

Vengono impiegati con successo in ambiti come logistica e ottimizzazione combinatoria (per la definizione di percorsi e assegnamenti complessi), ingegneria e design (dove consentono l'ottimizzazione di strutture e sistemi fisici soggetti a vincoli multipli) pianificazione e scheduling (per la gestione efficiente di risorse e vincoli temporali), robotica e sistemi di controllo (per la sintesi di traiettorie e comportamenti adattivi) e finanza (per l'ottimizzazione di portafogli e strategie di trading). In generale, ogni problema di allocazione ottimale di risorse sotto vincoli complessi rappresenta un candidato naturale all'impiego di metaeuristiche evolutive.

Capitolo 7: Swarm Intelligence

7.1 Intelligenza Collettiva e Auto-organizzazione

La **Swarm Intelligence** è una tecnica di ottimizzazione stocastica basata sulla popolazione, ispirata al comportamento sociale degli stormi di uccelli e dei banchi di pesci.

L'*ottimizzazione stocastica* è una branca dell'ottimizzazione matematica che risolve problemi caratterizzati da incertezza o casualità. A differenza dei modelli deterministici, include variabili probabilistiche nella funzione obiettivo o nei vincoli, consentendo di prendere decisioni strategiche robuste basate su scenari reali, come previsioni di mercato e di fornitura.

L'**Intelligenza Collettiva** è quel comportamento macroscopico, intelligente e globale che emerge spontaneamente dall'interazione di moltissimi agenti semplici. Nessuna singola formica sa come costruire un formicaio complesso o come trovare la via più breve per il cibo. Nessun uccello dirige lo stormo dicendo agli altri dove andare. Non c'è alcun controllore centrale. L'intelligenza non risiede nel singolo individuo, ma è una proprietà emergente del gruppo nel suo insieme.

Se l'intelligenza collettiva è il risultato, l'**Auto-organizzazione** è il meccanismo con cui la si ottiene. È un processo dinamico attraverso il quale un sistema disordinato acquisisce ordine e struttura in modo autonomo, basandosi esclusivamente su interazioni locali. Gli agenti reagiscono a semplici stimoli ambientali e alle azioni dei propri vicini immediati (esempio: "se l'uccello accanto a me vira a destra, viro a destra anche io", oppure "se sento una forte scia chimica di feromone, la seguo"). Questo interazione locale crea un'onda di organizzazione su scala globale.

Gli algoritmi di Swarm Intelligence traducono matematicamente queste dinamiche biologiche per risolvere problemi di ottimizzazione complessi (esempio: minimizzare una Loss Function in una Neural Network, o trovare il percorso logistico più breve per dei camion da consegna).

Questi algoritmi eccellono dove i metodi classici (come la Gradient Descent) falliscono:

- Non hanno bisogno che la funzione matematica sia derivabile;
- Esplorano parallelamente moltissime zone dello spazio di ricerca, riducendo drasticamente il rischio di rimanere intrappolati in minimi locali;
- Sanno bilanciare l'esplorazione di nuove aree (exploration) e lo sfruttamento delle aree promettenti appena scoperte (exploitation).

I Genetic Algorithms e la Swarm Intelligence sono meta-euristiche basate su popolazioni, ma differiscono nel meccanismo di ricerca. I Genetic Algorithms seguono un paradigma competitivo di tipo evolutivo, in cui le soluzioni migliori vengono selezionate e ricombinate tramite crossover e mutazione, mentre le peggiori vengono eliminate a ogni generazione, senza memoria esplicita degli individui passati. La Swarm Intelligence, invece, si basa su un paradigma cooperativo, in cui gli agenti non vengono mai eliminati, ma aggiornano iterativamente la propria posizione nello spazio delle soluzioni sfruttando memoria individuale e informazione globale condivisa.

7.2 Particle Swarm Optimization

La **Particle Swarm Optimization (PSO)** è stata introdotta nel 1995 da *Kennedy* (uno psicologo sociale) ed *Eberhart* (un ingegnere elettrico). È un algoritmo di ottimizzazione stocastica ispirato al comportamento sociale degli stormi di uccelli (*bird flocking*) o dei banchi di pesci (*fish schooling*). La PSO è nativamente concepita per lavorare in spazi continui (numeri reali).

Le potenziali soluzioni al problema non sono "cromosomi", ma **particelle** (*particle*). Nell'Hyperparameter Tuning, una particella rappresenta una specifica configurazione di una Neural Network da addestrare. Lo **sciame** (la popolazione, o *swarm*) è un insieme di queste particelle che "*volano*" attraverso lo spazio di ricerca multidimensionale. Se lo sciame ha 30 particelle, significa che stiamo addestrando in parallelo 30 configurazioni diverse.

In pratica abbiamo:

- **Particelle:** singoli punti nello spazio delle soluzioni che codificano una specifica configurazione di iperparametri di una Neural Network (un insieme di soluzioni);
- **Sciame:** Insieme di particelle che evolve simultaneamente esplorando lo spazio di ricerca;
- **Posizione (x_i):** È il vettore che contiene i valori attuali degli iperparametri di quella rete in un dato momento. Ad esempio, la "posizione" di una particella potrebbe essere un vettore a tre dimensioni:

[Learning Rate = 0.001, Numero di Livelli = 4, Dropout = 0.2]

Lo "spazio multidimensionale" è semplicemente lo spazio di ricerca di tutti i possibili iperparametri (di tutte le soluzioni);

- **Velocità (v_i):** Rappresenta il tasso di variazione degli iperparametri. Indica in quale direzione e di quanto stiamo modificando la configurazione per l'iterazione (o epoca) successiva. Ad esempio, una certa "velocità" potrebbe significare "aggiungi 0.005 al Learning Rate e toglì 0.1 al Dropout";
- **Personal Best ($pbest_i$):** È il set di iperparametri che ha fatto registrare la migliore metrica (la più alta Validation Accuracy o la Validation Loss più bassa) provata finora da quella specifica particella nel corso della sua ricerca;
- **Global Best ($gbest$):** rappresenta la configurazione di iperparametri migliore in assoluto scoperta tra tutte le Neural Network della popolazione.

A differenza della Random Search e della Grid Search, la PSO consente lo scambio indiretto di informazione tra soluzioni. Quando una particella individua una configurazione particolarmente efficace, questa viene registrata come Global Best e influenza l'evoluzione delle altre particelle, che vengono progressivamente attratte verso le regioni dello spazio di ricerca più promettenti, intensificando l'esplorazione locale attorno a tali valori.

Il flusso dell'algoritmo è il seguente:

1. Inizializza uno sciame di particelle con posizioni e velocità casuali;
2. Valuta la Fitness di ciascuna particella;
3. Aggiorna le posizioni migliori personali e globali;
4. Aggiorna le velocità e le posizioni delle particelle;
5. Ripeti i passaggi 2-4 fino al raggiungimento di un criterio di arresto.

Il nucleo dell'algoritmo risiede nell'aggiornamento iterativo della traiettoria di ciascuna particella nel tempo discreto t . Tale dinamica non è stocastica, ma determinata dalla combinazione vettoriale di tre componenti fondamentali che ne governano l'evoluzione nello spazio di ricerca.

Ad ogni passo t , per ogni dimensione j dello spazio, la velocità della particella i viene calcolata con questa equazione fondamentale:

$$v_i(t+1) = \underbrace{w \cdot v_i(t)}_{\text{Inerzia}} + \underbrace{c_1 \cdot r_1 \cdot (pbest_i - x_i(t))}_{\text{Componente Cognitiva}} + \underbrace{c_2 \cdot r_2 \cdot (gbest - x_i(t))}_{\text{Componente Sociale}}$$

Analizziamo i tre "motori" del movimento:

- **Inerzia** ($w \cdot v$): La tendenza della particella a continuare a muoversi nella stessa direzione in cui stava già andando. w è l'Inertia Weight (peso di inerzia). w controlla il bilanciamento tra esplorazione e sfruttamento, analogamente alla temperatura nel *Simulated Annealing*. Valori elevati (esempio: 0.9) favoriscono un'elevata esplorazione dello spazio di ricerca, mentre valori ridotti (esempio: 0.1) inducono un comportamento più conservativo, concentrato attorno a $pbest$ e $gbest$;
- **Componente Cognitiva** (c_1): La particella è attratta verso il suo miglior successo passato ($pbest$). Il coefficiente c_1 rappresenta la "fiducia in se stessi";
- **Componente Sociale** (c_2): La particella è attratta verso la posizione migliore trovata dal gruppo ($gbest$). Il coefficiente c_2 rappresenta la "fiducia nello sciame".

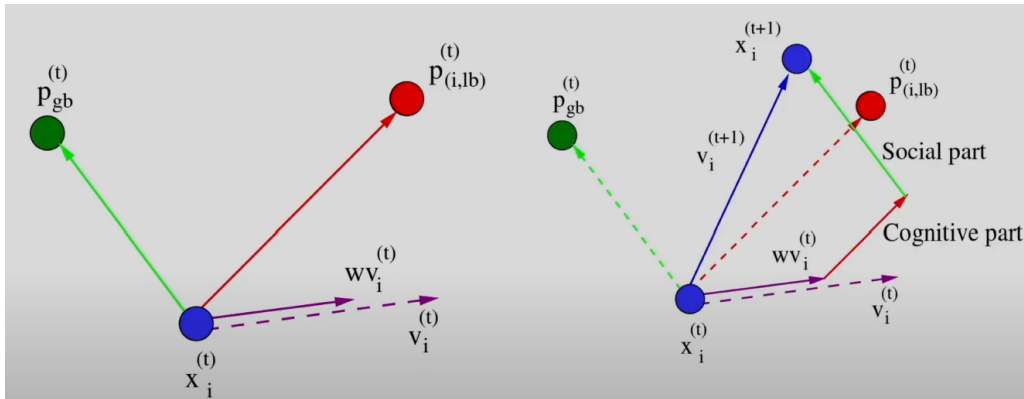
I valori r_1 e r_2 sono **numeri casuali** (tra 0 e 1) estratti ad ogni passo. Introducono la *stocasticità*, impedendo alle particelle di collidere tutte matematicamente nello stesso punto e rendono le traiettorie irregolari, come in un vero stormo.

Una volta calcolata la **nuova velocità**, la **nuova posizione** si ottiene con una semplice somma:

$$x_i(t+1) = x_i(t) + v_i(t+1)$$

Per evitare che l'accumulo delle tre forze spari le particelle fuori dai limiti dell'universo matematico, la velocità viene sempre "tagliata" (**Velocity Clamping**) entro un limite massimo prestabilito $[-V_{max}; V_{max}]$.

Abbiamo visto PSO, nel particolare riferito all'Hyperparameter Tuning, tuttavia non è il suo unico utilizzo. Può essere usato nei sistemi di controllo, nella robotica, nella pianificazione del percorso, eccetera ...



Vediamo un esempio più vicino al nostro dominio applicativo: il **Clustering**. Nel Clustering, l'obiettivo è raggruppare un set di dati non etichettati in K gruppi, trovando il "centro" (chiamato centroide) perfetto per ciascun gruppo in modo che i dati siano il più vicini possibile al proprio centroide.

Ecco come la PSO viene modellata per risolvere questo problema:

- **Particella:** In questo caso, una singola particella rappresenta le coordinate spaziali di tutti i K centroidi contemporaneamente. Se i dati hanno 2 dimensioni e cerchiamo 3 cluster, la particella sarà un vettore di 6 numeri (le coordinate x,y dei 3 centri);
- **Funzione di Fitness:** È la funzione di errore del Clustering. Si calcola misurando la distanza totale tra ogni singolo punto del dataset e il centroide (della particella) a cui è più vicino. L'obiettivo dello sciame è minimizzare questa distanza totale;
- **Velocità:** Ad ogni iterazione, le particelle "volano" spostando letteralmente le coordinate di questi centroidi fittizi lungo lo spazio dei dati;
- **Global Best (*gbest*):** Ad esempio, se la Particella 5 trova casualmente una disposizione dei centroidi che divide i dati in modo nettamente migliore delle altre, le sue coordinate diventano il nuovo *gbest*. Le altre particelle inizieranno a curvare la loro traiettoria verso quella disposizione, affinandola sempre di più.

La PSO, grazie alla sua capacità di esplorare lo spazio globalmente (grazie all'inerzia e allo sciame), riesce a evitare i minimi locali ed è meno sensibile all'inizializzazione. Molto spesso, nel **Data Mining** moderno, si usa un approccio ibrido: si lancia prima la PSO per trovare a grandi linee la zona ottimale dei centroidi, e poi si fa un fine-tuning veloce usando un algoritmo di Clustering classico, come K-Means, partendo da quelle coordinate.

PSO ha alcuni **vantaggi**:

- Facile da implementare e comprendere;
- Efficiente per problemi di ottimizzazione continua;
- Richiede un numero inferiore di parametri rispetto agli algoritmi genetici;
- Ottimo nella gestione di funzioni non lineari e non differenziabili.

Tuttavia presenta anche degli importanti **limiti**:

- Potrebbe convergere prematuramente all'ottimo locale;
- Le prestazioni dipendono molto dalla giusta impostazione dei parametri;
- Meno efficace per problemi di ottimizzazione discreta;
- La mancanza di diversità nelle iterazioni successive può ridurre l'esplorazione.

Codice

```

1  # Define the objective function
2  def objective_function(params):
3      """
4      Computes the objective function value for given parameters.
5
6      Parameters:
7      params : numpy.ndarray
8      Array of shape (n_particles, 3) containing particle positions
9
10     Returns:
11     numpy.ndarray

```



```

12     Array of shape (n_particles,) containing cost for each particle
13     """
14     return (params[:, 0] - 4)**2 + (params[:, 1] - 5)**2 + (params[:, 2] + 6)**2
15
16
17 # Additional utility: Function to run PSO with different parameters
18 def run_pso(n_particles=10, max_iter=30, w=0.5, c1=0.8, c2=0.9, bounds=None, obj_func=None):
19     """
20     Complete PSO implementation as a reusable function.
21
22     Parameters:
23     - n_particles: Number of particles
24     - max_iter: Maximum iterations
25     - w: Inertia weight
26     - c1: Cognitive coefficient
27     - c2: Social coefficient
28     - bounds: Search space bounds
29     - obj_func: Objective function
30
31     Returns:
32     - g_best_position: Best found solution
33     - global_best_cost: Best found cost
34     - cost_history: Array of best costs per iteration
35     """
36     if bounds is None:
37         bounds = np.array([[ -10, -10, -10], [10, 10, 10]])
38     if obj_func is None:
39         obj_func = objective_function
40
41     # Initialize particles and velocities
42     particles = bounds[0] + np.random.rand(n_particles, 3) * (bounds[1] - bounds[0])
43     velocities = np.zeros((n_particles, 3))
44
45     # Initialize best positions
46     p_best_positions = particles.copy()
47     best_costs = obj_func(particles)
48     g_best_idx = np.argmin(best_costs)
49     g_best_position = p_best_positions[g_best_idx].copy()
50     global_best_cost = best_costs[g_best_idx]
51
52     cost_history = []
53
54     for iteration in range(max_iter):
55         r1 = np.random.rand(n_particles, 3)
56         r2 = np.random.rand(n_particles, 3)
57
58         cognitive_component = c1 * r1 * (p_best_positions - particles)
59         social_component = c2 * r2 * (g_best_position - particles)
60
61         velocities = w * velocities + cognitive_component + social_component
62         particles = particles + velocities
63
64         particles = np.maximum(particles, bounds[0])
65         particles = np.minimum(particles, bounds[1])
66
67         costs = obj_func(particles)
68
69         improved_indices = costs < best_costs
70         p_best_positions[improved_indices] = particles[improved_indices]
71         best_costs[improved_indices] = costs[improved_indices]
72
73         current_best_idx = np.argmin(best_costs)
74         if best_costs[current_best_idx] < global_best_cost:
75             g_best_position = p_best_positions[current_best_idx].copy()
76             global_best_cost = best_costs[current_best_idx]
77

```

```

78     cost_history.append(global_best_cost)
79
80     return g_best_position, global_best_cost, cost_history
81
82 # Example usage:
83 best_pos, best_cost, history = run_pso(n_particles=20, max_iter=50, w=0.7)

```

7.3 Ant Colony Optimization

L'**Ant Colony Optimization (ACO)**, introdotto da *Marco Dorigo* nei primi anni '90, è un algoritmo progettato per problemi di ottimizzazione discreta su grafi che trova applicazione in problemi come il Traveling Salesman Problem, il routing e la progettazione di reti.

Le formiche reali sono quasi cieche e non comunicano direttamente tra loro, poiché non esiste un controllo centrale. Si coordinano invece tramite la **Stigmergia**, un meccanismo di comunicazione indiretta basato sull'ambiente. Durante il cammino rilasciano feromoni e quando una formica trova una fonte di cibo vicina, torna al nido più rapidamente, rafforzando la traccia su quel percorso. Le altre formiche, attratte da essa, la seguono e la rinforzano ulteriormente. Si crea così un feedback positivo che porta l'intera colonia a convergere sul percorso più efficiente.

L'algoritmo è il seguente:

1. **Inizializzazione:** Su ogni arco $(i; j)$ del grafo vengono inizializzati due valori:
 - τ_{ij} (Tau): La *Pheromone Matrix* che misura il livello di feromone (dinamico, inizializzato a un valore piccolo);
 - η_{ij} (Eta): La *Heuristic Matrix* che rappresenta informazione euristica (statica, tipicamente l'inverso della distanza: $\eta_{ij} = 1/d_{ij}$). Equivale a un approccio greedy (più la città è vicina, più è desiderabile).
2. **Costruzione del percorso (Regola di Transizione Probabilistica):** Ogni formica costruisce una soluzione muovendosi di nodo in nodo. La decisione non è deterministica, ma calcolata *on-the-fly* tramite una "ruota della roulette pesata". La probabilità p_{ij} che una formica nel nodo i scelga di andare nel nodo j è data da:

$$p_{ij} = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{u \in Allowed_i} [\tau_{iu}]^\alpha [\eta_{iu}]^\beta}$$

- α (**Influenza del feromone**): Quanto la formica si fida delle "scoperte" passate della colonia (**Exploitation**);
 - β (**Influenza dell'euristica**): Quanto la formica si affida alla convenienza immediata locale, come la distanza breve (**Exploration**).
3. **Aggiornamento dei Feromoni:** Una volta che tutte le formiche hanno costruito i loro percorsi, la matrice dei feromoni τ viene aggiornata:

$$\tau_{ij}(t+1) = (1 - \rho)\tau_{ij}(t) + \sum_{k=1}^m \Delta\tau_{ij}^{(k)}(t)$$

- **Evaporazione** $(1 - \rho)$: Il parametro ρ regola la diminuzione dei feromoni, permettendo di abbandonare progressivamente i percorsi meno efficaci ed evitando convergenze premature.
 - **Deposizione** $(\sum \Delta\tau)$: Quantità di feromone depositata dalla formica k sull'arco $(i; j)$.
4. **Iterazione:** Il processo si ripete finché non viene raggiunto un criterio di arresto (esempio: numero massimo di iterazioni).

Per fare un esempio pratico prendiamo sempre l'Hyperparameter Tuning. Mentre la PSO naviga fluidamente negli spazi continui, l'ACO è formidabile quando lo spazio degli iperparametri è discreto o categorico (esempio: la funzione di attivazione, il numero esatto di layer, o un set predefinito di learning rate).

Per usare l'ACO nell'Hyperparameter Tuning, dobbiamo costruire un "**Grafo delle Scelte**". Immagina un percorso a tappe sequenziali. Ogni "tappa" (o livello) del grafo rappresenta un iperparametro da scegliere, e i "nodi" in quella tappa sono i possibili valori. Ad esempio:

- **Tappa 1 (Learning Rate):** Nodi disponibili $\rightarrow [0.1, 0.01, 0.001]$;
- **Tappa 2 (Batch Size):** Nodi disponibili $\rightarrow [16, 32, 64]$;
- **Tappa 3 (Funzione di Attivazione):** Nodi disponibili $\rightarrow [ReLU, Tanh, Sigmoid]$

Una singola formica rappresenta la fase di costruzione di un'architettura. La formica parte dall'inizio e deve attraversare tutte le tappe. A ogni tappa, sceglie un nodo in base alla quantità di feromone presente su di esso. Un percorso completo (esempio: $0.01 \rightarrow 64 \rightarrow ReLU$) genera una configurazione completa di una Neural Network. Una volta che la formica ha costruito la rete, la Neural Network viene addestrata. L'Accuracy o la loss ottenuta sul Validation Set rappresenta la qualità del percorso. Se l'Accuracy è altissima (esempio: 95%) o la loss è bassa (si può usare l'inverso), la formica tornerà indietro e lascerà una grande quantità di feromone sui nodi 0.01 , 64 e $ReLU$.

Iterazione dopo iterazione, le formiche successive sentiranno un forte odore di feromone sulle scelte vincenti. L'intera colonia smetterà di esplorare i valori inutili (come potrebbero essere *Sigmoid* o 0.1) e convergerà rapidamente sull'architettura ottimale, ottimizzando il modello.

Analizziamo meglio le formule che abbiamo appena visto.

Quando la formica artificiale k si trova nella città i , deve scegliere la prossima città j . Non sceglie in modo deterministico la più vicina, ma usa una regola probabilistica (**Roulette Wheel**). La probabilità $p_{ij}^k(t)$ di scegliere il nodo j è definita dalla formula:

$$p_{ij}^{(k)}(t) = \frac{[\tau_{ij}(t)]^\alpha \cdot [\eta_{ij}]^\beta}{\sum_{u \in J_k(i)} [\tau_{iu}(t)]^\alpha \cdot [\eta_{iu}]^\beta}$$

Dove:

- $J_k(i)$: L'insieme dei nodi non ancora visitati dalla formica k (quelli non presenti nella sua memoria/Tabu list). Se un nodo è già stato visitato, la probabilità di tornarci è forzata a 0;
- $\tau_{ij}(t)$ (Tau): La concentrazione di feromone sull'arco (i, j) al tempo t ;
- η_{ij} (Eta): La visibilità (euristica), tipicamente calcolata come $1/d_{ij}$ (dove d è la distanza);
- α e β : I parametri che controllano il peso relativo del feromone rispetto alla distanza visibile.

Quando tutte le m formiche della colonia hanno completato i loro percorsi, si aggiorna globalmente la matrice dei feromoni per il ciclo successivo $(t+1)$. Questa fase è l'anima **Reinforcement Learning** e si compone di due forze opposte: **Evaporazione** e **Deposizione**.

La formula generale di aggiornamento per l'arco $(i; j)$ è:

$$\tau_{ij}(t+1) = (1 - \rho) \cdot \tau_{ij}(t) + \Delta\tau_{ij}$$

Dove:

- **Evaporazione:** ρ è l'*Evaporation Rate* (un valore tra 0 e 1, solitamente 0.5). Se l'evaporazione non ci fosse, gli archi iniziali scelti per caso accumulerebbero subito troppo feromone, costringendo l'algoritmo in un minimo locale. L'evaporazione permette alla colonia di "dimenticare" i percorsi peggiori.
- **Deposizione:** $\Delta\tau_{ij} = \sum_{k=1}^m \Delta\tau_{ij}^k$. È la quantità totale di nuovo feromone depositato su quell'arco da tutte le formiche che ci sono passate sopra. Quanto feromone deposita la singola formica k ? Se la formica non è passata su quell'arco, deposita 0. Se ci è passata, deposita una quantità pari a Q/L_k , dove Q è una costante e L_k è la lunghezza totale del tour appena completato. *Poiché c'è L_k al denominatore, le formiche che hanno trovato un giro brevissimo scaricheranno una densità altissima di feromone.*

ACO utilizza una matrice di feromoni τ che codifica la desiderabilità appresa delle soluzioni. Il **warm-start** consiste nell'inizializzare il processo di ottimizzazione a partire da una configurazione informata anziché casuale. Nei problemi dinamici, l'ACO può adattarsi rapidamente riutilizzando la struttura dei feromoni esistenti. Questa capacità di riutilizzo rende il warm-start particolarmente efficiente rispetto a molte altre meta-euristiche.

In uno scenario di *edge deletion*, un arco $(i; j)$ viene rimosso dal grafo a seguito di una modifica della struttura del problema. La matrice dei feromoni viene aggiornata per riflettere tale cambiamento, ponendo $\tau_{ij} = 0$ oppure eliminando completamente la corrispondente riga e colonna, se richiesto dalla rappresentazione adottata. L'algoritmo ACO viene quindi riavviato utilizzando la matrice aggiornata, in modo che le formiche non possano più selezionare l'arco eliminato e ne evitino automaticamente l'utilizzo grazie all'assenza di feromoni associati.

ACO ha alcuni **vantaggi**:

- Buona capacità di esplorazione dello spazio delle soluzioni;
- Efficiente per problemi di ottimizzazione discreta e su grafi;
- Capacità di adattamento a scenari dinamici;
- Possibilità di warm-start tramite feromoni preesistenti.

Tuttavia presenta anche degli importanti **limiti**:

- Convergenza lenta rispetto ad altri metaeuristici;
- Sensibilità alla scelta dei parametri (α, β, ρ) ;
- Elevato costo computazionale su istanze grandi;
- Performance meno efficace su problemi continui rispetto a quelli discreti.

Codice

```

1  class Node:
2      """Represents a city/node in the graph."""
3      def __init__(self, x, y):
4          self.x = x
5          self.y = y
6
7  class Graph:
8      """Represents the entire graph with nodes and edges."""
9      def __init__(self, x_coords, y_coords):
10         self.n = len(x_coords) # Number of nodes
11         self.nodes = [Node(x_coords[i], y_coords[i]) for i in range(self.n)]
12         self.edges = np.full((self.n, self.n), np.inf) # Adjacency matrix for edge weights
13
14     def add_edge(self, node1_idx, node2_idx, weight):
15         """Adds an edge between two nodes with a given weight."""
16         # Adjust indices to be 0-based for Python
17         self.edges[node1_idx - 1, node2_idx - 1] = weight
18         self.edges[node2_idx - 1, node1_idx - 1] = weight # Assuming an undirected graph
19
20     def create_graph(x_coords, y_coords):
21         """
22         Initializes the graph with given coordinates and predefined edge weights.
23         """
24         graph = Graph(x_coords, y_coords)
25
26         # Add edges with predefined weights (as per MATLAB script)
27         graph.add_edge(1, 2, 8)
28         graph.add_edge(1, 3, 3)
29         graph.add_edge(1, 4, 5)
30         graph.add_edge(2, 3, 6)
31         graph.add_edge(2, 5, 1)
32         graph.add_edge(2, 6, 6)
33         graph.add_edge(3, 4, 4)
34         graph.add_edge(3, 6, 6)
35         graph.add_edge(3, 7, 5)
36         graph.add_edge(4, 7, 4)
37         graph.add_edge(4, 8, 5)
38         graph.add_edge(5, 6, 1)
39         graph.add_edge(5, 9, 9)
40         graph.add_edge(5, 10, 4)
41         graph.add_edge(6, 7, 7)
42         graph.add_edge(6, 10, 2)
43         graph.add_edge(6, 11, 9)
44         graph.add_edge(7, 8, 1)
45         graph.add_edge(7, 11, 10)
46         graph.add_edge(7, 12, 4)
47         graph.add_edge(8, 12, 5)
48         graph.add_edge(8, 13, 10)
49         graph.add_edge(9, 10, 8)
50         graph.add_edge(9, 14, 7)
51         graph.add_edge(10, 11, 5)
52         graph.add_edge(10, 14, 7)
53         graph.add_edge(10, 15, 2)
54         graph.add_edge(11, 12, 9)
55         graph.add_edge(11, 15, 8)
56         graph.add_edge(11, 16, 7)
57         graph.add_edge(12, 13, 3)
58         graph.add_edge(12, 16, 8)
59         graph.add_edge(12, 17, 5)
60         graph.add_edge(13, 17, 9)
61         graph.add_edge(14, 15, 6)
62         graph.add_edge(14, 18, 2)
63         graph.add_edge(15, 16, 8)

```

```

64     graph.add_edge(15, 18, 7)
65     graph.add_edge(15, 19, 9)
66     graph.add_edge(16, 17, 2)
67     graph.add_edge(16, 19, 9)
68     graph.add_edge(16, 20, 3)
69     graph.add_edge(17, 20, 5)
70     graph.add_edge(18, 19, 10)
71     graph.add_edge(18, 21, 4)
72     graph.add_edge(19, 20, 9)
73     graph.add_edge(19, 21, 3)
74     graph.add_edge(20, 21, 8)
75     return graph
76
77 def fitness_function(tour, graph):
78     """
79     Calculates the total length (fitness) of a given tour.
80     A tour is a sequence of node indices (0-based).
81     """
82     fitness = 0
83     for i in range(len(tour) - 1):
84         current_node = tour[i]
85         next_node = tour[i+1]
86         fitness += graph.edges[current_node, next_node]
87     return fitness
88
89 def roulette_wheel(probabilities):
90     """
91     Selects an item based on its probability using the roulette wheel selection method.
92     """
93     cumsum_probabilities = np.cumsum(probabilities)
94     r = np.random.rand()
95     # Find the first index where r is less than or equal to the cumulative sum
96     next_node_idx = np.where(r <= cumsum_probabilities)[0][0]
97     return next_node_idx
98
99 class Ant:
100     """Represents an individual ant and its tour."""
101     def __init__(self):
102         self.tour = []
103         self.fitness = np.inf
104
105 class Colony:
106     """Manages a collection of ants."""
107     def __init__(self, ant_no):
108         self.ants = [Ant() for _ in range(ant_no)]
109         self.queen = Ant() # The best ant found so far
110
111 def create_colony(graph, ant_no, tau, eta, alpha, beta):
112     """
113     Creates tours for all ants in the colony. Each ant constructs a complete tour
114     probabilistically based on pheromone and desirability.
115     """
116     colony = Colony(ant_no)
117     node_no = graph.n
118
119     for i in range(ant_no):
120         # Select a random initial node (0-based index)
121         initial_node = np.random.randint(0, node_no)
122         colony.ants[i].tour = [initial_node]
123         current_node = initial_node
124         visited_nodes = {initial_node} # Use a set for efficient lookup
125
126         # Construct the primary tour visiting all nodes exactly once
127         while len(visited_nodes) < node_no:
128             current_node = colony.ants[i].tour[-1]

```

```

130     # Calculate probabilities for next step
131     # P_allNodes = (tau[current_node, :] ** alpha) * (eta[current_node, :] ** beta)
132     # Ensure correct element-wise multiplication for numpy arrays
133     pheromone_influence = tau[current_node, :] ** alpha
134     desirability_influence = eta[current_node, :] ** beta
135     P_allNodes = pheromone_influence * desirability_influence
136
137     # Assign almost zero probability to visited nodes and infinite edges
138     for j in range(node_no):
139         if j in visited_nodes or np.isinf(graph.edges[current_node, j]):
140             P_allNodes[j] = 1e-10 # Small non-zero value to avoid division by zero if sum is 0
141
142     # Normalize probabilities
143     sum_P_allNodes = np.sum(P_allNodes)
144     if sum_P_allNodes == 0: # Handle cases where no valid next move exists (shouldn't happen in a
connected graph)
145         unvisited = [n for n in range(node_no) if n not in visited_nodes and not np.isinf(graph.
edges[current_node, n])]
146         if unvisited:
147             next_node = np.random.choice(unvisited)
148         else:
149             break
150     else:
151         P = P_allNodes / sum_P_allNodes
152         next_node = roulette_wheel(P)
153
154     colony.ants[i].tour.append(next_node)
155     visited_nodes.add(next_node)
156
157     # Complete the tour by returning to the initial node
158     colony.ants[i].tour.append(initial_node) # Explicitly close the loop
159
160     return colony
161
162 def update_pheromone(tau, colony):
163     """
164     Updates the pheromone matrix based on the tours completed by the ants.
165     Pheromone is deposited inversely proportional to the tour's fitness.
166     """
167     # Initialize delta_tau for this iteration
168     delta_tau = np.zeros_like(tau)
169     ant_no = len(colony.ants)
170
171     for i in range(ant_no): # For each ant
172         tour = colony.ants[i].tour
173         fitness = colony.ants[i].fitness
174
175         if fitness == 0: # Avoid division by zero for perfect tours (unlikely in TSP)
176             pheromone_deposit = np.inf
177         else:
178             pheromone_deposit = 1.0 / fitness
179
180         for j in range(len(tour) - 1): # For each edge in the tour (excluding the last return to start)
181             node1 = tour[j]
182             node2 = tour[j+1]
183
184             # Deposit pheromone on both directions of the edge
185             delta_tau[node1, node2] += pheromone_deposit
186             delta_tau[node2, node1] += pheromone_deposit
187
188     # Add the deposited pheromone to the existing pheromone matrix
189     tau += delta_tau
190     return tau
191
192
193

```

```

194 def aco_loop(graph, max_iter, ant_no, tau, eta, alpha, beta, rho, edge_eliminated=False,
195             eliminated_nodes=None):
196     """
197     Main loop of the Ant Colony Optimization algorithm.
198     Iteratively creates ant tours, updates pheromones, and performs evaporation.
199     Returns the final pheromone matrix, best tour, AND the Plotly figure object.
200     """
201     best_fitness = np.inf
202     best_tour = []
203
204     for t in range(1, max_iter + 1):
205         # Create Ants and their tours
206         colony = create_colony(graph, ant_no, tau, eta, alpha, beta)
207
208         # Calculate the fitness values of all ants
209         for ant in colony.ants:
210             ant.fitness = fitness_function(ant.tour, graph)
211
212         # Find the best ant (queen) in the current iteration
213         all_ants_fitness = [ant.fitness for ant in colony.ants]
214         min_val = np.min(all_ants_fitness)
215         min_index = np.argmin(all_ants_fitness)
216
217         if min_val < best_fitness:
218             best_fitness = min_val
219             best_tour = colony.ants[min_index].tour
220
221         colony.queen.tour = best_tour
222         colony.queen.fitness = best_fitness
223
224         # Update pheromone matrix based on current iteration's tours
225         tau = update_pheromone(tau, colony)
226
227         # Evaporation: Pheromone trails gradually evaporate
228         tau = (1 - rho) * tau
229
230         # Display results and visualize progress
231         print(f"Iteration #{t}, Shortest length = {colony.queen.fitness:.2f}")
232
233     return tau, best_tour, fig # Return the figure object

```


Capitolo 8: Advanced Deep Learning

8.1 La necessità di ottimizzare il Deep Learning

Il **Deep Learning** ha determinato una trasformazione significativa nel campo dell'**Intelligenza Artificiale**. Tuttavia, il suo notevole successo ha incentivato lo sviluppo di modelli progressivamente sempre più complessi e di grandi dimensioni. È pertanto necessario analizzare i due principali colli di bottiglia che caratterizzano il Deep Learning contemporaneo.

Il primo problema è quello delle **Hand-crafted Architectures**. Fino a pochi anni fa, lo sviluppo di architetture (esempio: VGG, ResNet, Inception) era un processo manuale basato sull'intuizione e su estese fasi di *trial-and-error*. Questo approccio presenta diversi limiti. In primo luogo, introduce un bias umano verso architetture simmetriche e ripetitive, sebbene lo spazio delle possibili soluzioni sia molto più ampio e non vincolato a tali strutture. Inoltre, i tempi di progettazione, addestramento e validazione sono elevati, richiedendo mesi di lavoro. Infine, per garantire le prestazioni, si tende spesso a ricorrere a modelli sovradimensionati, con un elevato numero di parametri potenzialmente ridondanti.

Il secondo problema è quello del **collo di bottiglia nella Memoria e nel Calcolo**. Abbiamo infatti un problema di natura hardware. Le Neural Network moderne hanno raggiunto dimensioni molto elevate, con centinaia di milioni o miliardi di parametri, generando tre principali vincoli:

1. **Il primo riguarda la memoria:** la sola memorizzazione dei pesi (esempio: Float32) e dei gradienti richiede quantità di RAM/VRAM spesso proibitive;
2. **Il secondo è computazionale:** l'elaborazione di un singolo input implica miliardi di operazioni floating point, con conseguenti elevati tempi di inferenza senza hardware specializzato;
3. **Il terzo riguarda l'edge computing:** l'esecuzione su dispositivi con risorse limitate (smartphone, wearable, veicoli autonomi) è penalizzata da consumo energetico, vincoli di memoria e latenza, rendendo i modelli di grandi dimensioni difficilmente deployabili in ambienti real-time.

Ad esempio prendiamo i **Large Language Model**. L'addestramento di un modello linguistico basato su Transformer richiede grandi quantità di dati testuali, provenienti sia da fonti web (esempio: Common Crawl) sia da corpora curati come libri e Wikipedia. L'addestramento si divide in due fasi:

- **Pre-Training:** è la fase in cui un modello viene addestrato da zero su una grande quantità di dati generici. Il modello impara conoscenze generali sulla struttura dei dati;
- **Fine-Tuning:** è la fase successiva in cui un modello già pre-addestrato viene adattato ad un task specifico. Si parte da un modello generale e lo si rende *domain-specific*.

Durante il pre-training, il modello apprende a predire parole a partire dal contesto, acquisendo conoscenze sintattiche, semantiche e fattuali e la capacità di generare frasi corrette. Tuttavia, tale fase fornisce solo conoscenza generale della lingua, senza specializzazione di dominio. La specializzazione avviene

tramite fine-tuning, ossia un ulteriore addestramento su un dataset più piccolo e specifico, che aggiorna i parametri del modello per adattarlo a contesti applicativi mirati.

Questo metodo di addestramento non viene applicato solo alla **Natural Language Processing**, ma anche alla **Computer Vision**, ai **sistemi industriali** e in generale ai **modelli generativi**. Questo impiega enormi quantità di tempo e di risorse.

Ovviamente, queste considerazioni si applicano anche nel caso in cui il modello sia sottoposto esclusivamente a **una fase di training specifica** ed anzi, è proprio quest'ultimo scenario quello su cui ci concentreremo principalmente.

8.2 Low-Rank Adaptation per il Fine-Tuning

Il fine-tuning tradizionale aggiorna tutti i parametri del modello, quindi: è costoso in memoria, richiede GPU potenti, può "distruggere" la conoscenza generale (*catastrophic forgetting*). La **Low-Rank Adaptation (LoRA)** è una tecnica di fine-tuning efficiente. Invece di modificare tutti i pesi del modello, si modificano solo piccole "correzioni". Un layer lineare standard è:

$$W \cdot x$$

Con LoRA si scrive:

$$W \cdot x + \Delta W \cdot x$$

Tuttavia $\Delta W = A \cdot B$, dove A e B sono matrici a basso rango molto più piccole di W . Invece di cambiare tutta la conoscenza del modello, aggiungi una modifica leggera sopra il modello originale. Quindi nel LoRA non aggiorni la matrice originale W , ma addestri solo le matrici A e B .

Attualmente rappresenta il golden standard di riferimento per il fine-tuning, poiché consente una significativa riduzione dei tempi di addestramento e delle risorse computazionali necessarie. Tuttavia, è necessario considerare anche la fase di pre-training: qualora il numero di parametri da aggiornare sia molto elevato, il consumo di tempo e risorse può rimanere comunque rilevante.

8.3 Neural Architecture Search

Tre Pilastri

La **Neural Architecture Search (NAS)** è il processo che mira ad automatizzare completamente la progettazione delle Neural Network. Invece di far disegnare i layer a un ingegnere umano, si demanda il compito a un algoritmo, con l'obiettivo di trovare architetture che superino in efficacia ed efficienza quelle "artigianali".

Formalmente, un sistema NAS si articola tipicamente in tre componenti principali:

- **uno spazio di ricerca (search space):** che definisce l'insieme delle architetture ammissibili;
- **una strategia di ricerca (search strategy):** che governa l'esplorazione di tale spazio;
- **una strategia di valutazione (performance estimation strategy):** che consente di stimare l'efficacia delle architetture candidate.

La Search Space definisce l'insieme di tutte le possibili architetture che l'algoritmo può scoprire. Più è vasto, più è difficile da esplorare.

Si divide in due tipologie:

- **Macro-architettura:** L'algoritmo decide l'intera struttura della rete strato per strato (esempio: quanti layer convoluzionali, come connetterli);
- **Micro-architettura (Cell-based):** Ispirata alle reti umane moderne (come ResNet o Inception), la NAS non inventa tutta la rete, ma inventa solo due piccoli blocchi chiamati Normal Cell (che mantiene la dimensione spaziale) e Reduction Cell (che dimezza la dimensione spaziale). La rete finale si ottiene semplicemente impilando queste due celle ripetutamente. Questo riduce drasticamente lo spazio di ricerca.

Una volta definito cosa cercare, serve un algoritmo di Search Strategy e successivamente un modo per capire se la rete trovata è buona (Performance Estimation). Poiché addestrare ogni singola rete trovata per misurarne l'efficacia richiederebbe decenni, la ricerca si è divisa in tre grandi approcci.

Uso dei Genetic Algorithms

L'utilizzo dei **Genetic Algorithms** nella Neural Architecture Search è iniziato negli anni '90. Nella NAS, ogni individuo della popolazione codifica una possibile architettura neurale, che viene valutata in base alle prestazioni ottenute su un compito specifico. Attraverso operatori di selezione, crossover e mutazione, le architetture migliori vengono progressivamente combinate e modificate, con l'obiettivo di migliorare la qualità delle soluzioni nel tempo. Questi metodi si sono dimostrati efficaci nell'ottimizzazione di diversi aspetti strutturali delle Neural Network, come il numero di hidden layer, il numero di neuroni per layer, il tipo di funzione di attivazione e la struttura delle connessioni. Tuttavia, uno dei principali limiti degli approcci basati su algoritmi evolutivi è il costo computazionale elevato, poiché la valutazione di ciascun individuo richiede tipicamente l'addestramento (anche parziale) della rete corrispondente.

Per mitigare tale problema, sono stati proposti approcci ibridi, tra cui i **Memetic Algorithms**, che combinano la ricerca globale tipica degli algoritmi genetici con procedure di ottimizzazione locale, come la Gradient Descent.

- **Pro:** Altamente flessibile, ottima capacità di Global Search (non si blocca nei minimi locali).
- **Contro:** Costo computazionale proibitivo. Generare, addestrare e scartare migliaia di Neural Network richiede enormi cluster di GPU.

Reinforcement Learning

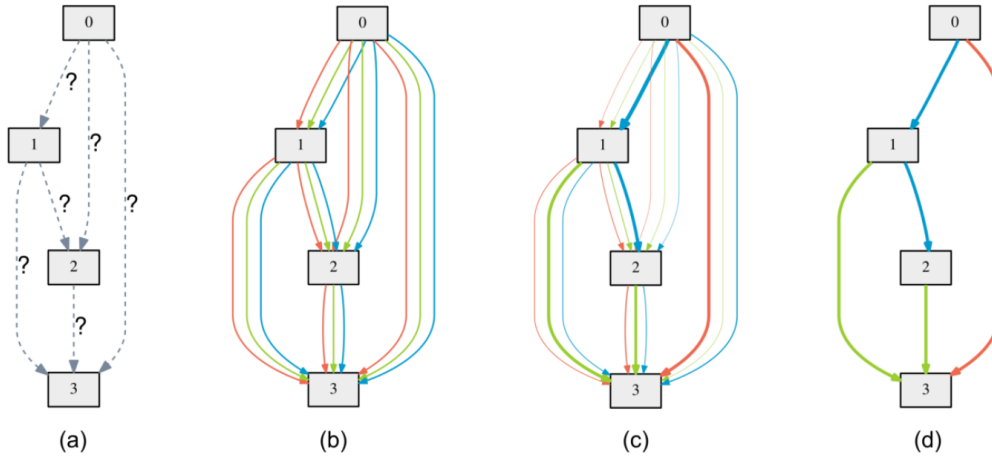
Il **Reinforcement Learning** applicato alla Neural Architecture Search consiste nel formulare la progettazione di Neural Network come un problema decisionale sequenziale, in cui un agente apprende a costruire architetture ottimali massimizzando una funzione di reward.

Una delle formulazioni più note è quella proposta da *Zoph e Le* (in *Google* nel 2017) dove si usa un **Recurrent Neural Network (RNN)** chiamata **Controller**. Il Controller genera sequenzialmente i parametri dell'architettura (le azioni). La rete generata viene addestrata e la sua Accuracy finale viene usata come Segnale di Ricompensa (**Reward**). Il Controller viene aggiornato tramite **Policy Gradient** per generare architetture sempre migliori.

L'obiettivo del Reinforcement Learning è quindi massimizzare il valore atteso del reward. Di conseguenza, le architetture migliori, che ottengono un reward più alto, avranno una maggiore probabilità di essere campionate in futuro, al contrario, le architetture peggiori, associate a un reward più basso, verranno progressivamente abbandonate.

- **Pro:** Ha generato reti storiche (come NASNet) che hanno battuto quelle umane.
- **Contro:** Ancora più costoso dei Genetic Algorithms. La prima NASNet richiese 800 GPU che lavorarono ininterrottamente per 28 giorni (migliaia di ore di calcolo).

Differentiable Architecture Search



Per risolvere il problema dei costi insostenibili, *Liu* ha ideato la **Differentiable Architecture Search (DARTS)**.

L'idea centrale è trasformare lo spazio discreto delle architetture (scelta di operazioni e connessioni tra nodi) in uno spazio continuo. Questo avviene tramite una rilassazione continua: invece di scegliere un'unica operazione tra più candidate (esempio: convoluzione, pooling, skip connection), si considera una combinazione pesata di tutte le operazioni disponibili.

Formalmente, per un arco tra due nodi i e j , l'output non è dato da una singola operazione $o_k(x)$, ma da una combinazione lineare pesata:

$$o_{i,j}(x) = \sum_{k=1}^K \alpha_{i,j,k} \cdot o_k(x)$$

dove i coefficienti $\alpha_{i,j,k}$ sono parametri continui, tipicamente normalizzati tramite Softmax, e rappresentano l'importanza relativa delle diverse operazioni candidate. In questo modo, anche la struttura della rete diventa differenziabile rispetto a tali parametri architetturali.

Durante la fase di search, vengono ottimizzati simultaneamente:

- i pesi della rete (weight parameters),
- i parametri architetturali (architecture parameters).

Questo schema è tipicamente formulato come un problema **bi-livello**, in cui i pesi del modello vengono ottimizzati sul training set, mentre i parametri architetturali vengono aggiornati sul validation set, al fine di ridurre il rischio di overfitting.

Al termine della fase di ricerca, l'architettura viene **discretizzata**, selezionando per ogni connessione le operazioni con peso massimo (operatore $\arg \max$), ottenendo così una rete neurale effettivamente implementabile.

- **Pro:** Riduce i tempi di ricerca da migliaia di giorni/GPU a una frazione infinitesimale, potendo girare su una singola GPU.
- **Contro:** Dovendo calcolare tutte le operazioni in parallelo (Supernetwork) prima di scartare quelle inutili, richiede moltissima RAM/VRAM. Inoltre quando alla fine si "pota" la rete scartando le operazioni con α basso, le prestazioni possono crollare perché la rete si era abituata ad avere tutte le operazioni attive.

Aspect	GAs	RL	DARTS
Search Efficiency	Low	Medium	High
Computational Cost	High	High	Medium
Global Search Capability	Yes	Partial	No
Implementation Complexity	Medium	High	Medium
Suitability for Large Models	Challenging	Challenging	More Feasible

8.4 Knowledge Distillation

Un **ensemble** di modelli (o "*ensemble learning*") è una tecnica di Machine Learning in cui più modelli vengono combinati per ottenere una previsione finale migliore rispetto a quella di un singolo modello. L'idea di base è che modelli diversi commettono errori diversi. Combinandoli, gli errori tendono a compensarsi, migliorando accuratezza e stabilità.

I modelli di Deep Learning moderni e gli Ensemble raggiungono precisioni incredibili, ma sono troppo pesanti, lenti e costosi per essere eseguiti su constrained device (smartphone, sensori).

La **Knowledge Distillation (KD)** è una tecnica introdotta per risolvere questo problema. Invece di cercare una nuova architettura da zero (come fa la NAS), la Knowledge Distillation prende un modello gigantesco già addestrato e ne "spreme" la conoscenza dentro un modello molto più piccolo.

L'architettura si basa su un framework Teacher-Student:

- **Teacher:** È un modello grande, complesso e ingombrante (o un ensemble di modelli). È già stato addestrato ed è altamente accurato ed efficace. Il suo compito è fare da guida;
- **Student:** È un modello piccolo, leggero e veloce. Deve essere addestrato da zero, ma invece di imparare solo dai dati grezzi, impara cercando di imitare il comportamento del Teacher.

In pratica la Knowledge Distillation consente di comprimere un modello riducendo dimensioni e costi computazionali, rendendolo adatto a dispositivi edge e mobile. Migliora la velocità di inferenza e può preservare prestazioni vicine a quelle del modello insegnante. Inoltre, favorisce una migliore generalizzazione dello Student grazie al segnale più informativo fornito dal Teacher, e può accelerare la fase di addestramento.

Per la seguente discussione sulla Classificazione, considereremo principalmente uno scenario di Classificazione multiclasse, mutuamente esclusiva. Prendiamo un esempio concreto, per capire meglio: un teacher model che riconosce foto di auto, camion e moto.

Quando il teacher model analizza una foto di una BMW, il suo output non ha praticamente mai un 1 netto. L'output tipico (i Soft Targets) potrebbe essere:

[Auto: 0.90, Camion: 0.09, Moto: 0.01]

Geoffrey Hinton ha coniato il termine **Dark Knowledge** per descrivere proprio queste probabilità apparentemente sbagliate. Il fatto che il modello assegni un 9% al Camion e solo l'1% alle Moto

contiene un'informazione strutturale: il Teacher ha imparato che un'auto assomiglia molto di più a un camion che a una moto. Se costringiamo lo Student a imparare solo dagli "Hard Targets" (gli 1 e gli 0 del dataset), questa mappa relazionale verrebbe ignorata. Lo scopo della Knowledge Distillation è anche quello di trasferire questa Dark Knowledge allo Student.

Alcuni moderni teacher model producono, tramite Softmax, distribuzioni di probabilità eccessivamente concentrate su una label (esempio: $[0.999, 0.001, 0.000]$), riducendo l'informazione sulle relazioni tra le classi non corrette. Per "ammorbidire" queste probabilità ed estrarre la Dark Knowledge, si introduce un parametro chiamato **Temperatura** (T) nella formula della Softmax:

$$q_i = \frac{e^{(z_i/T)}}{\sum_j e^{(z_j/T)}}$$

Da notare che:

- Se $T = 1$: È la Softmax standard. Produce distribuzioni appuntite (Hard/Sharp);
- Se $T > 1$ (esempio: $T = 3$ o $T = 5$): La distribuzione si "scioglie" e si appiattisce, rivelando chiaramente le probabilità nascoste delle classi secondarie.

C'è un'analogia alla temperatura che abbiamo visto nel Simulated Annealing. Nel Machine Learning, ogni volta che vedi una variabile chiamata *Temperature* (temperatura), il suo scopo è sempre uno solo: regolare quanto la nostra distribuzione di probabilità deve essere "piatta/uniforme" (Alta Temperatura) rispetto a quanto deve essere "appuntita/deterministica" (Bassa Temperatura). E qui non fa eccezione.

Cosa esattamente stiamo cercando di far copiare allo Student?

Nella Knowledge Distillation, il segnale di distillazione può essere estratto da diverse componenti della rete Teacher e trasferito allo Student attraverso strategie differenti:

- **Soft Target Matching:** Questo approccio consiste nel far apprendere allo Student le distribuzioni di probabilità generate dal Teacher dopo l'applicazione della funzione Softmax con temperatura. In questo modo vengono trasferite non solo le predizioni finali, ma anche la Dark Knowledge. L'allineamento tra le distribuzioni dei due modelli viene generalmente realizzato tramite la *Kullback-Leibler Divergence*;
- **Direct Output Matching:** In questo caso lo Student non imita le probabilità finali, ma direttamente i logits prodotti dal Teacher, ossia i valori grezzi in uscita dall'ultimo livello della rete. La corrispondenza viene ottenuta minimizzando una misura di distanza, tipicamente l'MSE, tra i vettori dei logits. Questo metodo è particolarmente diffuso nei problemi di Regressione, nei modelli linguistici di grandi dimensioni e negli scenari in cui non si desidera introdurre il parametro di temperatura;
- **Intermediate Feature Matching:** Oltre a replicare l'output finale, è possibile richiedere allo Student di riprodurre le rappresentazioni interne apprese dal Teacher, confrontando le attivazioni degli hidden layer. Poiché Teacher e Student spesso hanno dimensioni differenti, si introducono livelli di proiezione temporanei che trasformano le caratteristiche dello Student nello stesso spazio dimensionale del Teacher. L'errore tra le rappresentazioni viene quindi minimizzato durante l'addestramento, mentre i livelli di proiezione vengono rimossi al termine del processo;
- **Input Gradient Matching:** Questa famiglia di tecniche mira a trasferire non soltanto le risposte del Teacher, ma anche il suo comportamento rispetto agli input. Lo Student viene addestrato a produrre mappe di attenzione o gradienti simili a quelli del Teacher, imparando così a concentrarsi sulle stesse regioni informative dei dati. Approcci più avanzati estendono il confronto alla geometria delle rappresentazioni interne, ad esempio imponendo allo Student di replicare la matrice Jacobiana del Teacher. Ciò favorisce una maggiore robustezza e una risposta più coerente alle variazioni degli input.

L'addestramento dello studente è in genere guidato da una Loss Function composita. Lo Student deve contemporaneamente accontentare i dati reali e l'Insegnante:

- **Student Loss:** La normale *Cross-Entropy* calcolata tra l'output dello Student ($T = 1$) e le etichette reali (*Ground Truth*) del dataset. Assicura che lo Student apprenda correttamente il task supervisionato e rimanga allineato ai dati di addestramento;
- **Distillation Loss:** La *Kullback-Leibler Divergence* calcolata tra l'output dello Student "ammorbidito" (con Temperatura T) e l'output del Teacher "ammorbidito" (con Temperatura T). Costringe lo studente a copiare i ragionamenti sfumati del Teacher.

La Loss finale è una somma pesata tramite un parametro α (che decide quanta importanza dare al Teacher rispetto ai dati reali):

$$\mathcal{L}_{total} = (1 - \alpha) \cdot \mathcal{L}_{Student} + \alpha \cdot T^2 \cdot \mathcal{L}_{Distillation}$$

La Distillation Loss viene moltiplicata per T^2 per bilanciare i gradienti, poiché dividere i logits per T riduce l'ampiezza delle derivate.

Per la Regressione, una strategia consiste nel combinare la Loss standard del task con le previsioni raffinate del Teacher:

$$\mathcal{L}_{total} = \alpha \cdot \mathcal{L}_{task} + \beta \cdot \mathcal{L}_{HP}$$

Dove:

- $\mathcal{L}_{task}$ è l'MSE tra le previsioni dello student e la ground truth;
- $\mathcal{L}_{HP}$ è l'MSE che spinge le previsioni dello student a corrispondere alle previsioni raffinate del teacher, detta High Precision Loss;
- α e β sono due iperparametri per bilanciare l'importanza delle due loss.

8.5 Forward Automatic Differentiation

Forward Mode e Reverse Mode

L'**Automatic Differentiation (AD)** è una tecnica computazionale essenziale per calcolare le derivate delle funzioni definite da programmi informatici. L'idea di base dell'AD è che qualsiasi programma, per quanto complesso, esegue in realtà una sequenza di operazioni aritmetiche di base (addizione, sottrazione, moltiplicazione, divisione) e funzioni matematiche elementari (esponenziali, logaritmi, funzioni trigonometriche).

Esistono due modalità principali di AD:

- **Reverse Mode AD:** Più efficiente per le "Encoder networks" (molti input, pochi output);
- **Forward Mode AD (FM-AD):** Più efficiente per le "Decoder networks" (pochi input, molti output).

Tendenzialmente è più utilizzata la Reverse Mode AD, ma noi ci concentreremo sulla Forward Mode AD implementata tramite i Numeri Duali.

Numeri Duali

I **Numeri Duali** sono un'estensione dell'insieme dei numeri Reali ($\mathbb{R}$). Matematicamente, un numero duale z si esprime nella forma:

$$z = a + b\epsilon$$

Dove:

- a è la Parte Primale (un numero reale standard);
- b è la Parte Duale (anch'essa un numero reale);
- ϵ (epsilon) è l'unità infinitesimale.

La proprietà fondamentale e rivoluzionaria dell'unità ϵ è che è nilpotente: il suo quadrato è pari a zero ($\epsilon^2 = 0$), pur essendo $\epsilon \neq 0$. Questa definizione è concettualmente simile all'aggiunta dell'unità immaginaria i nei numeri complessi, dove $i^2 = -1$.

Dati due numeri duali $z_1 = a_1 + b_1\epsilon$ e $z_2 = a_2 + b_2\epsilon$:

- **Addizione:** $(a_1 + a_2) + (b_1 + b_2)\epsilon$;
- **Sottrazione:** $(a_1 - a_2) + (b_1 - b_2)\epsilon$;
- **Moltiplicazione:** $(a_1 a_2) + (a_1 b_2 + a_2 b_1)\epsilon$. Nota che il termine $b_1 b_2 \epsilon^2$ scompare grazie alla regola $\epsilon^2 = 0$;
- **Divisione:** $\frac{a_1}{a_2} + \left(\frac{a_2 b_1 - a_1 b_2}{a_2^2} \right) \epsilon$ (definita solo se $a_2 \neq 0$). Da qui si evince che i numeri duali puri (con parte reale nulla) non sono invertibili.

Automatic Differentiation

I numeri duali hanno una connessione profonda con lo **sviluppo in serie di Taylor**. Data una funzione reale generica e analitica $f(x)$, se estendiamo il suo dominio ai numeri duali calcolando $f(a + b\epsilon)$, lo sviluppo di Taylor risulta:

$$f(a + b\epsilon) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (b\epsilon)^n$$

Poiché per definizione $\epsilon^2 = 0, \epsilon^3 = 0$, ecc., tutti i termini della serie dal secondo grado in poi si annullano istantaneamente. L'equazione si semplifica drasticamente in:

$$f(a + b\epsilon) = f(a) + b f'(a) \epsilon$$

Se impostiamo la parte duale $b = 1$, calcolando la funzione sul numero duale $x + \epsilon$, otteniamo:

$$f(x + \epsilon) = f(x) + f'(x) \epsilon$$

Questo significa che, eseguendo la funzione standard su un input duale, il risultato conterrà:

- **Nella Parte Primale:** il valore della funzione valutata nel punto x ;
- **Nella Parte Duale:** il valore esatto della derivata della funzione nel punto x .

La derivata verrebbe così calcolata "*automaticamente*" **al solo costo di un'operazione matematica duale**.

Tuttavia, funzioni come e^x , necessarie per la **Sigmoide** standard $S(x) = \frac{1}{1+e^{-x}}$, richiederebbero un'implementazione complessa dell'overloading per i numeri duali basandosi sull'espansione di Taylor. Inoltre c'è un problema anche con il **valore assoluto**, che è una funzione fondamentale per creare attivazioni non lineari, ma non è differenziabile in zero e non può essere espressa con la semplice aritmetica di base.

Per risolvere il problema del valore assoluto, questo viene esteso al dominio duale in modo coerente con il concetto che la sua derivata è la funzione sign:

$$|z| = |a| + b \cdot \text{sign}(a)\epsilon$$

Avendo definito il valore assoluto, si può aggirare la complessità della Sigmoide classica utilizzandone una versione approssimata che richiede solo addizioni, sottrazioni, divisioni e il valore assoluto:

$$S_\epsilon(x) = \frac{1}{2} \left[\frac{x}{1 + |x|} + 1 \right]$$

Questa **sigmoide approssimata**:

- Non richiede di calcolare esponenziali duali;
- Offre un'accuratezza comparabile all'originale;
- Ha una derivata che decresce verso lo zero più lentamente rispetto all'originale. Questa proprietà rende la rete neurale moderatamente più robusta al problema del Vanishing Gradient, in quanto favorisce una propagazione dell'errore più consistente in Backpropagation.

Eseguendo il Forward Pass fornendo input duali, la rete calcola contemporaneamente sia l'attivazione dei neuroni (parte primale) sia la loro derivata (parte duale). Questo valore derivato può essere salvato e riutilizzato gratuitamente durante il Backward Pass, evitando calcoli dispendiosi aggiuntivi.

Abbiamo quindi dimostrato che il calcolo delle derivate tramite numeri duali può essere utilizzato nell'addestramento di un Multilayer Perceptron, con una precisione paragonabile all'implementazione originale.

8.6 Multi-Task Learning

Task

Un **task** (traducibile come "compito" o "problema") è lo specifico obiettivo pratico che il modello è chiamato a risolvere e su cui viene addestrato. In termini più formali e tecnici, un task è definito dalla relazione che si vuole mappare tra una serie di dati e un risultato atteso. È caratterizzato da tre elementi fondamentali:

- **Dati di Input:** Le informazioni di partenza che vengono fornite alla rete neurale (esempio: i pixel di un'immagine, le parole di un documento testuale, le misurazioni di un sensore);
- **Forma dell'Output:** La previsione, Classificazione o generazione che il modello deve produrre partendo dall'input (esempio: l'etichetta "gatto", il prezzo stimato di una casa, un testo tradotto);
- **Loss Function:** L'equazione matematica che valuta l'Accuracy del modello su quello specifico problema. Misura l'errore tra la previsione della rete e il risultato reale atteso, guidando l'addestramento.

Ad esempio, per la Computer Vision, un task può essere la Image Classification (potremmo identificare se un'immagine contiene un'auto, una moto o una bicicletta), o la Semantic Segmentation (colorare e separare ogni singolo pixel di un'immagine a seconda dell'oggetto a cui appartiene).

Invece nel Natural Language Processing possiamo trovare come task la Sentiment Analysis (stabilire se la recensione di un ristorante esprime un'emozione positiva, neutra o negativa), o la Machine Translation (ad esempio: tradurre un testo dall'italiano all'inglese).

Singolo modello con più task

Dati m learning task $\{\mathcal{T}_i\}_{i=1}^m$, il **Multi-Task Learning (MTL)** mira a migliorare l'addestramento di un modello per ciascun task $\mathcal{T}_i$ utilizzando la conoscenza condivisa tra tutti o alcuni degli altri task.

Si basa sull'idea che task correlati presentino spesso strutture o rappresentazioni di feature sottostanti comuni. A differenza del *Single-Task Learning*, che prevede l'addestramento di un modello indipendente per ciascun problema, il Multi-Task Learning cerca di unificare l'apprendimento per trarre vantaggio da queste comunanze.

Addestrare un modello su più task simultaneamente offre diversi benefici:

- **Migliore Generalizzazione:** Il processo funge da tecnica di regolarizzazione implicita. Questo riduce il rischio di overfitting su un singolo task, rendendo il modello più robusto quando incontra nuovi dati;
- **Efficienza e Velocità:** Il trasferimento di conoscenza tra task può accelerare la convergenza durante l'addestramento. Un modello condiviso risulta inoltre più compatto in termini di parametri e meno dispendioso computazionalmente rispetto a molti modelli separati;
- **Gestione di dati eterogenei:** Il MTL è in grado di operare con input di diversa natura (immagini, testo, audio), gestendo spazi di feature variabili tramite rappresentazioni condivise.

Il **Transfer Learning** mira principalmente a migliorare le prestazioni per uno specifico task target. Ciò si ottiene sfruttando le conoscenze acquisite da uno o più task sorgente diversi ma correlati, per i quali è stato pre-addestrato un modello.

Tuttavia il Multi-Task Learning è diverso dal Transfer Learning, che è a tutti gli effetti una **tecnica di Fine-Tuning**. Invece il Multi-Task Learning è un vero e proprio metodo di addestramento.

Quando si apprendono più compiti insieme, tre domande fondamentali guidano la condivisione efficace delle conoscenze:

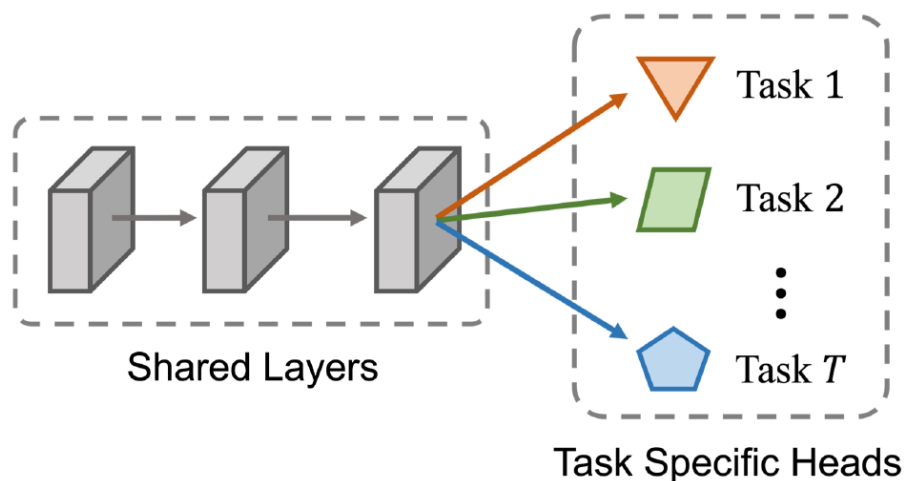
- **Quando condividere?** La scelta tra MTL e modelli separati dipende dal grado di correlazione dei task. Il Multi-Task Learning offre vantaggi massimi quando i task mostrano una forte sinergia reciproca. In assenza di essa, i benefici possono annullarsi;
- **Cosa condividere?** Si possono condividere le *Feature* (selezionando o trasformando l'input in modo che sia utile a più task), i *Parametri del modello* (usando la regolarizzazione per forzare similitudini strutturali) oppure le *Istanze* (pesando diversamente specifici dati di un task in base a quanto sono rilevanti per un altro);
- **Come condividere?** Avviene principalmente tramite la *condivisione di layer* (solitamente i primi del network) per apprendere rappresentazioni comuni, o tramite l'uso di architetture che favoriscono lo scambio esplicito di informazioni tra task.

Non esiste un'unica architettura MTL adatta a ogni scenario. Ogni strategia deve essere adattata alla specifica combinazione di attività, al volume e alla diversità dei dati disponibili e alle risorse

computazionali. Quando le attività mostrano una sinergia reciproca, la condivisione di parametri rigidi può produrre rappresentazioni congiunte efficaci, ma per obiettivi più eterogenei o contrastanti, i meccanismi di condivisione flessibile potrebbero essere d'aiuto. Le tre strategie principali sono:

- **Hard Parameter Sharing:** condivisione dei livelli inferiori tra tutti task con livelli di output specifici per ciascun task;
- **Soft Parameter Sharing:** ogni task ha il proprio modello, ma c'è una regolarizzazione delle differenze tra i parametri;
- **Hybrid Models:** combinazione di livelli condivisi con ramificazioni specifiche per ciascun task o meccanismi di attenzione.

Hard Parameter Sharing

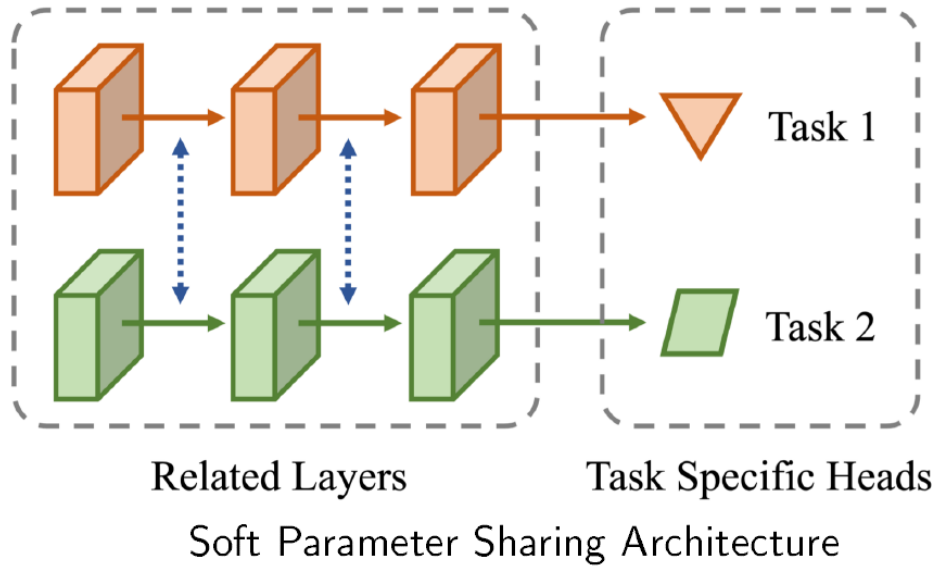


Hard Parameter Sharing Architecture

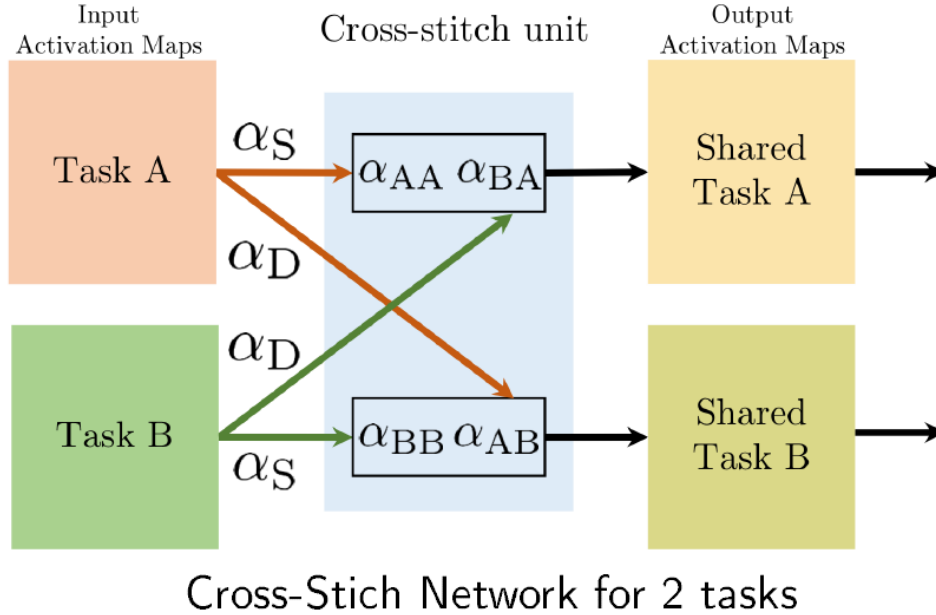
Nell'**Hard Parameter Sharing** i layer iniziali del modello (il "**backbone**") sono identici e condivisi da tutti i task per imparare feature di carattere generale. Successivamente, la rete si dirama in "**head**" specifiche per gestire le peculiarità finali di ogni task. È l'approccio più comune e storicamente consolidato nel Deep Learning per il multi-tasking. Si assume che i task siano fortemente sinergici e correlati. Di conseguenza, le rappresentazioni apprese dai primi livelli della rete possono essere utili a tutti i compiti simultaneamente.

Ha una altissima efficienza computazionale e un discreto risparmio di memoria, poiché gran parte della rete è condivisa. Inoltre ha anche un forte effetto di regolarizzazione implicita, che riduce drasticamente il rischio di overfitting sul singolo task. Tuttavia si rischia di fallire o di causare un trasferimento negativo (negative transfer) se i task hanno obiettivi contrastanti o se i dati sono troppo eterogenei.

Soft Parameter Sharing



A differenza dell'Hard Sharing, il **Soft Parameter Sharing** non forza i task a condividere rigidamente gli stessi identici layer matematici, prediligendo la flessibilità. Ogni task mantiene il proprio modello separato con un set distinto di parametri. **La condivisione della conoscenza avviene, tipicamente, tramite meccanismi di regolarizzazione che penalizzano l'eccessiva distanza tra i pesi dei modelli, oppure tramite layer di comunicazione espliciti.**



Un esempio di Soft Parameter Sharing Neural Network sono le **Cross-Stitch Networks**. In questa architettura, i task avanzano in parallelo tramite stream specifici. Periodicamente, vengono inserite delle unità **cross-stitch** (*unità di "intreccio"*) che combinano linearmente le mappe di attivazione dei due stream per calcolare le attivazioni successive ($\tilde{x}_A$ e $\tilde{x}_B$). Matematicamente, per due task (A e B), l'operazione è regolata da una matrice 2×2 di pesi apprendibili (α):

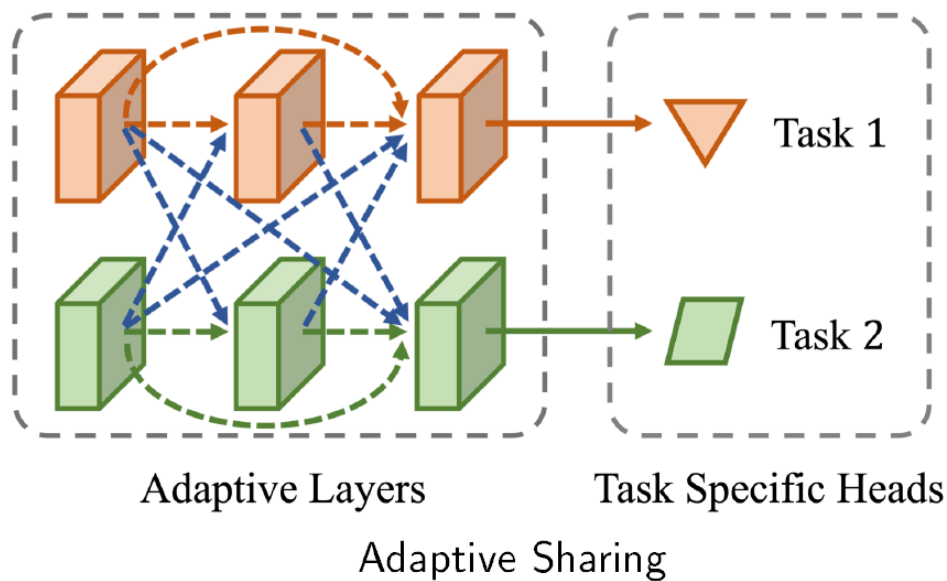
$$\begin{bmatrix} \tilde{x}_A \\ \tilde{x}_B \end{bmatrix} = \begin{bmatrix} \alpha_{AA} & \alpha_{AB} \\ \alpha_{BA} & \alpha_{BB} \end{bmatrix} \begin{bmatrix} x_A \\ x_B \end{bmatrix}$$

Dove:

- I termini sulla diagonale (α_{AA}, α_{BB}) controllano quanto il task si affida alle sue stesse feature (*same-task flow*);
- I termini fuori diagonale (α_{AB}, α_{BA}) determinano l'influenza delle feature dell'altro task (*cross-task flow*).

La rete impara autonomamente se un livello deve essere altamente specifico (portando i valori fuori diagonale vicino a zero) o se deve favorire la condivisione. I pesi vengono solitamente inizializzati tra $[0; 1]$ tramite Softmax per garantire stabilità.

Hybrid Architectures



Le **Architetture Ibride e Adattive** permettono di trovare un equilibrio apprendendo dinamicamente cosa e quanto condividere. Possono utilizzare meccanismi di attenzione o condividere solo sottoinsiemi molto specifici di layer. Rappresentano la frontiera più avanzata, combinando elementi rigidi (hard) e flessibili (soft) per permettere al modello stesso di decidere la topologia di condivisione ottimale.

Ha due strategie principali:

- **Partial Hard Sharing:** Invece di condividere un intero blocco profondo, il modello condivide rigidamente solo un piccolo sottoinsieme di layer iniziali (quelli dedicati alle feature di primissimo livello, come i bordi in un'immagine), lasciando i layer successivi parzialmente indipendenti o governati da altre logiche;
- **Learned Sharing:** Vengono integrati componenti dinamici intelligenti per decidere l'estensione e la natura dello scambio di informazioni. Tra questi spiccano i Meccanismi di Attenzione e i modelli *Mixture-of-Experts* (MoE) con reti di *Gating*. Nelle MoE, diversi blocchi di layer ("esperti") si specializzano su sottomansioni o sotto-pattern, e una rete di gating decide dinamicamente quali esperti attivare e combinare per un determinato input e per uno specifico task.

Offrono la massima adattabilità possibile, massimizzando il guadagno derivante dalla cooperazione tra task e azzerando quasi completamente l'interferenza distruttiva grazie alla specializzazione dinamica.

Approccio di Scalarizzazione Standard

Il fulcro delle strategie di addestramento nel Multi-Task Learning risiede nella necessità di **ottimizzare contemporaneamente molteplici Loss Function**, bilanciando i requisiti e le dinamiche di apprendimento di ogni singolo task. Il metodo più diffuso per affrontare i problemi MTL è l'**approccio di scalarizzazione**, che consiste nel formulare l'ottimizzazione complessiva come una combinazione lineare delle funzioni di perdita dei singoli task.

La loss totale viene espressa matematicamente come:

$$L_{total} = \sum_{i=1}^T \lambda_i L_i$$

Dove:

- T indica il numero totale di task eseguiti in parallelo;
- I parametri $\{L_i\}_{i=1}^T$ rappresentano le Loss Function specifiche per ogni rispettivo problema;
- I coefficienti $\{\lambda_i\}_{i=1}^T$ sono pesi assegnati a ciascun task per codificare l'importanza o le preferenze del sistema rispetto alle diverse mansioni.

I parametri del modello (θ) vengono aggiornati mediante la Gradient Descent seguendo la regola: $\theta \leftarrow \theta + \eta d$. In questo contesto, η rappresenta il learning rate, mentre la direzione di ricerca d è tipicamente definita come la somma pesata dei gradienti negativi delle singole loss: $d = -\sum_{i=1}^T \lambda_i \nabla L_i(\theta)$.

Questa formulazione lineare introduce due sfide fondamentali ed estremamente critiche per la stabilità dell'addestramento: il **problema dei gradienti dominanti** e il **problema dei gradienti in conflitto**.

Il **Problema dei Gradienti Dominanti** si manifesta quando la norma geometrica dei gradienti associati ad alcune funzioni di perdita è significativamente più grande rispetto a quella degli altri task. Di conseguenza, la direzione di aggiornamento combinata d risulta fortemente polarizzata e distorta a favore esclusivo dei task che possiedono i gradienti con la norma maggiore.

Il **Problema dei Gradienti in Conflitto**, invece, emerge nel momento in cui l'aggiornamento dei parametri finalizzato a ottenere un miglioramento delle prestazioni su un determinato task causa, come effetto collaterale, un degradamento o un peggioramento delle performance di un altro task concorrente.

Per risolvere questi problemi si utilizzano due macro-categorie di soluzioni: le tecniche di **Gradient Correction** e i metodi di **Dynamic Weighting**.

Tecniche di Gradient Correction

L'**algoritmo PCGrad** è progettato per mitigare il trasferimento negativo di conoscenza intervenendo direttamente sulla geometria dei gradienti in conflitto. L'obiettivo fondamentale è rimuovere le componenti di aggiornamento di un task che lavorano attivamente contro i requisiti di un altro. Gli step sono i seguenti:

1. **Rilevamento del Conflitto:** Un conflitto tra il gradiente g_i del task i e il gradiente g_j del task j (calcolati rispetto ai parametri condivisi del network) viene formalmente identificato se il loro prodotto scalare è negativo, ovvero quando: $g_i^T g_j < 0$;
2. **Proiezione del Gradiente:** Se viene rilevato un conflitto, la componente del gradiente g_i che si oppone direttamente a g_j viene eliminata. Questo processo si realizza proiettando matematicamente

g_i sul piano normale di g_j attraverso la formula:

$$g_i \leftarrow g_i - \frac{g_i^T g_j}{\|g_j\|_2^2} g_j$$

A livello algoritmico, il sistema calcola i gradienti individuali su un batch, inizializza le variabili corrette, esamina i task in ordine casuale per verificare la presenza di conflitti ed esegue la proiezione geometrica. La direzione di aggiornamento finale corrisponde alla sommatoria di tutti i gradienti così corretti.

Altre tecniche di correzione sono:

- **GradDrop:** Questa metodologia azzera in modo probabilistico gli elementi dei gradienti dei singoli task qualora i loro segni matematici risultino incoerenti con i segni degli elementi corrispondenti negli altri gradienti, forzando gli aggiornamenti solo verso direzioni di accordo generale;
- **CAGrad:** Configura un problema di ottimizzazione per individuare una direzione di aggiornamento comune che massimizzi il miglioramento minimo possibile tra tutti i task, minimizzando al contempo il conflitto globale;
- **Recon:** Identifica in una fase iniziale di addestramento i layer condivisi che esibiscono un tasso elevato di conflitto tra i gradienti dei task. Successivamente, converte in modo definitivo tali layer problematici in componenti specifiche e non condivise prima di rieseguire l'addestramento;
- **Adversarial Gradient Modulation:** Utilizza una rete discriminatrice ausiliaria per rendere i gradienti provenienti dai diversi decoder specifici statisticamente indistinguibili tra loro, allineandone implicitamente le traiettorie.

GradNorm per la Dynamic Weighting

GradNorm è un algoritmo che bilancia l'addestramento sintonizzando dinamicamente le ampiezze dei gradienti dei singoli task affinché questi si trovino su una scala comune e procedano a tassi di apprendimento simili.

L'algoritmo si basa sulle seguenti definizioni analitiche:

- θ_{sh} : rappresenta il sottoinsieme di pesi condivisi della rete a cui viene applicato il bilanciamento;
- $G_{\theta_{sh}}^{(i)}(t) = \|\nabla_{\theta_{sh}} \lambda_i(t) L_i(t)\|_2$: è la norma L_2 del gradiente della loss pesata del singolo task i rispetto ai parametri condivisi θ_{sh} al passo temporale t ;
- $\overline{G}_{\theta_{sh}}(t) = E_{task}[G_{\theta_{sh}}^{(i)}(t)]$: rappresenta la norma media dei gradienti calcolata su tutti i task al passo t .

Per valutare il ritmo di apprendimento si definisce il rapporto di perdita per il task i come: $\tilde{L}_i(t) = L_i(t)/L_i(0)$, dove $L_i(0)$ corrisponde alla loss iniziale del task. Valori inferiori di $\tilde{L}_i(t)$ indicano una velocità di addestramento maggiore. Il tasso relativo di apprendimento inverso viene quindi calcolato come: $r_i(t) = \tilde{L}_i(t)/E_{task}[\tilde{L}_i(t)]$.

Il nucleo operativo di GradNorm stabilisce che la norma del gradiente desiderata per un determinato task debba tendere al target: $\overline{G}_{\theta_{sh}}(t) \times [r_i(t)]^\alpha$. In questa formulazione, α è un iperparametro che definisce l'intensità della forza di ripristino per riportare i task a una velocità comune. Un valore di $\alpha = 0$ impone che i gradienti di tutti i task assumano una norma identica e pari alla media.

L'ottimizzazione dei pesi avviene minimizzando una loss ausiliaria L_{grad} definita come una perdita L_1 tra le norme effettive e i target stabiliti:

$$L_{grad}(t; \lambda_i(t)) = \sum_i |G_{\theta_{sh}}^{(i)}(t) - \overline{G}_{\theta_{sh}}(t) \times [r_i(t)]^\alpha|$$

Durante la derivazione di L_{grad} , il valore target viene trattato come una costante fissa e la differenziazione viene eseguita esclusivamente rispetto ai coefficienti λ_i . Subito dopo l'aggiornamento, i pesi $\lambda_i(t)$ vengono rinormalizzati affinché la loro sommatoria rispetti il vincolo $\sum_i \lambda_i(t) = T$, isolando il bilanciamento dall'effetto del learning rate globale.

Multi-Objective Optimization

In alternativa alla fusione delle loss in un unico scalare (Approccio di Scalarizzazione Standard), l'approccio di **Multi-Objective Optimization** tratta la perdita di ogni singolo task come un obiettivo matematico del tutto indipendente e distinto. Il problema viene formalizzato cercando di minimizzare simultaneamente un vettore di funzioni di costo: $\min_{\theta} \mathcal{L}(\theta) = (\mathcal{L}^{(1)}(\theta), \dots, \mathcal{L}^{(T)}(\theta))^T$ all'interno di un dominio ammissibile.

Conclusioni

Il Multi-Task Learning può offrire vantaggi significativi in termini di efficienza computazionale e generalizzazione del modello, principalmente a causa del suo effetto di regolarizzazione implicito. Questo effetto si verifica quando il modello impara a svolgere diversi compiti contemporaneamente, limitando così lo spazio delle ipotesi condivise, che può essere utile anche quando i compiti non mostrano una sinergia reciproca. Tuttavia, le applicazioni pratiche e le prove empiriche, comprese le osservazioni tratte da tutorial illustrativi, evidenziano che MTL non è una soluzione universalmente ottimale e il suo successo non è garantito. Infatti, in determinate condizioni, in particolare con compiti non corrispondenti o strategie di condivisione non ottimali, MTL può produrre risultati contrari ai miglioramenti desiderati.

Parte II

Laboratorio

Capitolo 9: PyTorch

9.1 Autograd

Un problema onnipresente nel **Machine Learning** è la necessità di trovare il valore ottimale dei parametri w di un modello per minimizzare una Loss Function $\mathcal{L}$. Per fare ciò, utilizziamo metodi di ottimizzazione come la Stochastic Gradient Descent (SGD), la cui regola di aggiornamento prevede di fare dei piccoli passi nella direzione opposta a quella del gradiente:

$$w_{t+1} = w_t + \eta \left(-\frac{\partial \mathcal{L}}{\partial w_t} \right)$$

Il cuore del problema computazionale è: come calcoliamo in modo efficiente questo gradiente $\frac{\partial \mathcal{L}}{\partial w}$ per una Deep Neural Network che possiede milioni o addirittura miliardi di parametri?

Prima dell'introduzione dei moderni framework, il calcolo delle derivate presentava notevoli difficoltà. La **derivazione analitica** e l'**implementazione manuale** delle espressioni dei gradienti risultano impraticabili per modelli complessi, oltre a essere estremamente soggette a errori. Inoltre, l'utilizzo di sistemi di algebra computazionale per ottenere la forma esplicita delle derivate conduce spesso al fenomeno noto come *expression swell*, per cui la complessità delle espressioni cresce rapidamente fino a renderne inefficiente la memorizzazione e la manipolazione.

Un'alternativa consiste nell'**approssimare la derivata mediante il rapporto incrementale**:

$$\frac{f(x+h) - f(x)}{h}$$

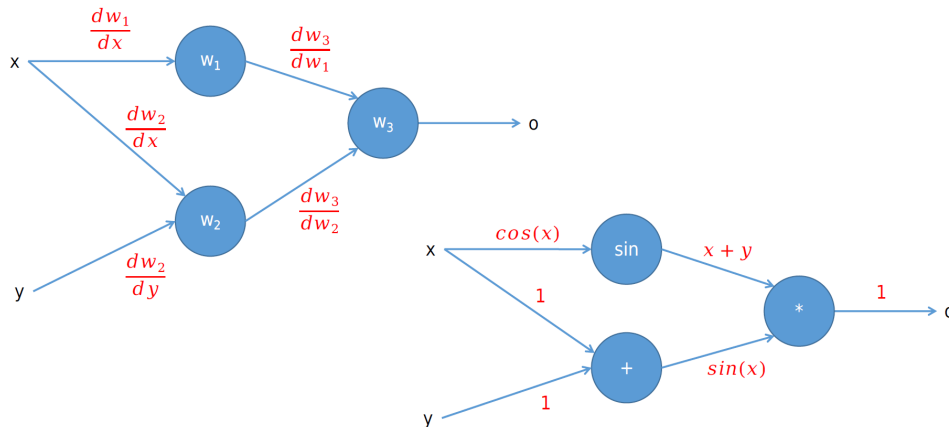
Tale approccio, tuttavia, richiede di ricalcolare l'intera rete in avanti due volte per ogni singolo parametro. Nel caso di modelli caratterizzati da milioni o miliardi di pesi, il costo computazionale diventa proibitivo. Inoltre, l'approssimazione è soggetta a errori numerici dovuti alla **rappresentazione in virgola mobile** (*floating-point*).

Autograd è il sistema di differenziazione automatica utilizzato da **PyTorch** per calcolare in modo efficiente i gradienti necessari durante l'addestramento delle Neural Network ed è la soluzione a questo problema.

Autograd si basa sulla rappresentazione di qualsiasi funzione matematica (per quanto complessa) sotto forma di un **Grafo Computazionale**. Si tratta di un **grafo diretto aciclico (DAG)** in cui:

- I nodi rappresentano le variabili (input, parametri) o le operazioni matematiche elementari (addizione, moltiplicazione, seno, logaritmo);
- Gli archi orientati rappresentano il flusso dei dati numerici da un'operazione all'altra.

Computational Graphs: Generalization



Scomponendo un'equazione immensa in una sequenza di operazioni base elementari, il framework sa sempre calcolare la derivata di ogni singolo minuscolo passaggio, poiché le regole di derivazione di somma e prodotto sono banali e note a priori.

La **Chain Rule** è il principio matematico alla base di Autograd. Essa consente di calcolare la derivata di una funzione composta scomponendo il problema in una sequenza di derivate locali. Se una variabile y dipende da u , che a sua volta dipende da x :

$$y = f(u) \quad u = g(x)$$

allora la derivata di y rispetto a x è:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

Esistono due modalità di attraversamento:

- **Forward Mode:** Calcola la derivata di tutti i possibili output rispetto a un singolo input alla volta. Si viaggia nel grafo da sinistra verso destra. Questa modalità è ideale per funzioni matematiche che hanno pochissimi input e tantissimi output ($f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ con $n \ll m$);
- **Reverse Mode:** Calcola le derivate di un singolo output rispetto a tutti gli input contemporaneamente. Si viaggia nel grafo a ritroso, da destra verso sinistra.

Le Loss Function hanno tipicamente questa forma: $\mathcal{L} : \mathbb{R}^n \rightarrow \mathbb{R}$. Abbiamo un numero spropositato di input (n , ossia i miliardi di pesi della rete) e un solo output scalare (l'errore calcolato, ad esempio l'MSE). Il Reverse Mode è l'approccio computazionalmente perfetto per questo scenario: con un solo "viaggio" all'indietro nel grafo, riusciamo a raccogliere in un colpo solo i gradienti di tutti i miliardi di parametri rispetto all'errore. Questo concetto coincide esattamente con l'algoritmo di **Backpropagation**.

In ambiente **PyTorch**, il processo diventa del tutto trasparente. È sufficiente istanziare i tensori di input attivando la flag di tracciamento (`requires_grad=True`).

9.2 Tensori

Fino ad ora abbiamo visto la teoria dietro le Neural Network. Adesso invece passiamo a qualcosa di più pratico. Da qui si richiede di avere un minimo di dimestichezza con **Python** e la libreria **numpy**.

Un **tensore** è un array n -dimensionale i cui elementi hanno tutti lo stesso tipo di dato. Il valore n viene chiamato *rango* o *dimensione* del tensore.

Un tensore in **PyTorch** è molto simile a un array multidimensionale di **NumPy**: in entrambi i casi gli elementi sono omogenei e le rispettive librerie offrono funzioni e classi efficienti per manipolarli. La differenza principale è che i tensori di **PyTorch** possono eseguire le operazioni in modo trasparente sia su CPU sia su GPU. L'implementazione su GPU è molto efficiente e, inoltre, **PyTorch** consente di distribuire memorizzazione e calcolo su più CPU, GPU e persino su macchine diverse.

Di solito i tensori contengono numeri reali, per questo motivo è naturale riferirsi ai tensori di rango 0, 1 e 2 come oggetti matematici:

- un tensore a 0 dimensioni è uno *scalare*;
- un tensore a 1 dimensione è un *vettore*;
- un tensore a 2 dimensioni è una *matrice*;
- un tensore a 3 dimensioni può essere visto come un vettore di matrici tutte della stessa forma;
- un tensore a 4 dimensioni può essere visto come una matrice di matrici tutte della stessa forma, oppure come un vettore di tensori di rango 3.

Un tensore è definito dal tipo dei suoi elementi e dalla sua *shape*, cioè dal numero di elementi lungo ogni dimensione. Per esempio, un'immagine RGB con h righe e w colonne di pixel può essere rappresentata come un tensore a 3 dimensioni. Una sequenza di b immagini può essere rappresentata come un tensore a 4 dimensioni.

Tutti i tensori derivano dalla classe base `torch.Tensor`. Essi contengono sia i dati sia alcune informazioni di stato interne.

Per usare **PyTorch** bisogna importare il pacchetto corrispondente:

```
1 import torch
```

Creazione

I tensori possono essere creati a partire da strutture dati già esistenti, come tuple e liste. I tensori creati con `torch.tensor()` fanno sempre una copia dei dati di input. Per convertire un tensore `w` in una lista Python si usa il metodo `w.tolist()`. Per convertire un tensore a 0 dimensioni `w` in un valore scalare Python si usa il metodo `w.item()`.

I tensori possono essere creati anche tramite funzioni dedicate, specificandone la forma mediante un intero, una tupla o una lista. Le più comuni sono riportate di seguito.

Funzione	Descrizione
<code>torch.empty(size)</code>	Crea un tensore con elementi non inizializzati.
<code>torch.zeros(size)</code>	Crea un tensore con elementi impostati a 0.0.
<code>torch.ones(size)</code>	Crea un tensore con elementi impostati a 1.0.
<code>torch.full(size, fill_value)</code>	Crea un tensore con elementi impostati a <code>fill_value</code> .

A volte un tensore deve contenere una sequenza di numeri, per esempio tutti gli interi da 1 a 10. In questi casi si può specificare l'inizio dell'intervallo, la fine dell'intervallo e il passo tra due valori consecutivi.

PyTorch offre funzioni per creare tensori inizializzati con dati casuali. Le funzioni più comuni per creare tensori con valori casuali sono le seguenti.

Funzione	Descrizione
<code>torch.rand(size)</code>	Estrae valori da una distribuzione uniforme in $[0; 1]$.
<code>torch.randn(size)</code>	Estrae valori da una distribuzione normale con media 0 e varianza 1.
<code>torch.normal(mean, std, size)</code>	Estrae valori da una distribuzione normale con media e varianza specificate.
<code>torch.randint(low=0, high, size)</code>	Estrae interi tramite campionamento uniforme negli estremi indicati.
<code>torch.bernoulli(input)</code>	Estrae valori binari da una distribuzione di Bernoulli con probabilità data.

Quando si lavora con la casualità è importante poter riprodurre i comportamenti casuali. In altre parole, bisogna poter ottenere gli stessi risultati, o risultati molto simili, su computer diversi. La funzione `torch.manual_seed(seed)` serve proprio a questo. Per le operazioni in Python e NumPy si consiglia di impostare anche i rispettivi semi di generazione casuale.

In alcuni casi è utile creare e inizializzare un tensore con proprietà simili a quelle di un altro tensore, inclusi shape, tipo di dato e dispositivo, così da facilitare i calcoli. Molte delle funzioni di creazione a valore costante viste sopra hanno una corrispondente variante con suffisso `_like()`, che rende l'operazione molto semplice. Per esempio, `torch.empty_like(w)` crea un tensore vuoto con la stessa shape di `w`.

Dopo la creazione, possiamo ottenere rapidamente informazioni su un tensore `x` accedendo ai suoi attributi:

- `x.dtype`: restituisce il tipo di dato dei suoi elementi;
- `x.device`: restituisce la risorsa su cui è memorizzato il tensore, cioè CPU o GPU;
- `x.ndim`: restituisce il numero di dimensioni del tensore, cioè il suo rango;
- `x.shape`: restituisce la lunghezza di ciascuna dimensione sotto forma di tupla.

PyTorch offre anche funzioni per creare nuovi tensori unendo, in un unico tensore, sequenze di due o più tensori. Le funzioni più comuni sono riportate di seguito.

Funzione	Descrizione
<code>torch.cat(seq, dim=0)</code>	Concatena la sequenza di tensori <code>seq</code> lungo una dimensione <i>esistente</i> <code>dim</code> .
<code>torch.concat(seq, dim=0)</code>	Alias di <code>torch.cat()</code> .
<code>torch.concatenate(seq, dim=0)</code>	Alias di <code>torch.cat()</code> .
<code>torch.stack(seq, dim=0)</code>	Concatena la sequenza di tensori <code>seq</code> lungo una <i>nuova</i> dimensione <code>dim</code> .
<code>torch.vstack(seq)</code>	Alias di <code>torch.stack()</code> lungo il primo asse (per righe).
<code>torch.hstack(seq)</code>	Alias di <code>torch.stack()</code> lungo il secondo asse (per colonne).

Tipi di Dato

Tutti gli elementi di un tensore hanno lo stesso tipo di dato. La tabella seguente riporta i tipi di dato più comuni disponibili in PyTorch e il relativo `dtype`.

Tipo	dtype
Floating point a 32 bit (predefinito)	<code>torch.float32</code> , <code>torch.float</code>
Floating point a 64 bit	<code>torch.float64</code> , <code>torch.double</code>
Floating point a 16 bit	<code>torch.float16</code> , <code>torch.half</code>
Complesso a 32 bit	<code>torch.complex32</code> , <code>torch.chalf</code>
Complesso a 64 bit	<code>torch.complex64</code> , <code>torch.cfloat</code>
Complesso a 128 bit	<code>torch.complex128</code> , <code>torch.cdouble</code>
Intero senza segno a 8 bit	<code>torch.uint8</code>
Intero con segno a 8 bit	<code>torch.int8</code>
Intero con segno a 16 bit	<code>torch.int16</code> , <code>torch.short</code>
Intero con segno a 32 bit	<code>torch.int32</code> , <code>torch.int</code>
Intero con segno a 64 bit	<code>torch.int64</code> , <code>torch.long</code>
Booleano	<code>torch.bool</code>

È possibile specificare il tipo di dato al momento della creazione del tensore tramite il parametro `dtype`. Se non viene specificato, il tipo viene dedotto dai dati di input quando possibile. In caso contrario si usa il tipo predefinito `torch.float32`. Una volta creato, il tipo di dato di un tensore si ottiene con l'attributo `w.dtype`.

PyTorch offre funzionalità di interoperabilità con NumPy. Anche `torch.tensor()` può essere usato per creare tensori a partire da array NumPy.

Serializzazione

Un tensore può essere facilmente salvato su file e caricato da file tramite `torch.save()` e `torch.load()`. Entrambe le funzioni usano di default il pacchetto Pickle e i file PyTorch usano le estensioni `.pt` o `.pth`.

CUDA

CUDA (*Compute Unified Device Architecture*) è una piattaforma di calcolo parallelo e un modello di programmazione sviluppato da *NVIDIA* che permette di usare la GPU non solo per la grafica, ma anche per eseguire calcoli generici ad alte prestazioni.

Per impostazione predefinita un tensore viene creato sulla CPU. Usando il parametro `device` è possibile creare un tensore sulla GPU. Una volta creato, il dispositivo su cui è memorizzato si ottiene con l'attributo `w.device`.

Per verificare se PyTorch ha accesso a una GPU si usa la funzione `torch.cuda.is_available()`. Se restituisce `True`, PyTorch può vedere e usare la o le GPU, altrimenti restituisce `False`.

Indicizzazione

I tensori possono essere **indicizzati** con la sintassi standard `x[obj]`, dove `x` è il tensore e `obj` è l'oggetto di selezione. A seconda di `obj` si distinguono diversi tipi di indicizzazione, come l'indicizzazione di base e quella avanzata.

Il caso del **singolo elemento** si verifica quando `obj` è un intero o una tupla di interi. In Python, `x[(exp1, exp2, ..., expN)]` è equivalente a `x[exp1, exp2, ..., expN]`. La seconda forma è solo una scorciatoia sintattica della prima. L'indicizzazione di singoli elementi funziona come per le sequenze Python standard: si parte da 0 e sono ammessi indici negativi per contare dalla fine del tensore. Se un tensore multidimensionale viene accesso con meno indici rispetto alle sue dimensioni, si ottiene un tensore di dimensione inferiore. L'indicizzazione di un singolo elemento restituisce sempre un tensore, anche quando il risultato è un solo valore. Come visto in precedenza, per convertire un tensore a 0 dimensioni in un valore numerico Python occorre usare `x.item()`. In questo caso il tensore restituito è una vista, non una copia del tensore originale.

Il caso dello **Slicing** si verifica quando `obj` è uno *slice*. Un tensore può essere indicizzato per ottenere una porzione dei suoi valori usando la forma `[start:end:step]`, in modo analogo a liste e array NumPy. Le regole standard dello slicing valgono per ogni dimensione separatamente. L'ellissi `...` si espande nel numero di oggetti : necessari per costruire una tupla di selezione che indicizzi tutte le dimensioni. Nella maggior parte dei casi, ciò significa che la lunghezza della tupla espansa è `x.ndim`. Può comparire una sola ellissi. Il valore `None` può essere usato nella tupla di selezione per aggiungere al risultato una dimensione di lunghezza unitaria. La nuova dimensione viene inserita nella posizione occupata da `None` nella tupla. Tutti i tensori ottenuti tramite slicing sono sempre viste del tensore originale.

Il caso dell'**Indicizzazione Avanzata** si verifica quando `obj` è un tensore di tipo intero o booleano, oppure una tupla che contiene almeno un oggetto di tipo sequenza. L'accesso ai contenuti di un tensore tramite indicizzazione avanzata restituisce sempre una copia dei valori selezionati. Usando un tensore di interi come indice si possono accedere ai contenuti di un tensore in modo arbitrario, anche fuori ordine e con ripetizioni. L'array o tensore di indici può avere una shape arbitraria, il tensore risultante avrà la stessa shape. Per eseguire questo tipo di indicizzazione su un tensore n -dimensionale è necessario specificare n tensori indice, uno per ogni dimensione. I tensori indice devono avere tutti la stessa shape, e tale shape comune determina la shape del tensore risultante. Usando un tensore booleano come indice è possibile accedere ai contenuti di un tensore in modo arbitrario, in base ai valori dell'indice. Se `x` è un tensore n -dimensionale e `ind` è un tensore booleano della stessa shape di `x`, allora `x[ind]` restituisce un array monodimensionale costruito scorrendo `x` e `ind` in ordine row-major. Ogni volta che un elemento di `ind` è `True`, il corrispondente elemento di `x` viene aggiunto in coda al tensore risultante.

Oltre all'indicizzazione avanzata, PyTorch offre alcune funzioni per creare nuovi tensori a partire da valori selezionati da altri tensori. Le funzioni più comuni sono riassunte qui sotto.

Funzione	Descrizione
<code>torch.index_select(input, dim, index)</code>	Seleziona elementi lungo una dimensione.
<code>torch.masked_select(input, mask)</code>	Seleziona elementi tramite una maschera booleana.
<code>torch.nonzero(input)</code>	Restituisce gli indici degli elementi non nulli.
<code>torch.where(C, input, other)</code>	Seleziona elementi in base a una condizione.

9.3 Operazioni sui Tensori

PyTorch include un ampio numero di operazioni e funzioni matematiche sui tensori, tra cui operazioni elemento per elemento, funzioni di riduzione, operatori di confronto e operazioni di algebra lineare.

Operazioni elemento per elemento

Le operazioni *element-wise* eseguono un'operazione su ciascun elemento di uno o due tensori e restituiscono un nuovo tensore con lo stesso rango e le stesse dimensioni degli input, memorizzato sulla stessa risorsa. Due tensori coinvolti nella stessa operazione matematica devono essere memorizzati sulla stessa risorsa.

Le operazioni elemento per elemento più comuni sono elencate nella tabella seguente.

Categoria	Esempi
Matematiche	<code>add()</code> , <code>div()</code> , <code>mul()</code> , <code>sub()</code> , <code>neg()</code> , <code>true_divide()</code> , oppure <code>+</code> , <code>/</code> , <code>*</code> , <code>-</code>
Troncamento	<code>ceil()</code> , <code>floor()</code> , <code>round()</code> , <code>trunc()</code>
Numeri complessi	<code>abs()</code> , <code>angle()</code> , <code>conj()</code> , <code>imag()</code> , <code>real()</code>
Trigonometria	<code>acos()</code> , <code>asin()</code> , <code>atan()</code> , <code>cos()</code> , <code>cosh()</code> , <code>sin()</code> , <code>sinh()</code> , <code>tan()</code> , <code>tanh()</code>
Potenze e logaritmi	<code>exp()</code> , <code>expm1()</code> , <code>log()</code> , <code>log10()</code> , <code>log1p()</code> , <code>log2()</code> , <code>pow()</code> , <code>sqrt()</code>
Logiche	<code>logical_and()</code> , <code>logical_not()</code> , <code>logical_or()</code> , <code>logical_xor()</code>
Cumulative	<code>addcdiv()</code> , <code>addcmul()</code>
Bitwise	<code>bitwise_and()</code> , <code>bitwise_not()</code> , <code>bitwise_or()</code> , <code>bitwise_xor()</code>

La funzione `true_divide()` converte gli elementi dei tensori in valori floating point prima di eseguire la divisione, ed è quindi utile quando si dividono interi e si vogliono risultati di divisione corretti in virgola mobile.

La maggior parte delle operazioni sui tensori è disponibile in tre forme diverse. I tensori supportano l'*operator overloading*, quindi possiamo usare direttamente operatori come `z = x + y` al posto di `z = torch.add(x, y)`. Inoltre esiste la sintassi *in-place*, ottenuta aggiungendo il suffisso `_` all'operazione. L'operazione in-place non crea un nuovo tensore, ma sostituisce il contenuto del tensore su cui viene invocata. Per esempio, `y.add_(x)` somma i tensori `x` e `y` e memorizza il risultato in `y`.

Operazioni di riduzione

Le operazioni di riduzione *comprimono* un insieme di numeri in un singolo numero o in un insieme più piccolo. In altre parole, riducono il numero di dimensioni, o il rango, di un tensore. Queste operazioni includono funzioni per calcolare massimo e minimo, statistiche come media e varianza, e così via.

Le principali operazioni di riduzione disponibili in PyTorch sono le seguenti.

Funzione	Descrizione
<code>torch.all(x)</code>	Verifica se tutti gli elementi di <code>x</code> valutano a <code>True</code> .
<code>torch.any(x)</code>	Verifica se almeno un elemento di <code>x</code> valuta a <code>True</code> .
<code>torch.argmax(x)</code> , <code>torch.argmin(x)</code>	Restituiscono gli indici del massimo/minimo di tutti gli elementi di <code>x</code> .
<code>torch.max(x)</code> , <code>torch.min(x)</code>	Restituiscono il valore massimo/minimo di tutti gli elementi di <code>x</code> .
<code>torch.mean(x)</code>	Restituisce la media di tutti gli elementi di <code>x</code> .
<code>torch.median(x)</code>	Restituisce la mediana dei valori di <code>x</code> .
<code>torch.prod(x)</code>	Restituisce il prodotto di tutti gli elementi di <code>x</code> .
<code>torch.std(x)</code>	Restituisce la deviazione standard di tutti gli elementi di <code>x</code> .
<code>torch.sum(x)</code>	Restituisce la somma di tutti gli elementi di <code>x</code> .
<code>torch.unique(x)</code>	Restituisce gli elementi unici di <code>x</code> .
<code>torch.var(x)</code>	Restituisce la varianza di tutti gli elementi di <code>x</code> .
<code>torch.count_nonzero(x)</code>	Conta il numero di valori diversi da zero in <code>x</code> .

Molte operazioni di riduzione accettano il parametro opzionale `dim`, che specifica la dimensione su cui effettuare la riduzione per i tensori multidimensionali. Se il parametro non è presente, la riduzione avviene su tutte le dimensioni. Se invece è presente, il suo valore rappresenta la dimensione “da collassare”, cioè la dimensione lungo la quale i valori vengono accumulati. Per esempio, in un tensore a 2 dimensioni, `dim=0` indica di collassare tutte le righe insieme, quindi la riduzione avviene lungo le colonne. Analogamente, con `dim=1` si collassano le colonne e la riduzione avviene lungo le righe.

Operatori di confronto

Gli operatori di confronto confrontano in genere tutti gli elementi di un tensore, oppure gli elementi di un tensore rispetto a quelli di un altro tensore. Possono restituire un singolo valore booleano oppure un tensore di valori booleani.

Gli operatori e le funzioni di confronto più comuni sono riportati qui sotto.

Operazioni o funzioni	Esempi
Confronto tra tensori	<code>eq()</code> , <code>ne()</code> , <code>le()</code> , <code>ge()</code> , <code>lt()</code> , <code>gt()</code> , oppure <code>==</code> , <code>!=</code> , <code><=</code> , <code>>=</code> , <code><</code> , <code>></code>
Condizioni sui tensori	<code>isclose()</code> , <code>isfinite()</code> , <code>isinf()</code> , <code>isnan()</code>
Confronto su un tensore	<code>allclose()</code> , <code>equal()</code>
Ricerca di valori in un tensore	<code>argsort()</code> , <code>sort()</code> , <code>kthvalue()</code> , <code>topk()</code>

Le operazioni di confronto sono intuitive. La funzione `torch.eq()` e l'operatore `==` restituiscono un tensore con un elemento booleano per ciascun elemento del tensore confrontato. Al contrario, la funzione `torch.equal()` restituisce `True` se i tensori hanno le stesse dimensioni e se tutti gli elementi sono uguali elemento per elemento.

Operazioni di algebra lineare

Le operazioni di algebra lineare implementano in modo efficiente molte operazioni di matrice, estendendole ai tensori n -dimensionali.

Le operazioni di algebra lineare più comuni in **PyTorch** sono le seguenti.

Funzione	Descrizione
<code>torch.matmul()</code> , <code>@</code>	Calcola il prodotto matriciale di due tensori con broadcasting.
<code>torch.mm()</code>	Calcola il prodotto matriciale di due tensori senza broadcasting.
<code>torch.bmm(A, B)</code>	Calcola il prodotto batch di matrici 3D con lo stesso numero di matrici in A e B.
<code>torch.dot()</code>	Calcola il prodotto scalare di due tensori monodimensionali.

È importante ricordare che `A * B` calcola il prodotto elemento per elemento di A e B, mentre la moltiplicazione matriciale si ottiene con `A @ B` oppure `torch.matmul(A, B)`.

Broadcasting

NumPy ha introdotto il meccanismo di *broadcasting* per eseguire operazioni matematiche su array n -dimensionali con shape diverse. Molte operazioni di **PyTorch** lo supportano a loro volta. In breve, se un'operazione **PyTorch** supporta il broadcasting, i suoi argomenti possono essere espansi automaticamente fino ad avere la stessa shape, senza copiare fisicamente i dati.

Il broadcasting non può gestire qualunque combinazione di shape. Per esempio, un tensore di shape (7;5) è incompatibile con un tensore di shape (11;3). Provare ad aggiungere due tensori del genere produrrebbe un errore `ValueError`.

Per stabilire se due tensori sono *broadcast-compatible*, si allineano le loro shape facendo coincidere le dimensioni finali e si controlla che ogni coppia di dimensioni allineate soddisfi una delle seguenti condizioni:

1. le dimensioni allineate hanno la stessa ampiezza;
2. una delle due dimensioni ha ampiezza 1.

I due tensori sono broadcast-compatible se ogni coppia di dimensioni allineate soddisfa una di queste condizioni.

Se due tensori x e y sono broadcast-compatible, le dimensioni dei tensori broadcastati si calcolano così:

1. se il numero di dimensioni di x e y è diverso, si aggiungono tante dimensioni di ampiezza 1 al tensore più corto da rendere uguale il numero di dimensioni;
2. per ogni dimensione, la dimensione del risultato è il massimo tra le due dimensioni corrispondenti.

Quando un tensore viene broadcastato, viene trattato come se il suo contenuto fosse replicato lungo le dimensioni appropriate, in modo che la shape del nuovo tensore a dimensione superiore sia adatta all'operazione matematica che si sta eseguendo.

Si noti che le operazioni in-place non consentono di modificare la shape di un tensore come risultato del broadcasting.

Rimodellamento

La modifica dello **shape** di un tensore senza alterarne il contenuto si chiama operazione di **Rearrange**. La regola aurea di questa operazione è che **il numero totale di elementi deve rimanere lo stesso**: il prodotto delle dimensioni del tensore di partenza deve essere uguale al prodotto delle dimensioni del tensore di arrivo. Ad esempio, un tensore bidimensionale di forma (6; 2) contenente 12 elementi in totale può essere riorganizzato in un tensore di forma (3; 4), (2; 6) o persino (12; 1), ma non in un (5; 3).

Quando si cambia la forma di un tensore, bisogna stabilire con quale logica gli elementi originali vadano a riempire i nuovi "spazi". PyTorch utilizza l'algebra tensoriale **row-major** (ordinamento per righe). Questo significa che gli elementi vengono prelevati dal tensore originale e riposizionati nel nuovo tensore leggendoli rigorosamente da sinistra verso destra e dall'alto verso il basso, cominciando sempre dalla prima dimensione (le righe). L'ordine sequenziale e cronologico degli elementi viene quindi sempre mantenuto durante l'operazione di rearrange.

Il framework offre due metodi distinti per compiere il rimodellamento, ognuno con conseguenze diverse sulla gestione della memoria della macchina:

- **reshape**: Costruisce un nuovo tensore e provvede a riorganizzare fisicamente e materialmente gli elementi all'interno della memoria RAM o VRAM. Questo spostamento fisico rende l'operazione in sé più lenta, ma garantisce che l'accesso ai dati nei passaggi successivi sia estremamente veloce.
- **view**: Non tocca minimamente la disposizione in memoria dei dati originali, limitandosi ad alterare il "punto di vista" o l'indicizzazione con cui PyTorch andrà a leggere quegli stessi valori. Questa peculiarità rende il rearrange praticamente istantaneo, ma l'accesso in lettura durante i calcoli successivi potrebbe risultare leggermente penalizzato in termini di velocità.

Riduzione

Un'operazione di **Reduce** prende un gruppo di elementi all'interno di una specifica dimensione del tensore, li aggrega matematicamente e li "collassa" generando un unico nuovo elemento in uscita. L'effetto pratico è la riduzione della dimensionalità complessiva della struttura dati.

Immaginiamo di lavorare su un tensore bidimensionale di forma $(3, 4)$:

- Possiamo decidere di schiacciarlo verticalmente, riducendo la prima dimensione (le righe) e trasformando il blocco in un tensore "piatto" di forma $(1, 4)$;
- In alternativa, possiamo schiacciarlo orizzontalmente, riducendo la seconda dimensione (le colonne) per ottenere un vettore colonna di forma $(3, 1)$.

In ambiente **PyTorch**, l'asse lungo il quale si desidera applicare la contrazione viene specificato attraverso la keyword `dim`. Le operazioni di riduzione più diffuse messe a disposizione dalla libreria sono:

- **Max / Min**: Estraggono rispettivamente il valore massimo o minimo lungo l'asse selezionato;
- **Sum**: Somma aritmeticamente tutti i valori presenti nella dimensione;
- **Mean**: Calcola la media dei valori;
- **Std**: Calcola la deviazione standard per analizzare la dispersione dei dati.

A livello di codice, richiamare la funzione `a.mean(dim=0)` su una matrice a imporrà alla rete di calcolare la media colonna per colonna (poiché il comando dice di "collassare" la dimensione 0, ovvero le righe). Al contrario, `a.mean(dim=1)` calcolerà la media riga per riga annientando le colonne.

9.4 Dati

Il primo passo in qualsiasi applicazione di Neural Network in **PyTorch** è preparare i dati di input per l'elaborazione. Le Neural Network devono essere addestrate e testate su tensori. **PyTorch** offre classi predefinite per caricare e gestire diversi tipi di dati:

- **Dataset**: per definire come accedere ai dati da una sorgente e come pre-elaborarli;
- **Sampler**: per definire come selezionare i dati da un dataset e creare batch;
- **DataLoader**: per combinare un dataset e un meccanismo di selezione e iterare automaticamente su un insieme di batch.

Dataset

Un oggetto **Dataset** ha il compito di caricare i dati da una sorgente e trasformarli in tensori elaborabili da **PyTorch**.

PyTorch supporta due tipi di dataset:

- **map-style dataset**, in cui ogni campione può essere acceduto indipendentemente e in qualunque ordine;
- **iterable-style dataset**, adatti a dati letti in sequenza.

I **dataset di tipo map-style** sono particolarmente efficaci quando l'ordine dei dati non influenza il risultato, per esempio nel training con stochastic gradient descent. L'accesso casuale aumenta la varietà dei campioni visti durante l'addestramento e può migliorare la capacità di generalizzazione del modello.

I **dataset di tipo iterable-style** sono adatti a dati che vengono letti in sequenza, come stream, file molto grandi e server remoti, cioè in tutti i casi in cui l'accesso casuale è costoso.

Uno dei passaggi più importanti in una applicazione di rete neurale è dividere il dataset in training set e test set e, quando necessario, in validation set.

Ogni suddivisione del dataset ha uno scopo preciso:

- **training set:** la rete neurale apprende i propri parametri da questi dati;
- **validation set:** la rete neurale adatta gli iperparametri su questi dati;
- **test set:** la rete neurale viene valutata su questi dati per verificare ciò che ha imparato.

Quando si lavora con dati reali, il test set dovrebbe essere sempre mantenuto separato da tutti gli altri dati. Vogliamo che la rete impari sui dati di training e poi venga valutata sui dati di test per misurare quanto generalizza su esempi mai visti. I dati di validazione servono per più scopi, ad esempio per scegliere iperparametri come il learning rate o il numero di epoche, oppure per controllare l'overfitting.

Sampler

Per i dataset iterable-style, l'ordine di caricamento dei dati è completamente controllato dall'iterable definito dall'utente. Per i dataset map-style, PyTorch fornisce un meccanismo di campionamento per specificare la sequenza di indici o chiavi usati durante il caricamento dei dati.

Questo meccanismo è definito tramite una classe derivata dalla classe base **Sampler**. Ogni sampler deve implementare `__iter__()` per specificare come iterare sugli indici degli elementi del dataset e `__len__()` per restituire la lunghezza dell'iteratore generato.

Si può costruire un semplice batch sampler per creare un sampler deterministico che restituisce campioni di dimensione fissa. Il modulo `torch.utils.data` fornisce anche diverse classi di sampler:

- **SequentialSampler(dataset):** campiona gli elementi in ordine sequenziale, sempre nello stesso modo.
- **SubsetRandomSampler(indices):** campiona casualmente elementi da una lista specificata di indici, senza reinserimento.
- **RandomSampler(dataset, replacement=False, num_samples=None):** campiona elementi casualmente. Se `replacement=False`, i campioni vengono scelti dal dataset mescolato. Se `replacement=True`, è possibile specificare il numero di campioni da generare.
- **WeightedRandomSampler(weights, replacement=False, num_samples=None):** campiona elementi dall'insieme `[0, ..., len(weights)-1]` con probabilità determinate dai pesi.
- **BatchSampler(sampler, batch_size, drop_last):** avvolge un altro sampler per generare batch.

DataLoader

La classe `Dataset` restituisce un oggetto che incapsula i dati e le informazioni sui dati. La classe `Sampler` restituisce i dati stessi in un ordine predeterminato o casuale. La classe `DataLoader` combina `Dataset` e `Sampler` per restituire un iterabile. `Dataset` e `Sampler`, presi singolarmente, non sono iterabili e quindi non possono essere usati direttamente insieme in un ciclo `for`. Il `Dataloader` risolve questo problema.

Il prototipo semplificato del costruttore di un `Dataloader` è il seguente:

```
DataLoader(dataset, shuffle=False, sampler=None,
batch_size=1, drop_last=False, collate_fn=None,
batch_sampler=None)
```

La classe `DataLoader` rappresenta un iterabile sul dataset passato come argomento `Dataset`, sia esso map-style o iterable-style. Quando il dataset è di tipo map-style, il dataloader utilizza il sampler per determinare l'ordine in cui i campioni vengono recuperati. Se il dataset è di tipo iterable-style, il dataloader usa direttamente l'iteratore del dataset.

Dopo aver ottenuto una lista di campioni dal batch sampler, il dataloader usa la funzione passata nel parametro `collate_fn` per raccogliere i campioni e organizzarli in batch. Quando il batching automatico è disabilitato, cioè `batch_size=None`, la funzione `collate_fn` viene invocata su ogni singolo campione e si limita a convertire gli array di input in tensori `PyTorch`. Quando il batching automatico è attivo, `collate_fn` viene invocata su una lista di campioni alla volta. Per esempio, se ogni campione è composto da un'immagine a 3 canali e da un'etichetta di classificazione, cioè se ogni elemento del dataset è una tupla (`image`, `label`), la funzione `collate_fn` di default aggrega una lista di tali tuple in una singola tupla, composta da un tensore batch di immagini e da un tensore batch di etichette. In particolare, se il campione è un dizionario, il risultato è un dizionario con le stesse chiavi, ma con tensori batch come valori. Lo stesso vale per liste, tuple, named tuple e così via.

I programmatori possono personalizzare la funzione `collate_fn` per implementare meccanismi di batching specifici, ad esempio il padding per sequenze di lunghezza diversa all'interno dello stesso batch.

9.5 Moduli

Un **modulo** `PyTorch` è il blocco fondamentale per implementare le Neural Network in `PyTorch`. In termini generali, un modulo corrisponde a un layer o a un blocco di un'architettura neurale.

In termini matematici, un modulo rappresenta una funzione parametrizzata $y = f(x, w)$. Un modulo riceve il tensore di input x e calcola il tensore di output y , ma può anche contenere stato interno, come tensori che rappresentano *parametri* apprendibili w . I parametri specificano esplicitamente quali tensori devono essere trattati come parametri apprendibili del modulo, così che vengano aggiornati durante il training.

Per usare i moduli preesistenti di `PyTorch` è necessario importare i pacchetti seguenti:

```
import torch.nn as nn
import torch.nn.functional as F
```

Il pacchetto `torch.nn` include classi Python che rappresentano i moduli di `PyTorch`. Il pacchetto `torch.nn.functional` include le funzioni utilizzate per implementare i moduli contenuti in `torch.nn`. Tutti i moduli derivano dalla classe base `nn.Module`.

I parametri w di un modulo sono rappresentati come attributi del modulo stesso, con tipo `nn.Parameter`. Un oggetto `nn.Parameter` è semplicemente un tensore con il flag `requires_grad` impostato a `True`.

I parametri, come attributi, vengono definiti nel costruttore `__init__()` del modulo. Per default, i parametri sono tensori floating point a 32 bit memorizzati sulla CPU.

Quando un oggetto `nn.Parameter` viene assegnato come attributo del modulo `m`, viene aggiunto automaticamente alla lista dei parametri e comparirà, ad esempio, nell'iteratore `m.parameters()`.

Ogni volta che si assegna un parametro a un attributo del modulo, esso viene registrato con un nome. Il parametro assume sempre lo stesso nome dell'attributo. Per iterare su tutti i parametri e sui loro nomi associati si usa l'iteratore `m.named_parameters()`.

In un modulo possono esistere tensori che non richiedono gradienti, e quindi non sono parametri. Un tensore di questo tipo si chiama **buffer**. Ogni modulo può contenere buffer: per memorizzare un buffer `x` in un modulo `m` con nome `n`, si usa il metodo `m.register_buffer(n, x)`.

Per iterare su tutti i buffer si usa l'iteratore `m.buffers()`, mentre per iterare su tutti i buffer e sui rispettivi nomi si usa `m.named_buffers()`.

Ogni modulo `m` deve implementare il metodo `m.forward(x)`, che definisce come il tensore di input `x` viene elaborato dalla rete neurale rappresentata dal modulo. Il metodo `m.forward(x)` non dovrebbe mai essere chiamato esplicitamente: viene invocato automaticamente dal metodo `m.__call__()`, quindi quando si esegue `m(x)`.

Mentre il metodo `m.forward(x)` è responsabile del calcolo dell'output del modulo `m` a partire dall'input x e della costruzione del grafo computazionale associato alla funzione f e all'output y , il calcolo dei gradienti per w è svolto automaticamente da `autograd` chiamando il metodo `y.backward()`, disponibile in ogni tensore.

I moduli non devono per forza avere parametri, cioè non devono necessariamente essere stati. Un modulo può contenere anche uno o più altri moduli, detti sub-moduli, come attributi. `PyTorch` è in grado di tenere traccia automaticamente dei loro parametri.

Per ottenere i sub-moduli direttamente annidati in un altro modulo `m` si usa il generatore `m.children()`.

Per scendere nei moduli profondamente annidati si può usare il metodo `m.modules()`, che itera ricorsivamente su tutti i sub-moduli di `m`, incluso il modulo stesso.

Analogamente, il metodo `m.parameters()` elenca ricorsivamente tutti i parametri contenuti in un modulo e nei suoi sub-moduli.

Si può spostare facilmente un modello `m`, cioè tutti i suoi parametri e buffer e quelli dei suoi sub-moduli, da un dispositivo a un altro usando il metodo `m.to(device)`.

Si possono anche cambiare i tipi di dato dei parametri e dei buffer con il metodo `m.to(dtype)`.

In alternativa, si può usare `m.cpu()` per spostare un modello sulla CPU e `m.cuda(device)` per spostarlo su una GPU.

I sub-moduli non possono far parte di liste o dizionari Python standard. Per organizzare moduli in liste o dizionari e consentire a `PyTorch` di gestirli automaticamente, esistono speciali **container modules**:

- `nn.Sequential`: permette di impilare un numero arbitrario di moduli in sequenza. Si passa una lista di moduli al metodo `__init__()` e i moduli vengono aggiunti al contenitore nell'ordine in cui sono stati forniti. Una volta aggiunti, si può chiamare `forward()` per far passare un tensore di input attraverso la rete.

- **nn.ModuleList**: permette di memorizzare una lista di moduli. È simile a una lista Python, ma garantisce che i parametri dei moduli nella lista vengano registrati correttamente, fornisce metodi come `append()`, `insert()` e `remove()`, e consente l'iterazione tramite un normale ciclo `for`.
- **nn.ModuleDict**: permette di memorizzare un dizionario di moduli. È simile a un dizionario Python, ma offre alcune funzionalità aggiuntive, garantisce la registrazione corretta dei parametri dei moduli nel dizionario, fornisce metodi come `get()`, `set()` e `items()`, e consente l'iterazione tramite un normale ciclo `for`.

Tutti i parametri e i buffer di un modulo ne costituiscono lo **stato**. Ogni modulo **PyTorch** `m` ha l'attributo `m.state_dict`, che contiene il dizionario con tutti i nomi e i valori dei parametri e buffer. Le uniche eccezioni sono i buffer creati con il parametro `persistent=False`, che non vengono inclusi in `m.state_dict`.

Per salvare e caricare modelli in **PyTorch** esistono due metodi principali:

- salvare e caricare l'**intero modello**: è il metodo più intuitivo e richiede meno codice;
- salvare e caricare il **model state**: è il metodo consigliato.

Salvare un modello in questo modo salva l'intero modulo usando il modulo `pickle` di Python. Lo svantaggio di questo approccio è che i dati serializzati dipendono dalle classi specifiche e dalla struttura delle directory usata al momento del salvataggio. Questo accade perché `pickle` non salva la classe del modello in sé, ma solo il percorso del file che la contiene, e tale percorso viene usato in fase di caricamento. Per questo, il codice può rompersi in altri progetti o dopo refactoring.

Quando si salva un modello per l'inferenza, è sufficiente salvare i parametri appresi dal modello. Quando si carica lo stato di un modello, bisogna prima creare un modello adatto e poi impostarne lo stato con lo stato caricato.

9.6 Loss Function e Optimizer

Una volta definito il modello **PyTorch** di una rete neurale, possiamo creare in memoria i parametri del modello, inizializzarne i valori e far elaborare al modello i dati di input in base a tali valori. Gli elementi fondamentali di un training loop standard in **PyTorch**, oltre al modello, sono:

- la **loss function**, che determina come misurare la performance del modello e calcola l'errore tra la performance ottenuta dal modello corrente e quella attesa dai dati di input;
- l'**optimizer**, che determina come aggiornare i parametri del modello durante il training.

Loss function

La **loss function** è una funzione dei parametri di una rete neurale, cioè dipende dai dati **Parameter** di un oggetto **Module** in **PyTorch**. Perciò, le loss function si comportano come un layer che prende in input l'output del modello e produce un singolo valore reale. Quando eseguiamo la backpropagation, la eseguiamo sulla loss function, non sul modulo.

PyTorch offre molte loss function integrate nel pacchetto `torch.nn`. In generale, le loss function in **PyTorch** si dividono in tre grandi categorie: **regression losses**, **classification losses** e **ranking losses**.

Le regression losses si usano quando il modello predice un valore continuo, come l'età di una persona. Le classification losses si usano quando il modello predice un valore discreto, come stabilire se una e-mail è spam oppure no. Le ranking losses si usano quando il modello predice le distanze relative tra input, per esempio per ordinare prodotti in base alla loro rilevanza in una pagina di ricerca e-commerce.

Le loss function più comuni in PyTorch sono le seguenti:

- **Mean Absolute Error (MAE) loss**, detta anche **L1 loss** (`nn.L1Loss`): calcola la media delle differenze assolute tra valori reali e valori predetti. È una loss di regressione, adatta soprattutto quando la variabile target contiene outlier, cioè valori molto distanti dalla media.
- **Mean Squared Error (MSE) loss**, detta anche **L2 loss** (`nn.MSELoss`): calcola la media delle differenze al quadrato tra valori reali e valori predetti. La presenza del quadrato fa sì che errori grandi vengano penalizzati più di quelli piccoli. È una loss di regressione, ed è la funzione di loss predefinita nella maggior parte dei problemi di regressione in PyTorch.
- **Negative Log-Likelihood (NLL) loss** (`nn.NLLLoss`): si applica solo a modelli che usano la funzione softmax come attivazione in output. In NLL, la loss è grande per predizioni corrette con probabilità bassa e piccola per predizioni corrette con probabilità alta. È una loss di classificazione, usata nei problemi di classificazione multi-classe.
- **Cross Entropy loss** (`nn.CrossEntropyLoss`): calcola la differenza tra due distribuzioni di probabilità per un insieme di variabili casuali fornito. Viene usata per calcolare la differenza media tra valori predetti e valori reali. La Cross Entropy penalizza fortemente i casi in cui il modello è molto sicuro ma sbaglia. A differenza della NLL Loss, che non penalizza in base alla confidenza della predizione, la Cross Entropy penalizza sia le predizioni sbagliate ma molto sicure, sia quelle corrette ma poco sicure. È una loss di classificazione usata nei problemi multi-classe. Una variante molto diffusa è la **Binary Cross-Entropy (BCE) loss** (`nn.BCELoss`), usata soprattutto per la classificazione binaria, cioè con sole due classi.
- **Kullback-Leibler (KL) Divergence** (`nn.KLDivLoss`): calcola la differenza tra due distribuzioni di probabilità. Se la distribuzione predetta è molto lontana da quella vera, il valore della loss è grande. Se la KL Divergence è zero, le distribuzioni coincidono. La KL Divergence si comporta in modo simile alla Cross Entropy Loss, ma con una differenza importante: la Cross Entropy penalizza il modello in base alla confidenza delle predizioni, mentre la KL Divergence no. La KL Divergence valuta soltanto quanto la distribuzione predetta differisce da quella del ground truth. È una loss di classificazione, usata nei problemi multi-classe.
- **Hinge Embedding loss** (`nn.HingeEmbeddingLoss`): si usa quando si ha un tensore di input e un tensore di etichette con valori in $\{1, -1\}$, ed è quindi adatta alla classificazione binaria. È utile soprattutto per determinare se due input sono simili o dissimili.
- **Margin Ranking loss** (`nn.MarginRankingLoss`): è un criterio per prevedere le distanze relative tra input. La loss si calcola avendo due input x_1 e x_2 e un'etichetta y con valori 1 o -1 . Quando $y = 1$, l'input x_1 viene considerato maggiore e quindi classificato sopra x_2 . Se $y = -1$, allora viene classificato sopra x_2 . È una ranking loss, usata in problemi di ranking.

Una loss personalizzata non può essere scritta in NumPy, perché deve operare su tensori sui quali devono essere calcolati i gradienti lungo tutta la rete durante la backpropagation.

Scrivere correttamente una loss in PyTorch richiede attenzione: se si chiama una funzione che modifica gli input senza usare completamente i metodi numerici di PyTorch, il tensore si stacca dal grafo che lo collega alla rete neurale ai fini della backpropagation, rendendo la loss inutilizzabile.

Optimizer

La **backpropagation** è l'algoritmo che calcola il gradiente dei parametri di una rete neurale rispetto alla loss. **PyTorch** implementa la backpropagation tramite il pacchetto **autograd**.

Gli algoritmi di ottimizzazione basati sul gradient descent usano il gradiente per capire come aggiornare i parametri e ridurre la loss.

Il gradient descent stocastico è l'algoritmo di ottimizzazione più semplice. Nel suo nucleo, si prendono i parametri correnti del modello e si sottrae il gradiente di tali parametri moltiplicato per il **learning rate** η , che controlla l'entità dell'aggiornamento. Se il learning rate è troppo piccolo, gli aggiornamenti saranno troppo piccoli per addestrare il modello in tempi ragionevoli. Al contrario, se il learning rate è troppo grande, gli aggiornamenti saranno così grandi che l'apprendimento diventerà instabile.

Gli algoritmi di gradient descent procedono per passi: a ogni passo t , i parametri del modello w_t vengono aggiornati rispetto ai valori precedenti w_{t-1} come segue:

$$w_t = w_{t-1} - \eta \nabla_w \mathcal{L}(w_{t-1}),$$

dove $\mathcal{L}(w_{t-1})$ è il gradiente della loss rispetto al parametro di input w_t calcolato in w_{t-1} .

Generiamo alcuni dati sintetici:

Dato un learning rate `lr` e una loss function `loss_fn`, possiamo calcolare a mano gli aggiornamenti del gradient descent del modello nel modo seguente.

Prima di chiamare `loss.backward()` per attivare il calcolo dei gradienti, è necessario abilitare l'accumulazione dei gradienti sui tensori parametro del modello. Inoltre, poiché **autograd** accumula sempre i gradienti sui tensori, li azzeriamo impostandoli a `None`, così da ricominciare ad accumulare da 0. Dopo `loss.backward()`, abbiamo il gradiente e dobbiamo aggiornare i parametri del modello. Prima di farlo, disabilitiamo il calcolo dei gradienti sui tensori parametro, altrimenti le operazioni verranno registrate nel grafo computazionale, e poi applichiamo il gradient descent usando i valori correnti dei parametri, i gradienti calcolati e il learning rate `lr`.

Queste azioni sono implementate in **PyTorch** tramite meccanismi specifici:

- `m.zero_grad()` imposta a 0 il gradiente di tutti i parametri del modello `m`.
- `torch.no_grad()` disabilita il calcolo dei gradienti. In questa modalità, il risultato di ogni computazione avrà `requires_grad=False`, anche quando gli input hanno `requires_grad=True`. Fa eccezione il caso delle factory functions, cioè funzioni che creano un nuovo tensore e accettano un argomento `requires_grad`, che non vengono influenzate da questa modalità.

Si può racchiudere il codice precedente in una classe Python e usarlo in questo modo.

PyTorch include varie implementazioni di algoritmi di ottimizzazione nel pacchetto `torch.optim`. Per usare `torch.optim` bisogna creare un oggetto **Optimizer** che conserva lo stato corrente e aggiorna i parametri in base ai gradienti calcolati. Ogni optimizer si costruisce passando un iterable contenente i parametri da ottimizzare. Poi si possono specificare opzioni proprie dell'optimizer, come learning rate, weight decay e altro.

Tutti gli optimizer implementano il metodo `step()`, che aggiorna i parametri. Il metodo può essere chiamato una volta calcolati i gradienti con `loss.backward()`.

Un'operazione comune durante l'ottimizzazione di una rete neurale è modificare il learning rate durante il processo, riducendolo lentamente mentre l'ottimizzazione procede verso la fine. Questa procedura si chiama **learning rate scheduling**.

Il modulo `torch.optim.lr_scheduler` fornisce diversi metodi per regolare il learning rate in base al numero di iterazioni eseguite dall'algoritmo di ottimizzazione. Lo scheduling del learning rate dovrebbe essere applicato dopo l'aggiornamento dell'optimizer.

Ad esempio:

```
from torch.optim.lr_scheduler import *

optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
scheduler = ExponentialLR(optimizer, gamma=0.9)

for epoch in range(20):
    for input, target in dataset:
        optimizer.zero_grad()
        output = model(input)
        loss = loss_fn(output, target)
        loss.backward()
        optimizer.step()
        scheduler.step()
```

Ora abbiamo tutti gli elementi per scrivere una funzione `training_step()` che esegua un singolo passo di training, chiamato anche *epoch*. Si assume di avere una rete neurale codificata in `module` e un dataset da iterare in `dataset`.

La funzione `training_step()` enumera i dati di `dataset` e compie le seguenti operazioni:

1. ottiene un batch di dati;
2. azzeri i gradienti dell'optimizer;
3. esegue il forward pass, cioè ottiene le predizioni del modello per il batch in input;
4. calcola la loss;
5. esegue il backward pass, cioè calcola il gradiente sui parametri della rete;
6. chiede all'optimizer di compiere un passo di ottimizzazione, cioè di aggiornare i parametri della rete in base ai gradienti calcolati per quel batch.

Dopo aver addestrato il modello e tentato di minimizzare la loss function, bisogna valutare le prestazioni del modello su dati che non ha mai visto. La progettazione di un modello neurale include, oltre ai dati di training, anche dati di validazione e dati di test. I dati di validazione vengono usati in un validation loop per evitare l'overfitting, mentre i dati di test vengono usati in un test loop per misurare le prestazioni del modello su dati nuovi.

Quando si usano dati di validazione e test, il modello deve essere messo nella modalità appropriata. Alcuni moduli, come dropout e normalizzazione, vengono usati solo durante il training e non durante la valutazione. Per configurare un modello `m` per il training, si dovrebbe chiamare `m.train()` prima di iniziare il training loop. Per configurare il modello per la valutazione, sia durante la validazione sia durante il test, si dovrebbe chiamare `m.eval()` prima di iniziare i corrispondenti loop. Anche se non è sempre strettamente necessario, è buona pratica usare sempre questi metodi.

Durante la validazione e il test, viene calcolato solo l'output del modello senza modificare in alcun modo i parametri. Per questo motivo, non è necessario tracciare i gradienti né le funzioni per calcolarli.

9.7 Maschere Booleane

Quando si lavora con i tensori, un modo potentissimo per manipolarli senza usare lenti cicli `for` è l'uso delle **maschere booleane**. Una maschera booleana è un tensore composto esclusivamente da valori `True` o `False`, avente la stessa forma (shape) del tensore originale, generato attraverso una condizione logica. Immagina di avere una matrice `a` e di voler azzerare tutti i valori superiori a 0.5. Invece di controllare ogni singolo numero, puoi creare una maschera e applicarla istantaneamente:

```
import torch

a = torch.rand(3, 3)
mask = a > 0.5 # Creates a tensor of True/False
a[mask] = 0.0  # Instantaneously zeroes out only the True elements
```

PyTorch permette anche di combinare queste maschere usando la logica binaria (& per AND, | per OR, ~ per NOT), rendendo operazioni complesse incredibilmente veloci e compatte.

Perché dobbiamo usare le maschere invece dei classici cicli `for` quando calcoliamo la Loss?

Vediamo due esempi.

Questo codice è **ERRATO**:

```
errors = torch.zeros(y_model.shape[0]) # Tensor initialization
for i in range(0, y_model.shape[0]):
    if y_model[i] > y[i] and c[i] == 1:
        errors[i] = y_model[i] - y[i] # Manual assignment
    if y_model[i] > y[i] and c[i] == 0:
        errors[i] = y[i] - y_model[i] # Manual assignment
loss = torch.mean(errors)
```

Questo codice genera un `RuntimeError`. Il motivo risiede nel modo in cui l'Autograd traccia le operazioni. Inizializzare un tensore vuoto e riempirlo manualmente elemento per elemento "spezza" il Grafo Computazionale. Nella Backpropagation (la fase backward), il flusso del gradiente si ferma bruscamente ai singoli elementi assegnati manualmente (e1, e2, eccetera) e non riesce a risalire verso i parametri del Modello.

Vediamo come si fa:

```
errors = torch.abs(y_model - y)
errors[predicted_above == actual_above] = 0 # Masking
errors[predicted_below == actual_below] = 0 # Masking
loss = torch.mean(errors)
```

Questa soluzione funziona perfettamente. In PyTorch, "**impostare a zero tramite una maschera**" è formalmente riconosciuta come un'operazione differenziabile.

Quando applichiamo la maschera `errors[...] = 0`, PyTorch inserisce un nodo speciale nel Grafo Computazionale. Durante il Backward Pass, quando il gradiente arriva a quel nodo, PyTorch sa semplicemente che deve "impostare i gradienti a zero" per quegli specifici elementi filtrati dalla maschera, lasciando fluire inalterati tutti gli altri gradienti verso i parametri del Modello.

Capitolo 10: Convolutional Neural Network

10.1 Matematica della Convoluzione

La **Convoluzione** tra due funzioni unidimensionali $k(x)$ e $f(x)$ è definita come:

$$(k * f)(x) = \int_{-\infty}^{+\infty} k(x - \tilde{x}) f(\tilde{x}) d\tilde{x}$$

La funzione $k(x)$ è detta **kernel**. Operativamente, la convoluzione consiste nel ribaltare il kernel, traslarlo nella posizione x e calcolare l'integrale del prodotto con f . Essa misura la corrispondenza locale tra le due funzioni. In **Computer Vision**, la convoluzione consente di estrarre caratteristiche locali da un'immagine tramite un filtro che scorre l'immagine e risponde a pattern come bordi, linee o texture.

Le principali proprietà sono:

- **Commutatività:** $k * f = f * k$
- **Associatività:** $(k * f) * g = k * (f * g)$
- **Distributività:** $k * (f + g) = k * f + k * g$
- **Omogeneità:** $\alpha(k * f) = (\alpha k) * f$
- **Differenziazione:** $(k * f)' = k' * f = k * f'$

La convoluzione bidimensionale è definita come:

$$(k * f)(x; y) = \iint k(x - \tilde{x}, y - \tilde{y}) f(\tilde{x}; \tilde{y}) d\tilde{x} d\tilde{y}$$

Un'immagine digitale è una funzione $f : \mathbb{Z}^2 \rightarrow \mathbb{Z}$, mentre una *multispectral image* è $f : \mathbb{Z}^2 \rightarrow \mathbb{Z}^d$, con $d > 1$. Spesso, è necessario manipolare le immagini per estrarre informazioni utili o per trasformarle al fine di modificarne/migliorarne le proprietà. Un **filtro** (o sistema) è una scatola nera che converte un'immagine di input $f[x; y]$ in un'immagine di output $g[x; y]$. Un filtro è identificato in modo univoco dal suo operatore di filtro $\mathcal{S}$.

Un filtro che ha queste 4 proprietà è chiamato **Liner Shift-Invariance Filter (LSI)**:

- **Additività:** $\mathcal{S}(f[x; y] + g[x; y]) = \mathcal{S}(f[x; y]) + \mathcal{S}(g[x; y])$
- **Omogeneità:** $\mathcal{S}(\alpha f[x; y]) = \alpha \mathcal{S}(f[x; y])$, $\alpha \in \mathbb{R}$
- **Superposizione:** $\mathcal{S}(\alpha f[x; y] + \beta g[x; y]) = \alpha \mathcal{S}(f[x; y]) + \beta \mathcal{S}(g[x; y])$, $\alpha, \beta \in \mathbb{R}$
- **Invarianza di Traslazione:** $g[x - x_0] = \mathcal{S}(f[x - x_0, y - y_0])$

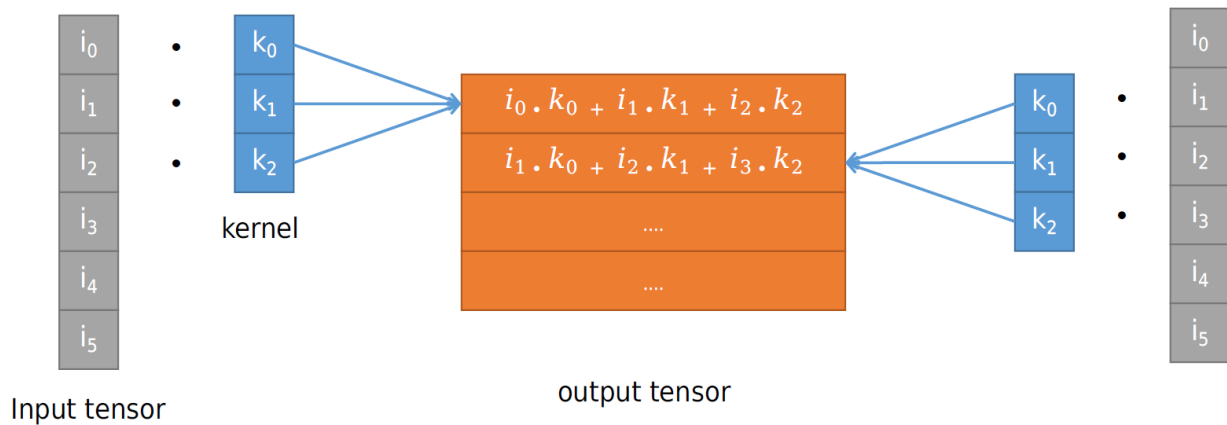
La risposta all'impulso $h[x; y]$ è l'uscita del filtro per un ingresso impulsivo e lo caratterizza completamente. Per sistemi LSI, l'uscita generale è: $g = f * h$

10.2 Convoluzione 2D

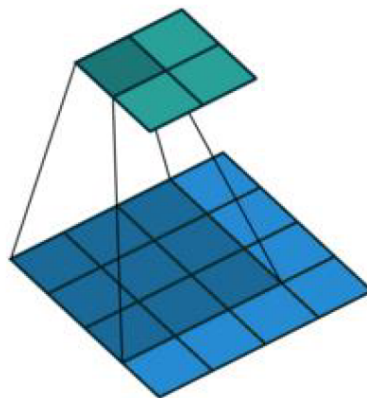
Nel caso dei filtri per immagini, la risposta all'impulso coincide con il kernel: un impulso produce una copia traslata del kernel. Poiché lavoriamo con pixel digitali, l'integrale si trasforma in una sommatoria. La **convoluzione discreta 2D** diventa:

$$(k * f)[x, y] = \sum_{i=-\infty}^{+\infty} \sum_{j=-\infty}^{+\infty} k[x - i; y - j] f[i; j]$$

L'operatore di convoluzione funziona moltiplicando una piccola finestra dei dati (l'intorno dei pixel) con un kernel (una matrice di pesi), e sommando i risultati per produrre un singolo valore. La convoluzione applica un kernel ai dati di ingresso calcolando, in ogni posizione, la somma dei prodotti tra il kernel e la regione locale considerata. Ripetendo tale operazione su tutto l'ingresso, si ottiene un tensore di uscita i cui elementi rappresentano le risposte del kernel nelle diverse posizioni. In questo modo vengono estratte le caratteristiche locali più rilevanti dei dati.



Il **kernel size** è la dimensione del kernel o dei kernel usati nella convoluzione. Le dimensioni del kernel determinano l'area dell'input che viene considerata durante la convoluzione, e quindi influenzano la dimensione del tensore di uscita.



10.3 Convolutional Layer

Abbiamo visto i **Multi-Layer Perceptron** (MLP), in cui ogni neurone di un livello è connesso a tutti i neuroni del livello precedente (strati fully connected o densi). Questo approccio funziona bene per dati tabulari, ma non è abbastanza efficace quando si tratta di elaborare immagini.

Ad esempio, un'immagine RGB di dimensione 256×256 elaborata da uno strato con 1000 neuroni richiederebbe circa $256 \times 256 \times 3 \times 1000 \approx 1.97 \times 10^8$ pesi, con conseguenti problemi di costo computazionale e overfitting.

Per gestire dati caratterizzati da una struttura spaziale, come le immagini, sono state introdotte le **Convolutional Neural Networks** (CNN), che sfruttano connessioni locali e condivisione dei pesi per ridurre drasticamente il numero di parametri.

Il nucleo di una CNN è il **Convolutional Layer**. Invece di usare una matrice di pesi globale, la CNN utilizza un piccolo "filtro", ossia il Kernel (ad esempio una matrice 3×3 o 5×5). Questo kernel scorre letteralmente sopra l'immagine di input, partendo dall'angolo in alto a sinistra fino a quello in basso a destra. Ad ogni passo, esegue un'operazione matematica semplice: moltiplica i valori dei pixel coperti in quel momento per i pesi del kernel e li somma, producendo un singolo numero in uscita. L'insieme di tutti questi numeri generati dal passaggio del kernel forma una nuova mappa, chiamata **Feature Map**.

Le Feature Map sono i tensori di output di un Convolutional Layer. Ogni canale corrisponde all'applicazione di un diverso kernel e rappresenta la risposta del relativo filtro all'input. Ogni valore della feature map indica il livello di attivazione di una specifica caratteristica. Nei primi strati vengono estratte feature di basso livello (bordi, linee), mentre negli strati più profondi emergono caratteristiche più astratte (forme e oggetti). Il numero di feature map è un iperparametro: aumentarlo consente di apprendere più caratteristiche, ma incrementa la complessità del modello e il rischio di overfitting. Le feature map rappresentano quindi le informazioni rilevanti estratte dall'input e vengono propagate ai layer successivi.

Invece di imparare un peso per ogni singolo pixel dell'immagine (come nell'MLP), la rete impara un solo set di piccoli pesi (il kernel) e lo applica ovunque. Questo riduce drasticamente i parametri. Inoltre, sfrutta il principio di "località": in un'immagine, i pixel vicini sono fortemente correlati e formano strutture (come bordi, angoli, texture) che il piccolo kernel è in grado di riconoscere.

```
import torch
```

```
conv = torch.nn.Conv2d(3, 96, (11, 11), stride=4)
```

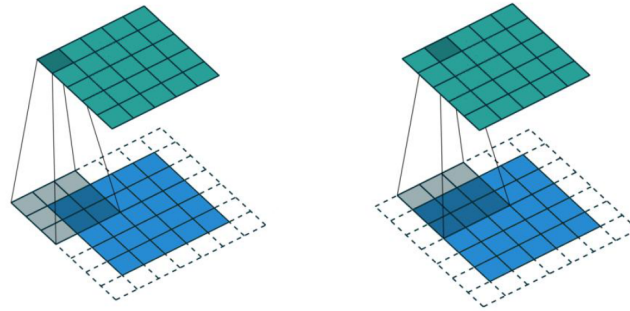
Nell'esempio sopra il layer prende un'immagine RGB (3 canali), applica 96 filtri di dimensione 11×11 che scorrono con passo 4, ed estrae 96 mappe di caratteristiche (feature maps). La convoluzione è calcolata in parallelo essendo la stessa operazione da fare su input diversi. Può essere usato il paradigma SIMD, e le GPU sono perfette per questo tipo di operazione.

I canali in una CNN sono i diversi "strati" del tensore di uscita di un Convolutional Layer. Ogni canale corrisponde al risultato di una convoluzione con un kernel diverso. I canali permettono alla CNN di catturare diverse caratteristiche dell'input, e di apprendere gerarchie di feature complesse.

Mentre il kernel scorre, l'immagine in uscita tende naturalmente a restringersi (poiché il kernel non può sporgere oltre i bordi dell'immagine originale senza "cadere nel vuoto"). Per controllare la geometria della Feature Map, si usano due parametri:

- **Padding:** l'aggiunta di extra pixel all'input così da garantire che la convoluzione processi l'intero ingresso, specialmente nei bordi. Questo permette al kernel di centrare anche i pixel più esterni, mantenendo inalterata la risoluzione spaziale dell'immagine in uscita;

- **Stride:** il numero di pixel che il kernel salta mentre scorre l'input. Più il passo è grande, più la dimensione del tensore di uscita sarà ridotta. Questo accelera il calcolo e riduce le dimensioni (sottocampionamento) della Feature Map in uscita, comportandosi come un compressore.



Per capire come impostare **Kernel Size**, **Padding** e **Stride** dobbiamo prima fare chiarezza sulla matematica che governa il ridimensionamento dei tensori. L'operazione di convoluzione fa scorrere un filtro lungo le dimensioni del tensore di input. Esiste una formula esatta che collega l'altezza (e la larghezza) del tensore di input con quella del tensore di output:

$$H_{out} = \lfloor \frac{H_{in} + 2P - K}{S} \rfloor + 1$$

Dove H_{in} è l'altezza di input, P è il padding, K è la dimensione del kernel, S è lo stride, e $\lfloor \dots \rfloor$ indica l'arrotondamento. Il Kernel Size deve essere sempre dispari.

Le **funzioni di attivazione sono utilizzate per introdurre non linearità in una CNN**, permettendo al modello di approssimare funzioni complesse. **Questo viene fatto perché di per se la convoluzione è un'operazione lineare**, e dunque una composizione di convoluzione può essere ridotta ad una singola convoluzione. Facendo seguire ogni operazione lineare da una funzione di attivazione, si risolve questo problema. Le funzioni di attivazione più comuni sono la *ReLU*, *Sigmoid* e la *Tanh*, eccetera ...

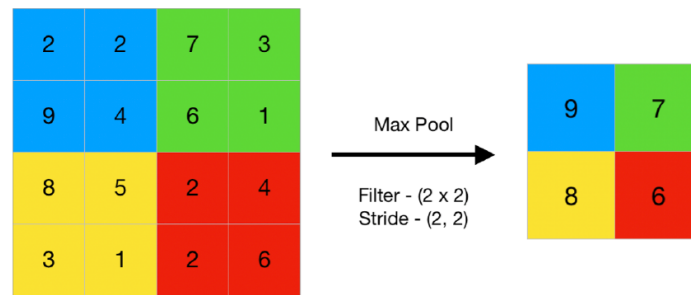
10.4 Pooling Layer

Per mantenere le informazioni più importanti riducendo il numero di dati da elaborare, si utilizzano i **Pooling Layer**. Dopo una convoluzione, l'immagine viene trasformata in una o più feature map e se continuiamo ad applicare convoluzioni senza ridurre le dimensioni aumentano memoria e tempi di calcolo, crescono i parametri e aumenta il rischio di overfitting. Il pooling risolve questo problema comprimendo l'informazione.

Ci sono due tipi di Pooling: l'**Average Pooling** e il **Max Pooling**.

Quando la finestra scorre sui dati, l'**Average Pooling** calcola la **media aritmetica** di tutti i valori presenti all'interno della finestra. Questo metodo "spalma" l'informazione. Nessun dettaglio viene scartato completamente, ma tutto viene fuso in un unico valore rappresentativo. Ideale per situazioni in cui è importante mantenere un'idea generale del "contesto" di una regione, piuttosto che cercare una singola caratteristica forte.

Il **Max Pooling** è il tipo più utilizzato. Quando la finestra scorre sui dati, **seleziona semplicemente il valore massimo** presente al suo interno e scarta tutti gli altri. I valori alti in una mappa delle caratteristiche di solito indicano la forte presenza di una "feature" specifica (come un bordo netto, un colore acceso o un pattern particolare). Prendendo il massimo, la rete si assicura che se una caratteristica è stata rilevata in quella regione, l'informazione venga passata allo strato successivo, ignorando il "rumore" di fondo. Riconoscimento di bordi, contrasti netti e caratteristiche salienti.



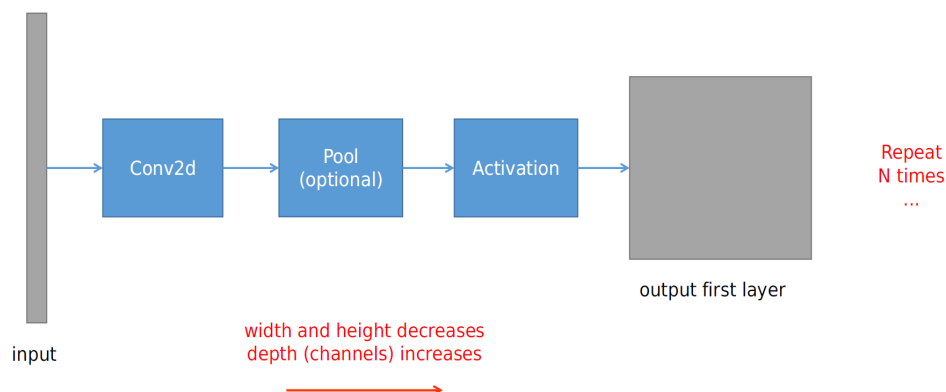
```
import torch

maxpool = torch.nn.MaxPool2d((2, 2), stride=1)
input = torch.rand(1, 3, 3)
output = maxpool(input)
```

Qua il codice applica un Max Pooling 2×2 con stride 1 a un tensore 3×3 , producendo una mappa più compatta che conserva solo i valori massimi locali.

10.5 Anatomia di una CNN

Anatomy of a Convolutional Neural Network



Le CNN si comportano come un sistema visivo artificiale. La sua funzione primaria è prendere un dato grezzo (come un'immagine fatta di migliaia di pixel), sgranarlo gradualmente per trovare gli schemi geometrici fondamentali, e infine usare queste informazioni per prendere una decisione (esempio: "In questa foto c'è un gatto").

Per fare una Classificazione, l'anatomia di una CNN è divisa rigorosamente in due "organi" sequenziali: il **blocco di Feature Extraction** (la vista) e il **blocco di Classification** (il cervello).

Il blocco di Feature Extraction è formato da una sequenza ripetuta di tre strati:

1. **Livello Convolutionale (Conv2D):** Come visto prima, applica numerosi filtri (kernel) all'immagine. I primi strati convoluzionali imparano a "vedere" cose molto semplici: linee verticali, linee orizzontali, bordi e gradienti di colore. Gli strati convoluzionali più profondi combinano queste linee per "vedere" forme complesse (es. un cerchio, un occhio, una ruota). L'output di questo strato è un volume di Feature Map;

2. **Livello di Attivazione:** Immediatamente dopo la convoluzione, si applica una funzione non lineare come la ReLU. La sua funzione pratica è azzerare tutti i valori negativi. Se un filtro stava cercando una linea verticale e non l'ha trovata, il risultato negativo viene "spento", lasciando accesi solo i "veri positivi" (i tratti distintivi trovati);
3. **Livello di Max Pooling:** Interviene per comprimere e riassumere l'informazione. Rimpicciolisce la mappa tenendo solo i segnali più forti. Questo fa due cose: riduce drasticamente il carico computazionale e rende la rete resistente a piccole traslazioni (se il gatto si sposta un po' a destra nella foto, il Max Pooling cattura comunque i suoi tratti salienti).

Dopo diversi passaggi di $\text{Conv} \rightarrow \text{ReLU} \rightarrow \text{MaxPool}$, le nostre mappe sono diventate molto piccole spazialmente (esempio: 5×5 pixel), ma molto "profonde" (esempio: 256 mappe diverse, ognuna che rappresenta un concetto visivo complesso). A questo punto, la struttura 2D non ci serve più. Lo strato di **Flatten** prende questa matrice e la fa diventare un lunghissimo vettore monodimensionale.

Ora che i pixel grezzi sono stati trasformati in un vettore ordinato contenente solo le "caratteristiche pure" dell'immagine, entrano in gioco i classici Multi-Layer Perceptron.

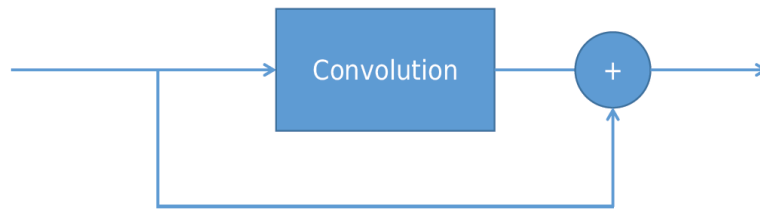
```
class CNN(nn.Module):
    def __init__( self ):
        super( CNN , self ). __init__(input_w, input_h, num_classes)
        self.cnn = nn.Sequential(
            nn.Conv2d(kernel_size = (3 , 3), in_channels = 1, out_channels = 32),
            nn.LeakyReLU(),
            nn.MaxPool2d(kernel_size =(2 , 2)),
            nn.Conv2d(kernel_size = (3 , 3), in_channels = 32, out_channels = 64),
            nn.LeakyReLU(),
            nn.MaxPool2d (kernel_size =(2 , 2)),
            nn.Flatten()
        )
        # Fully connected layers (MLP)
        self.ff = nn.Sequential(
            nn.Linear(64 * 6 * 6, 128),
            nn.LeakyReLU(),
            nn.Linear(128, num_classes)
        )

    def forward(self , x):
        x = self. cnn(x)
        x = self.ff(x)
        return x
```

10.6 Residual Networks

Le **Residual Networks (ResNet)** sono delle Neural Network progettate per affrontare il problema del **Vanishing Gradient**. Si è visto che aggiungere più Convolutional Layer a una rete tradizionale causa spesso una diminuzione progressiva dei gradienti, propagandosi all'indietro, diventano sempre più piccoli fino quasi ad annullarsi, impedendo al modello di imparare correttamente.

In un'architettura standard, l'input passa attraverso una serie di trasformazioni sequenziali. In una ResNet, l'input di un blocco viene invece combinato direttamente con il suo output finale attraverso le cosiddette **skip connections**.



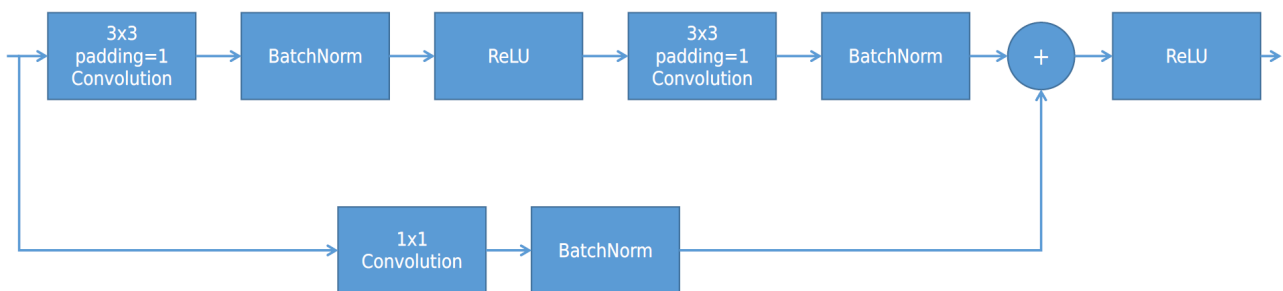
Le skip connections sono connessioni dirette fra due layer di una Neural Network e **creano un percorso alternativo che permette al gradiente di fluire in modo molto più diretto all'indietro attraverso la rete durante la Backpropagation**.

Questo collegamento diretto fra ingresso e uscita è poi sommato all'uscita del layer, permettendo al gradiente di propagarsi direttamente. Abbiamo però il problema che il tensore di uscita, una volta elaborato dalla Neural Network, avrà potenzialmente dimensioni che non corrispondono più a quelle del tensore di input, e quindi è impossibile fare la somma. Affinché l'operazione di somma tra l'input originale e l'output trasformato sia matematicamente possibile, i due tensori devono avere esattamente la stessa forma spaziale (stessa larghezza e altezza). Questo problema viene superato con due passaggi:

1. **Half-Padding:** Si calcola la convoluzione col kernel, con un padding pari alla metà della dimensione del kernel, così da preservare le dimensioni spaziali del tensore. Si applica esclusivamente ai layer convoluzionali del cammino principale;
2. **1x1 Convolution:** Si applica una convoluzione con kernel 1x1 in skip connection, così da riportare il tensore di ingresso allo stesso numero di canali del tensore d'uscita. Un altro modo più banale è quello di usare lo Zero-Padding.

Abbiamo poi anche la **Batch Normalization**, essenziale nella ResNet poiché opera su batch di dati e normalizza l'input sottraendo la media e dividendo per la deviazione standard, secondo la formula $y = \frac{x - E[x]}{\sqrt{Var[x] + \epsilon}} * \gamma + \beta$. Questo passaggio è fondamentale per stabilizzare e accelerare l'addestramento.

Ogni rete con una skip connection è tecnicamente una ResNet. Solitamente un blocco residuo "standard" ha questo aspetto:



In pratica la struttura di una ResNet differisce da una CNN standard per la presenza di skip connections, che permettono al gradiente di propagarsi direttamente, senza il rischio di attenuazione durante l'attraversamento inverso della rete. Questo permette di addestrare reti molto più profonde.

10.7 Codice

```

1  model = torch.nn.Sequential(
2      torch.nn.Conv2d(1, 16, (3, 3), padding=1),
3      torch.nn.LeakyReLU(),
4      torch.nn.Conv2d(16, 32, (3, 3), padding=1),
5      torch.nn.LeakyReLU(),
6      torch.nn.MaxPool2d((2, 2)),
7      torch.nn.Conv2d(32, 64, (3, 3)),
8      torch.nn.LeakyReLU(),
9      torch.nn.MaxPool2d((2, 2)),
10     torch.nn.Conv2d(64, 64, (3, 3), stride=2),
11     torch.nn.LeakyReLU(),
12     torch.nn.Flatten(),
13 )

1  import random
2  import torch
3  import torch.nn as nn
4  from torch.utils.data import IterableDataset
5
6  class FollowingNumber(IterableDataset):
7      def __init__(self, dataset):
8          super().__init__()
9          self.dataset = dataset
10
11     def __iter__(self):
12         return self
13
14     def _sample(self):
15         return random.choice(self.dataset)
16
17     def __next__(self):
18         x, y = self._sample()
19         while y == 9:
20             x, y = self._sample()
21
22         next_x, next_y = self._sample()
23         while next_y != y + 1:
24             next_x, next_y = self._sample()
25
26         return x, next_x
27
28  class ResidualBlock(nn.Module):
29      def __init__(self, in_c, out_c):
30          super().__init__()
31
32          self.conv1 = nn.Sequential(
33              nn.Conv2d(in_c, out_c, kernel_size=3, padding=1),
34              nn.BatchNorm2d(out_c),
35              nn.ReLU(),
36          )
37          self.conv2 = nn.Sequential(
38              nn.Conv2d(out_c, out_c, kernel_size=3, padding=1),
39              nn.BatchNorm2d(out_c),
40          )
41
42          self.shortcut = nn.Identity()
43          if in_c != out_c:
44              self.shortcut = nn.Sequential(
45                  nn.Conv2d(in_c, out_c, kernel_size=1),
46                  nn.BatchNorm2d(out_c),
47              )

```

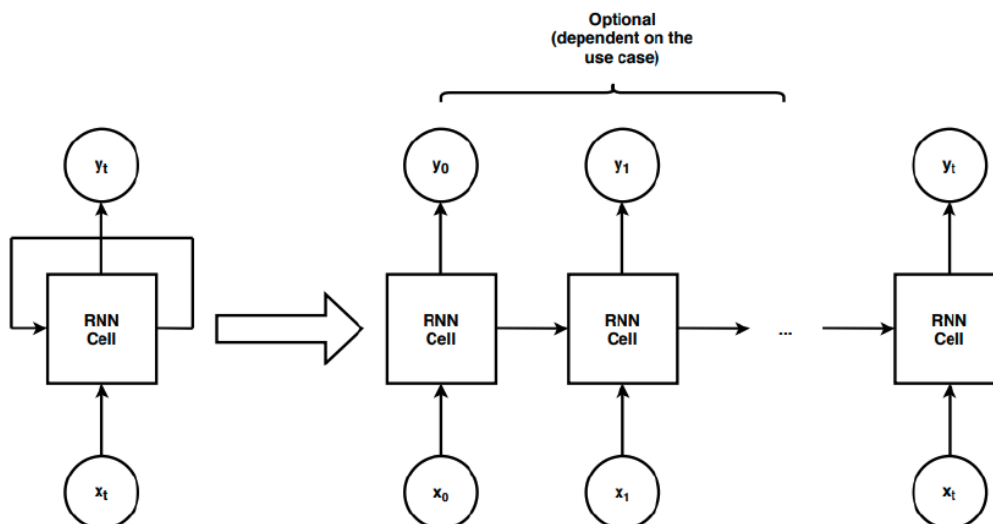
```
48
49     def forward(self, x):
50         out = self.conv1(x)
51         out = self.conv2(out)
52         out = out + self.shortcut(x)
53         return torch.relu(out)
54
55
56 class ResNet(nn.Module):
57     def __init__(self):
58         super().__init__()
59         self.net = nn.Sequential(
60             ResidualBlock(1, 16),
61             ResidualBlock(16, 32),
62             ResidualBlock(32, 64),
63             ResidualBlock(64, 64),
64             ResidualBlock(64, 64),
65             ResidualBlock(64, 64),
66             ResidualBlock(64, 32),
67             ResidualBlock(32, 16),
68             ResidualBlock(16, 1),
69         )
70
71     def forward(self, x):
72         return self.net(x)
```

Capitolo 11: Recurrent Neural Network

11.1 Meccanismo della Memoria

Una **Recurrent Neural Network (RNN)** è una tipologia di Neural Network caratterizzata dalla presenza di connessioni ricorrenti, che le consentono di elaborare sequenze di dati nel tempo (come le Time Series). Il suo funzionamento si basa sull'utilizzo ripetuto degli stessi pesi su input che variano in funzione della dimensione temporale, permettendo così di modellare dipendenze tra elementi successivi di una sequenza.

La struttura fondamentale di una RNN è la cosiddetta **cella**. Ogni cella è caratterizzata da tre elementi chiave: un **input**, un **hidden state** e un **output**. A ogni passo temporale, la cella non elabora l'input in modo isolato: l'input corrente viene fuso con l'hidden state generato nel passo precedente. L'hidden state precedente h_{t-1} rappresenta un riassunto di tutto ciò che la rete ha visto fino all'istante $t - 1$. Questo meccanismo permette alla rete di sviluppare una **memoria interna**, catturando così i pattern e le dipendenze temporali intrinseche alla sequenza. La struttura "input, hidden, output" è solo una "semplificazione" descrittiva. Non si tratta di tre layer distinti come in una feed-forward network, ma di un singolo modulo che combina input e memoria (hidden state) per produrre un aggiornamento dello stato. Questo significa che l'input e l'output della rete possono cambiare ad ogni passo temporale. L'hidden state, quindi, contiene informazioni riguardo le elaborazioni precedenti temporalmente.



11.2 Famiglie di Applicazione delle RNN

Le RNN sono estremamente versatili e si dividono in due famiglie di applicazione:

- **Sequence to Task (Seq2Task):** La rete processa un'intera sequenza di dati in ingresso e produce un singolo risultato in uscita che è poi usata da un'altra Neural Network per compiere un task specifico. È l'approccio standard per compiti di Classificazione di sequenze o di Regressione.
- **Sequence to Sequence (Seq2Seq):** La rete riceve una sequenza in ingresso e genera a sua volta una sequenza in uscita, spesso di lunghezza variabile. Questo modello è alla base dei moderni sistemi di traduzione automatica o di text summarization.

La differenza principale tra le due famiglie risiede nella struttura dell'output: nel caso *Seq2Task* l'intera sequenza viene ridotta a una singola rappresentazione finale, mentre nel caso *Seq2Seq* la rete produce una sequenza completa in uscita, spesso attraverso un processo di generazione step-by-step.

Negli approcci *Seq2Seq* più avanzati, vengono spesso introdotti miglioramenti architetturali come i meccanismi di *attention*, che permettono al decoder di accedere selettivamente a diverse parti della sequenza di input invece di dipendere esclusivamente da una singola rappresentazione compressa.

11.3 Backpropagation Through Time

Le RNN possiedono dei cicli interni, in quanto l'hidden state viene passato al passo temporale successivo. La Backpropagation standard, concepita per architetture feed-forward acicliche, non è direttamente applicabile a tale struttura.

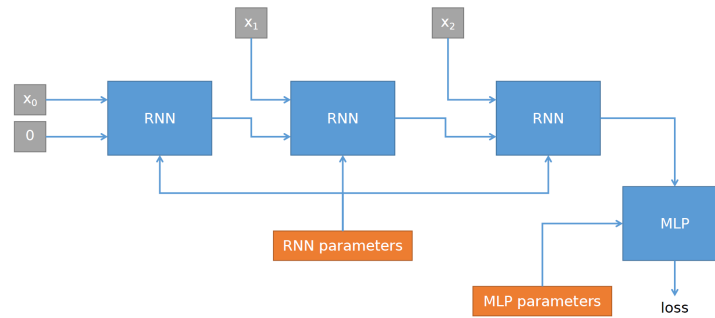
Per risolvere questo problema si utilizza la **Backpropagation Through Time (BPTT)**, che si basa su un'operazione di unfolding (srotolamento) della rete lungo la dimensione temporale. In questo modo, una RNN viene trasformata in una rete feed-forward equivalente, in cui ogni passo temporale corrisponde a un livello distinto della rete.

Il processo si divide in tre fasi principali:

- **Forward Pass:** La rete riceve la sequenza in ingresso passo dopo passo. Calcola l'hidden state e l'output per ogni istante t , fino ad arrivare alla fine della sequenza dove viene calcolato la Loss rispetto al target atteso;
- **Backward Pass:** L'errore finale viene preso e propagato letteralmente "all'indietro nel tempo", partendo dall'ultimo istante (esempio: $t = 10$) fino al primo istante ($t = 1$). Ad ogni passo a ritroso lungo i livelli srotolati, si calcola il gradiente, ovvero di quanto i pesi hanno contribuito a quell'errore;
- **Aggiornamento dei Pesi:** Poiché nella RNN i pesi fisici sono condivisi in tutti i passi temporali, i gradienti calcolati per ogni singolo passo della rete srotolata vengono sommati tra loro per ottenere un unico gradiente cumulativo. Infine, questo gradiente totale viene usato dall'optimizer (come lo Stochastic Gradient Descent) per aggiornare i pesi effettivi della rete.

In un'architettura Sequence to Task (esempio: Classificazione di un'intera sequenza), la rete elabora tutti i dati in ingresso passo dopo passo, ma la predizione vera e propria avviene solo all'ultimo istante di tempo. La Loss Function viene calcolata una sola volta alla fine della sequenza, confrontando l'output finale con il target. Il gradiente nasce unicamente dall'ultimo nodo e fluisce all'indietro (verso sinistra nel grafo srotolato) in modo lineare.

Recurrent Neural Networks: Computational Graph (end loss)



Esempio codice Seq2Task:

```
# The loss is calculated ONLY at the end of the sequence
current_hidden = torch.zeros(30)

for t in range(5):
    current_input = input_sequence[t]
    network_input = torch.concat((current_input, current_hidden), dim=0)

    # Step through time, but we don't calculate loss yet
    current_hidden = rnn(network_input)

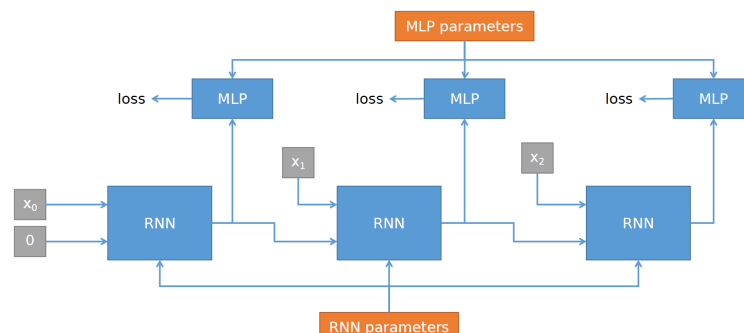
# LOOP ENDS. Now we make the final prediction and calculate the SINGLE loss
final_output = mlp(current_hidden)
loss = loss_fn(final_output, target)

# BPTT flows backward from this single end point
loss.backward()
```

In un'architettura Sequence to Sequence (esempio: traduzione linguistica), la rete deve produrre un output valido a ogni singolo passo temporale. La Loss non viene calcolata solo alla fine, ma viene calcolata a ogni singolo istante t (si ha una predizione per ogni input). Alla fine, l'errore totale è solitamente la media o la somma di tutte queste Loss intermedie. Il gradiente diventa molto più complesso. A un dato istante t , la rete riceve errore da due direzioni contemporaneamente:

- Il gradiente generato dalla Loss locale calcolata in quell'esatto istante t ;
- Il gradiente che proviene dal futuro, ovvero dall'istante $t + 1$.

Recurrent Neural Networks: Computational Graph (step loss)



Esempio codice Seq2Seq:

```
# The loss is calculated at EVERY step and accumulated
current_hidden = torch.zeros(30)
losses_list = []

for t in range(5):
    current_input = input_sequence[t]
    current_target = targets[t] # We have a target for every single step

    network_input = torch.concat((current_input, current_hidden), dim=0)
    current_hidden = rnn(network_input)

    # We make a prediction at THIS specific time step
    current_output = mlp(current_hidden)

    # Calculate the local loss for this step and save it
    step_loss = loss_fn(current_output, current_target)
    losses_list.append(step_loss)

# Average all the step losses into a single total loss
total_loss = torch.stack(losses_list).mean()

# BPTT flows backward from multiple points simultaneously
total_loss.backward()
```

11.4 Problema del Vanishing Gradient

Il problema del **Vanishing Gradient** si manifesta quando il gradiente associato alla Loss Function diventa progressivamente troppo piccolo durante la fase di Backpropagation, fino a risultare praticamente nullo. In tali condizioni, l'aggiornamento dei pesi della rete diventa inefficace o trascurabile.

Nelle RNN, questo fenomeno è particolarmente rilevante poiché l'errore viene propagato all'indietro nel tempo attraverso la procedura di BPTT. Tale processo comporta una lunga sequenza di moltiplicazioni di derivate e matrici jacobiane associate a ciascun passo temporale.

Quando i valori singolari delle matrici coinvolte sono inferiori a 1, il prodotto tende a decrescere esponenzialmente con l'aumentare della distanza temporale k , portando alla progressiva attenuazione del segnale di gradiente.

Di conseguenza, i parametri associati agli stati più remoti nel tempo ricevono aggiornamenti sempre più piccoli, fino a diventare sostanzialmente nulli. Questo impedisce alla rete di apprendere dipendenze a lungo termine all'interno della sequenza, limitandone la capacità di modellare correlazioni tra eventi distanti.

Tale limitazione strutturale della BPTT rappresenta uno dei principali motivi per cui sono state sviluppate architetture più avanzate, come le **Long Short-Term Memory** e le **Gated Recurrent Unit**. Questi modelli introducono meccanismi di gating che regolano il flusso delle informazioni e del gradiente, consentendo di preservare selettivamente l'informazione rilevante nel tempo e mitigando in modo significativo il problema del vanishing gradient.

11.5 Long Short-Term Memory

Le reti **Long Short-Term Memory (LSTM)** introducono il concetto di "cella di memoria" dotata di un meccanismo di gating, che permette alla rete di immagazzinare, leggere o cancellare informazioni nel tempo in modo selettivo.

Il flusso di informazioni all'interno della cella LSTM è governato da quattro componenti matematiche principali:

- **Input Gate (i_t):** Controlla quanta della nuova informazione corrente deve essere scritta nella memoria. È attivato da una funzione sigmoide (σ):

$$i_t = \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{t-1} + b_{hi})$$

- **Forget Gate (f_t):** Decide quale porzione delle informazioni passate deve essere scartata o "dimenticata" dalla cella. Utilizza la sigmoide (σ) per schiacciare i valori tra 0 (dimentica tutto) e 1 (ricorda tutto):

$$f_t = \sigma(W_{if}x_t + b_{if} + W_{hf}h_{t-1} + b_{hf})$$

- **Candidate Memory (g_t):** Anche detta *Cell Gate*. Calcola i nuovi valori candidati che potrebbero essere aggiunti alla memoria. Utilizza la tangente iperbolica ($\tanh$) per generare valori tra -1 e 1:

$$g_t = \tanh(W_{ig}x_t + b_{ig} + W_{hg}h_{t-1} + b_{hg})$$

- **Output Gate (o_t):** Determina quanta parte della memoria interna della cella debba essere esposta come output verso l'esterno e passata al prossimo stato nascosto:

$$o_t = \sigma(W_{io}x_t + b_{io} + W_{ho}h_{t-1} + b_{ho})$$

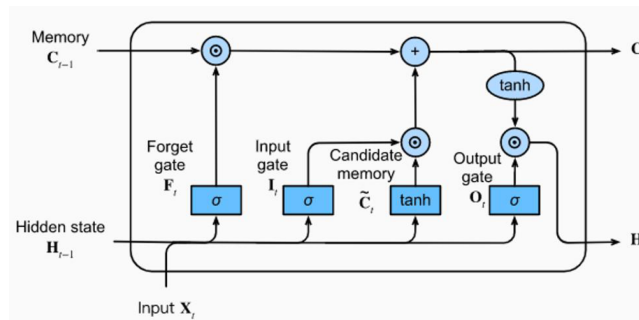
L'aggiornamento finale del sistema avviene in due fasi:

1. **Aggiornamento della Memoria (c_t):** La memoria precedente (c_{t-1}) viene moltiplicata per il forget gate e poi le viene sommata la nuova informazione candidata pesata dall'input gate:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

2. **Aggiornamento dello Stato Nascosto (h_t):** Il nuovo stato nascosto si ottiene moltiplicando l'output gate per la memoria appena calcolata (passata attraverso una $\tanh$):

$$h_t = o_t \odot \tanh(c_t)$$



Il nuovo Cell State C_t combina il passato filtrato dal Forget Gate f_t con una nuova informazione. Se $f_t \approx 1$, il gradiente attraversa il Cell State quasi invariato ($\frac{\partial C_t}{\partial C_{t-1}} \approx 1$), evitando il problema del Vanishing Gradient, in modo analogo alle Skip Connection delle ResNet.

11.6 Gated Recurrent Unit

Le **Gated Recurrent Unit (GRU)** nascono con lo stesso obiettivo delle LSTM, ma offrono un'architettura più semplice. A differenza delle LSTM, le GRU non possiedono una memoria cellulare separata dallo stato nascosto e riducono i meccanismi di controllo a soli tre gate: il **reset gate**, l'**update gate** e il **new gate**.

Il funzionamento delle porte nella GRU è il seguente:

- **Reset Gate (r_t):** Determina quanta parte del passato deve essere dimenticata dall'algoritmo. È attivato da una sigmoide, se il suo valore tende a zero l'informazione precedente viene cancellata:

$$r_t = \sigma(W_{ir}x_t + b_{ir} + W_{hr}h_{t-1} + b_{hr})$$

- **Update Gate (z_t):** Svolge un ruolo simile alla combinazione di input e forget gate della LSTM, stabilendo quanto del passato deve essere mantenuto attivo. Anch'esso è attivato dalla sigmoide:

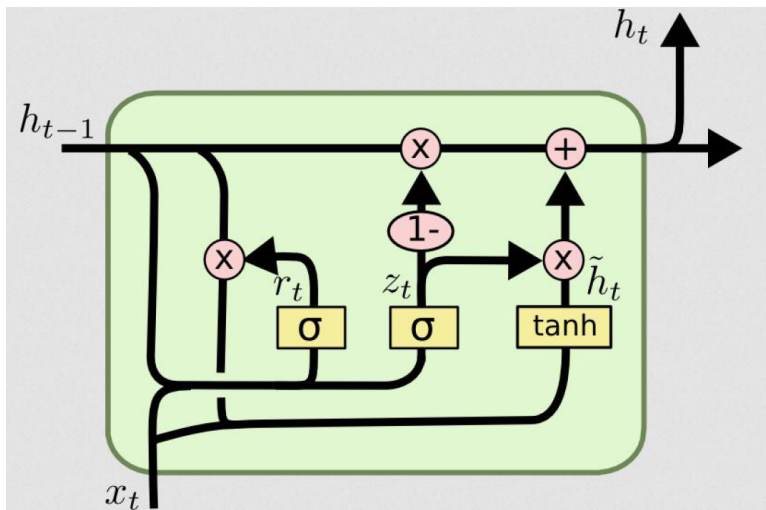
$$z_t = \sigma(W_{iz}x_t + b_{iz} + W_{hz}h_{t-1} + b_{hz})$$

- **New Gate (n_t):** Determina quanta parte del nuovo stato candidato deve essere aggiunta. Il suo calcolo dipende direttamente da ciò che il Reset Gate ha deciso di preservare del passato ed è attivato dalla tangente iperbolica (valori tra -1 e 1):

$$n_t = \tanh(W_{in}x_t + b_{in} + r_t \odot (W_{hn}h_{t-1} + b_{hn}))$$

L'aggiornamento finale è gestito interamente dall'Update Gate (z_t), che esegue un'interpolazione lineare tra il vecchio stato nascosto (h_{t-1}) e il nuovo stato candidato (n_t). **Next Hidden (h_t):**

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}$$



Il nuovo stato nascosto è una combinazione lineare (somma) tra il vecchio stato h_{t-1} e la nuova informazione $\tilde{h}_t$, controllata dalla porta di aggiornamento z_t (Update Gate). Se la rete reputa vitale ricordare il passato, l'Update Gate z_t tenderà a 1. Di nuovo, si instaura un flusso lineare. La derivata rispetto a h_{t-1} sarà vicinissima a 1, e il gradiente potrà scorrere all'indietro nel tempo superando il problema del Vanishing Gradient senza subire attenuazioni.

11.7 Codice

Sequence Classification

```

1  class RNNSeq2Task(torch.nn.Module):
2      def __init__(self, input_size, hidden_size, output_size):
3          super().__init__()
4
5          self.input_size = input_size
6          self.hidden_size = hidden_size
7          self.output_size = output_size
8
9          # Create the RNN cell schema
10         self.rnn = torch.nn.Sequential(
11             torch.nn.Linear(input_size + hidden_size, hidden_size),
12             torch.nn.LeakyReLU(),
13             torch.nn.Linear(hidden_size, hidden_size),
14             torch.nn.LeakyReLU(),
15             torch.nn.Linear(hidden_size, hidden_size)
16         )
17
18         # Feed-forward output layer for classification
19         self.ff = torch.nn.Linear(hidden_size, output_size)
20
21     def forward(self, x):
22         hidden_state = torch.zeros(
23             x.shape[0],
24             self.hidden_size,
25             device=x.device
26         )
27
28         for t in range(x.shape[1]):
29             # Concatenate the sequence point with the hidden state
30             input = torch.cat([x[:, t, :], hidden_state], dim=1)
31             # Apply RNN cell
32             hidden_state = self.rnn(input)
33
34         return self.ff(hidden_state)
35
36 # Training
37 dataloaders = nn.DataLoader(dataset, batch_size=32, shuffle=True)
38
39
40 model = Seq2Task()
41
42 loss_fn = nn.BCELoss()
43 optimizer = optim.Adam(model.parameters(), lr=0.01)
44
45 for epoch in range(num_epochs):
46     for input, cls in dataloaders:
47         output = forward(input)
48         loss = loss_fn(output, cls)
49
50         optimizer.zero_grad()
51         loss.backward()
52         optimizer.step()

```

Sequence Generation

```

1  class RNNSeq2Seq(nn.Module):
2      def __init__(self, input_size, hidden_size, output_size):
3          super().__init__()
4          self.input_size = input_size
5          self.hidden_size = hidden_size
6          self.output_size = output_size
7
8          # Custom RNN cell (similar to the provided code)
9          self.rnn_cell = nn.Sequential(
10             nn.Linear(input_size + hidden_size, hidden_size),
11             nn.LeakyReLU(),
12             nn.Linear(hidden_size, hidden_size),
13             nn.LeakyReLU(),
14             nn.Linear(hidden_size, hidden_size)
15         )
16
17         # Feed-forward output layer applied to each step
18         self.ff = nn.Linear(hidden_size, output_size)
19
20     def forward(self, x):
21         # x shape: (Batch, Sequence_Length, Features)
22         hidden_state = torch.zeros(x.shape[0], self.hidden_size, device=x.device)
23
24         # List to store the output of each time step
25         outputs = []
26
27         for t in range(x.shape[1]):
28             # Concatenate input at time t with the previous hidden state
29             combined_input = torch.cat([x[:, t, :], hidden_state], dim=1)
30
31             # Update hidden state
32             hidden_state = self.rnn_cell(combined_input)
33
34             # Compute output for the current time step
35             step_output = self.ff(hidden_state)
36             outputs.append(step_output)
37
38         # Stack outputs along the sequence dimension (dim=1)
39         # Final shape: (Batch, Sequence_Length, Output_Size)
40         return torch.stack(outputs, dim=1)

```

LSTM

```

1  class LSTMModel(nn.Module):
2      def __init__(self, input_size, hidden_size, output_size):
3          super().__init__()
4          self.hidden_size = hidden_size
5
6          # LSTM cell handles the complex gate mathematics internally
7          self.lstm_cell = nn.LSTMCell(input_size, hidden_size)
8
9          # Output layer
10         self.ff = nn.Linear(hidden_size, output_size)
11
12     def forward(self, x):
13         # LSTM requires TWO states: Hidden State (h) and Cell State (c)
14         h_t = torch.zeros(x.shape[0], self.hidden_size, device=x.device)
15         c_t = torch.zeros(x.shape[0], self.hidden_size, device=x.device)
16

```

```
17     for t in range(x.shape[1]):
18         # The cell takes the input and a tuple of the previous states
19         h_t, c_t = self.lstm_cell(x[:, t, :], (h_t, c_t))
20
21     # For Seq2Task, we only care about the very last hidden state
22     return self.ff(h_t)
```

GRU

```
1  class GRUModel(nn.Module):
2      def __init__(self, input_size, hidden_size, output_size):
3          super().__init__()
4          self.hidden_size = hidden_size
5
6          # GRU cell (more efficient than LSTM, merges cell and hidden states)
7          self.gru_cell = nn.GRUCell(input_size, hidden_size)
8
9          self.ff = nn.Linear(hidden_size, output_size)
10
11     def forward(self, x):
12         # GRU requires only ONE state (Hidden State)
13         h_t = torch.zeros(x.shape[0], self.hidden_size, device=x.device)
14
15         for t in range(x.shape[1]):
16             # Update hidden state
17             h_t = self.gru_cell(x[:, t, :], h_t)
18
19         return self.ff(h_t)
```

Capitolo 12: Variational Autoencoder

12.1 Autoencoder

Gli **Autoencoder** sono un'architettura per problemi di **unsupervised learning**. Il loro scopo non è classificare, ma imparare una rappresentazione compatta ed efficiente dei dati.

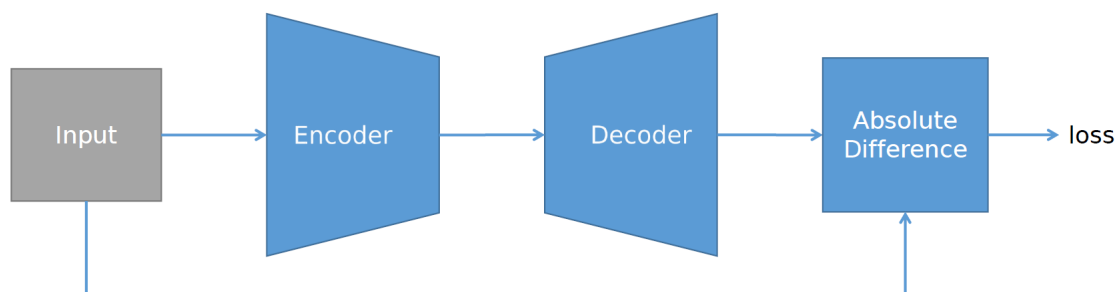
Un Autoencoder è progettato per comprimere l'input e poi ricostruirlo. È composto da due Neural Network contrapposte:

- **Encoder:** Prende il dato in input (esempio: un'immagine ad alta risoluzione) e lo comprime in una rappresentazione a bassa dimensionalità.
- **Decoder:** Prende questa rappresentazione compressa e cerca di ricostruire il dato originale con la massima fedeltà possibile.

L'output dell'Encoder è chiamato **Latent Space** (*Spazio Latente*). È una rappresentazione interna e compressa dei dati (un riassunto) che la Neural Network impara automaticamente. Cattura solo le caratteristiche (feature) più salienti e fondamentali dei dati, scartando i dettagli inutili.

L'addestramento avviene minimizzando il **Reconstruction Error** (*Errore di Ricostruzione*), ovvero la differenza assoluta (esempio: Mean Absolute Error) tra l'input originale passato all'Encoder e l'output generato dal Decoder. Questo errore misura quanto bene l'Autoencoder riesce a ricostruire l'input originale a partire dalla sua rappresentazione compatta nello spazio latente.

Il Decoder invece prende in ingresso i dati trasformati nello spazio latente e cerca di ricostruire l'input originale.



```

1  class Autoencoder(nn.Module):
2      def __init__(self, input_size, hidden_size):
3          super().__init__()
4          self.encoder = nn.Sequential(
5              nn.Linear(input_size, 300),
6              nn.LeakyReLU(),
7              nn.Linear(300, 300),
8              nn.LeakyReLU(),
9              nn.Linear(300, hidden_size)
10         )
11
12         self.decoder = nn.Sequential(
13             nn.Linear(hidden_size, 300),
14             nn.LeakyReLU(),
15             nn.Linear(300, 300),
16             nn.LeakyReLU(),
17             nn.Linear(300, input_size)
18         )
19
20     def forward(self, x):
21         latent = self.encoder(x)
22         reconstruction = self.decoder(latent)
23         return reconstruction

```

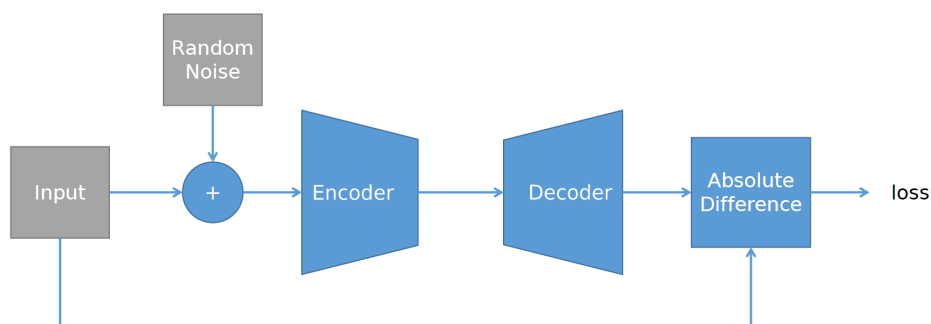
```

1  dataloaders = nn.DataLoader(dataset_MNIST, batch_size=32, shuffle=True)
2  model = Autoencoder()
3  loss_fn = nn.L1Loss() # Mean Absolute Error
4  optimizer = optim.Adam( model . parameters (), lr =0.01)
5
6  for epoch in range(num_epochs):
7      for input, _ in dataloaders:
8          output = model(input)
9          loss = loss_fn(output, input)
10         optimizer.zero_grad()
11         loss.backward()
12         optimizer.step()

```

12.2 Denoising Autoencoder

Il **Denoising Autoencoder** è una variante progettata per gestire dati rumorosi e imparare feature molto più robuste. Invece di passare l'input pulito all'Encoder, si aggiunge artificialmente del rumore casuale (Random Noise) all'input. Tuttavia, l'errore di ricostruzione finale viene calcolato confrontando l'output del Decoder con l'input originale pulito (senza rumore). In questo modo, la rete non si limita a memorizzare i pixel, ma impara attivamente a "pulire" i dati e a estrarre concetti generici e resistenti alle perturbazioni.



```

1  class DenoisingAutoencoder(nn.Module):
2      def __init__(self, , input_size, hidden_size):
3          super().__init__()
4          self.encoder = nn.Sequential(
5              nn.Linear(input_size, 300),
6              nn.LeakyReLU(),
7              nn.Linear(300, 300),
8              nn.LeakyReLU(),
9              nn.Linear(300, hidden_size)
10         )
11
12         self.decoder = nn.Sequential(
13             nn.Linear(hidden_size, 300),
14             nn.LeakyReLU(),
15             nn.Linear(300, 300),
16             nn.LeakyReLU(),
17             nn.Linear(300, input_size)
18         )
19
20     def forward(self, x):
21         noise_x = x + torch.randn(x.shape) # ADDING NOISE
22         latent = self.encoder(noise_x)
23         reconstruction = self.decoder(latent)
24         return reconstruction

```

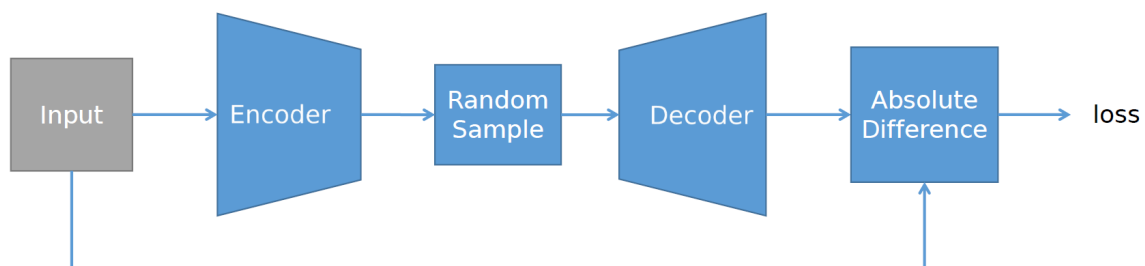
12.3 VAE come Modelli Generativi

I **Variational Autoencoders (VAE)** sono un tipo di Autoencoder progettati non solo per imparare una rappresentazione compatta dei dati, ma anche per cercare di **generare nuovi dati** a partire dalla rappresentazione nello spazio latente dell'input. Non sono più solo strumenti di compressione, ma sono vere e proprie *Architetture Generative*.

Per fare ciò i VAE utilizzano un approccio probabilistico, che permette di campionare nuovi punti nello spazio latente e generare nuovi dati. Lo spazio latente è continuo e non discreto. In un VAE, l'Encoder non mappa l'input in un singolo punto fisso nello spazio latente, ma produce i parametri di una distribuzione di probabilità (tipicamente una Gaussiana). Per ogni input, l'Encoder restituisce due vettori:

- La **Media** (μ);
- La **Deviazione Standard** (σ).

Il Decoder, per ricostruire l'immagine, non usa direttamente μ e σ , ma prende un punto campionato casualmente da questa distribuzione. Dato che l'Encoder genera un'area di probabilità e non un punto rigido, il Decoder impara a decodificare intere zone dello spazio latente, permettendo la generazione continua e fluida di nuovi contenuti.

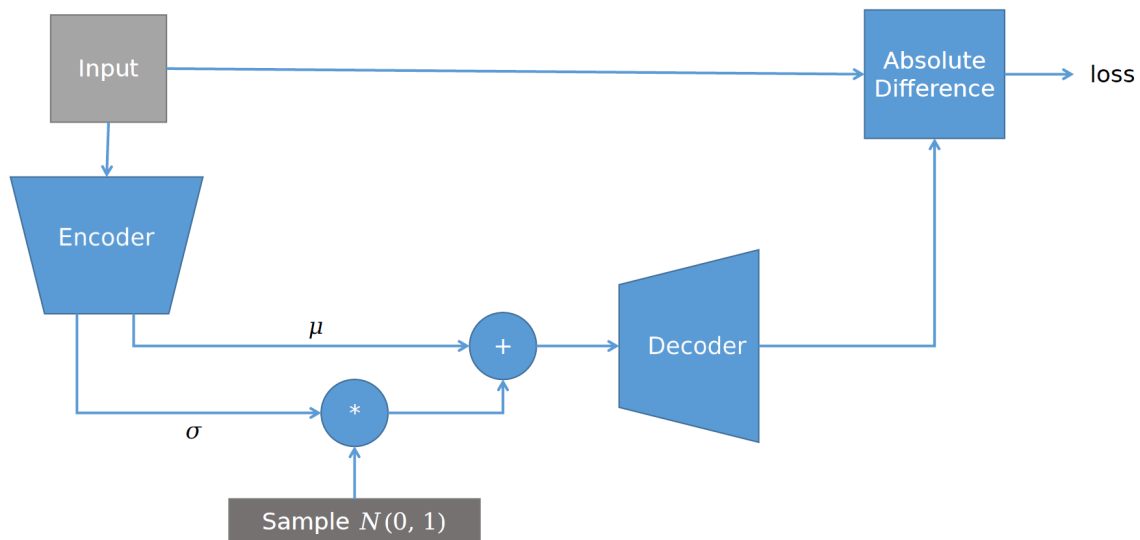


Questo introduce un problema: **il campionamento probabilistico non è differenziabile**, il che significa che la Backpropagation si bloccherebbe a quel livello, impedendo l'addestramento dell'Encoder.

Per risolvere questo blocco si usa il **Reparameterization Trick**: invece di campionare z direttamente dalla distribuzione complessa $\mathcal{N}(\mu; \sigma)$, si campiona un valore di rumore ε (o $\hat{z}$) da una distribuzione normale standard $\mathcal{N}(0; 1)$. Poi, si calcola il vettore latente z tramite operazioni matematiche base (somma e moltiplicazione):

$$z = \mu + \sigma \cdot \varepsilon$$

Poiché la somma e la moltiplicazione sono derivabili, il gradiente può ora fluire liberamente all'indietro attraverso μ e σ fino all'Encoder.



Di conseguenza non si può più utilizzare soltanto il Reconstruction Error come Loss Function, perché il modello ha natura generativa e l'obiettivo non è solo ricostruire correttamente gli input, ma anche apprendere una distribuzione latente che permetta di generare nuovi dati coerenti. L'addestramento dei VAE richiede di minimizzare due componenti:

- **Reconstruction Error:** Garantisce che il Decoder sia in grado di ricostruire correttamente i dati di input a partire dalla rappresentazione latente, mantenendo quindi informazione sufficiente nella codifica;
- **Kullback-Leibler Divergence (KLD):** È una formula matematica che misura quanto la distribuzione generata dall'Encoder ($\mu; \sigma$) si discosti da una distribuzione normale standard $\mathcal{N}(0; 1)$. Questa Loss costringe lo spazio latente a rimanere denso, regolare e centrato attorno allo zero, evitando che la rete "bari" mappando input diversi in zone isolate e lontanissime dello spazio (il che impedirebbe la generazione di nuovi dati coerenti).

$$\mathcal{L} = \alpha \cdot \text{Reconstruction_Error} + \beta \cdot \text{KLD}$$

Con:

$$\text{KLD}(q(z|x) \| p(z)) = -\frac{1}{2} \sum_{i=1}^d (1 + \log \sigma_i^2 - \mu_i^2 - \sigma_i^2)$$

```

1  class VAE(nn.Module):
2      def __init__(self, input_size, hidden_size):
3          super().__init__()
4
5          self.encoder = nn.Sequential(
6              nn.Linear(input_size, 300),
7              nn.LeakyReLU(),
8              nn.Linear(300, 300),
9              nn.LeakyReLU()
10         )
11
12         self.mu = nn.Linear(300, hidden_size)
13         self.std = nn.Linear(300, hidden_size)
14
15         self.decoder = nn.Sequential(
16             nn.Linear(hidden_size, 300),
17             nn.LeakyReLU(),
18             nn.Linear(300, 300),
19             nn.LeakyReLU(),
20             nn.Linear(300, input_size)
21         )
22
23     def encode(self, x):
24         latent = self.encoder(x)
25         mu = self.mu(latent)
26         std = torch.abs(self.std(latent))
27         return mu, std
28
29     def generate(self, mu, std):
30         z = mu + std * torch.randn_like(mu)
31         return self.decoder(z)
32
33     def forward(self, x, kl=False):
34         mu, std = self.encode(x)
35         reconstruction = self.generate(mu, std)
36         if kl:
37             kl_divergence = 0.5 * (std**2 + mu**2 - torch.log(std**2) - 1).sum(1).mean(dim=0)
38         return reconstruction, kl_divergence
39     return reconstruction

```

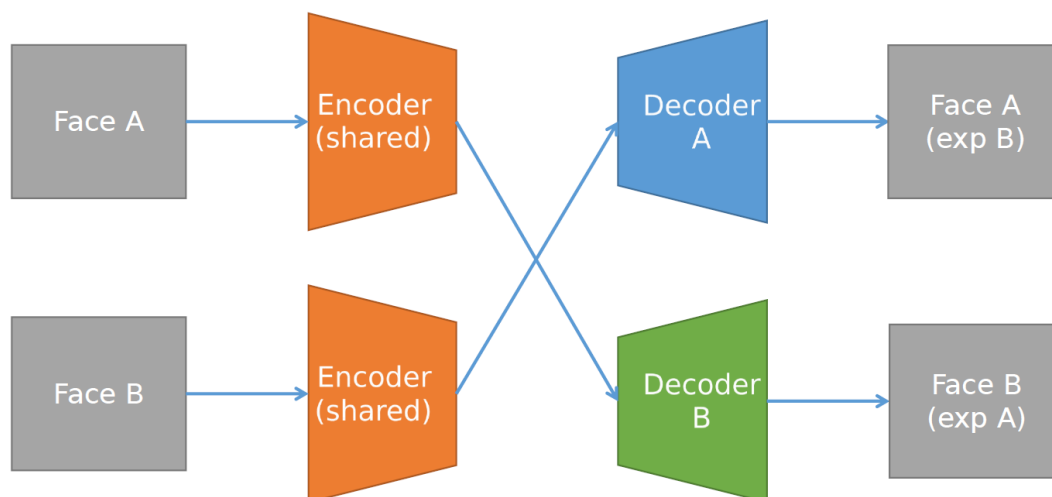
12.4 Face Swap

Il **Face Swap** è una tecnica basata sui VAE ed utilizzata per sintetizzare nuove immagini a partire da due differenti identità facciali. L'architettura tipica prevede l'impiego di due Decoder distinti associati a uno stesso Encoder condiviso.

Durante la fase di addestramento, il modello viene alimentato con immagini appartenenti a due soggetti differenti. L'Encoder apprende una rappresentazione latente comune, mentre ciascun Decoder viene ottimizzato per la ricostruzione del volto di uno specifico individuo (ad esempio, il Decoder A per il soggetto A e il Decoder B per il soggetto B).

In fase di inferenza, è possibile sfruttare la struttura condivisa dello spazio latente: un'immagine di un soggetto A viene codificata dall'Encoder e la sua rappresentazione latente viene fornita al Decoder del soggetto B, ottenendo così una ricostruzione che conserva la struttura del volto di partenza ma con le caratteristiche dell'identità target, e viceversa.

Questo meccanismo consente la generazione di immagini ibride che combinano in modo coerente attributi facciali provenienti da identità diverse.



12.5 Vector Quantized Variational Autoencoders

I **Vector Quantized Variational Autoencoder (VQ-VAE)** è un tipo di architettura di Neural Network che combina elementi dei Variational Autoencoder e di quantizzazione vettoriale per apprendere rappresentazioni discrete dei dati. È particolarmente utile per applicazioni in cui le rappresentazioni discrete o categoriali sono più *"vantaggiose"* di quelle continue.

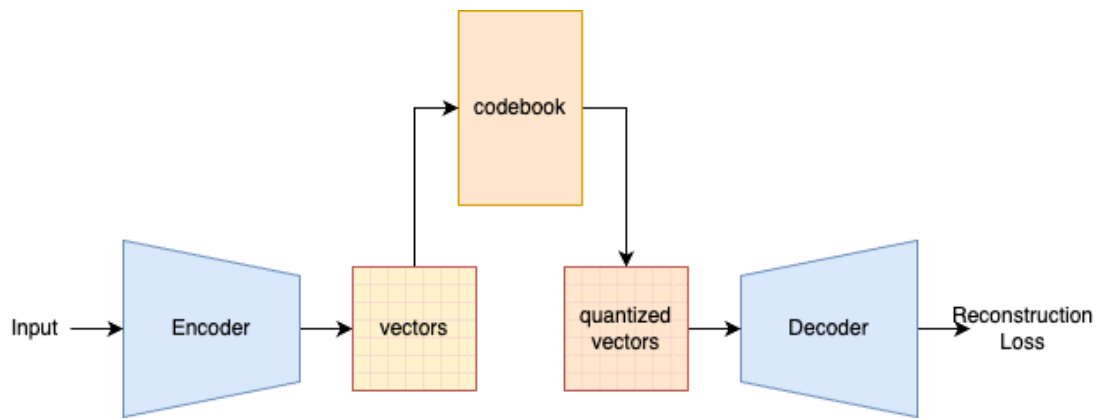
Quindi la principale differenza è che, mentre un VAE standard mappa i dati di input in uno spazio latente continuo (usando media e varianza), un VQ-VAE mappa i dati in uno spazio latente discreto (categorico).

L'architettura mantiene la struttura Encoder-Decoder, ma introduce un elemento cruciale al centro: il **Codebook**. Il Codebook è semplicemente una matrice (una lista) che contiene un numero fisso di vettori continui, chiamati *embeddings*. Ognuno di questi vettori rappresenta una specifica "categoria" o "codice". La rappresentazione continua è riorganizzata in una matrice, che ha le stesse colonne del Codebook, e il numero di righe è uguale al numero di valori della codifica dell'input.

Il flusso dei dati funziona in questo modo:

1. **Encoder:** Prende l'input (esempio: un'immagine) e lo trasforma in una rappresentazione vettoriale continua;
2. **Vector Quantization:** Il vettore continuo in uscita dall'Encoder viene confrontato con tutti i vettori presenti nel Codebook. L'algoritmo calcola la distanza (solitamente la distanza Euclidea tramite la funzione `cdist` in *PyTorch*) tra l'output dell'Encoder e i vettori del Codebook;
3. **Assegnazione (Argmin):** L'output dell'Encoder viene fisicamente sostituito dal vettore del Codebook che gli è più vicino (quello con la distanza minima);
4. **Decoder:** Il Decoder riceve questo vettore discreto pescato dal Codebook e lo usa per ricostruire una versione discreta dell'input originale.

Il Codebook è una tabella di vettori appresi, che rappresentano le possibili configurazioni discrete dello spazio latente. Non è costruito manualmente, ma viene inizializzato e poi ottimizzato durante l'addestramento insieme a Encoder e Decoder.



L'inizializzazione del Codebook può avvenire secondo due metodologie principali:

- **Random:** campionamento casuale dei vettori (tipicamente da una distribuzione normale). Risulta un approccio di semplice implementazione, ma caratterizzato da una minore stabilità;
- **Da Dati:** la pratica più diffusa, che consiste nell'elaborare un batch iniziale tramite l'Encoder, raccogliere i vettori $z_e(x)$ e applicare un algoritmo di Clustering (come il k-Means) per inizializzare i centroidi. Questa opzione garantisce una maggiore stabilità durante la fase di addestramento, poiché il Codebook viene generato in prossimità della distribuzione reale dello spazio latente.

Poi si prende il vettore continuo $z_e(x)$ dall'Encoder (la descrizione compressa dell'input) e si cerca il vettore del Codebook più simile. La similarità è misurata con la distanza euclidea:

$$\|z_e(x) - e_j\|^2$$

Per calcolare questo di solito si usa la `cdist` (come già accennato), che calcola la distanza tra due tensori e crea una matrice dove ogni elemento è la distanza fra due righe dei tensori. Vengono confrontate tutte le righe del primo tensore con tutte le righe del secondo tensore. Ogni riga del Codebook è un embedding e bisogna scegliere quella più simile, ossia:

$$k = \arg \min_j \|z_e(x) - e_j\|^2$$

Questo significa che si confronta $z_e(x)$ con tutti i vettori del Codebook, si trova quello più vicino e si prende il suo indice (di riga) k .

Quest'ultimo passaggio ($\arg \min$) **non è differenziabile**, il che significa che la Backpropagation si bloccherebbe al Codebook.

Per superare questo ostacolo si utilizza il **Quantization Trick**, che consiste nello scollegare dal grafo computazionale l'arco che calcola la differenza. Sia x l'uscita continua dell'Encoder, e sia q il vettore discreto scelto dal Codebook. Invece di passare semplicemente q al Decoder, si passa la seguente formula:

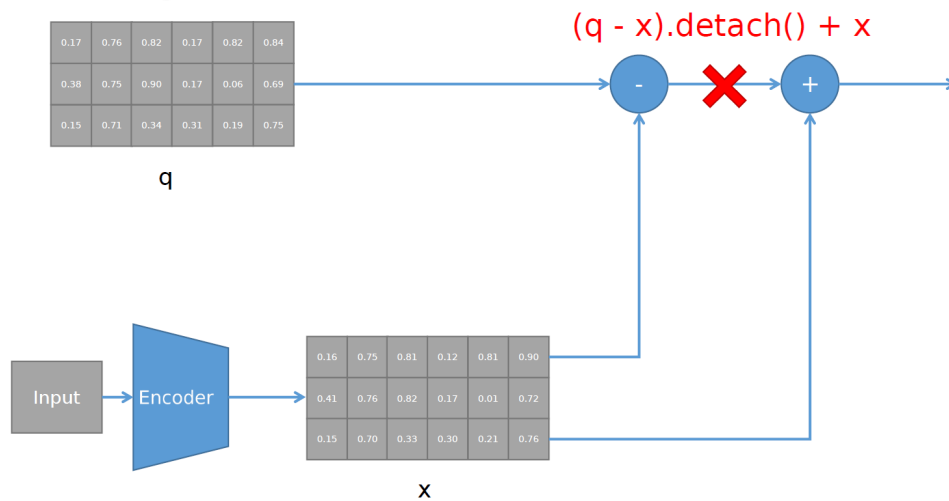
$$Output = (q - x).detach() + x$$

In pratica:

- **Nel Forward pass:** L'operazione `.detach()` in PyTorch "scollega" la variabile dal calcolo del gradiente, trattandola come una costante. Quindi matematicamente abbiamo $q - x + x$, che fa esattamente q . Il Decoder riceve correttamente il vettore quantizzato q ;
- **Nel Backward pass:** Durante il calcolo dei gradienti, la parte col `.detach()` viene completamente ignorata (come se avesse derivata 0). Quindi la derivata "scivola" attraverso $+x$. In altre parole, il gradiente calcolato dal Decoder viene copiato e incollato direttamente sull'Encoder, bypassando del tutto il blocco non-differenziabile del Codebook.

Da notare che questo trucco introduce una stima del gradiente, che è euristica e non corrisponde al gradiente esatto.

VQVAE: quantization trick



Il Decoder di un VQ-VAE è responsabile di ricostruire l'input a partire dalla rappresentazione discreta dello spazio latente, che sono poi i vettori del codebook più vicini all'input. Il Decoder prende i vettori del codebook e li trasforma in un'output, che è la ricostruzione dell'input originale.

Poiché abbiamo introdotto il Codebook, non basta più il Reconstruction Error. L'addestramento di un VQ-VAE deve bilanciare tre diverse funzioni di errore:

- **Reconstruction Error:** La classica differenza assoluta (MAE) o quadratica (MSE) tra l'immagine originale in input e l'immagine ricostruita dal Decoder;
- **Codebook Loss:** Misura la differenza tra i vettori del codebook e i vettori dell'input. È necessario che i vettori del dizionario si "*muovano*" verso gli output dell'Encoder per rappresentarli meglio. Si calcola come la distanza al quadrato tra l'output dell'Encoder (bloccato col `.detach()`) e il vettore del Codebook;
- **Commitment Loss:** Serve a evitare che l'output dell'Encoder fluttui impazzito tra un codice e l'altro. Costringe l'Encoder a "*impegnarsi*" (commit) verso il vettore del Codebook che ha scelto. Si calcola come la distanza al quadrato tra l'output dell'Encoder e il vettore del Codebook (bloccato col `.detach()`).

Essendo un modello generativo, il VQ-VAE permette di creare immagini inedite. Per farlo, non usiamo più l'Encoder. Generiamo semplicemente una griglia di codici casuali (esempio: numeri interi da 0 a 511, se il Codebook ha 512 vettori). Peschiamo i vettori corrispondenti dal Codebook e li passiamo direttamente al Decoder, che genererà una nuova immagine basata su quella specifica combinazione di concetti discreti.

vector-quantize-pytorch è una libreria e implementazione Python che facilita l'utilizzo dei VQ-VAE in PyTorch.

```

1  class VQVAE(torch.nn.Module):
2      def __init__(self, *, input_dim, codebook_dim, vector_dim, num_embeddings):
3          super().__init__()
4
5          self.input_dim = input_dim
6          self.codebook_dim = codebook_dim
7          self.vector_dim = vector_dim
8          self.num_embeddings = num_embeddings
9
10         # ENCODER
11         self.encoder = torch.nn.Sequential(
12             torch.nn.Linear(input_dim, 300),
13             torch.nn.LeakyReLU(),
14             torch.nn.Linear(300, 300),
15             torch.nn.LeakyReLU(),
16             torch.nn.Linear(300, vector_dim * num_embeddings),
17         )
18
19         self.vq = VectorQuantize(
20             dim=vector_dim,
21             codebook_size=codebook_dim,
22             decay=0.8,
23             commitment_weight=1.0
24         )
25
26         # DECODER
27         self.decoder = torch.nn.Sequential(
28             torch.nn.Linear(vector_dim * num_embeddings, 300),
29             torch.nn.LeakyReLU(),
30             torch.nn.Linear(300, 300),
31             torch.nn.LeakyReLU(),
32             torch.nn.Linear(300, input_dim),
33             torch.nn.Sigmoid()
34         )
35
36         def encode(self, x):
37             embeddings = self.encoder(x)
38             embeddings = embeddings.view(embeddings.shape[0], self.num_embeddings, self.vector_dim)
39             return embeddings
40
41         def quantize(self, embeddings):
42             quantized, codes, commit_loss = self.vq(embeddings)
43             return quantized, codes, commit_loss
44
45         def decode(self, x):
46             return self.decoder(x)
47
48         def decode_from_codes(self, codes):
49             embeddings = self.vq.codebook[codes]
50             flat_quantized = embeddings.view(embeddings.shape[0], -1)
51             return self.decoder(flat_quantized)
52
53         def forward(self, x, return_codes=False):
54             embeddings = self.encode(x)
55             quantized, codes, commit_loss = self.quantize(embeddings)
56             flat_quantized = quantized.view(quantized.shape[0], -1)
57             reconstructed = self.decode(flat_quantized)
58
59             if return_codes:
60                 return reconstructed, codes, commit_loss
61
62             return reconstructed, commit_loss

```

Capitolo 13: Advanced Models

13.1 Generative Adversarial Networks

Una **Generative Adversarial Network (GAN)** è un modello di Deep Neural Network progettata per generare nuovi dati (spesso nuove immagini).

La GAN non è una singola rete, ma è composta da due reti neurali distinte in competizione tra loro:

- **Discriminatore:** Funziona come un classico Classificatore binario. È responsabile di distinguere i dati reali da quelli generati. Nel caso delle immagini, prende in input un'immagine e deve decidere se quell'immagine è "Reale" (proveniente dal dataset originale, output 1) oppure "Falsa" (creata dal generatore, output 0). Il suo unico obiettivo è non farsi ingannare;
- **Generatore:** È il "falsario". Non vede mai le immagini originali. È responsabile di generare nuovi dati a partire da un vettore di rumore casuale. Nel caso delle immagini, prende in input un vettore di rumore casuale (random noise z) e cerca di modellarlo per generare un'immagine sintetica. Il suo obiettivo è produrre immagini così realistiche da ingannare il Discriminatore, costringendolo a classificarle come "Reali" (output 1).

Allenare una GAN è un processo delicato, poiché le due reti si addestrano a turni alterni. Quando se ne addestra una, l'altra deve essere rigorosamente **congelata**.

L'addestramento del Discriminatore funziona così:

1. Si genera un batch di immagini false passando del rumore al Generatore;
2. Si prende un batch di immagini reali dal dataset;
3. Si passano entrambe al Discriminatore;
4. Si calcola la Loss tramite **Binary Cross Entropy**. Il Discriminatore viene penalizzato se non assegna 1 alle immagini reali e 0 a quelle false;
5. Si aggiornano solo i pesi del Discriminatore. Il Generatore non viene toccato.

L'addestramento del Generatore funziona così:

1. Si genera un nuovo batch di immagini false partendo da nuovo rumore z ;
2. Si passano queste immagini al Discriminatore;
3. Questa volta, il target viene impostato forzatamente a 1 (Reale). Se il Discriminatore restituisce un valore basso (esempio 0.1, capendo che è falsa), il Generatore riceve un errore enorme;
4. Si aggiornano solo i pesi del Generatore per spingerlo a produrre immagini che otterranno un punteggio più alto. Il Discriminatore non viene toccato.

```

1  def train_discriminator(*, optimizer, generator, discriminator, real_images):
2      batch_size = real_images.shape[0]
3      device = real_images.device
4
5      z = torch.randn(batch_size, generator.latent_dim).to(device)
6      fake_images = generator(z)
7
8      fake_label = torch.zeros(batch_size).to(device) # 0 = fake
9      real_label = torch.ones(batch_size).to(device) # 1 = real
10
11     fake_logits = discriminator(fake_images)[: , 0]
12     real_logits = discriminator(real_images)[: , 0]
13
14     fake_loss = torch.nn.functional.binary_cross_entropy(fake_logits, fake_label)
15     real_loss = torch.nn.functional.binary_cross_entropy(real_logits, real_label)
16
17     loss = (fake_loss + real_loss) / 2
18
19     loss.backward()
20     optimizer.step()
21
22     return fake_loss.item(), real_loss.item(), fake_accuracy.item(), real_accuracy.item()

```

```

1  def train_generator(*, optimizer, generator, discriminator, real_images):
2      batch_size = real_images.shape[0]
3      device = real_images.device
4
5      z = torch.randn(batch_size, generator.latent_dim).to(device)
6      fake_images = generator(z)
7
8      real_label = torch.ones(batch_size).to(device)
9
10     fake_logits = discriminator(fake_images)[: , 0]
11     loss = torch.nn.functional.binary_cross_entropy(fake_logits, real_label)
12
13     loss.backward()
14     optimizer.step()

```

```

1  generator = Generator(latent_dim=64, output_dim=28).to("mps")
2  discriminator = Discriminator(input_dim=28).to("mps")
3
4  train_dl = torch.utils.data.DataLoader(train_mnist, batch_size=64, shuffle=True)
5
6  generator_optimizer = torch.optim.Adam(generator.parameters(), lr=2e-4)
7  discriminator_optimizer = torch.optim.Adam(discriminator.parameters(), lr=2e-4)
8
9  history = []
10
11  for epoch in range(0, 20):
12      for real_images, _ in tqdm(train_dl):
13          real_images = real_images.to("mps")
14
15          discriminator_optimizer.zero_grad()
16          generator_optimizer.zero_grad()
17
18          d_fake_loss, d_real_loss, d_fake_acc, d_real_acc = train_discriminator(
19              generator=generator,
20              discriminator=discriminator,
21              optimizer=discriminator_optimizer,
22              real_images=real_images
23          )
24
25          discriminator_optimizer.zero_grad()

```



```

26     generator_optimizer.zero_grad()
27
28     g_loss, g_accuracy = train_generator(
29         generator=generator,
30         discriminator=discriminator,
31         optimizer=generator_optimizer,
32         real_images=real_images
33     )

```

Trovare un equilibrio perfetto tra Generatore e Discriminatore è estremamente difficile. Se il Discriminatore diventa troppo bravo troppo in fretta, il Generatore non riceverà gradienti utili per imparare (il gradiente si annulla).

Per stabilizzare l'addestramento, sono nate le **Deep Convolutional GAN (DCGAN)**, che usano reti convoluzionali al posto di reti lineari. Vediamo alcuni trucchi per farle funzionare:

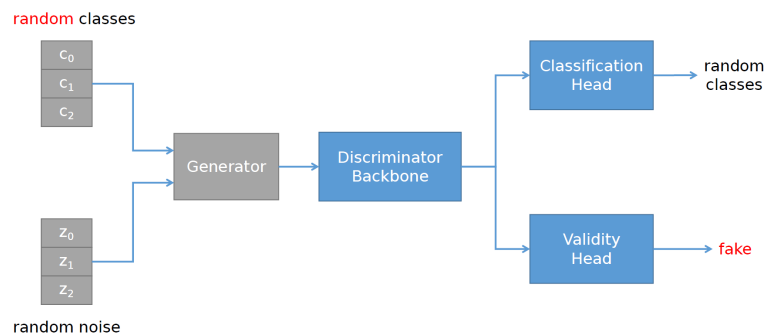
- Normalizzare le immagini tra -1 e 1;
- Preferire l'uso di CNN rispetto ai classici MLP;
- Inserire un layer **BatchNorm2d** dopo ogni convoluzione;
- Evitare i layer di **MaxPooling**, preferendo l'uso dello stride per ridurre le dimensioni;
- Inizializzare i pesi delle convoluzioni vicino allo 0 (esempio: $\mathcal{N}(0; 0.02)$) e quelli della BatchNorm a 1 ($\mathcal{N}(1; 0.02)$);
- Usare un Learning Rate molto piccolo e delicato (esempio: 0.0002).

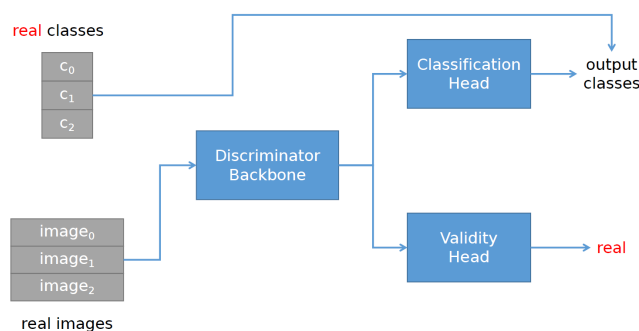
Un'**Auxiliary Classifier Generative Adversarial Network (ACGAN)** è una variante di GAN dove il discriminatore non solo cerca di distinguere i dati reali da quelli generati, ma anche di classificare le immagini in diverse classi. Questo permette di generare immagini che appartengono a classi specifiche.

Una GAN standard genera immagini casuali (non puoi chiederle di generare "un gatto" anziché "un cane"). Le ACGAN risolvono questo problema. In una ACGAN, il Discriminatore viene diviso in due "Head":

- **Validity Head:** Dice se l'immagine è Reale (1) o Falsa (0);
- **Classification Head:** Cerca di indovinare a quale classe (esempio: "cane", "auto") appartiene l'immagine.

ACGANs: Discriminator Training (generated images)



ACGANs: Discriminator Training (real images)

Come si può mantenere l'equilibrio tra casualità e informazioni sulla classe in un ACGAN?

Questo può essere fatto semplicemente sfruttando il `torch.nn.Embedding` che permette di proiettare in un vettore il numero di classi memorizzandolo all'interno di un dizionario. Nell'operazione forward del Generatore, per il quale passeremo le classi e un po' di rumore casuale come input, creeremo innanzitutto l'Embedding a partire dalle classi e poi lo moltiplicheremo per il rumore. Può essere bilanciato anche dall'utilizzo di due loss: una per la Classificazione binaria e una per la Classificazione delle classi delle immagini.

Finiamo introducendo **HuggingFace**, il più grande repository open-source (come un "GitHub per l'IA"). Offre l'accesso immediato a migliaia di modelli pre-addestrati (per testo, immagini, audio) e dataset, permettendo agli sviluppatori di usare modelli avanzati con pochissime righe di codice (grazie alla libreria transformers).

13.2 You Only Look Once

Il Deep Learning ha trasformato radicalmente il campo della **Computer Vision**, in particolare nella **Object Detection**, cioè il problema di individuare e classificare oggetti all'interno di un'immagine o di un flusso video. Tra i modelli più influenti e utilizzati in questo ambito si trova **You Only Look Once (YOLO)**, una famiglia di architetture introdotta per la prima volta nel 2015 da Joseph Redmon e successivamente evoluta in numerose varianti fino alle versioni più recenti come YOLOv8 e oltre. YOLO rappresenta un cambio di paradigma rispetto ai metodi precedenti perché affronta il problema come una singola Regressione end-to-end, invece di suddividerlo in più fasi indipendenti.

Prima dell'introduzione di YOLO, i sistemi di object detection erano generalmente basati su approcci a due stadi. Modelli come R-CNN, Fast R-CNN e Faster R-CNN funzionavano prima proponendo regioni candidate nell'immagine e poi classificandole singolarmente. Questo approccio era accurato ma computazionalmente costoso, poiché l'immagine veniva analizzata molte volte per ogni possibile regione. Di conseguenza, questi metodi risultavano poco adatti ad applicazioni in **tempo reale**, come la guida autonoma o la videosorveglianza.

YOLO introduce invece un'idea radicale: l'intera immagine viene analizzata **una sola volta** da una rete neurale convoluzionale (CNN), da cui il nome "You Only Look Once". Questo approccio a singolo passaggio (**single-stage detector**) consente di ottenere una velocità molto elevata mantenendo una buona precisione, rendendo il modello particolarmente adatto a sistemi real-time.

Dal punto di vista architetturale, YOLO utilizza una **Convolutional Neural Network (CNN)** profonda che prende in input un'immagine e produce in output un tensore strutturato che rappresenta simultaneamente la posizione e la classe degli oggetti presenti. L'immagine viene suddivisa in una

griglia (grid) di celle, e ogni cella è responsabile della predizione degli oggetti il cui centro cade al suo interno.

Per ogni cella della griglia, YOLO esegue tre compiti fondamentali in parallelo, integrati in un'unica funzione di Regressione:

- **Objectness score** (P_o): rappresenta la probabilità che nella cella sia presente un oggetto. Questo valore è tipicamente modellato come una probabilità tramite una funzione logistica (sigmoid), e permette alla rete di distinguere tra background e oggetti reali.
- **Localizzazione** (X, Y, W, H): la rete predice le coordinate del bounding box, cioè la posizione del centro dell'oggetto (X, Y) e le sue dimensioni (larghezza W e altezza H). Questi valori sono regressi come numeri continui normalizzati rispetto alla dimensione dell'immagine.
- **Classificazione** ($C_1, C_2, \dots, C_n$): ogni bounding box è associato a una distribuzione di probabilità sulle classi possibili (ad esempio persona, auto, cane, bicicletta). Questo è ottenuto tramite una funzione softmax o varianti multi-label nelle versioni più recenti.

Il risultato finale del modello è quindi una lista di bounding box, ciascuno associato a una probabilità di presenza dell'oggetto e a una classe specifica. L'aspetto chiave è che tutto questo viene predetto in un singolo forward pass della rete, senza necessità di moduli separati per proposal e Classificazione.

Un elemento fondamentale dell'evoluzione di YOLO riguarda il miglioramento della sua capacità di gestire oggetti di dimensioni diverse. Le versioni successive hanno introdotto concetti come **anchor boxes**, ovvero box predefiniti con proporzioni diverse che aiutano la rete a prevedere meglio oggetti con forme variabili. Inoltre, architetture più recenti hanno integrato backbone più potenti, spesso ispirati a modelli come Darknet, CSPDarknet o backbone derivati da EfficientNet e Transformer, migliorando significativamente la capacità di estrazione delle feature.

Un problema intrinseco nella Object Detection è la presenza di molte predizioni sovrapposte per lo stesso oggetto. Poiché diverse celle della griglia o diversi anchor possono rilevare lo stesso oggetto, YOLO genera spesso più bounding box per una singola entità. Per risolvere questo problema viene utilizzato un algoritmo di post-processing chiamato **Non-Maximum Suppression (NMS)**.

La Non-Maximum Suppression funziona in modo relativamente semplice ma efficace. Prima vengono ordinate tutte le predizioni in base al loro livello di confidenza. Successivamente, si seleziona il bounding box con la confidenza più alta e si eliminano tutti gli altri box che hanno un'elevata sovrapposizione con esso, misurata tramite l'**Intersection over Union (IoU)**. Questo processo viene ripetuto iterativamente fino a eliminare tutte le duplicazioni, lasciando solo le predizioni più affidabili e rappresentative.

Nel corso degli anni YOLO ha subito numerose evoluzioni. Le prime versioni erano estremamente veloci ma meno precise nella localizzazione degli oggetti piccoli. YOLOv2 e YOLOv3 hanno migliorato la stabilità del training e la qualità delle predizioni introducendo tecniche come batch normalization, multi-scale training e miglioramenti nella gestione delle classi. Le versioni successive, fino a YOLOv5, YOLOv7 e YOLOv8, hanno ulteriormente ottimizzato il rapporto tra velocità e accuratezza, rendendo il modello uno standard industriale in molti settori.

YOLOv8, in particolare, rappresenta una delle implementazioni più avanzate e diffuse. È caratterizzato da un design modulare, da un miglioramento della gestione degli anchor (in alcuni casi anche anchor-free), e da una maggiore efficienza computazionale. Inoltre, integra miglioramenti nei processi di training e ottimizzazione che riducono il tempo necessario per addestrare il modello su dataset complessi come COCO, uno dei benchmark più utilizzati per la valutazione dei sistemi di object detection.

Le applicazioni di YOLO sono estremamente ampie. Nel campo della **guida autonoma**, viene utilizzato per rilevare veicoli, pedoni e segnali stradali in tempo reale. Nella **videosorveglianza** permette il riconoscimento di comportamenti sospetti o la presenza di persone in aree sensibili. In

ambito industriale viene impiegato per il controllo qualità e il rilevamento di difetti nei prodotti. Anche in ambito medico, alcune varianti di YOLO vengono utilizzate per identificare anomalie in immagini radiologiche o scansioni.

Uno dei principali vantaggi di YOLO è il suo equilibrio tra velocità e accuratezza. Essendo un modello **single-stage**, è significativamente più veloce rispetto ai modelli a due stadi, pur mantenendo performance competitive. Tuttavia, questa scelta progettuale comporta anche alcuni limiti: la precisione nella localizzazione di oggetti molto piccoli o molto vicini tra loro può essere inferiore rispetto a modelli più complessi, e il modello può avere difficoltà in scenari con forte sovrapposizione di oggetti.

Dal punto di vista teorico, YOLO può essere interpretato come un problema di **Regressione multi-task**, in cui la rete apprende simultaneamente localizzazione e Classificazione. Questo approccio è reso possibile dalla capacità delle reti neurali profonde di apprendere rappresentazioni condivise tra compiti diversi, sfruttando feature comuni estratte nei livelli convoluzionali iniziali e intermedi.

13.3 Faster R-CNN

Nel panorama della **Object Detection**, un ruolo fondamentale è ricoperto da **FasterRegion-based Convolutional Neural Network (R-CNN)**, uno dei modelli più influenti sviluppati dopo le prime architetture R-CNN e Fast R-CNN. A differenza di approcci come YOLO, ottimizzati principalmente per la **velocità** e per applicazioni in tempo reale, Faster R-CNN è stato progettato con un obiettivo diverso: massimizzare la **precisione** nella localizzazione e Classificazione degli oggetti, anche a costo di una maggiore complessità computazionale.

L'idea centrale di Faster R-CNN è quella di suddividere il problema dell'object detection in due fasi strettamente integrate ma concettualmente distinte: la generazione di possibili regioni contenenti oggetti e la loro successiva Classificazione. Questa separazione permette al modello di concentrarsi prima su "dove guardare" e poi su "cosa c'è" all'interno di quelle regioni, migliorando significativamente la qualità delle predizioni rispetto ai metodi precedenti.

L'architettura di Faster R-CNN si basa su tre componenti principali. La prima è una **CNN backbone**, tipicamente rappresentata da reti profonde come **ResNet** o varianti più moderne ottimizzate per la feature extraction. Questa rete ha il compito di trasformare l'immagine in ingresso in una **feature map**, cioè una rappresentazione compatta e informativa che conserva le caratteristiche spaziali e semantiche dell'immagine. In questa fase, l'immagine non viene più analizzata a livello di pixel, ma attraverso pattern astratti come bordi, texture e forme.

Sopra questa rappresentazione si innesta la seconda componente fondamentale: la **Region Proposal Network (RPN)**. Questa rete è una delle innovazioni chiave introdotte da Faster R-CNN rispetto ai modelli precedenti come Fast R-CNN, dove le regioni candidate venivano generate da algoritmi esterni come Selective Search, molto lenti e non ottimizzati per il learning end-to-end. La RPN invece è una rete neurale completamente integrata che analizza la feature map e genera automaticamente un insieme di **regioni di interesse**.

La RPN funziona scorrendo una piccola finestra convoluzionale sulla feature map e, per ogni posizione, genera un insieme di **anchor boxes**, ovvero rettangoli predefiniti con diverse scale e aspect ratio. Per ciascun anchor, la rete predice due informazioni fondamentali: la probabilità che contenga un oggetto (**objectness score**) e una correzione delle coordinate per adattare meglio il bounding box all'oggetto reale. Questo approccio permette al modello di proporre regioni altamente candidate in modo efficiente e completamente apprendibile.

Una volta generate le regioni candidate, queste vengono passate alla terza componente del modello: il **ROI Pooling** (o nelle versioni più recenti ROI Align). Questo modulo ha il compito di trasformare regioni di dimensioni variabili in feature di dimensione fissa, rendendole compatibili con i livelli fully connected successivi. Il ROI Pooling suddivide ogni regione in una griglia e applica operazioni di

pooling per estrarre una rappresentazione uniforme, preservando le informazioni più rilevanti.

Le feature estratte vengono quindi inviate a un classificatore finale, che esegue due compiti simultanei: la **Classificazione dell'oggetto** e la **Regressione del bounding box**. In questo modo, per ogni regione proposta, il modello determina sia la classe dell'oggetto (ad esempio persona, auto, animale) sia una raffinazione delle coordinate per migliorare la precisione della localizzazione.

Uno degli aspetti più importanti di Faster R-CNN è il fatto che l'intero sistema può essere addestrato in modo **end-to-end**, almeno nelle versioni più evolute. Questo significa che la CNN backbone, la RPN e il classificatore condividono un processo di ottimizzazione comune, basato sulla backpropagation, migliorando progressivamente la qualità delle feature e delle proposte regionali.

Dal punto di vista prestazionale, Faster R-CNN rappresenta un compromesso opposto rispetto a YOLO. Se YOLO privilegia la velocità con un approccio single-stage, Faster R-CNN adotta un paradigma **two-stage**, che introduce un costo computazionale maggiore ma garantisce generalmente una precisione superiore, soprattutto nella rilevazione di oggetti piccoli o parzialmente occlusi. Questo lo rende particolarmente adatto a contesti in cui la precisione è più importante della latenza, come l'analisi di immagini mediche, la ricerca scientifica e alcuni sistemi di sorveglianza avanzata.

Nel corso del tempo, Faster R-CNN è stato ulteriormente migliorato attraverso diverse innovazioni. Una delle più importanti è stata la sostituzione del ROI Pooling con il **ROI Align**, introdotto in modelli successivi come Mask R-CNN. Questa modifica elimina la quantizzazione approssimata delle regioni, migliorando la precisione spaziale delle predizioni. Inoltre, l'evoluzione delle backbone CNN verso architetture più profonde e efficienti, come ResNet e ResNeXt, ha aumentato significativamente le prestazioni complessive del sistema.

Un altro aspetto rilevante riguarda l'utilizzo degli **anchor boxes**. La scelta delle dimensioni e proporzioni degli anchor influenza fortemente la qualità delle region proposals. Una configurazione ben progettata permette di adattarsi meglio a dataset complessi come COCO, dove gli oggetti possono variare enormemente in scala e forma.

13.4 U-Net

L'architettura **U-Net** rappresenta uno dei modelli più importanti nel campo della **Semantic Image Segmentation**, una branca della visione artificiale che ha l'obiettivo di assegnare una classe a ogni singolo pixel di un'immagine. A differenza della **Object Detection**, che si limita a individuare gli oggetti tramite bounding box rettangolari, la segmentazione semantica produce una mappa dettagliata in cui ogni pixel viene etichettato con una categoria specifica, come "strada", "auto", "pedone" o "sfondo". Questo livello di granularità rende U-Net particolarmente utile in applicazioni in cui la precisione spaziale è fondamentale.

U-Net è stata introdotta nel 2015 da *Olaf Ronneberger* e colleghi per applicazioni di **biomedical image segmentation**, in particolare per l'analisi di immagini microscopiche. La sua efficacia in questo ambito ha rapidamente portato alla sua adozione in numerosi altri settori, dalla guida autonoma all'analisi satellitare, grazie alla sua capacità di funzionare bene anche con dataset relativamente piccoli, un problema comune nel dominio medico.

L'architettura prende il nome dalla sua forma a "U", che deriva dalla struttura simmetrica composta da due rami principali: un percorso di compressione e uno di espansione. Il primo è chiamato **contraction path** o **encoder**, mentre il secondo è noto come **expansion path** o **decoder**. Questa struttura simmetrica è fondamentale per il funzionamento del modello, poiché consente di combinare informazioni di alto livello semantico con dettagli spaziali precisi.

L'**Encoder** ha il compito di estrarre caratteristiche sempre più astratte dall'immagine in ingresso. Questo processo avviene attraverso una sequenza di operazioni convoluzionali seguite da funzioni di attivazione non lineare e operazioni di pooling. Ad ogni livello, la risoluzione spaziale dell'immagine

viene progressivamente ridotta, mentre il numero di canali aumenta. Questo permette al modello di catturare informazioni sempre più complesse, passando da semplici bordi e texture a strutture più globali e semantiche. In questa fase, il modello si concentra sul **cosa** è presente nell'immagine, perdendo gradualmente parte dell'informazione spaziale fine.

Il **Decoder**, invece, ha il compito opposto: ricostruire la risoluzione originale dell'immagine a partire dalle feature compresse. Questo avviene attraverso operazioni di upsampling, spesso realizzate tramite convoluzioni trasposte o interpolazioni seguite da convoluzioni standard. L'obiettivo è quello di riportare la rappresentazione a una dimensione pixel-wise, in modo da poter assegnare una classe a ciascun pixel. Questa fase si concentra quindi sul **dove** si trovano gli oggetti all'interno dell'immagine.

Il vero elemento innovativo di U-Net è rappresentato dalle **skip connections**. Queste connessioni orizzontali collegano direttamente i livelli corrispondenti dell'encoder e del decoder, trasferendo informazioni ad alta risoluzione che altrimenti andrebbero perse durante la fase di downsampling. In pratica, mentre il percorso di compressione riduce i dettagli spaziali per estrarre feature astratte, le skip connections reintroducono informazioni locali precise, permettendo al decoder di ricostruire bordi e strutture con maggiore accuratezza. Questo meccanismo è fondamentale per ottenere segmentazioni precise, soprattutto nei contorni degli oggetti.

Dal punto di vista computazionale, U-Net è un'architettura completamente **fully convolutional**, il che significa che non utilizza layer fully connected e può quindi operare su immagini di dimensioni variabili. Questa caratteristica la rende estremamente flessibile e adatta a diversi tipi di dati. Inoltre, l'uso esclusivo di convoluzioni consente di preservare la struttura spaziale dell'immagine lungo tutto il network.

13.5 CLIP

Contrastive Language-Image Pretraining (CLIP) è un modello introdotto da OpenAI che ha rappresentato un cambiamento significativo nel modo in cui la **Computer Vision** può essere integrata con il **Natural Language Processing**. A differenza dei modelli tradizionali di Classificazione delle immagini, che vengono addestrati su dataset con etichette rigide e predefinite (ad esempio "cane = classe 0", "gatto = classe 1"), CLIP utilizza un approccio più flessibile e generale basato su coppie di dati **immagine-testo** raccolte su larga scala dal web.

L'idea fondamentale alla base di CLIP è quella di imparare una rappresentazione condivisa tra due modalità diverse: le immagini e il linguaggio naturale. Questo significa che il modello non si limita a riconoscere categorie fisse, ma apprende a collegare descrizioni testuali a contenuti visivi in modo più semantico e generalizzato. Per farlo, viene addestrato su milioni di coppie (immagine, didascalia), dove ogni immagine è associata a una descrizione testuale naturale, spesso rumorosa ma estremamente ricca dal punto di vista informativo.

Il cuore del funzionamento di CLIP è una **loss contrastiva**, una funzione di ottimizzazione che spinge il modello a massimizzare la similarità tra rappresentazioni corrette e a minimizzare quella tra rappresentazioni errate. In pratica, CLIP apprende a proiettare sia le immagini sia i testi in uno stesso **spazio vettoriale condiviso**. In questo spazio, elementi semanticamente correlati si trovano vicini tra loro: ad esempio, l'immagine di un gatto e la frase "un gatto seduto su un divano" avranno vettori molto simili, mentre una frase non correlata come "un aereo che vola nel cielo" sarà distante.

Architettonicamente, CLIP è composto da due encoder separati ma simmetricamente progettati: un **image encoder**, spesso basato su reti convoluzionali (CNN) o Transformer visivi, e un **text encoder**, tipicamente basato su Transformer simili a quelli utilizzati nei modelli di linguaggio. Entrambi gli encoder trasformano i rispettivi input in embedding vettoriali di dimensione fissa. Successivamente, viene calcolata la similarità tra questi embedding, spesso tramite prodotto scalare normalizzato o cosine similarity.

Durante l'addestramento, il modello riceve batch contenenti molte coppie immagine-testo. Per ogni immagine, il modello deve identificare quale descrizione testuale è quella corretta tra molte possibili alternative, e viceversa. Questo approccio contrastivo permette di costruire uno spazio semantico robusto, in cui la relazione tra visione e linguaggio emerge in modo emergente dai dati stessi, senza bisogno di annotazioni manuali estremamente dettagliate.

Uno degli aspetti più innovativi di CLIP è la capacità di effettuare **zero-shot learning**. Questo significa che il modello può riconoscere categorie mai viste durante l'addestramento, semplicemente fornendo una descrizione testuale. Ad esempio, se si vuole classificare immagini di animali in categorie nuove come “koala” o “panda rosso”, basta fornire queste etichette testuali al modello: CLIP confronterà l'immagine con le embedding delle descrizioni e selezionerà la più simile. Questo approccio elimina la necessità di riaddestrare il modello ogni volta che si introduce una nuova classe.

Questa capacità deriva direttamente dalla natura del suo spazio latente condiviso, che non è vincolato a un set fisso di categorie, ma è invece strutturato in base alla semantica del linguaggio naturale. In questo senso, CLIP si avvicina a una forma di comprensione più generale rispetto ai modelli tradizionali supervisionati.

Dal punto di vista dei dati, CLIP si distingue anche per la scala e la varietà del dataset utilizzato. Le coppie immagine-testo vengono raccolte direttamente da Internet, il che introduce una grande diversità ma anche una certa quantità di rumore nei dati. Tuttavia, proprio questa eterogeneità contribuisce alla robustezza del modello, che impara a generalizzare su contesti molto diversi tra loro.

Parte III

Esame

Capitolo 14: Domanda 1

14.1 MLP_MSELIN

14.1.1 Domanda Standard

Descrivi la teoria e il codice di un Multi-Layer Perceptron per problemi di Regressione a singola uscita, utilizzando l'errore quadratico medio (MSE) come Loss Function. Spiega in dettaglio il forward pass e il backward pass.

Il **Multilayer Perceptron (MLP)** è una catena di Perceptron collegati tra loro e organizzata in strati: uno di input, uno o più strati nascosti e uno di output. L'introduzione degli strati nascosti e di funzioni di attivazione non lineari consente al modello di apprendere relazioni complesse e non lineari nei dati, superando i limiti del singolo Perceptron.

MLP_MSELIN identifica una specifica configurazione di una rete neurale Multi-Layer Perceptron per risolvere problemi di Regressione. L'acronimo evidenzia due scelte architetturali fondamentali del modello:

- **MSE (Mean Squared Error):** indica l'utilizzo dell'errore quadratico medio come loss function.
- **LIN (Linear):** indica l'impiego di una funzione di attivazione lineare nell'ultimo strato (output layer).

Ogni neurone di un livello è tipicamente connesso a tutti i neuroni del livello successivo (architettura fully connected o densa).

La fase di addestramento del modello si basa sul Gradient Descent iterativo e comprende:

- **Inizializzazione:** pesi W_1 e W_2 inizializzati tramite un metodo di inizializzazione (ad esempio: randomica, o Xavier Initialization);
- **Forward pass:** I dati di input vengono moltiplicati per W_1 e passati attraverso la funzione di attivazione non lineare per produrre le uscite dello strato nascosto. Queste uscite vengono poi moltiplicate per W_2 . L'output finale è calcolato in modo puramente lineare;
- **Loss Function:** calcolo della MSE tra predizione e target;
- **Backward pass:** Si calcolano i gradienti dell'errore. Grazie all'utilizzo dell'attivazione lineare in output, il gradiente dell'MSE rispetto all'ultimo strato assume una forma matematica estremamente pulita e diretta: è semplicemente la differenza tra la predizione e il valore reale ($\hat{y} - y$). Questo errore viene propagato all'indietro attraverso la rete, moltiplicandolo per le derivate delle funzioni di attivazione del hidden layer;
- **Aggiornamento:** aggiornamento dei pesi tramite learning rate η .

Forward Pass:

$$rA0 = X^T \quad rA0 \in \mathbb{R}^{D \times B}$$

$$A0 = \text{ext}(rA0) \quad A0 \in \mathbb{R}^{D+1 \times B}$$

ext inserisci una riga di solo 1 (quindi un numero B di 1) in cima alla matrice

$$rZ1 = W1 \cdot A0 \quad rZ1 \in \mathbb{R}^{H \times B}$$

$$rA1 = \sigma(rZ1) = \frac{1}{1 + e^{-rZ1}} \quad rA1 \in \mathbb{R}^{H \times B}$$

$$A1 = \text{ext}(rA1) \quad A1 \in \mathbb{R}^{(H+1) \times B}$$

$$rZ2 = W2 \cdot A1$$

$$rA2 = rZ2 \quad rA2 \text{ è una matrice, ma un vettore in questo caso}$$

I dati trasposti $rA0$ vengono estesi con la riga di unitari per generare la matrice $A0$. Questa subisce una trasformazione affine tramite la matrice dei pesi del primo strato $W1 \in \mathbb{R}^{H \times (D+1)}$. Al risultato $rZ1$ viene applicata, elemento per elemento, la funzione di attivazione non lineare logistica (sigmoide), producendo la matrice delle attivazioni latenti $rA1$. A seguito di un'ulteriore estensione di bias ($A1$), il segnale viene proiettato scalarmente tramite la matrice d'uscita $W2 \in \mathbb{R}^{1 \times (H+1)}$. Poiché il livello di output adotta un'attivazione lineare (la funzione identità), lo stato di pre-attivazione $rZ2$ coincide direttamente con le predizioni del modello $rA2$.

Loss Function: Ottenuta la matrice delle predizioni $rA2$, la rete quantifica lo scostamento rispetto al vettore riga dei target reali $Y \in \mathbb{R}^{1 \times B}$ calcolando l'Errore Quadratico Medio:

$$\mathcal{L} = \frac{1}{2B} (rA2 - Y)(rA2 - Y)^T$$

Il fattore di normalizzazione $\frac{1}{2B}$ assolve una duplice funzione: mediare la penalità sulla dimensione del batch (rendendo l'apprendimento invariante alla cardinalità del campione) ed elidere algebricamente il coefficiente quadratico in sede di derivazione.

Backward Pass:

$$\frac{\partial \mathcal{L}}{\partial rZ2} = rA2 - y$$

$$\frac{\partial \mathcal{L}}{\partial W2} = \frac{\partial \mathcal{L}}{\partial rZ2} A1^T$$

$$\frac{\partial \mathcal{L}}{\partial A1} = \frac{\partial \mathcal{L}}{\partial rZ2} W2^T$$

$$\frac{\partial \mathcal{L}}{\partial rZ1} = \text{red}\left(\frac{\partial \mathcal{L}}{\partial A1}\right) \odot \sigma'(rZ1)$$

red rimuove la prima riga dalla matrice

$$\sigma'(rZ1) = \sigma(rZ1) \odot (1 - \sigma(rZ1))$$

$$\frac{\partial \mathcal{L}}{\partial W1} = \frac{\partial \mathcal{L}}{\partial rZ1} A0^T$$

Ove il simbolo $\odot$ indica il prodotto di Hadamard, o moltiplicazione elemento per elemento, ossia primo elemento per il primo, il secondo per il secondo e così via.

Grazie alla linearità dello strato di output, il gradiente locale dell'errore $\frac{\partial \mathcal{L}}{\partial rZ2}$ assume una forma strettamente analitica, pari al puro residuo della predizione ($\hat{y} - y$). Questo segnale viene retroproiettato verso lo strato nascosto premoltiplicandolo per la trasposta dei pesi $W2$. Successivamente, l'operatore di riduzione scarta il gradiente associato al nodo di bias (non retro-addestrabile), e il segnale risultante viene modulato dalla derivata prima della funzione sigmoide.

Aggiornamento:

$$W1 \leftarrow W1 - \eta \frac{\partial \mathcal{L}}{\partial W1}$$

$$W2 \leftarrow W2 - \eta \frac{\partial \mathcal{L}}{\partial W2}$$

```

1  import numpy as np
2  np.random.seed(42)
3
4  def MLP_initialize_weights(model):
5      h = model['n_hidden']
6      f = model['n_features']
7      o = model['n_output']
8      W1 = 2 * np.random.rand(h, f + 1) - 1
9      W2 = 2 * np.random.rand(o, h + 1) - 1
10     return W1, W2
11
12 def MLP_sigmoid(z):
13     return 1.0 / (1.0 + np.exp(-z))
14
15 def MLP_sigmoid_derivative(rZ):
16     s = MLP_sigmoid(rZ)
17     return s * (1.0 - s)
18
19 def MLP_MSE_cost(y_true, y_pred):
20     B = y_true.shape[1] # Batch size
21     return np.sum((y_true - y_pred)**2) / (2 * B)
22
23 def MLP_extend(rA):
24     B = rA.shape[1]
25     return np.vstack((np.ones(B), rA))
26
27 def MLP_MSELIN_forward(X_input, W1, W2):
28     A0 = MLP_extend(X_input.T)
29     rZ1 = W1 @ A0
30     rA1 = MLP_sigmoid(rZ1)
31     A1 = MLP_extend(rA1)
32     rA2 = W2 @ A1
33     return rA2, A1, A0, rZ1
34
35 def MLP_MSELIN_backprop(rA2, A1, A0, rZ1, Y_true, W1, W2):
36     dL_dZ2 = rA2 - Y_true
37     dL_dW2 = dL_dZ2 @ A1.T
38     dL_dA1 = W2.T @ dL_dZ2
39     dL_drZ1 = dL_dA1[1:] * MLP_sigmoid_derivative(rZ1) # [1:] drops the bias row
40     dL_dW1 = dL_drZ1 @ A0.T
41     return dL_dW1, dL_dW2
42
43 def MLP_MSELIN_train(X_train, y_train, model):
44     W1, W2 = MLP_initialize_weights(model)
45
46     eta = model['eta']
47     epochs = model['epochs']
48     m_batches = model['minibatches']
49
50     model['cost_history'] = []
51     N = X_train.shape[0]
52
53     for epoch in range(epochs):
54         perm = np.random.permutation(N)
55         X_shuf, y_shuf = X_train[perm], y_train[perm]
56
57         X_batches = np.array_split(X_shuf, m_batches)
58         y_batches = np.array_split(y_shuf, m_batches)
59

```

```

60     for X_mb, y_mb in zip(X_batches, y_batches):
61         y_mb = y_mb.T
62         rA2, A1, A0, rZ1 = MLP_MSELIN_forward(X_mb, W1, W2)
63
64         cost = MLP_MSE_cost(y_mb, rA2)
65         model['cost_history'].append(cost)
66
67         delta_W1, delta_W2 = MLP_MSELIN_backprop(rA2, A1, A0, rZ1, y_mb, W1, W2)
68
69         W1 = W1 - eta * delta_W1
70         W2 = W2 - eta * delta_W2
71
72     model['W1'] = W1
73     model['W2'] = W2
74     return model, W1, W2

```

14.2 MLP_BCESIG

14.2.1 Domanda Standard

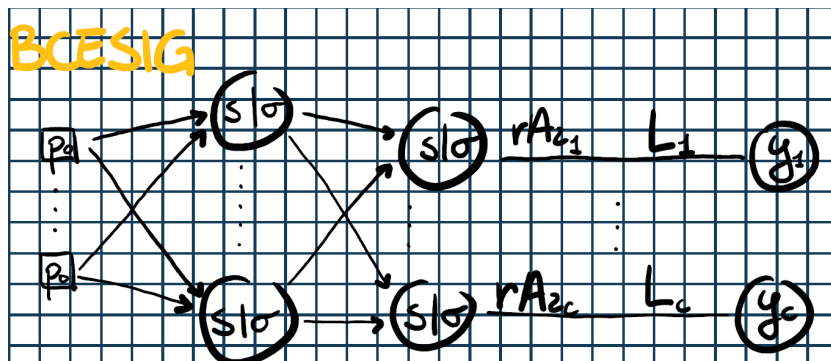
Descrivi la teoria e lo pseudocodice di un Multi-Layer Perceptron per il problema della Classificazione Multi-Label (BCESIG), quando si utilizza la funzione di attivazione sigmoid. Fornisci la formula della Binary Cross Entropy. Spiega in dettaglio sia il forward pass che il backward pass.

Il **Multilayer Perceptron (MLP)** è una catena di Perceptron collegati tra loro e organizzata in strati: uno di input, uno o più strati nascosti e uno di output. L'introduzione degli strati nascosti e di funzioni di attivazione non lineari consente al modello di apprendere relazioni complesse e non lineari nei dati, superando i limiti del singolo Perceptron.

MLP_BCESIG è una rete MLP per problemi di classificazione **multi-label**. Il nome evidenzia le due componenti principali:

- **BCE (Binary Cross-Entropy):** È la Loss Function utilizzata per misurare la discrepanza tra la previsione e la realtà.
- **SIG (Sigmoid):** È la funzione di attivazione impiegata, in particolare nello strato di output.

Sebbene BCE e Sigmoid siano tipicamente associate alla classificazione binaria, risultano adatte anche al contesto multi-label. In questo caso, l'ultimo strato contiene C neuroni, uno per ciascuna classe. Ogni neurone produce una previsione indipendente che indica la presenza o l'assenza della relativa label.



La fase di addestramento del modello si basa sul Gradient Descent iterativo e comprende:

- **Inizializzazione:** pesi W_1 e W_2 inizializzati tramite un metodo di inizializzazione (ad esempio: randomica, o Xavier Initialization);
- **Forward pass:** I dati di input vengono moltiplicati per W_1 e passati attraverso la funzione di attivazione non lineare per produrre le uscite dello strato nascosto. Queste uscite vengono poi moltiplicate per W_2 , fatte passare per un'altra sigmoide e infine restituite come vettore di label;
- **Loss Function:** calcolo della BCE tra predizione e target;
- **Backward pass:** Si calcolano i gradienti dell'errore. Questo errore viene propagato all'indietro attraverso la rete, moltiplicandolo per le derivate delle funzioni di attivazione del l'hidden layer;
- **Aggiornamento:** aggiornamento dei pesi tramite learning rate η .

D : dimensione di input (feature)

B : numero di dati nel mini-batch corrente

H : numero di neuroni nell'hidden layer

C : numero di neuroni nell'output layer

$$X \in \mathbb{R}^{D \times B}$$

$$W1 \in \mathbb{R}^{H \times (1+D)}$$

$$W2 \in \mathbb{R}^{C \times (1+H)}$$

$$Y_{enc} \in \mathbb{R}^{C \times B}$$

Forward Pass:

$$rA0 = X^T \in \mathbb{R}^{D \times B}$$

$$A0 = \text{ext}(rA0) \in \mathbb{R}^{D+1 \times B}$$

ext inserisci una riga di solo 1 (quindi un numero B di 1) in cima alla matrice

$$rZ1 = W1 \cdot A0 \in \mathbb{R}^{H \times B}$$

$$rA1 = \sigma(rZ1) = \frac{1}{1 + e^{-rZ1}} \in \mathbb{R}^{H \times B}$$

$$A1 = \text{ext}(rA1) \in \mathbb{R}^{H+1 \times B}$$

$$rZ2 = W2 \cdot A1 \in \mathbb{R}^{C \times B}$$

$$rA2 = \sigma(rZ2) \in \mathbb{R}^{C \times B}$$

Loss Function: La funzione di perdita globale $\mathcal{L}$ è la somma delle singole Binary Cross-Entropy calcolate per ciascuna delle C classi sull'intero mini-batch di dimensione B :

$$\mathcal{L} = \sum_{c=1}^C \mathcal{L}_c$$

$$\mathcal{L}_c = - \sum_{b=1}^B (Y_{enc}(c; b) \cdot \log(rA2(c; b)) + (1 - Y_{enc}(c; b)) \cdot \log(1 - rA2(c; b)))$$

(Ove $Y_{enc} \in \mathbb{R}^{C \times B}$ è la matrice dei target reali con codifica multi-hot).

Backward Pass: Grazie alla combinazione matematica della BCE con l'attivazione Sigmoidale in output, i termini complessi si elidono e la derivata rispetto allo stato di pre-attivazione d'uscita assume una forma algebrica pulita:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial rZ2} &= rA2 - Y_{\text{enc}} \in \mathbb{R}^{C \times B} \\ \frac{\partial \mathcal{L}}{\partial W2} &= \frac{\partial \mathcal{L}}{\partial rZ2} A1^T \in \mathbb{R}^{C \times (H+1)} \\ \frac{\partial \mathcal{L}}{\partial A1} &= W2^T \frac{\partial \mathcal{L}}{\partial rZ2} \in \mathbb{R}^{(H+1) \times B} \\ \frac{\partial \mathcal{L}}{\partial rZ1} &= \text{red} \left(\frac{\partial \mathcal{L}}{\partial A1} \right) \odot \sigma'(rZ1) \in \mathbb{R}^{H \times B}\end{aligned}$$

red rimuove la prima riga dalla matrice

$$\begin{aligned}\sigma'(rZ1) &= \sigma(rZ1) \odot (1 - \sigma(rZ1)) \\ \frac{\partial \mathcal{L}}{\partial W1} &= \frac{\partial \mathcal{L}}{\partial rZ1} A0^T \in \mathbb{R}^{H \times (D+1)}\end{aligned}$$

Aggiornamento:

$$\begin{aligned}W1 &\leftarrow W1 - \eta \frac{\partial \mathcal{L}}{\partial W1} \\ W2 &\leftarrow W2 - \eta \frac{\partial \mathcal{L}}{\partial W2}\end{aligned}$$

```

1  import numpy as np
2
3  def MLP_initialize_weights(model):
4      h = model['n_hidden']
5      f = model['n_features']
6      o = model['n_output']
7
8      W1 = 2 * np.random.rand(h, f + 1) - 1
9      W2 = 2 * np.random.rand(o, h + 1) - 1
10     return W1, W2
11
12 def MLP_sigmoid(z):
13     return 1.0 / (1.0 + np.exp(-z))
14
15 def MLP_sigmoid_gradient(Z):
16     sig = MLP_sigmoid(Z)
17     return sig * (1.0 - sig)
18
19 def encode_labels(y, k):
20     onehot = np.zeros((k, len(y)), dtype=int)
21     onehot[y, np.arange(len(y))] = 1
22     return onehot
23
24 def MLP_extend(A):
25     return np.vstack((np.ones((1, A.shape[1])), A))
26
27 def MLP_BCESIG_forward(X, W1, W2):
28     A0 = MLP_extend(X)
29     rZ1 = W1 @ A0
30     A1 = MLP_extend(MLP_sigmoid(rZ1))
31     rZ2 = W2 @ A1
32     rA2 = MLP_sigmoid(rZ2)
33     return rA2, A1, A0, rZ1, rZ2
34
35 def get_BCE_cost(y_enc, y_pred):
36     y_pred = np.clip(y_pred, 1e-12, 1.0 - 1e-12)
37     term1 = -y_enc * np.log(y_pred)

```

```

38     term2 = (1.0 - y_enc) * np.log(1.0 - y_pred)
39     return np.sum(term1 - term2)
40
41 def BCESIG_backward(rA2, A1, A0, rZ1, Y_enc, W2):
42     dL_dZ2 = rA2 - Y_enc
43     dL_dW2 = dL_dZ2 @ A1.T
44     dL_dA1 = W2.T @ dL_dZ2
45     dL_drZ1 = dL_dA1[1:, :] * MLP_sigmoid_gradient(rZ1)
46     dL_dW1 = dL_drZ1 @ A0.T
47     return dL_dW1, dL_dW2
48
49 def MLP_BCESIG_train(X_train, y_train, model):
50     W1, W2 = MLP_initialize_weights(model)
51
52     eta = model['eta']
53     epochs = model['epochs']
54     n_output = model['n_output']
55     m_batches = model['minibatches']
56
57     y_enc = encode_labels(y_train, n_output)
58
59     model['cost_history'] = []
60     N = X_train.shape[0]
61
62     for e in range(1, epochs + 1):
63         perm = np.random.permutation(N)
64         X_shuf = X_train[perm].T
65         Y_shuf = Y_enc[:, perm]
66
67         X_batches = np.array_split(X_shuf, m_batches, axis=1)
68         Y_batches = np.array_split(Y_shuf, m_batches, axis=1)
69
70         for X_mb, y_mb in zip(X_batches, y_batches):
71             rA2, A1, A0, rZ1, rZ2 = MLP_BCESIG_forward(X_mb, W1, W2)
72
73             cost = get_BCE_cost(y_mb, rA2)
74             model['cost_history'].append(cost)
75
76             dL_dW1, dL_dW2 = BCESIG_backward(rA2, A1, A0, rZ1, y_mb, W2)
77
78             W1 = W1 - eta * dL_dW1
79             W2 = W2 - eta * dL_dW2
80
81     model['W1'] = W1
82     model['W2'] = W2
83     return model, W1, W2

```

14.2.2 Domanda Tanh

Descrivi i passaggi matematici e lo pseudocodice di un Multi-Layer Perceptron per un problema di Classificazione Non Esclusiva (Multi-Label), utilizzando la funzione di attivazione sigmoid sul livello di output e la funzione di attivazione tangente iperbolica (tanh) nel hidden layer. Esponi i passaggi in dettaglio, esplicitando la derivata della funzione tanh.

Il Multilayer Perceptron (MLP) è una catena di Perceptron collegati tra loro e organizzata in strati: uno di input, uno o più strati nascosti e uno di output. L'introduzione degli strati nascosti e di funzioni di attivazione non lineari consente al modello di apprendere relazioni complesse e non lineari nei dati, superando i limiti del singolo Perceptron.

MLP_BCESIG è una rete MLP per problemi di classificazione **multi-label**. Il nome evidenzia

le due componenti principali:

- **BCE (Binary Cross-Entropy):** È la Loss Function utilizzata per misurare la discrepanza tra la previsione e la realtà.
- **SIG (Sigmoid):** È la funzione di attivazione impiegata, in particolare nello strato di output.

Sebbene BCE e Sigmoid siano tipicamente associate alla classificazione binaria, risultano adatte anche al contesto multi-label. In questo caso, l'ultimo strato contiene C neuroni, uno per ciascuna classe. Ogni neurone produce una previsione indipendente che indica la presenza o l'assenza della relativa label.

La fase di addestramento del modello si basa sul Gradient Descent iterativo e comprende:

- **Inizializzazione:** pesi W_1 e W_2 inizializzati tramite un metodo di inizializzazione (ad esempio: randomica, o Xavier Initialization);
- **Forward pass:** I dati di input vengono moltiplicati per W_1 e passati attraverso la funzione di attivazione non lineare per produrre le uscite dello strato nascosto. Queste uscite vengono poi moltiplicate per W_2 , fatte passare per un'altra sigmoide e infine restituite come vettore di label;
- **Loss Function:** calcolo della BCE tra previsione e target;
- **Backward pass:** Si calcolano i gradienti dell'errore. Questo errore viene propagato all'indietro attraverso la rete, moltiplicandolo per le derivate delle funzioni di attivazione del l'hidden layer;
- **Aggiornamento:** aggiornamento dei pesi tramite learning rate η .

D : dimensione di input (feature)

B : numero di dati nel mini-batch corrente

H : numero di neuroni nell'hidden layer

C : numero di neuroni nell'output layer

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\tanh'(z) = 1 - \tanh^2(z) = 1 - \left(\frac{e^z - e^{-z}}{e^z + e^{-z}}\right)^2$$

$$X \in \mathbb{R}^{D \times B}$$

$$W1 \in \mathbb{R}^{H \times (1+D)}$$

$$W2 \in \mathbb{R}^{C \times (1+H)}$$

$$Y_{enc} \in \mathbb{R}^{C \times B}$$

Forward Pass:

$$rA0 = X^T \in \mathbb{R}^{D \times B}$$

$$A0 = \text{ext}(rA0) \in \mathbb{R}^{D+1 \times B}$$

ext inserisci una riga di solo 1 (quindi un numero B di 1) in cima alla matrice

$$rZ1 = W1 \cdot A0 \in \mathbb{R}^{H \times B}$$

$$rA1 = \tanh(rZ1) = \frac{e^{rZ1} - e^{-rZ1}}{e^{rZ1} + e^{-rZ1}} \in \mathbb{R}^{H \times B}$$

$$A1 = \text{ext}(rA1) \in \mathbb{R}^{H+1 \times B}$$

$$rZ2 = W2 \cdot A1 \in \mathbb{R}^{C \times B}$$

$$rA2 = \sigma(rZ2) \in \mathbb{R}^{C \times B}$$

Loss Function: La funzione di perdita globale $\mathcal{L}$ è la somma delle singole Binary Cross-Entropy calcolate per ciascuna delle C classi sull'intero mini-batch di dimensione B :

$$\mathcal{L} = \sum_{c=1}^C \mathcal{L}_c$$

$$\mathcal{L}_c = - \sum_{b=1}^B (Y_{\text{enc}}(c; b) \cdot \log(rA2(c; b)) + (1 - Y_{\text{enc}}(c; b)) \cdot \log(1 - rA2(c; b)))$$

(Ove $Y_{\text{enc}} \in \mathbb{R}^{C \times B}$ è la matrice dei target reali con codifica multi-hot).

Backward Pass: Grazie alla combinazione matematica della BCE con l'attivazione Sigmoide in output, i termini complessi si elidono e la derivata rispetto allo stato di pre-attivazione d'uscita assume una forma algebrica pulita:

$$\frac{\partial \mathcal{L}}{\partial rZ2} = rA2 - Y_{\text{enc}} \in \mathbb{R}^{C \times B}$$

$$\frac{\partial \mathcal{L}}{\partial W2} = \frac{\partial \mathcal{L}}{\partial rZ2} A1^T \in \mathbb{R}^{C \times (H+1)}$$

$$\frac{\partial \mathcal{L}}{\partial A1} = W2^T \frac{\partial \mathcal{L}}{\partial rZ2} \in \mathbb{R}^{(H+1) \times B}$$

$$\frac{\partial \mathcal{L}}{\partial rZ1} = \text{red} \left(\frac{\partial \mathcal{L}}{\partial A1} \right) \odot \tanh'(rZ1) = \text{red} \left(\frac{\partial \mathcal{L}}{\partial A1} \right) \odot (1 - \tanh^2(rZ1)) \in \mathbb{R}^{H \times B}$$

red rimuove la prima riga dalla matrice

$$\sigma'(rZ1) = \sigma(rZ1) \odot (1 - \sigma(rZ1))$$

$$\frac{\partial \mathcal{L}}{\partial W1} = \frac{\partial \mathcal{L}}{\partial rZ1} A0^T \in \mathbb{R}^{H \times (D+1)}$$

Aggiornamento:

$$W1 \leftarrow W1 - \eta \frac{\partial \mathcal{L}}{\partial W1}$$

$$W2 \leftarrow W2 - \eta \frac{\partial \mathcal{L}}{\partial W2}$$

```

1  import numpy as np
2
3  def MLP_initialize_weights(model):
4      h = model['n_hidden']
5      f = model['n_features']
6      o = model['n_output']
7
8      W1 = 2 * np.random.rand(h, f + 1) - 1
9      W2 = 2 * np.random.rand(o, h + 1) - 1
10     return W1, W2
11
12 def MLP_tanh(z):
13     return (np.exp(z) - np.exp(-z)) / (np.exp(z) + np.exp(-z))
14
15 def MLP_sigmoid_gradient(z):
16     t = MLP_tanh(z)
17     return (1.0 - t**2)
18
19 def encode_labels(y, k):
20     onehot = np.zeros((k, len(y)), dtype=int)
21     onehot[y, np.arange(len(y))] = 1
22     return onehot
23
24 def MLP_extend(A):
25     return np.vstack((np.ones((1, A.shape[1])), A))
26
27 def MLP_BCESIG_forward(X, W1, W2):
28     def MLP_BCESIG_forward(X, W1, W2):
29         A0 = MLP_extend(X)
30         rZ1 = W1 @ A0
31         A1 = MLP_extend(MLP_tanh(rZ1))
32         rZ2 = W2 @ A1
33         rA2 = MLP_sigmoid(rZ2)
34         return rA2, A1, A0, rZ1, rZ2
35
36 def get_BCE_cost(y_enc, y_pred):
37     y_pred = np.clip(y_pred, 1e-12, 1.0 - 1e-12)
38     term1 = -y_enc * np.log(y_pred)
39     term2 = (1.0 - y_enc) * np.log(1.0 - y_pred)
40     return np.sum(term1 - term2)
41
42 def BCESIG_backward(rA2, A1, A0, rZ1, Y_enc, W2):
43     dL_dZ2 = rA2 - Y_enc
44     dL_dW2 = dL_dZ2 @ A1.T
45     dL_dA1 = W2.T @ dL_dZ2
46     dL_drZ1 = dL_dA1[1:, :] * MLP_tanh_gradient(rZ1)
47     dL_dW1 = dL_drZ1 @ A0.T
48     return dL_dW1, dL_dW2
49
50 def MLP_BCESIG_train(X_train, y_train, model):
51     W1, W2 = MLP_initialize_weights(model)
52
53     eta = model['eta']
54     epochs = model['epochs']
55     n_output = model['n_output']
56     m_batches = model['minibatches']
57
58     y_enc = encode_labels(y_train, n_output)
59
60     model['cost_history'] = []
61     N = X_train.shape[0]
62
63     for e in range(1, epochs + 1):
64         perm = np.random.permutation(N)
65         X_shuf = X_train[perm].T
66         Y_shuf = Y_enc[:, perm]

```

```

67
68
69 X_batches = np.array_split(X_shuf, m_batches, axis=1)
70 Y_batches = np.array_split(Y_shuf, m_batches, axis=1)
71
72 for X_mb, y_mb in zip(X_batches, y_batches):
73     rA2, A1, A0, rZ1, rZ2 = MLP_BCESIG_forward(X_mb, W1, W2)
74
75     cost = get_BCE_cost(y_mb, rA2)
76     model['cost_history'].append(cost)
77
78     dL_dW1, dL_dW2 = BCESIG_backward(rA2, A1, A0, rZ1, y_mb, W2)
79
80     W1 = W1 - eta * dL_dW1
81     W2 = W2 - eta * dL_dW2
82
83 model['W1'] = W1
84 model['W2'] = W2
85 return model, W1, W2

```

14.3 MLP_CCESM

14.3.1 Domanda Standard

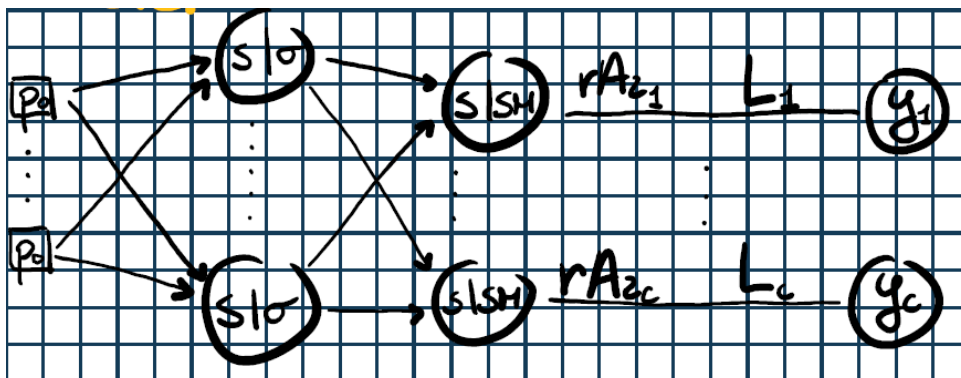
Descrivi la teoria e lo pseudocodice di un Multi-Layer Perceptron per il problema della Classificazione Multi-Classe (CCESM), quando si utilizza la funzione di attivazione SoftMax. Fornisci la formula della Categorical Cross Entropy. Spiega in dettaglio sia il forward pass che il backward pass.

Il **Multilayer Perceptron (MLP)** è una catena di Perceptron collegati tra loro e organizzata in strati: uno di input, uno o più strati nascosti e uno di output. L'introduzione degli strati nascosti e di funzioni di attivazione non lineari consente al modello di apprendere relazioni complesse e non lineari nei dati, superando i limiti del singolo Perceptron.

MLP_CCESM è una rete MLP per problemi di classificazione **multi-classe mutuamente esclusiva**, in cui ogni campione appartiene a una sola classe.

Il nome del modello deriva dalle componenti utilizzate nello strato di output:

- **CCE (Categorical Cross-Entropy)**: È la Loss Function utilizzata per calcolare l'errore.
- **SM (SoftMax)**: È la funzione di attivazione vettoriale applicata ai neuroni finali.



L'ultimo strato contiene C neuroni, uno per ciascuna classe, mentre le etichette sono rappresentate tramite **One-Hot Encoding**. La SoftMax trasforma i logits in una distribuzione di probabilità:

$$a_c = \frac{e^{z_c}}{\sum_{k=1}^C e^{z_k}}$$

Gli output sono compresi tra 0 e 1 e la loro somma è pari a 1. La probabilità più alta identifica la classe predetta.

L'errore viene calcolato mediante la **Categorical Cross-Entropy**:

$$L_{\text{CCE}} = - \sum_{c=1}^C y_c \log(\hat{y}_c)$$

Poiché il vettore target è codificato in One-Hot Encoding, la loss considera soltanto la probabilità assegnata alla classe corretta, penalizzando maggiormente le previsioni errate o poco confidenti.

D : dimensione di input (feature)

B : numero di dati nel mini-batch corrente

H : numero di neuroni nell'hidden layer

C : numero di neuroni nell'output layer

$X \in \mathbb{R}^{D \times B}$

$W1 \in \mathbb{R}^{H \times (1+D)}$

$W2 \in \mathbb{R}^{C \times (1+H)}$

$Y_{\text{enc}} \in \mathbb{R}^{C \times B}$

Forward Pass:

$$rA0 = X^T \in \mathbb{R}^{D \times B}$$

$$A0 = \text{ext}(rA0) \in \mathbb{R}^{(D+1) \times B}$$

ext inserisce una riga di soli 1 (quindi un numero B di 1) in cima alla matrice

$$rZ1 = W1 \cdot A0 \in \mathbb{R}^{H \times B}$$

$$rA1 = \sigma(rZ1) = \frac{1}{1 + e^{-rZ1}} \in \mathbb{R}^{H \times B}$$

$$A1 = \text{ext}(rA1) \in \mathbb{R}^{(H+1) \times B}$$

$$rZ2 = W2 \cdot A1 \in \mathbb{R}^{C \times B}$$

$$rA2 = \text{SoftMax}(rZ2) \in \mathbb{R}^{C \times B}$$

Ove la funzione SoftMax agisce colonna per colonna, ossia normalizzando le probabilità su ciascun singolo campione b : $\text{SoftMax}(z_c) = \frac{e^{z_c}}{\sum_{k=1}^C e^{z_k}}$

Loss Function:

La funzione di perdita è la Categorical Cross-Entropy sommata sull'intero mini-batch di dimensione B :

$$\mathcal{L}_{\text{CCE}} = \sum_{c=1}^C \sum_{b=1}^B -Y_{\text{enc}}(c; b) \cdot \log(rA2(c; b))$$

Ove $Y_{\text{enc}} \in \mathbb{R}^{C \times B}$ è la matrice dei target con codifica one-hot: presenta esattamente un solo 1 per ogni colonna b , mentre i restanti elementi sono 0.

Backward Pass:

La combinazione algebrica tra la derivata della CCE e la derivata del SoftMax fa collassare il gradiente d'uscita nella pura differenza tra predizione e valore reale:

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial rZ2} &= rA2 - Y_{\text{enc}} \in \mathbb{R}^{C \times B} \\ \frac{\partial \mathcal{L}}{\partial W2} &= \frac{\partial \mathcal{L}}{\partial rZ2} A1^T \in \mathbb{R}^{C \times (H+1)} \\ \frac{\partial \mathcal{L}}{\partial A1} &= W2^T \frac{\partial \mathcal{L}}{\partial rZ2} \in \mathbb{R}^{(H+1) \times B} \\ \frac{\partial \mathcal{L}}{\partial rZ1} &= \text{red} \left(\frac{\partial \mathcal{L}}{\partial A1} \right) \odot \sigma'(rZ1) \in \mathbb{R}^{H \times B} \\ \text{red} &\text{ rimuove la prima riga dalla matrice} \\ \sigma'(rZ1) &= \sigma(rZ1) \odot (1 - \sigma(rZ1)) \\ \frac{\partial \mathcal{L}}{\partial W1} &= \frac{\partial \mathcal{L}}{\partial rZ1} A0^T \in \mathbb{R}^{H \times (D+1)}\end{aligned}$$

Aggiornamento:

$$\begin{aligned}W1 &\leftarrow W1 - \eta \frac{\partial \mathcal{L}}{\partial W1} \\ W2 &\leftarrow W2 - \eta \frac{\partial \mathcal{L}}{\partial W2}\end{aligned}$$

```

1  import numpy as np
2
3  def MLP_initialize_weights(model):
4      h = model['n_hidden']
5      f = model['n_features']
6      o = model['n_output']
7
8      W1 = 2 * np.random.rand(h, f + 1) - 1
9      W2 = 2 * np.random.rand(o, h + 1) - 1
10     return W1, W2
11
12 def MLP_sigmoid(z):
13     return 1.0 / (1.0 + np.exp(-z))
14
15 def MLP_sigmoid_gradient(Z):
16     s = MLP_sigmoid(Z)
17     return s * (1.0 - sig)
18
19 def MLP_softmax(rZ):
20     exp_rZ = np.exp(rZ - np.max(rZ, axis=0, keepdims=True))
21     return exp_rZ / np.sum(exp_rZ, axis=0, keepdims=True)
22
23 def encode_labels(y, k):
24     onehot = np.zeros((k, len(y)), dtype=int)
25     onehot[y, np.arange(len(y))] = 1
26     return onehot
27
28 def MLP_extend(A):
29     return np.vstack((np.ones((1, A.shape[1])), A))
30
31 def MLP_CCESM_forward(X, W1, W2):
32     A0 = MLP_extend(X)

```

```

33     rZ1 = W1 @ A0
34     A1_ext = MLP_extend(MLP_sigmoid(rZ1))
35     rZ2 = W2 @ A1_ext
36     rA2 = MLP_softmax(rZ2)
37     return rA2, A1_ext, A0, rZ1, rZ2
38
39 def get_CCE_cost(y_enc, y_pred):
40     y_pred = np.clip(y_pred, 1e-12, 1.0 - 1e-12)
41     return -np.sum(y_enc * np.log(y_pred))
42
43 def MLP_CCESM_backward(rA2, A1_ext, A0, rZ1, Y_enc, W2):
44     dL_dZ2 = rA2 - Y_enc
45     dL_dW2 = dL_dZ2 @ A1_ext.T
46     dL_dA1 = W2.T @ dL_dZ2
47     dL_drZ1 = dL_dA1[1:, :] * MLP_sigmoid_gradient(rZ1)
48     dL_dW1 = dL_drZ1 @ A0.T
49     return dL_dW1, dL_dW2
50
51 def MLP_CCESM_train(X_train, y_train, model):
52     W1, W2 = MLP_initialize_weights(model)
53
54     eta = model['eta']
55     epochs = model['epochs']
56     n_output = model['n_output']
57     m_batches = model['minibatches']
58
59     Y_enc = encode_labels(y_train, n_output)
60
61     model['cost_history'] = []
62     N = X_train.shape[0]
63
64     for e in range(epochs):
65         perm = np.random.permutation(N)
66         X_shuf = X_train[perm].T
67         Y_shuf = Y_enc[:, perm]
68
69         X_batches = np.array_split(X_shuf, m_batches, axis=1)
70         Y_batches = np.array_split(Y_shuf, m_batches, axis=1)
71
72         for X_mb, Y_mb in zip(X_batches, Y_batches):
73             rA2, A1_ext, A0, rZ1, rZ2 = MLP_CCESM_forward(X_mb, W1, W2)
74
75             cost = get_CCE_cost(Y_mb, rA2)
76             model['cost_history'].append(cost)
77
78             dL_dW1, dL_dW2 = MLP_CCESM_backward(rA2, A1_ext, A0, rZ1, Y_mb, W2)
79
80             W1 = W1 - eta * dL_dW1
81             W2 = W2 - eta * dL_dW2
82
83     model['W1'] = W1
84     model['W2'] = W2
85     return model, W1, W2

```

14.3.2 Domanda Tanh e Layer in più

Descrivi la teoria e lo pseudocodice di un Multi-Layer Perceptron per il problema della Classificazione Multi-Classe (CCESM), utilizzando la funzione di attivazione SM sul livello di output, la funzione di attivazione tangente iperbolica (tanh) nel primo hidden layer e la funzione di attivazione sigmoide (σ) nel secondo hidden layer. Esponi i passaggi in dettaglio, esplicitando la derivata della funzione tanh e σ .

Il **Multilayer Perceptron (MLP)** è una catena di Perceptron collegati tra loro e organizzata in strati: uno di input, uno o più strati nascosti e uno di output. L'introduzione degli strati nascosti e di funzioni di attivazione non lineari consente al modello di apprendere relazioni complesse e non lineari nei dati, superando i limiti del singolo Perceptron.

MLP_CCESM è una rete MLP per problemi di classificazione **multi-classe mutuamente esclusiva**, in cui ogni campione appartiene a una sola classe.

Il nome del modello deriva dalle componenti utilizzate nello strato di output:

- **CCE (Categorical Cross-Entropy):** È la Loss Function utilizzata per calcolare l'errore.
- **SM (SoftMax):** È la funzione di attivazione vettoriale applicata ai neuroni finali.

L'ultimo strato contiene C neuroni, uno per ciascuna classe, mentre le etichette sono rappresentate tramite **One-Hot Encoding**. La SoftMax trasforma i logits in una distribuzione di probabilità:

$$a_c = \frac{e^{z_c}}{\sum_{k=1}^C e^{z_k}}$$

Gli output sono compresi tra 0 e 1 e la loro somma è pari a 1. La probabilità più alta identifica la classe predetta.

L'errore viene calcolato mediante la **Categorical Cross-Entropy**:

$$L_{CCE} = - \sum_{c=1}^C y_c \log(\hat{y}_c)$$

Poiché il vettore target è codificato in One-Hot Encoding, la loss considera soltanto la probabilità assegnata alla classe corretta, penalizzando maggiormente le previsioni errate o poco confidenti.

D : dimensione di input (feature)

B : numero di dati nel mini-batch corrente

$H1$: numero di neuroni nel primo hidden layer

$H2$: numero di neuroni nel secondo hidden layer

C : numero di neuroni nell'output layer

$$X \in \mathbb{R}^{D \times B}$$

$$W1 \in \mathbb{R}^{H1 \times (1+D)}$$

$$W2 \in \mathbb{R}^{H2 \times (1+H1)}$$

$$W3 \in \mathbb{R}^{C \times (1+H2)}$$

$$Y_{enc} \in \mathbb{R}^{C \times B}$$

Forward Pass:

$$rA0 = X^T \in \mathbb{R}^{D \times B}$$

$$A0 = ext(rA0) \in \mathbb{R}^{(D+1) \times B}$$

ext inserisce una riga di soli 1 (quindi un numero B di 1) in cima alla matrice

$$rZ1 = W1 \cdot A0 \in \mathbb{R}^{H \times B}$$

$$\begin{aligned}
rA1 &= \tanh(rZ1) = \frac{e^{rZ1} - e^{-rZ1}}{e^{rZ1} + e^{-rZ1}} \in \mathbb{R}^{H1 \times B} \\
A1 &= \text{ext}(rA1) \in \mathbb{R}^{(H1+1) \times B} \\
rZ2 &= W2 \cdot A1 \in \mathbb{R}^{H2 \times B} \\
rA2 &= \sigma(rZ2) \in \mathbb{R}^{H2 \times B} \\
A2 &= \text{ext}(rA2) \in \mathbb{R}^{(H2+1) \times B} \\
rZ3 &= W3 \cdot A2 \\
rA3 &= \text{SoftMax}(rZ3)
\end{aligned}$$

Ove la funzione SoftMax agisce colonna per colonna, ossia normalizzando le probabilità su ciascun singolo campione b : $\text{SoftMax}(z_c) = \frac{e^{z_c}}{\sum_{k=1}^C e^{z_k}}$

Loss Function:

La funzione di perdita è la Categorical Cross-Entropy sommata sull'intero mini-batch di dimensione B :

$$\mathcal{L}_{\text{CCE}} = \sum_{c=1}^C \sum_{b=1}^B -Y_{\text{enc}}(c; b) \cdot \log(rA2(c; b))$$

Ove $Y_{\text{enc}} \in \mathbb{R}^{C \times B}$ è la matrice dei target con codifica one-hot: presenta esattamente un solo 1 per ogni colonna b , mentre i restanti elementi sono 0.

Backward Pass:

La combinazione algebrica tra la derivata della CCE e la derivata del SoftMax fa collassare il gradiente d'uscita nella pura differenza tra predizione e valore reale:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial rZ3} &= rA3 - Y_{\text{enc}} \in \mathbb{R}^{C \times B} \\
\frac{\partial \mathcal{L}}{\partial W3} &= \frac{\partial \mathcal{L}}{\partial rZ3} A2^T \in \mathbb{R}^{C \times (H2+1)} \\
\frac{\partial \mathcal{L}}{\partial A2} &= W3^T \frac{\partial \mathcal{L}}{\partial rZ3} \in \mathbb{R}^{(H2+1) \times B} \\
\frac{\partial \mathcal{L}}{\partial rZ2} &= \text{red} \left(\frac{\partial \mathcal{L}}{\partial A2} \right) \odot \sigma'(rZ2) \in \mathbb{R}^{H2 \times B} \\
\frac{\partial \mathcal{L}}{\partial rZ2} &= \text{red} \left(\frac{\partial \mathcal{L}}{\partial A2} \right) \odot (rA2 \odot (1 - rA2)) \\
\frac{\partial \mathcal{L}}{\partial W2} &= \frac{\partial \mathcal{L}}{\partial rZ2} A1^T \in \mathbb{R}^{H2 \times (H1+1)} \\
\frac{\partial \mathcal{L}}{\partial A1} &= W2^T \frac{\partial \mathcal{L}}{\partial rZ2} \in \mathbb{R}^{(H1+1) \times B} \\
\frac{\partial \mathcal{L}}{\partial rZ1} &= \text{red} \left(\frac{\partial \mathcal{L}}{\partial A1} \right) \odot \tanh'(rZ1) \in \mathbb{R}^{H1 \times B} \\
\frac{\partial \mathcal{L}}{\partial rZ1} &= \text{red} \left(\frac{\partial \mathcal{L}}{\partial A1} \right) \odot (1 - rA1 \odot rA1) \\
\frac{\partial \mathcal{L}}{\partial W1} &= \frac{\partial \mathcal{L}}{\partial rZ1} A0^T \in \mathbb{R}^{H1 \times (D+1)}
\end{aligned}$$

Aggiornamento:

$$W1 \leftarrow W1 - \eta \frac{\partial \mathcal{L}}{\partial W1}$$

$$W2 \leftarrow W2 - \eta \frac{\partial \mathcal{L}}{\partial W2}$$

$$W3 \leftarrow W3 - \eta \frac{\partial \mathcal{L}}{\partial W3}$$

Capitolo 15: Domanda 2

15.1 Radial Basis Function Network

15.1.1 Domanda Standard

Spiega le Radial Basis Function Network (RBFN). Fornisci quanti più dettagli interessanti possibile. Puoi elencare alcune tecniche che possono essere utilizzate per inizializzare i centri?

Teorema (Teorema di Cover sulla separabilità dei pattern). *Un problema di Classificazione complesso (non linearmente separabile) in uno spazio a bassa dimensionalità ha un'altissima probabilità di diventare linearmente separabile se viene proiettato in modo non lineare in uno spazio a dimensionalità molto più alta.*

Le **Radial Basis Function (RBF)** sono funzioni non lineari che applicano questo teorema: prendono gli input, li proiettano in uno spazio più grande in maniera non lineare, permettendo così all'ultimo strato della rete di risolvere il problema con una semplice retta o iperpiano.

La RBF più utilizzata è la **Funzione Gaussiana**:

$$\phi_j(x) = \exp\left(-\frac{\|x - \mu_j\|^2}{2\sigma_j^2}\right) = e^{-\frac{\|x - \mu_j\|^2}{2\sigma_j^2}}$$

Una **Radial Basis Function Network (RBFN)** ha una topologia fissa e shallow, composta rigorosamente da tre livelli:

- **Input Layer:** Composto da N nodi che si limitano a passare il vettore di input x al livello successivo, senza fare calcoli.
- **Hidden Layer:** Contiene K neuroni. Qui non ci sono pesi da moltiplicare. Ogni neurone è caratterizzato da un Centro (μ) e da una Larghezza (σ). Il neurone calcola la distanza euclidea tra l'input x e il suo centro μ , passandola poi a una funzione radiale (tipicamente Gaussiana).
- **Output Layer:** È un livello puramente lineare che esegue una semplice somma pesata (prodotto scalare) delle attivazioni ricevute dal livello nascosto.

Il più grande vantaggio storico delle RBFN è la loro velocità di addestramento. Poiché la rete è divisa in una parte non lineare e una lineare, si può evitare la lenta retropropagazione (Backpropagation) iterativa in favore di un metodo ibrido a due fasi:

1. **Apprendimento Non Supervisionato (Hidden Layer):** In questa fase, senza guardare le etichette target (y), si posizionano i centri μ_j nello spazio degli input.

Per trovare i centri:

- Si fa Random Subset Selection scegliendo casualmente K punti direttamente dal dataset;
- Si usa un algoritmo di Clustering come K-Means;
- Si fa Orthogonal Least Squares provando ogni singolo dato del dataset come potenziale nuovo centro;
- Si mettono come parametri addestrabili e si calcolano con la Gradient Descent tramite Backpropagation.

Lo spread σ , invece, si calcola in base alla distanza media tra i centri appena trovati (esempio: K-Nearest Neighbors) affinché le campane gaussiane si sovrappongano dolcemente;

2. **Apprendimento Supervisionato (Output Layer):** Una volta fissati i centri e le larghezze, il livello nascosto viene "congelato". Ora la rete si comporta come un semplice modello di Regressione lineare: $Y = \Phi W$. I pesi W del livello di output possono essere trovati analiticamente e istantaneamente usando la *Matrice Pseudo-Inversa di Moore-Penrose*: $W = (\Phi^T \Phi)^{-1} \Phi^T Y$ oppure, in alternativa, si può usare la *Gradient Descent*.

Condividono la stessa formula matematica (le campane gaussiane) con la **Kernel Density Estimation**. Tuttavia, la KDE è un metodo statistico in cui si piazza un centro su ogni singolo dato del dataset per misurare la densità di probabilità (senza ottimizzazione). Nelle RBF, i centri sono molti meno dei dati ($K \ll N$) e si ottimizzano per fare previsioni predittive.

Le RBFN sono particolarmente efficaci in problemi a *bassa dimensionalità, di controllo e di approssimazione di funzioni*, soprattutto quando è *richiesta un'elevata velocità di addestramento*. Per dati ad alta dimensionalità e problemi complessi si continuano a preferire Multi-Layer Perceptron.

15.2 Evolutionary Algorithms

15.2.1 Differenza tra Grid Search, Random Search e Genetic Algorithms

Spiega la differenza tra Grid Search, Random Search e Genetic Algorithms. Quali sono i passaggi che richiedono più tempo in un Genetic Algorithm? Cosa faresti per ridurre il tempo di esecuzione?

Mettiamo di avere una rete neurale. I suoi parametri sono i pesi v e i bias b che vengono aggiornati e migliorati attraverso il meccanismo della Backpropagation. Tuttavia oltre ai parametri abbiamo anche quelli che sono chiamati "*iperparametri*" e non sono ottimizzati dall'addestramento della rete. Questi iperparametri possono essere: il learning rate, il numero di neuroni, il numero di layer, le funzioni di attivazione, la Loss Function, eccetera ...

Scegliere degli iperparametri ottimali è un problema convesso non lineare, la cui risoluzione può cambiare anche di molto le performance della rete neurale. Questo processo di scoperta degli iperparametri ottimali è detto **Hyperparameter Tuning**.

Innanzitutto abbiamo bisogno di una Fitness Function, ossia di una funzione scalare che ci faccia da metrica per analizzare le performance di una configurazione di iperparametri. Può essere la Loss Function (l'inverso se si vuole una misura che più sale, meglio è), l'Accuracy o qualsiasi altra metrica adeguata alla valutazione del problema.

Un possibile metodo per eseguire l'Hyperparameter Tuning è l'Exhaustive Search, dove si fa una ricerca utilizzando tutti le configurazioni possibili e scegliendo la migliore, ma è un approccio completamente non sostenibile, che nemmeno i migliori supercomputer riescono a fare in tempi brevi.

Di conseguenza abbiamo tre alternative:

- **Grid Search:** Si discretizzano gli iperparametri continui e si settano manualmente tutti i valori degli iperparametri da testare (oppure si stabilisce un range e uno step) e si valuta la Fitness di ogni singola configurazione. Sfortunatamente non è un approccio scalabile;
- **Random Search:** Si stabilisce lo spazio degli iperparametri, si determina un metodo di ricerca stocastica sfruttando un buon algoritmo di generazione dei numeri casuali. Maggiormente scalabile, ma con efficacia pessima;
- **Genetic Algorithms:** Gli algoritmi genetici sono meta-euristiche ispirate alla biologia, che attraverso meccanismi di crossover e di mutazione riescono ad esplorare in maniera rapida ed efficace lo spazio degli iperparametri, introducendo anche varietà ed provando anche combinazioni nuove senza averle settate. Se due configurazioni hanno ottenuto ottimi risultati, gli algoritmi genetici le selezionano e ne incrociano i parametri (Crossover), sperando che il figlio erediti il meglio di entrambe. Infine, applicano lievi modifiche casuali (Mutazione) per esplorare nuove strade.

L'algoritmo evolve una popolazione di soluzioni attraverso le seguenti fasi:

1. **Inizializzazione:** generazione casuale della popolazione iniziale;
2. **Valutazione:** calcolo della fitness di ogni individuo;
3. **Selezione:** scelta dei genitori in base alla fitness;
4. **Crossover:** combinazione dei genitori per generare nuovi individui;
5. **Mutazione:** introduzione di variazioni casuali nei figli;
6. **Sostituzione:** aggiornamento della popolazione con la nuova generazione;
7. **Arresto:** ripetizione del ciclo fino al soddisfacimento di una condizione di terminazione.

Il passaggio più dispendioso è la Fitness Evaluation, poiché significa dover costruire fisicamente la rete neurale con quegli specifici iperparametri, addestrarla e validarla.

Potremmo pensare di parallelizzare la Fitness Evaluation (quindi la costruzione di reti neurali), ma ciò può richiedere un enorme uso della memoria.

Il metodo più efficace è quello dei **Surrogate Model**. Si costruisce un modello che approssimi la Fitness Evaluation, alleggerendo di conseguenza quel passaggio poiché verrebbe stimata senza all'atto pratico svolgerla. Un Surrogate Model può essere un'ulteriore rete neurale (addestrata per questo compito), una Radial Basis Function Network (addestrabile velocemente), Kriging (che oltre al valore di Fitness Function ti dice quanto è sicuro della stima di questo valore) e Polynomial Regression (non ottimale per rappresentare relazioni non lineari).

Quindi la Fitness Evaluation con un Surrogate Model diventerebbe:

1. Si inizializza la popolazione;
2. Per un certo numero di iterazioni si stima la Fitness Function;
3. Con i dati raccolti, si costruisce il Surrogate Model per stimare la Fitness Evaluation senza applicarla;
4. Per un certo numero di iterazioni si usa il Surrogate Model per la Fitness Evaluation;

5. Dopo un certo numero di iterazioni si rifanno una serie di Fitness Evaluation per confrontarle con il Surrogate Model:
 - Se il "fenomeno" sottostante è ancora ben rappresentato, il Surrogate Model rimane invariato;
 - Se invece c'è una certa discrepanza, allora si riaddestra il Surrogate Model con i dati raccolti fino ad ora e si ricomincia.

15.2.2 Evolutionary Strategies

Cosa sono le Evolutionary Strategies? Fornisci lo pseudocodice e alcuni operatori di mutazione. Ora modifica lo pseudocodice per affrontare problemi di ottimizzazione vincolata, ipotizzando che i vincoli siano hard constraints (ovvero debbano essere rigorosamente soddisfatti affinché la soluzione sia ammissibile).

Le **Evolution Strategies (ES)** rappresentano una delle forme più semplici di algoritmi evolutivi e sono impiegate principalmente per affrontare problemi di ottimizzazione numerica.

La **(1+1)-ES** è la variante più semplice: mantiene una sola soluzione corrente (*parent*) e ne genera una sola nuova (*offspring*) a ogni iterazione. Il figlio viene ottenuto tramite mutazione del genitore, quindi valutato insieme ad esso: se risulta migliore, lo sostituisce, altrimenti viene scartato. Il processo si ripete fino al soddisfacimento di una condizione di arresto.

Questa strategia è affine al *Simulated Annealing* per l'uso di perturbazioni casuali, ma se ne distingue perché adotta un criterio elitario, accettando solo soluzioni migliorative.

($\mu + 1$)-ES estende il caso precedente mantenendo μ genitori e generando un solo figlio per iterazione. Il nuovo individuo compete con l'intera popolazione corrente e viene inserito solo se migliora la soluzione meno adatta, introducendo così una forma di memoria sulle soluzioni migliori già trovate.

La **($\mu + \lambda$)-ES** mantiene una popolazione di μ genitori e produce, a ogni iterazione, λ figli tramite mutazione. I figli vengono valutati e, nella fase di selezione, i migliori μ individui tra genitori e discendenti formano la generazione successiva. Questo schema elitario preserva le soluzioni migliori e rende il parametro λ determinante per il bilanciamento tra esplorazione ed efficienza.

La disponibilità di una popolazione rende inoltre possibile l'uso del crossover, avvicinando queste strategie al comportamento dei Genetic Algorithms.

Nei problemi di ottimizzazione reale, un hard constraint è una condizione imprescindibile (esempio: "la somma delle probabilità deve essere esattamente 1", oppure "il peso non deve superare i 100 kg"). Se un individuo viola questa regola, la sua soluzione è inammissibile (infeasible).

La tecnica più elegante e utilizzata nelle ES per gestire questi vincoli è la Death Penalty (o Continuous Resampling): se un figlio mutato viola anche un solo vincolo, viene "ucciso" immediatamente, senza nemmeno calcolarne la fitness.

Per la mutazione possiamo usare:

- **Uniform Mutation:** sostituisce un valore con un valore casuale uniforme in $[L; U]$;
- **Gaussian Mutation:** applica una perturbazione gaussiana $x' = x + \mathcal{N}(0; \sigma^2)$, favorendo variazioni locali e fine-tuning.

```

1  import numpy as np
2
3  def ES_mu_plus_lambda(mu, lambda, ngenes, evaluate_fitness, mutation, selection_method,
4                        check_hard_constraints, termination_condition):
5      P = initialize_population(mu, ngenes)
6      fit_P = evaluate_fitness(P)
7      g = 0
8
9      while not termination_condition(g, P, fit_P):
10         Q = []
11
12         while len(Q) < lambda:
13             parent = selection_method(P, fit_P)
14             offspring = mutation(parent)
15             # CONSTRAINT CHECK
16             if check_hard_constraints(offspring):
17                 Q.append(offspring)
18             # You can remove the check and use only Q.append(offspring) for the standard version
19
20         Q = np.array(Q)
21         fit_Q = evaluate_fitness(Q)
22
23         R = np.vstack((P, Q))
24         fit_R = np.concatenate((fit_P, fit_Q))
25
26         best_idx = np.argsort(-fit_R)[:mu]
27         P = R[best_idx]
28         fit_P = fit_R[best_idx]
29
30         g += 1
31
32     best_x = P[0]
33     fit_x = fit_P[0]
34     return best_x, fit_x

```

15.3 Genetic Algorithms

15.3.1 TSP

Supponi di dover risolvere un problema di ottimizzazione NP-Hard, come il Problema del Commesso Viaggiatore (TSP). In tali problemi vogliamo scoprire la permutazione ottimale. Supponi inoltre di voler utilizzare un Algoritmo Evolutivo. Che tipo di rappresentazione utilizzeresti per il cromosoma? E quali operatori di crossover e mutazione useresti? Rifletti sulla possibilità di introdurre operatori di crossover e mutazione personalizzati, inventati da te, che siano idonei alla rappresentazione del cromosoma che hai selezionato.

Il **TSP** consiste nel trovare il percorso più breve che tocca una serie di tappe, visitando ciascun luogo esattamente una volta e tornando al punto di partenza.

Per risolvere il TSP un Genetic Algorithm standard non basta. I **Memetic Algorithms** (*algoritmi memetici*) sono meta-euristiche ibride che integrano una ricerca globale, generalmente evolutiva, con procedure di ricerca locale per il miglioramento delle soluzioni. L'obiettivo è combinare capacità esplorativa ed efficacia di raffinamento nelle regioni promettenti dello spazio delle soluzioni. Questo tipo di algoritmi sono quelli più adatti per risolvere il problema del TSP.

Si rappresenta il TSP come un **Permutation Encoding** (nessun numero può essere ripetuto e nessun numero può mancare), in cui un **cromosoma rappresenta una sequenza di interi che rappresentano gli identificativi univoci delle città**. Esempio:

Cromosoma = [3, 0, 4, 1, 2]

Un singolo gene rappresenta l'ID univoco di una città (esempio: 4). La posizione del gene nel cromosoma indica in quale ordine cronologico la città viene visitata. L'obiettivo del TSP è minimizzare la distanza totale percorsa. **Pertanto la Fitness Function sarà l'inverso del costo totale del percorso.**

Se usassimo il classico Single Point Crossover, tagliando a metà due genitori (esempio: P1 = [0, 1, 2, 3] e P2 = [3, 2, 1, 0]), il figlio erediterebbe [0, 1] da P1 e [1, 0] da P2. Il risultato [0, 1, 1, 0] è invalido (la città 1 è visitata due volte, la 2 e la 3 mancano). Lo stesso problema si verificherebbe mutando un gene a caso.

Un genitore potrebbe aver ottimizzato il segmento iniziale del percorso, mentre l'altro quello finale. Facciamo che il figlio eredita il blocco iniziale dal primo genitore e il blocco finale dal secondo, evitando duplicazioni. Le città rimanenti vengono inserite nella parte centrale preservando l'ordine con cui compaiono nel primo genitore.

L'algoritmo globale sarà un Genetic Algorithm, mentre quello locale è chiamato **2-opt**. Funziona così:

1. Utilizziamo un Genetic Algorithm per trovare un percorso;
2. Prima di passare il percorso alla Fitness Function, il 2-opt lo analizza e cerca due archi che si incrociano (formando una X sulla mappa). Se invertendo l'ordine di visita delle città in quel tratto il costo totale diminuisce, il 2-opt modifica il cromosoma;
3. Si valuta attraverso la Fitness Function e si ripete il procedimento fino al criterio di stop.

In pratica, dopo la valutazione del costo del tour, il metodo 2-opt considera ogni coppia di archi e ne valuta lo scambio, accettandolo qualora produca una riduzione del costo complessiva del percorso. Viene usato come meccanismo di mutazione.

15.3.2 Genetic Algorithm al posto della Backpropagation

Descrivi in che modo addestreresti un Multi-Layer Perceptron generico utilizzando un Genetic Algorithm al posto della classica backpropagation.

Un MLP è una rete neurale costituita da layer di nodi completamente connessi e risolve problemi di Classificazione o Regressione apprendendo relazioni non lineari attraverso l'algoritmo di Backpropagation.

Un MLP si articola in tre categorie principali di strati (livelli):

- **Input Layer:** riceve i dati grezzi in ingresso. Ogni nodo rappresenta una singola feature del dataset;
- **Hidden Layer:** elaborano i dati ricevuti dagli strati precedenti applicando pesi, bias e funzioni di attivazione non lineari (esempio: ReLU, Sigmoid, Tanh);
- **Output Layer:** produce il risultato finale, che può essere una classe prevista (Classificazione) o un valore continuo (Regressione).

I **Genetic Algorithms** sono meta-euristiche ispirate ai principi dell'evoluzione naturale, impiegate per la ricerca di soluzioni ottimali. Le soluzioni candidate sono rappresentate come individui di una popolazione, ciascuno codificato mediante un cromosoma. La popolazione evolve nel tempo attraverso operatori di selezione, crossover e mutazione.

Un MLP è definito da matrici di pesi e vettori di bias. Utilizziamo il **Real Encoding**, in cui tutti questi parametri vengono concatenati in un unico vettore unidimensionale di valori reali, che rappresenta il **cromosoma** dell'individuo. Ogni numero è un gene.

La **Fitness Function** è una funzione scalare che valuta la qualità di ciascun cromosoma. In questo caso è l'inverso della **Loss Function**.

La **Tournament Selection** è un metodo di selezione in cui si estraggono casualmente un numero fissato di individui dalla popolazione e si sceglie come genitore quello con fitness migliore all'interno del gruppo. Scegliamo così i genitori prima di applicare i metodi di Crossover e di Mutazione per il Real Encoding.

Per il **crossover**:

- **Discrete Crossover:** estende l'uniform crossover ai domini reali, scambiando direttamente valori continui tra i genitori;
- **Arithmetic Crossover:** genera i figli come combinazioni lineari dei genitori, $z = \alpha x + (1 - \alpha)y$, con $\alpha \in [0; 1]$;
- **Convex Crossover:** caso particolare dell'aritmetico con α fissato;
- **Simulated Binary Crossover (SBX):** riproduce il comportamento statistico del crossover binario in domini continui. La dispersione dei figli è controllata dal *Distribution Index* η e dallo *Spread Factor* β , che determina la distanza relativa tra figli e genitori:

$$\beta = \begin{cases} (2u)^{\frac{1}{\eta+1}} & \text{se } u \leq 0.5 \\ \left(\frac{1}{2(1-u)}\right)^{\frac{1}{\eta+1}} & \text{se } u > 0.5 \end{cases}$$

I figli sono quindi definiti da:

$$c_1 = \frac{1}{2}[(1 + \beta)p_1 + (1 - \beta)p_2] \quad c_2 = \frac{1}{2}[(1 - \beta)p_1 + (1 + \beta)p_2]$$

con comportamento contrattivo per $\beta < 1$, espansivo per $\beta > 1$ e identico per $\beta = 1$.

Per la **mutazione**:

- **Uniform Mutation:** sostituisce il gene con un valore casuale uniforme in $[L; U]$;
- **Gaussian Mutation:** applica una perturbazione gaussiana $x' = x + \mathcal{N}(0; \sigma^2)$, favorendo variazioni locali e fine-tuning.

L'addestramento genetico è generalmente più lento della Backpropagation nelle reti di grandi dimensioni, poiché si basa su una ricerca stocastica anziché sull'ottimizzazione guidata dal gradiente. Tuttavia, non richiede funzioni differenziabili, è quindi compatibile con attivazioni non continue, ed è immune al problema del *vanishing gradient*.

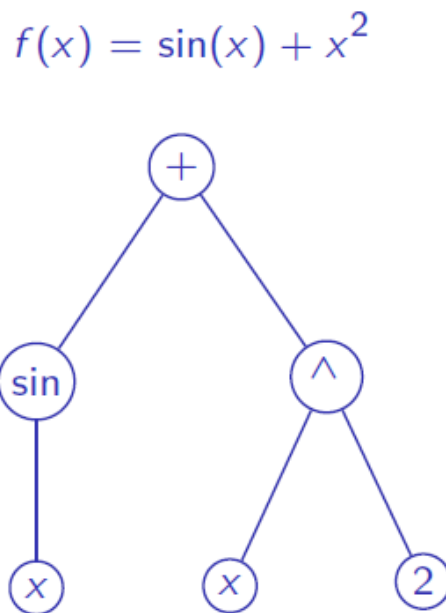
15.3.3 Genetic Programming

Spiega la Genetic Programming applicato al problema della Symbolic Regression. Che tipo di rappresentazione utilizzeresti per gli individui? Definisci un insieme plausibile di Funzioni e Terminali per un task di Regressione. Descrivi gli operatori di crossover e mutazione idonei a questa rappresentazione.

La **Genetic Programming** è una meta-euristica evolutiva che ottimizza automaticamente programmi per la risoluzione di problemi. A differenza dei Genetic Algorithms, che evolvono direttamente soluzioni candidate, la Genetic Programming evolve programmi che generano tali soluzioni. Gli individui sono rappresentati come **Syntax Tree**, mentre nei Genetic Algorithms sono tipicamente cromosomi a lunghezza fissa.

Mentre nella Regressione tradizionale la forma dell'equazione matematica è decisa a priori e l'algoritmo si limita a ottimizzarne i coefficienti (i pesi), nella **Regressione Simbolica** l'algoritmo scopre da solo sia la struttura dell'equazione sia i suoi coefficienti, partendo unicamente dai dati di Input e Output.

Un'equazione matematica ha una natura intrinsecamente gerarchica, che si mappa perfettamente su un albero. Ad esempio, l'equazione $\sin(x) + x^2$ viene rappresentata con la radice $+$, che ha come figlio sinistro un sotto-albero $\sin$ (con foglia x) e come figlio destro la foglia $\wedge$ (con foglia sinistra x e foglia destra 2).



Per costruire i **Syntax Trees**, si definiscono due insiemi fondamentali:

- **Function Set:** Sono i nodi interni dell'albero. Possono essere operatori matematici $(+, -, *, /)$, funzioni logiche (AND, OR), o funzioni trigonometriche (sin, cos). Ognuno ha una specifica "arietà" (il numero di figli che richiede, esempio: l'addizione richiede 2 figli);
- **Terminal Set:** Sono le foglie dell'albero, i nodi che non hanno figli. Includono le variabili di input del problema (esempio: X, Y) e costanti numeriche (esempio: $3.14, -5$).

Poiché lavoriamo con strutture ad albero, le classiche operazioni di incrocio e mutazione devono preservare la sintassi del programma:

- **Subtree Crossover:** È l'operatore principale di Crossover. Si prende un Genitore A e un Genitore B:
 1. Si seleziona a caso un nodo qualsiasi all'interno dell'albero A;
 2. Si seleziona a caso un nodo qualsiasi all'interno dell'albero B;
 3. Si tagliano fisicamente i due sotto-alberi radicati in quei nodi e si scambiano. Il risultato sono due nuovi alberi (figli) sintatticamente validi, ma che mescolano la logica dei genitori.
- **Subtree Mutation:** Si tratta di mutazione e non di crossover. Si sceglie un nodo casuale in un individuo, si taglia l'intero sotto-albero pendente da esso e lo si sostituisce con un albero del tutto nuovo generato casualmente. Inietta materiale genetico fresco.
- **Point Mutation:** Si tratta di mutazione e non di crossover. Si sostituisce un singolo nodo con un altro nodo dello stesso tipo e della stessa arietà (esempi: si cambia un operatore $+$ in un $-$, oppure la foglia x in una foglia y).

Nel corso dell'evoluzione, gli alberi tendono a crescere eccessivamente accumulando *introni*, ovvero codice ridondante che non altera l'output ma riduce l'efficienza computazionale e protegge porzioni funzionali da modifiche distruttive. Per limitare tale fenomeno si introduce la **Parsimony Pressure**, che penalizza la complessità strutturale nella funzione di fitness, favorendo soluzioni più compatte ed efficienti.

15.4 Simulated Annealing

15.4.1 Domanda Standard

Spiega la teoria e l'ispirazione fisica alla base dell'algoritmo Simulated Annealing (SA) per problemi di ottimizzazione. Qual è il ruolo del parametro Temperatura e del cooling schedule? Descrivi almeno una strategia di raffreddamento comune. Fornisci lo pseudocodice dell'algoritmo.

Il **Simulated Annealing** è un algoritmo di ottimizzazione probabilistica ispirato al processo di ricottura (*annealing*) dei metalli. È impiegato per risolvere problemi di ottimizzazione combinatoria, in particolare quando la funzione obiettivo presenta numerosi ottimi locali.

L'algoritmo riproduce il comportamento della ricottura: analogamente a un materiale che, raffreddandosi lentamente, tende verso configurazioni a energia sempre minore, il Simulated Annealing esplora lo spazio delle soluzioni accettando non solo miglioramenti, ma anche, con una certa probabilità, soluzioni peggiori. Tale probabilità è regolata da un parametro di **temperatura**, inizialmente elevato e progressivamente ridotto secondo una **cooling schedule**. Temperature elevate favoriscono l'esplorazione dello spazio delle soluzioni, mentre temperature basse rendono la ricerca sempre più selettiva, favorendo la convergenza.

In pratica, dopo aver inizializzato la temperatura, scelto una cooling schedule e aver trovato una soluzione di partenza, l'algoritmo esegue le seguenti iterazioni fino al raggiungimento del criterio di arresto:

1. Genera una soluzione vicina;
2. Valuta la funzione obiettivo per la nuova soluzione;

3. Confronta i valori della funzione obiettivo della soluzione corrente e della nuova soluzione:

- Se la nuova soluzione è migliore, accettarla come soluzione corrente;
- Se la nuova soluzione è peggiore, calcolare la probabilità di accettazione in base alla temperatura e a un criterio probabilistico (come il *criterio di Metropolis*). Quindi accettare la soluzione peggiore con una certa probabilità.

4. Regolare la temperatura in base alla cooling schedule.

L'accettazione di una soluzione peggiorativa è generalmente regolata dal **criterio di Metropolis**. Se il peggioramento è pari a $\Delta E > 0$, la probabilità di accettazione è:

$$P = e^{-\frac{\Delta E}{T}}$$

Con T temperatura corrente. Operativamente, si genera un numero casuale $r \in [0; 1]$ e la soluzione viene accettata se $r < P$.

La cooling schedule definisce l'evoluzione della temperatura nel corso dell'ottimizzazione, determinando il compromesso tra esplorazione e sfruttamento. Tra le strategie più comuni figurano il raffreddamento lineare, esponenziale e logaritmico.

```

1  import numpy as np
2
3  def perturb(current_x, a, b):
4      new_x = current_x + 0.2 * (b - a) * (np.random.rand() - 0.5)
5      return np.clip(new_x, a, b)
6
7  def SA_RE_1D(f, a, b, niter, lambda):
8      time = np.arange(1, niter + 1)
9      temperature = np.exp(-lambda * time)
10
11     current_x = np.random.rand() * (b - a) + a
12     current_y = f(current_x)
13
14     best_x = current_x
15     best_y = current_y
16
17     for i in range(1, niter):
18         new_x = perturb(current_x, a, b)
19         new_y = f(new_x)
20
21         if (new_y < current_y) or (np.random.rand() < temperature[i]):
22             current_x = new_x
23             current_y = new_y
24
25         if new_y < best_y:
26             best_x = new_x
27             best_y = new_y
28
29     return best_x, best_y

```

15.5 Particle Swarm Optimization

15.5.1 Domanda Standard

Spiega la teoria alla base della Particle Swarm Optimization (PSO). Descrivi in che modo le particelle esplorano lo spazio di ricerca e fornisci le formule matematiche utilizzate per aggiornare le loro velocità e posizioni. Quali sono i ruoli del peso inerziale, della componente cognitiva e della componente sociale?

La **Particle Swarm Optimization (PSO)** è un algoritmo di ottimizzazione stocastica ispirato al comportamento sociale degli stormi di uccelli (*bird flocking*) o dei banchi di pesci (*fish schooling*) per risolvere problemi in spazi continui (numeri reali). Nella PSO abbiamo:

- **Particelle:** Una particella rappresenta una soluzione al problema. Ogni particella "vola" all'interno dello spazio matematico multidimensionale del problema, cercando la soluzione migliore;
- **Sciame:** Insieme di particelle che evolve simultaneamente esplorando lo spazio di ricerca;
- **Posizione (x_i):** Le coordinate attuali nello spazio. Rappresenta la soluzione proposta in quel momento esatto;
- **Velocità (v_i):** La direzione e l'intensità (il "passo") del movimento;
- **Memoria Personale ($pbest_i$):** Le coordinate della posizione migliore che quella specifica particella ha visitato finora nella sua vita;
- **Memoria Globale ($gbest$):** Le coordinate della posizione migliore scoperta finora dall'intero stormo.

Il flusso dell'algoritmo è il seguente:

1. Inizializza uno sciame di particelle con posizioni e velocità casuali;
2. Valuta la Fitness di ciascuna particella;
3. Aggiorna le posizioni migliori personali e globali;
4. Aggiorna le velocità e le posizioni delle particelle;
5. Ripeti i passaggi 2-4 fino al raggiungimento di un criterio di arresto.

Il nucleo dell'algoritmo risiede nell'aggiornamento iterativo della traiettoria di ciascuna particella nel tempo discreto t . Tale dinamica non è stocastica, ma determinata dalla combinazione vettoriale di tre componenti fondamentali che ne governano l'evoluzione nello spazio di ricerca.

Ad ogni passo t , per ogni dimensione j dello spazio, la velocità della particella i viene calcolata con questa equazione fondamentale:

$$v_i(t+1) = \underbrace{w \cdot v_i(t)}_{\text{Inerzia}} + \underbrace{c_1 \cdot r_1 \cdot (pbest_i - x_i(t))}_{\text{Componente Cognitiva}} + \underbrace{c_2 \cdot r_2 \cdot (gbest - x_i(t))}_{\text{Componente Sociale}}$$

Analizziamo i tre "motori" del movimento:

- **Inerzia ($w \cdot v$):** La tendenza della particella a continuare a muoversi nella stessa direzione in cui stava già andando. w è l'Inertia Weight (peso di inerzia). w controlla il bilanciamento tra esplorazione e sfruttamento, analogamente alla temperatura nel *Simulated Annealing*. Valori elevati (esempio: 0.9) favoriscono un'elevata esplorazione dello spazio di ricerca, mentre valori ridotti (esempio: 0.1) inducono un comportamento più conservativo, concentrato attorno a $pbest$ e $gbest$;
- **Componente Cognitiva (c_1):** La particella è attratta verso il suo miglior successo passato ($pbest$). Il coefficiente c_1 rappresenta la "fiducia in se stessi";
- **Componente Sociale (c_2):** La particella è attratta verso la posizione migliore trovata dal gruppo ($gbest$). Il coefficiente c_2 rappresenta la "fiducia nello sciame".

I valori r_1 e r_2 sono **numeri casuali** (tra 0 e 1) estratti ad ogni passo. Introducono la *stocasticità*, impedendo alle particelle di collidere tutte matematicamente nello stesso punto e rendono le traiettorie irregolari, come in un vero stormo.

Una volta calcolata la **nuova velocità**, la **nuova posizione** si ottiene con una semplice somma:

$$x_i(t+1) = x_i(t) + v_i(t+1)$$

Per evitare che l'accumulo delle tre forze spari le particelle fuori dai limiti dell'universo matematico, la velocità viene sempre "tagliata" (**Velocity Clamping**) entro un limite massimo prestabilito $[-V_{max}; V_{max}]$.

PSO ha alcuni **vantaggi**:

- Facile da implementare e comprendere;
- Efficiente per problemi di ottimizzazione continua;
- Richiede un numero inferiore di parametri rispetto agli algoritmi genetici;
- Ottimo nella gestione di funzioni non lineari e non differenziabili.

Tuttavia presenta anche degli importanti **limiti**:

- Potrebbe convergere prematuramente all'ottimo locale;
- Le prestazioni dipendono molto dalla giusta impostazione dei parametri;
- Meno efficace per problemi di ottimizzazione discreta;
- La mancanza di diversità nelle iterazioni successive può ridurre l'esplorazione.

```

1 import numpy as np
2
3 def objective_function(params):
4     return (params[:, 0] - 4)**2 + (params[:, 1] - 5)**2 + (params[:, 2] + 6)**2
5
6 def run_pso(n_particles, max_iter, w, c1, c2, bounds, obj_func):
7     dim = len(bounds[0])
8
9     particles = bounds[0] + np.random.rand(n_particles, dim) * (bounds[1] - bounds[0])
10    velocities = np.zeros_like(particles)

```

```

11
12 p_best_pos = particles.copy()
13 p_best_costs = obj_func(particles)
14
15 g_best_idx = np.argmin(p_best_costs)
16 g_best_pos = p_best_pos[g_best_idx].copy()
17 g_best_cost = p_best_costs[g_best_idx]
18
19 cost_history = []
20
21 for _ in range(max_iter):
22     r1 = np.random.rand(n_particles, dim)
23     r2 = np.random.rand(n_particles, dim)
24
25     velocities = (w * velocities +
26                  c1 * r1 * (p_best_pos - particles) +
27                  c2 * r2 * (g_best_pos - particles))
28
29     particles = particles + velocities
30     particles = np.clip(particles, bounds[0], bounds[1])
31
32     costs = obj_func(particles)
33
34     improved = costs < p_best_costs
35     p_best_pos[improved] = particles[improved]
36     p_best_costs[improved] = costs[improved]
37
38     current_best_idx = np.argmin(p_best_costs)
39     if p_best_costs[current_best_idx] < g_best_cost:
40         g_best_pos = p_best_pos[current_best_idx].copy()
41         g_best_cost = p_best_costs[current_best_idx]
42
43     cost_history.append(g_best_cost)
44
45     return g_best_pos, g_best_cost, cost_history

```

15.6 Ant Colony Optimization

15.6.1 Domanda Standard

Spiega i principi della Ant Colony Optimization (ACO) per la risoluzione di problemi di routing sui grafi. Descrivi in dettaglio la regola della probabilità di transizione utilizzata dalle formiche per selezionare il nodo successivo e l'equazione di aggiornamento dei feromoni, definendo esplicitamente i ruoli di tutti i parametri (esempio: tasso di evaporazione, informazione euristica). Cosa succede in uno scenario di edge deletion?

L'Ant Colony Optimization (ACO) è una meta-euristica ispirata al comportamento foraggiatore delle formiche, perfetta per risolvere problemi su grafi discreti come il *Traveling Salesman Problem* (TSP) o l'instradamento dei pacchetti nelle reti di telecomunicazione.

Le formiche reali, quasi cieche e prive di un controllo centrale, si coordinano tramite la **stigmergia**: rilasciano feromoni lungo il percorso e, quando trovano cibo, rafforzano la traccia al ritorno. Le altre formiche seguono e intensificano il segnale, creando un feedback positivo che porta la colonia a convergere sul percorso più efficiente.

L'algoritmo è il seguente:

1. **Inizializzazione:** Su ogni arco $(i; j)$ del grafo vengono inizializzati due valori:

- τ_{ij} (Tau): La *Pheromone Matrix* che misura il livello di feromone (dinamico, inizializzato a un valore piccolo);
- η_{ij} (Eta): La *Heuristic Matrix* che rappresenta informazione euristica (statica, tipicamente l'inverso della distanza: $\eta_{ij} = 1/d_{ij}$). Equivale a un approccio greedy (più la città è vicina, più è desiderabile).

2. **Costruzione del percorso (Regola di Transizione Probabilistica):** Ogni formica costruisce una soluzione muovendosi di nodo in nodo. La decisione non è deterministica, ma calcolata *on-the-fly* tramite una "ruota della roulette pesata". La probabilità p_{ij} che una formica nel nodo i scelga di andare nel nodo j è data da:

$$p_{ij} = \frac{[\tau_{ij}]^\alpha [\eta_{ij}]^\beta}{\sum_{u \in Allowed_i} [\tau_{iu}]^\alpha [\eta_{iu}]^\beta}$$

- α (**Influenza del feromone**): Quanto la formica si fida delle "scoperte" passate della colonia (**Exploitation**);
- β (**Influenza dell'euristica**): Quanto la formica si affida alla convenienza immediata locale, come la distanza breve (**Exploration**).

3. **Aggiornamento dei Feromoni:** Una volta che tutte le formiche hanno costruito i loro percorsi, la matrice dei feromoni τ viene aggiornata:

$$\tau_{ij}(t+1) = (1 - \rho)\tau_{ij}(t) + \sum_{k=1}^m \Delta\tau_{ij}^{(k)}(t)$$

- **Evaporazione** $(1 - \rho)$: Il parametro ρ regola la diminuzione dei feromoni, permettendo di abbandonare progressivamente i percorsi meno efficaci ed evitando convergenze premature.
 - **Deposizione** $(\sum \Delta\tau)$: Quantità di feromone depositata dalla formica k sull'arco $(i; j)$.
4. **Iterazione:** Il processo si ripete finché non viene raggiunto un criterio di arresto (esempio: numero massimo di iterazioni).

Una singola formica rappresenta la fase di costruzione di un'architettura. La formica parte dall'inizio e deve attraversare tutte le tappe. A ogni tappa, sceglie un nodo in base alla quantità di feromone presente su di esso. Iterazione dopo iterazione, le formiche successive sentiranno un forte odore di feromone sulle scelte vincenti. L'intera colonia smetterà di esplorare i valori inutili e convergerà rapidamente sull'architettura ottimale, ottimizzando il modello.

ACO utilizza una matrice di feromoni τ che codifica la desiderabilità appresa delle soluzioni. Il **warm-start** consiste nell'inizializzare il processo di ottimizzazione a partire da una configurazione informata anziché casuale. Nei problemi dinamici, l'ACO può adattarsi rapidamente riutilizzando la struttura dei feromoni esistenti. Questa capacità di riutilizzo rende il warm-start particolarmente efficiente rispetto a molte altre meta-euristiche.

In uno scenario di **edge deletion**, un arco $(i; j)$ viene rimosso dal grafo in seguito a una modifica della struttura del problema. Per riflettere tale variazione, la matrice dei feromoni viene aggiornata impostando $\tau_{ij} = 0$ oppure eliminando la corrispondente voce, a seconda della rappresentazione adottata. L'algoritmo ACO viene quindi rieseguito sulla nuova configurazione del grafo, impedendo alle formiche di selezionare l'arco rimosso e orientando la ricerca verso percorsi alternativi validi.

ACO ha alcuni **vantaggi**:

- Buona capacità di esplorazione dello spazio delle soluzioni;
- Efficiente per problemi di ottimizzazione discreta e su grafi;
- Capacità di adattamento a scenari dinamici;
- Possibilità di warm-start tramite feromoni preesistenti.

Tuttavia presenta anche degli importanti **limiti**:

- Convergenza lenta rispetto ad altri metaeuristici;
- Sensibilità alla scelta dei parametri (α , β , ρ);
- Elevato costo computazionale su istanze grandi;
- Performance meno efficace su problemi continui rispetto a quelli discreti.

15.7 Advanced Deep Learning

15.7.1 NAS

Spiega il concetto di Neural Architecture Search (NAS) e il suo obiettivo nell'automazione della progettazione delle Artificial Neural Networks.

Il limite delle **Hand-crafted Architectures** risiede nella progettazione manuale delle reti neurali, tradizionalmente guidata dall'intuizione e da lunghe fasi di trial-and-error. Questo approccio introduce bias verso strutture regolari, restringe l'esplorazione dello spazio architetturale e comporta tempi di sviluppo elevati. Inoltre, per garantire prestazioni adeguate, si ricorre spesso a modelli sovradimensionati, caratterizzati da un numero elevato di parametri e da possibili ridondanze.

La **Neural Architecture Search (NAS)** è il processo volto ad automatizzare la progettazione delle reti neurali artificiali, sostituendo le Hand-crafted Architectures fatte dall'uomo. Si usano tre approcci NAS.

Il primo è quello dei **Genetic Algorithms**:

- **Pro:** Offrono flessibilità e capacità di ricerca globale (Global Search).
- **Contro:** È un metodo computationally intensive (estremamente oneroso dal punto di vista computazionale).

Il secondo è quello del **Reinforcement Learning**. Abbiamo una rete Controller (una RNN) che genera sequenzialmente i parametri dell'architettura (le azioni). La rete generata viene addestrata e la sua Accuracy finale viene usata come Segnale di Ricompensa (**Reward**). Il Controller viene aggiornato tramite **Policy Gradient** per generare architetture sempre migliori. L'obiettivo del Reinforcement Learning è quindi massimizzare il valore atteso del reward.

- **Pro:** Ha generato reti storiche (come NASNet) che hanno battuto quelle umane.
- **Contro:** Ancora più costoso dei Genetic Algorithms. La prima NASNet richiese 800 GPU che lavorarono ininterrottamente per 28 giorni (migliaia di ore di calcolo).

Il terzo è **Differentiable Architecture Search (DARTS)**. Invece di fare una scelta discreta tra varie operazioni, DARTS applica un rilassamento continuo. Utilizzando una funzione Softmax, l'algoritmo "smussa" lo spazio di ricerca, permettendo di ottimizzare l'architettura simultaneamente ai pesi utilizzando le potenti tecniche basate sul gradiente (Gradient-Based Optimization). Alla fine discretizza la soluzione.

- **Pro:** Riduce i tempi di ricerca da migliaia di giorni/GPU a una frazione infinitesimale, potendo girare su una singola GPU.
- **Contro:** Dovendo calcolare tutte le operazioni in parallelo (Supernetwork) prima di scartare quelle inutili, richiede moltissima RAM/VRAM.

Non esiste un metodo perfetto in assoluto. Il metodo più appropriato dipende dalle risorse a disposizione, dalla complessità del problema e dai requisiti specifici del progetto.

15.7.2 Knowledge Distillation

Spiega i principi di funzionamento della Knowledge Distillation nelle Deep Neural Networks. Descrivi i ruoli dei modelli Teacher e Student. Come viene calcolata la loss totale durante l'addestramento della rete Student? Discuti il ruolo del parametro di temperatura (T) nella funzione softmax e in che modo influenza la generazione dei soft target. Spiega il concetto di "dark knowledge".

I moderni modelli di Deep Learning e gli ensemble raggiungono elevata accuratezza, ma risultano spesso troppo onerosi in termini di memoria, latenza e consumo energetico per dispositivi *edge* e mobile.

La **Knowledge Distillation (KD)** affronta questo problema trasferendo la conoscenza di un modello complesso (**Teacher**) a un modello più compatto (**Student**). Il Teacher è una rete già addestrata e altamente accurata, mentre lo Student viene addestrato a imitare il suo comportamento mantenendo costi computazionali ridotti.

La KD consente quindi di comprimere i modelli, accelerare l'inferenza e preservare prestazioni vicine a quelle del Teacher, favorendo inoltre una migliore generalizzazione.

Nel caso della Classificazione, il Teacher non produce solamente la classe corretta, ma una distribuzione di probabilità sulle classi. Ad esempio:

[Auto : 0.90; Camion : 0.09; Moto : 0.01]

Queste probabilità contengono informazioni sulle relazioni tra le classi, note come **Dark Knowledge**. Nell'esempio, il Teacher ha appreso che un'auto è più simile a un camion che a una moto. Tale informazione andrebbe persa utilizzando esclusivamente le etichette binarie del dataset.

Per rendere più evidenti queste relazioni si introduce una **temperatura T** nella Softmax:

$$q_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Con $T = 1$ si ottiene la Softmax standard, mentre per $T > 1$ la distribuzione diventa più uniforme, rendendo visibili le probabilità associate alle classi secondarie.

La conoscenza del Teacher può essere trasferita attraverso diverse strategie:

- **Soft Target Matching:** lo Student apprende le distribuzioni di probabilità del Teacher, generalmente tramite *Kullback-Leibler Divergence*;
- **Direct Output Matching:** lo Student imita direttamente i logits del Teacher, minimizzando tipicamente una MSE;
- **Intermediate Feature Matching:** vengono allineate le rappresentazioni interne dei due modelli mediante il confronto delle attivazioni degli hidden layer;
- **Input Gradient Matching:** lo Student apprende mappe di attenzione o gradienti simili a quelli del Teacher, replicandone il comportamento sugli input.

L'addestramento dello Student combina due obiettivi:

- **Student Loss:** Cross-Entropy tra le predizioni dello Student e le etichette reali;
- **Distillation Loss:** Kullback-Leibler Divergence tra le distribuzioni ammorbidite di Teacher e Student.

La loss complessiva è:

$$\mathcal{L}_{total} = (1 - \alpha) \cdot \mathcal{L}_{Student} + \alpha \cdot T^2 \cdot \mathcal{L}_{Distillation}$$

Dove α controlla il contributo della distillazione e il termine T^2 compensa la riduzione dei gradienti dovuta alla temperatura.

Nei problemi di Regressione, la distillazione può essere realizzata combinando la loss del task con una loss che forza lo Student a replicare le predizioni del Teacher:

$$\mathcal{L}_{total} = \alpha \cdot \mathcal{L}_{task} + \beta \cdot \mathcal{L}_{HP}$$

Dove $\mathcal{L}_{task}$ è l'MSE rispetto alla ground truth, $\mathcal{L}_{HP}$ è l'MSE rispetto alle predizioni del Teacher, mentre α e β ne bilanciano l'importanza.

Capitolo 16: Domanda 3

16.1 Autograd

16.1.1 Computational Graph

Spiega il concetto di Computational Graph e il suo ruolo fondamentale nell'Automatic Differentiation (Autograd) all'interno di framework moderni di Deep Learning come PyTorch. Discuti la differenza tra forward differentiation e backward differentiation.

Un problema fondamentale nel **Machine Learning** consiste nel determinare i parametri ottimali w minimizzando una Loss Function $\mathcal{L}$. A tal fine si utilizzano algoritmi di ottimizzazione, come la **Stochastic Gradient Descent (SGD)**, che aggiorna i pesi nella direzione opposta al gradiente:

$$w_{t+1} = w_t + \eta \left(-\frac{\partial \mathcal{L}}{\partial w_t} \right)$$

La difficoltà principale è il calcolo efficiente del gradiente $\frac{\partial \mathcal{L}}{\partial w}$ in reti neurali con milioni di parametri. La derivazione analitica e l'implementazione manuale risultano impraticabili, mentre l'algebra simbolica soffre del fenomeno di *expression swell*. L'approssimazione mediante rapporto incrementale $\frac{f(x+h)-f(x)}{h}$ richiede invece due passaggi forward per parametro ed è soggetta a errori numerici dovuti alla rappresentazione *floating-point*.

Autograd, il sistema di differenziazione automatica di **PyTorch**, supera questi limiti calcolando i gradienti in modo efficiente durante l'addestramento delle reti neurali.

Un **Grafo Computazionale** è una rappresentazione visiva e matematica di un'equazione complessa.

- **Nodi:** Rappresentano le variabili (input, parametri, costanti) o le singole operazioni matematiche (addizione, moltiplicazione, seno, coseno);
- **Archi:** Rappresentano il flusso dei dati (i Tensori). Indicano l'ordine in cui le operazioni devono essere eseguite.

Una volta costruito il grafo, ci sono due modi per calcolare le derivate:

- **Forward Mode Differentiation:** Valuta la funzione e la sua derivata simultaneamente, partendo dagli input e muovendosi in avanti (forward) verso l'output. Usa concettualmente i "numeri duali". È ideale quando abbiamo una funzione $f : \mathbb{R}^m \rightarrow \mathbb{R}^n$ dove il numero di input m è molto minore del numero di output n ($m \ll n$).
- **Backward Mode Differentiation:** Parte dall'output finale e torna indietro verso gli input, applicando ricorsivamente la *Chain Rule* del calcolo differenziale. Se una variabile y dipende da u , che a sua volta dipende da x :

$$y = f(u) \qquad u = g(x)$$

allora la derivata di y rispetto a x è:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

È ideale quando abbiamo una funzione $f : \mathbb{R}^n \rightarrow \mathbb{R}$ dove gli input n sono milioni (i parametri della rete) e l'output è uno solo (il valore scalare della Loss).

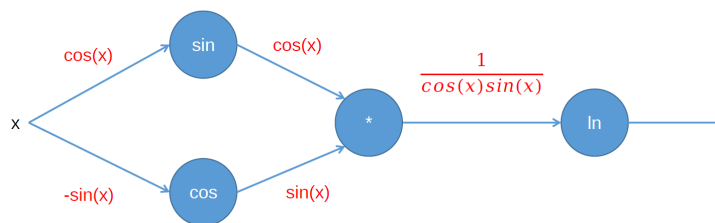
La Loss Function spesso è una funzione $f : \mathbb{R}^n \rightarrow \mathbb{R}$ con n il numero di pesi della rete e di conseguenza per noi è più conveniente la Backward Mode Differentiation.

Ecco il processo passo dopo passo della costruzione del grafo:

1. Quando dichiari un tensore in PyTorch, bisogna impostare il flag: `requires_grad=True`;
2. Nel momento in cui esegui un'operazione matematica (esempio: $z = x + y$), Autograd fa due cose contemporaneamente:
 - Calcola il risultato numerico di z ;
 - Istanza un oggetto "Operazione" per il Grafo Computazionale.
3. Quando chiami la funzione `out.backward()`, il motore Autograd prende questo grafo appena costruito e inizia a percorrerlo a ritroso costruendo il gradiente della Loss in base ai pesi.

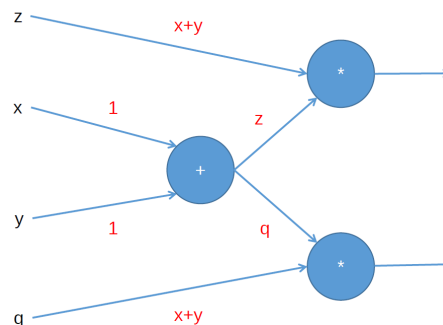
Ad esempio:

Exercises: $f(x) : \mathbb{R} \rightarrow \mathbb{R} : \ln(\sin(x) \cdot \cos(x))$

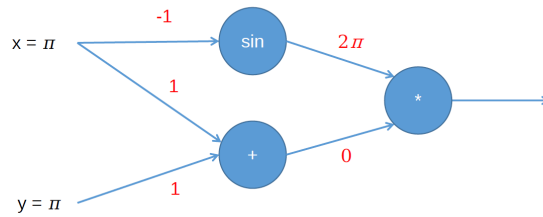


$$\cos(x)\cos(x) \cdot \frac{1}{\cos(x)\sin(x)} \cdot \sin(x)\sin(x) \cdot \frac{1}{\cos(x)\sin(x)} = \frac{\cos(x)}{\sin(x)} - \frac{\sin(x)}{\cos(x)} = \cotan(x) - \tan(x)$$

Exercises: $f(x, y, z, q) : \mathbb{R}^4 \rightarrow \mathbb{R}^2 : \begin{pmatrix} (x+y) \cdot z \\ (x+y) \cdot q \end{pmatrix}$



Exercises: $(x + y)\sin(x)$ *where $x = \pi$ and $y = \pi$*



16.2 PyTorch

16.2.1 Dataset, DataLoader e Optimizer

Delinea i passaggi previsti per l'addestramento di una rete neurale all'interno del framework PyTorch, concentrandoti in modo specifico sui ruoli di Dataset, DataLoader e Optimizer. Spiega i processi in dettaglio e fornisci lo pseudocodice della routine di ottimizzazione.

I tre pilastri di questo processo sono il **Dataset**, il **DataLoader** e l'**Optimizer**.

La classe **Dataset** è un'astrazione che rappresenta i tuoi dati grezzi (immagini, testi, numeri) e le rispettive etichette (le ground truth). Il suo unico scopo è dire a **PyTorch** come recuperare un singolo elemento di dato alla volta.

Per creare un Dataset personalizzato in **PyTorch**, devi sovrascrivere due metodi:

- `__len__(self)`: Restituisce il numero totale di campioni nel dataset;
- `__getitem__(self, index)`: Riceve un indice intero e restituisce il campione di dati corrispondente e la sua etichetta, convertiti in Tensori di **PyTorch**.

Mentre il Dataset sa come prendere un singolo dato, le reti neurali moderne vengono addestrate usando i Mini-Batch con il DataLoader:

- **Batching**: Raggruppa N campioni singoli estratti dal Dataset in un unico grande Tensore;
- **Shuffling**: Mescola casualmente l'ordine dei dati ad ogni epoca (`shuffle=True`), che è cruciale per evitare che la rete impari a memoria l'ordine del dataset e per inserire quel rumore stocastico necessario a sfuggire ai minimi locali;
- **Multiprocessing**: Utilizza più worker (processi CPU in parallelo) per pre-caricare i dati in memoria mentre la GPU sta addestrando il batch precedente. Questo evita che la GPU (velocissima) debba aspettare la CPU (lenta a leggere dal disco).

L'**Optimizer** è l'algoritmo che implementa la regola di aggiornamento dei pesi (esempio: SGD, Adam, RMSprop). Quando lo inizializzi, gli passi i parametri (pesi e bias) del tuo modello e un Learning Rate.

Il suo ruolo è prendere i gradienti calcolati dal motore di Autograd (memorizzati nell'attributo `.grad` di ogni tensore foglia) e usarli per sottrarre la giusta frazione ai pesi correnti, spingendo la rete verso il minimo della funzione di Loss.

All'interno del ciclo dei batch, si ripetono sempre, rigorosamente, questi 5 passaggi:

- `optimizer.zero_grad()`: PyTorch, per default, accumula (somma) i gradienti delle iterazioni precedenti in Autograd. Se non li azzeriamo all'inizio di ogni nuovo batch, la rete sommerà i gradienti vecchi a quelli nuovi, muovendosi in direzioni completamente sballate;
- **Forward Pass**: Si passano i dati di input al modello per ottenere le previsioni ($\hat{y}$);
- **Calcolo della Loss**: Si confrontano le previsioni con le etichette reali per calcolare l'errore scalare;
- `loss.backward()`: Percorre il grafo computazionale a ritroso e calcola la derivata esatta della loss rispetto a ogni singolo peso del modello, popolandolo i loro attributi `.grad`;
- `optimizer.step()`: L'optimizer guarda i gradienti appena calcolati e aggiorna fisicamente le matrici dei pesi della rete.

```

1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  from torch.utils.data import Dataset, DataLoader
5
6  class CustomDataset(Dataset):
7      def __init__(self, data, labels):
8          self.data = data
9          self.labels = labels
10
11     def __len__(self):
12         return len(self.data)
13
14     def __getitem__(self, idx):
15         return self.data[idx], self.labels[idx]
16
17 class SimpleModel(nn.Module):
18     def __init__(self, input_dim):
19         super().__init__()
20         self.network = nn.Sequential(
21             nn.Linear(input_dim, 32),
22             nn.ReLU(),
23             nn.Linear(32, 1),
24             nn.Sigmoid()
25         )
26
27     def forward(self, x):
28         return self.network(x)
29
30 def train_model(dataset, input_dim):
31     dataloader = DataLoader(dataset, batch_size=32, shuffle=True)
32
33     model = SimpleModel(input_dim=input_dim)
34     criterion = nn.BCELoss()
35     optimizer = optim.Adam(model.parameters(), lr=0.01)
36
37     model.train()
38
39     for epoch in range(5):
40         for features, labels in dataloader:
41             optimizer.zero_grad()
42             predictions = model(features)

```

```
43     loss = criterion(predictions, labels)
44     loss.backward()
45     optimizer.step()
46
47     return model
```

16.2.2 Tensori

Definisci il concetto di Tensore e i fondamenti dell'algebra dei tensori nei moderni framework di Deep Learning. Descrivi in dettaglio le meccaniche del broadcasting ed enuncia le regole rigorose che consentono la moltiplicazione element-wise tra tensori di forme diverse. Discuti le operazioni di rearrange (esempio: reshaping/permuting) e reduce (esempio: sum/mean su assi specifici).

Un **tensore** è un array n -dimensionale i cui elementi hanno tutti lo stesso tipo di dato. Il valore n viene chiamato *rango* o *dimensione* del tensore.

Di solito i tensori contengono numeri reali, per questo motivo è naturale riferirsi ai tensori di rango 0, 1 e 2 come oggetti matematici:

- un tensore a 0 dimensioni è uno *scalare*;
- un tensore a 1 dimensione è un *vettore*;
- un tensore a 2 dimensioni è una *matrice*;
- un tensore a 3 dimensioni può essere visto come un vettore di matrici tutte della stessa forma;
- un tensore a 4 dimensioni può essere visto come una matrice di matrici tutte della stessa forma, oppure come un vettore di tensori di rango 3.

La sua struttura è definita da tre attributi principali:

- **Rank:** Quanti assi (o indici) servono per individuare un singolo numero;
- **Shape:** Una tupla che indica esattamente quanti elementi ci sono lungo ogni asse (esempio: la shape di un'immagine FHD a colori è (3, 1080, 1920));
- **Dtype:** Il tipo di dato contenuto (esempio: torch.float32, torch.int64).

Nell'algebra lineare tradizionale, le operazioni elemento per elemento tra due matrici richiedono che queste abbiano la medesima dimensione. Nei framework di Deep Learning (ad esempio NumPy, PyTorch e TensorFlow), tale vincolo viene generalizzato mediante il meccanismo di **Broadcasting**.

Il **Broadcasting** consente di eseguire operazioni element-wise tra tensori con shape differenti, espandendo virtualmente il tensore di dimensioni inferiori senza allocare memoria aggiuntiva.

La **compatibilità tra due tensori** viene verificata confrontando le rispettive shape da destra verso sinistra. Due dimensioni sono considerate compatibili se soddisfano almeno una delle seguenti condizioni:

- sono uguali;
- una delle due è pari a 1, nel qual caso il tensore viene replicato virtualmente lungo tale asse;
- una delle due è assente, situazione equivalente all'aggiunta implicita di una dimensione unitaria nelle posizioni più esterne, fino a rendere i due tensori dello stesso rango.

Nelle reti neurali è spesso necessario riorganizzare i dati senza modificarne i valori. Le principali operazioni di manipolazione dei tensori sono:

- **Reshape / View:** modificano la shape del tensore mantenendo invariato il numero di elementi e la loro disposizione in memoria.
- **Permute / Transpose:** riordinano gli assi del tensore, modificandone la struttura semantica (ad esempio da [H, W, C] a [C, H, W]).
- **Squeeze / Unsqueeze:** rimuovono o aggiungono dimensioni unitarie (di ampiezza 1), adattando il tensore ai requisiti delle operazioni successive.

Le **operazioni di riduzione** (*sum, mean, max, min*, eccetera...) collassano una o più dimensioni di un tensore calcolando una statistica lungo l'asse specificato dal parametro `dim`. L'asse selezionato viene eliminato, salvo che sia impostato `keepdim=True`, nel qual caso viene mantenuto con dimensione unitaria, facilitando le successive operazioni di *Broadcasting*.

16.2.3 SGD

Spiega la teoria alla base della Stochastic Gradient Descent (SGD) per l'addestramento delle Deep Neural Networks. Discuti in dettaglio l'impatto fisico dei diversi valori di Learning Rate (η) sulla convergenza dell'algoritmo (esempio: cosa succede quando η è troppo alto o troppo basso?). Come mitigheresti i problemi associati a learning rate subottimali e il rischio di rimanere intrappolati in punti di sella o minimi locali?

La **Stochastic Gradient Descent (SGD)** è un algoritmo di ottimizzazione iterativo utilizzato per addestrare le reti neurali. Il suo obiettivo è minimizzare la Loss Function aggiornando i parametri del modello nella direzione opposta al gradiente della funzione di costo:

$$w_{t+1} = w_t + \eta \left(-\frac{\partial \mathcal{L}}{\partial w_t} \right)$$

Dove $\frac{\partial \mathcal{L}}{\partial w_t}$ è il gradiente (la pendenza) calcolato tramite Backpropagation, ed η (Eta) è il Learning Rate. Il Learning Rate (η) è l'iperparametro più critico di tutta l'architettura. Rappresenta la lunghezza del passo che l'algoritmo compie nella direzione indicata dal gradiente. Un learning rate basso ha passi microscopici, con un addestramento estremamente lento. Poiché il passo è minuscolo, se l'algoritmo incontra una piccolissima buca (minimo locale sub-ottimale), non avrà lo slancio sufficiente per uscirne e si fermerà lì per sempre. Un learning rate alto ha passi giganteschi. Può saltare completamente il punto di minimo globale, oppure rimbalza tra le pareti della Loss Function, oscillando violentemente. Il learning rate ottimale fa scendere l'errore in modo rapido e fluido nelle prime epoche, per poi assestarsi in modo stabile (plateau) al raggiungimento del minimo globale.

Negli spazi ad altissima dimensionalità (milioni di pesi), il vero nemico non sono i minimi locali, ma i **Punti di Sella** (Saddle Points): aree piatte dove la pendenza (il gradiente) è quasi zero in alcune direzioni, ma che non sono il vero fondo della valle. In un punto di sella, il gradiente diventa 0, e l'aggiornamento $w_{new} = w_{old} - \eta \cdot 0$ paralizza l'addestramento.

Per superare le aree piatte, si aggiunge un termine di **Inerzia** (Momentum) alla formula dello SGD. Invece di basarsi solo sul gradiente attuale, l'aggiornamento ricorda una frazione dei gradienti passati:

$$v_t = \gamma \cdot v_{t-1} + \eta \cdot \nabla L(w)$$

$$w_{new} = w_{old} - v_t$$

Inoltre possiamo adattare il learning rate mano a mano che le iterazioni vanno avanti.

16.3 CNN

16.3.1 Domanda Standard

Fornisci una panoramica dell'architettura generale di una CNN, descrivendo in dettaglio i ruoli dei convolutional layer, dei pooling layer e dei fully connected layer. Discuti come ciascun livello contribuisce all'estrazione delle feature e alla Classificazione nei task di riconoscimento delle immagini. Fornisci lo pseudocodice per un passaggio in avanti attraverso un convolutional layer seguito da un livello di max pooling, ponendo l'accento sulle operazioni di stride, padding e filter.

Le **Convolutional Neural Networks (CNN)** sono una particolare architettura di reti neurali progettata per elaborare dati che presentano una struttura spaziale, come immagini, video o immagini mediche.

L'idea fondamentale è che, invece di analizzare ogni pixel singolarmente come farebbe un MLP, una CNN sfrutta il fatto che pixel vicini sono correlati tra loro e cerca automaticamente pattern locali come bordi, texture e forme.

Ad esempio, un'immagine RGB (quindi 3 canali) di dimensione 256×256 elaborata da uno strato con 1000 neuroni richiederebbe circa $256 \times 256 \times 3 \times 1000 \approx 1.97 \times 10^8$ pesi, con conseguenti problemi di costo computazionale e overfitting.

Il nucleo di una CNN è il **Convolutional Layer** dove si utilizza un piccolo "filtro", ossia il Kernel (ad esempio una matrice 3×3 o 5×5).

Questo kernel scorre letteralmente sopra l'immagine di input, partendo dall'angolo in alto a sinistra fino a quello in basso a destra. Ad ogni passo, esegue un'operazione matematica semplice: moltiplica i valori dei pixel coperti in quel momento per i pesi del kernel e li somma, producendo un singolo numero in uscita. L'insieme di tutti questi numeri generati dal passaggio del kernel forma una nuova mappa, chiamata **Feature Map**. Ogni pixel della feature map indica la presenza del pattern cercato. Un solo kernel produce una feature map.

Poiché il filtro ha una certa larghezza, facendolo scivolare l'immagine si rimpicciolirebbe a ogni layer, perdendo i pixel sui bordi:

- **Padding:** Si risolve il problema aggiungendo una cornice attorno all'immagine di input che sia di zeri (Zero-Padding) o di una copia dei pixel nel bordo, permettendo al filtro di elaborare i bordi senza ridurre la dimensione spaziale dell'output.

- **Stride:** È il numero di pixel di cui il filtro si sposta ad ogni passo. Uno stride di 1 esplora ogni pixel (output grande). Uno stride di 2 fa "saltare" un pixel al filtro, dimezzando le dimensioni della Feature Map in output.

Abbiamo quindi che una rete convoluzionale è composta da:

- **Convolutional Layer:** Che applica la convoluzione, mano a mano a tutti i gruppi di pixel dell'immagine;
- **Pooling Layer:** Subito dopo l'estrazione delle feature (e l'applicazione di una funzione di attivazione come la ReLU), i dati passano a questo Layer, che esegue un sottocampionamento spaziale (downsampling) non lineare. Il metodo più comune è il Max Pooling, dove una finestra mobile (esempio: 2×2) seleziona il valore più alto, senza usare pesi da apprendere. In questo modo si riducono le dimensioni delle feature map, diminuendo il costo computazionale e il rischio di overfitting;
- **Fully Connected Layer:** Dopo che l'immagine è passata attraverso svariate fasi di Convoluzione e Pooling, si ottengono delle Feature Map molto piccole nello spazio, ma molto profonde. Questi tensori 3D vengono srotolati in un lunghissimo vettore 1D. Questo vettore viene passato ai Fully Connected Layers (i classici strati MLP). A questo punto, le relazioni spaziali non contano più. La rete ragiona globalmente. L'ultimo layer usa solitamente una funzione Softmax per fornire la distribuzione di probabilità finale tra le varie classi.

```

1  import torch
2  import torch.nn as nn
3
4  class CNN(nn.Module):
5      def __init__(self, num_classes):
6          super().__init__()
7
8          self.cnn = nn.Sequential(
9              nn.Conv2d(
10                 in_channels=3,
11                 out_channels=32,
12                 kernel_size=3,
13                 stride=1,
14                 padding=1
15             ),
16             nn.LeakyReLU(),
17             nn.MaxPool2d(kernel_size=2),
18             nn.Flatten()
19         )
20
21         self.ff = nn.Sequential(
22             nn.Linear(32 * 14 * 14, 128),
23             nn.LeakyReLU(),
24             nn.Linear(128, num_classes)
25         )
26
27     def forward(self, x):
28         x = self.cnn(x)
29         x = self.ff(x)
30         return x

```

Nel codice, si assume come input un'immagine RGB di dimensione $28 \times 28 \times 3$.

La prima operazione è la convoluzione:

- il kernel ha dimensione 3×3 ;
- con `stride = 1` e `padding = 1`, la formula

$$H_{out} = \frac{H_{in} + 2P - K}{S} + 1$$

garantisce che le dimensioni spaziali restino invariate.

Quindi:

$$28 \times 28 \times 3 \rightarrow 28 \times 28 \times 32$$

poiché vengono applicati 32 filtri distinti.

Successivamente, il **Max Pooling** con finestra 2×2 riduce le dimensioni spaziali di un fattore 2:

$$28 \times 28 \times 32 \rightarrow 14 \times 14 \times 32$$

Il tensore viene poi appiattito (Flatten) in un vettore 1D di dimensione:

$$32 \cdot 14 \cdot 14 = 6272$$

Questo spiega la scelta del primo layer fully connected:

```
nn.Linear(32 * 14 * 14, 128)
```

Riceve in input tutte le feature estratte e le combina per produrre una rappresentazione più compatta di dimensione 128, successivamente usata per la Classificazione finale.

16.3.2 ResNet e Skip Connection

Discuti il problema del Vanishing Gradient nelle reti neurali profonde. In che modo le skip connection contribuiscono a mitigare questo problema? Descrivi come vengono implementate le skip connection in ResNet. Qual è il ruolo delle half-padding convolution e 1x1?

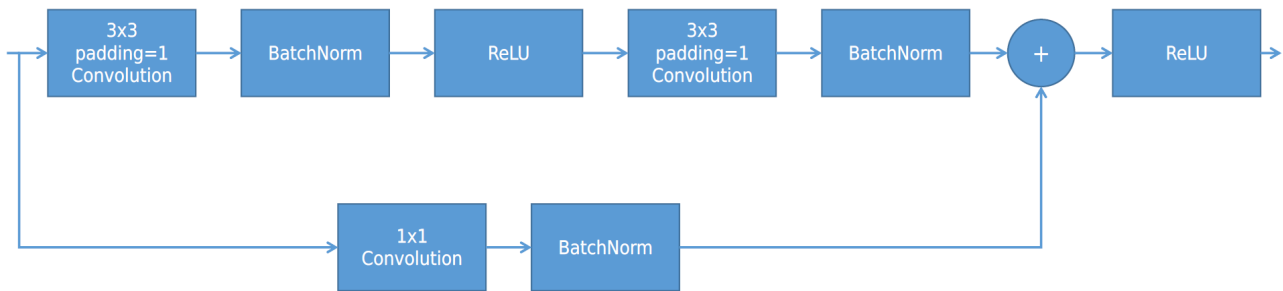
Durante la fase di addestramento (**Backward Pass**), i gradienti dell'errore vengono calcolati partendo dall'ultimo layer e propagati all'indietro verso il primo, utilizzando la **Chain Rule** delle derivate.

Matematicamente, la Chain Rule implica una lunga serie di moltiplicazioni. Se i pesi della rete e le derivate delle funzioni di attivazione (come Sigmoid o Tanh) hanno valori inferiori a 1, moltiplicare decine di numeri minori di 1 fa sì che il risultato tenda esponenzialmente a zero.

Di conseguenza i primi layer ricevono un gradiente piccolissimo e i loro pesi smettono di aggiornarsi, e la rete smette di imparare i pattern di base (come bordi e colori), paralizzando l'intera architettura.

L'architettura **ResNet** è una rete convoluzionale, con le **Skip Connections**. Invece di costringere un blocco di layer a imparare direttamente la mappatura desiderata $H(x)$, ResNet costringe il blocco a imparare una funzione "residua" $F(x) = H(x) - x$. Alla fine del blocco, l'input originale x viene aggiunto di nuovo all'output del blocco prima di applicare la ReLU finale:

$$Output = F(x) + x$$



Così, durante la Backpropagation, la derivata di una semplice addizione ($F(x) + x$) rispetto a x è 1. Il segnale di errore può "saltare" i layer problematici viaggiando indietro attraverso le Skip Connections senza subire alcuna moltiplicazione attenuante. I primi layer riceveranno sempre un gradiente sano e forte.

Per poter eseguire matematicamente l'operazione $F(x) + x$, c'è un requisito ferreo: il tensore $F(x)$ e il tensore x devono avere esattamente le stesse dimensioni (altezza, larghezza e numero di canali).

Se passassimo x attraverso una normale convoluzione 3×3 , i pixel sui bordi andrebbero persi e $F(x)$ sarebbe spazialmente più piccolo di x , rendendo impossibile l'addizione. È qui che entra in gioco l'Half-Padding (solitamente padding=1 per kernel 3×3). Aggiungendo una cornice di zeri all'immagine prima della convoluzione, si garantisce che la dimensione spaziale in uscita sia identica a quella in ingresso. Senza half-padding, la logica basilare delle ResNet crollerebbe a causa di un mismatch dimensionale.

Sebbene l'half-padding risolva il problema della dimensione spaziale ($W \times H$), in una CNN profonda ci sono momenti in cui dobbiamo cambiare le dimensioni del tensore (ad esempio, quando facciamo un sottocampionamento con stride=2 per dimezzare l'immagine, o quando raddoppiamo il numero di canali, o filtri, da 64 a 128).

Se x ha 64 canali e $F(x)$ ha 128 canali, l'addizione diretta è matematicamente impossibile. Come risolviamo? Si utilizza una Convoluzione 1×1 sulla Skip Connection (chiamata Projection Shortcut).

Un filtro 1×1 non analizza i pattern spaziali vicini (poiché guarda solo un pixel alla volta), ma "buca" l'immagine in profondità.

In questo modo, la convoluzione 1×1 adatta perfettamente il tensore x alla nuova forma di $F(x)$, permettendo alla preziosa addizione $F(x) + x$ di avvenire senza intoppi anche quando l'architettura si restringe e si approfondisce.

16.4 Variational AutoEncoder

16.4.1 Differenza fra AE e VAE

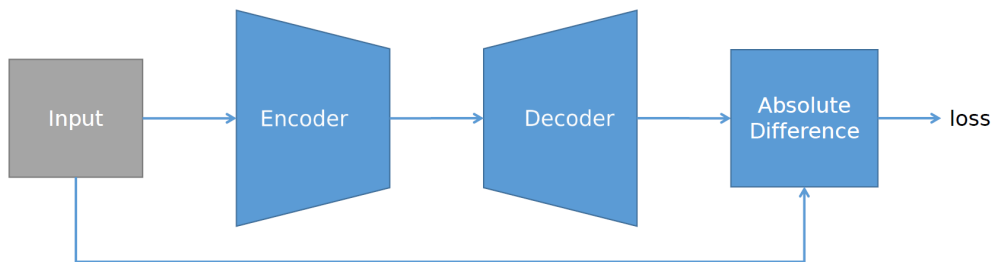
Descrivi le architetture degli AutoEncoder (AE) e dei Variational AutoEncoder (VAE). Evidenzia le principali differenze tra AE e VAE, discuti il reparameterization trick e spiega il ruolo della KL-Divergence loss.

Gli **Autoencoder** sono una particolare architettura di reti neurali utilizzate nell'**unsupervised learning**. Il loro scopo principale è imparare rappresentazioni efficienti e compresse dei dati, senza bisogno di etichette manuali.

Un Autoencoder è progettato per comprimere l'input e poi ricostruirlo. È composto da due Neural Network contrapposte:

- **Encoder:** Prende il dato in input (esempio: un'immagine ad alta risoluzione) e lo comprime in una rappresentazione a bassa dimensionalità.
- **Decoder:** Prende questa rappresentazione compressa e cerca di ricostruire il dato originale con la massima fedeltà possibile.

La rete viene addestrata minimizzando la **Reconstruction Loss**, ad esempio il Mean Absolute Error (MAE) tra l'input x e l'output $\hat{x}$. Il collo di bottiglia costringe la rete a scartare il rumore e a conservare solo le feature fondamentali necessarie per ricostruire il dato.



Il **Variational AutoEncoder (VAE)** mantiene la struttura Encoder-Decoder, ma cambia radicalmente la natura dello spazio latente. Non è più un modello di sola compressione, ma diventa un Modello Generativo (capace di creare dati nuovi, mai visti prima).

In un VAE, l'Encoder non mappa l'input x in un singolo punto deterministico z . Al contrario, mappa l'input nei parametri di una distribuzione di probabilità (una Gaussiana). Per ogni input, l'Encoder restituisce due vettori: il **vettore delle medie** (μ) e il **vettore delle deviazioni standard** (σ). Il vettore latente z che verrà passato al Decoder viene campionato (sampled) casualmente da questa distribuzione di probabilità $\mathcal{N}(\mu; \sigma)$.

L'operazione di campionamento stocastico non è differenziabile. Se la inserissimo così com'è, la Backpropagation si bloccherebbe. Per aggirare il problema si usa il Reparameterization Trick: Invece di campionare z direttamente dalla distribuzione complessa guidata dalla rete, si sposta la casualità in un nodo esterno e indipendente;

1. Si campiona un vettore di puro rumore ε da una Gaussiana standard fissa: $\varepsilon \sim \mathcal{N}(0; 1)$;
2. Si calcola z con operazioni matematiche semplici (moltiplicazione e somma):

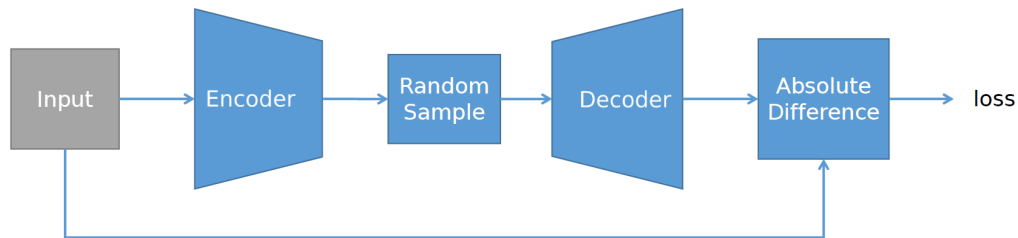
$$z = \mu + \sigma \cdot \varepsilon$$

In assenza di un'opportuna regolarizzazione, un VAE tenderebbe a convergere verso una soluzione degenera. Per minimizzare l'errore di ricostruzione, l'Encoder imparerebbe ad associare a ciascun campione una distribuzione con varianza pressoché nulla ($\sigma \approx 0$) e con medie fortemente separate tra loro. Di conseguenza, il modello si comporterebbe in modo analogo a un Autoencoder tradizionale, compromettendo la continuità e la struttura dello spazio latente.

Per impedirlo, la funzione di Loss totale del VAE è composta da due parti:

$$\text{Total Loss} = \text{Reconstruction Loss} + \text{KL Divergence}$$

La **Kullback-Leibler Divergence** è una misura di quanto due distribuzioni di probabilità siano diverse. Nel VAE, calcoliamo la KL-Divergence tra la distribuzione generata dall'Encoder $\mathcal{N}(\mu; \sigma)$ e una Gaussiana Standard $\mathcal{N}(0; 1)$. In questo modo gli impedisce di barare creando uno spazio latente denso e uniforme. Tira le medie (μ) di tutte le classi verso lo zero, ammassando le distribuzioni al centro dello spazio latente. Forza le varianze (σ) a rimanere vicine a 1, assicurandosi che i dati non collassino in punti singoli.



Capitolo 17: Domanda 4

17.1 RNN

17.1.1 Domanda su RNN

Discuti le Recurrent Neural Networks (RNNs) e fornisci lo pseudocodice per il passaggio in avanti. Come e quando sono influenzate dal problema del vanishing or exploding gradient? Illustra e spiega il grafo computazionale associato ai problemi sequence-to-task e sequence-to-sequence.

Una **Recurrent Neural Network (RNN)** è una rete neurale progettata per l'elaborazione di dati sequenziali. Il suo funzionamento si basa sull'utilizzo ripetuto degli stessi pesi su input che variano in funzione della dimensione temporale, permettendo così di modellare dipendenze tra elementi successivi di una sequenza.

La struttura fondamentale di una RNN è la cosiddetta **cella**. Ogni cella è caratterizzata da tre elementi chiave: un **input**, un **hidden state** e un **output**. A ogni passo temporale, la cella non elabora l'input in modo isolato: l'input corrente viene fuso con l'hidden state generato nel passo precedente.

Le RNN trovano applicazione principalmente in due scenari:

- **Seq2Task:** l'intera sequenza viene codificata in una rappresentazione finale utilizzata per compiti di Classificazione o Regressione.
- **Seq2Seq:** la rete trasforma una sequenza di input in una sequenza di output, spesso mediante meccanismi di *attention*.

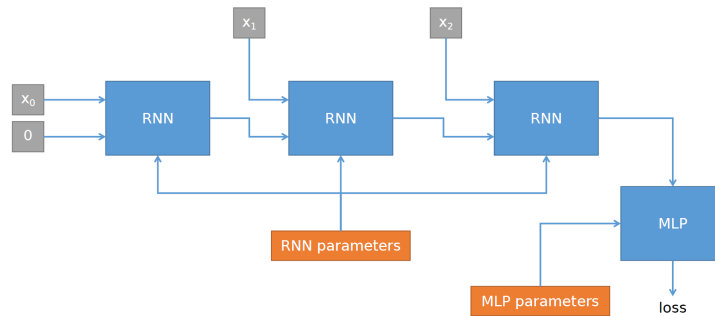
L'addestramento avviene mediante **Backpropagation Through Time (BPTT)**, che consiste nello "srotolare" la rete lungo la dimensione temporale e applicare la backpropagation alla rete equivalente. I gradienti calcolati a ciascun passo vengono quindi aggregati per aggiornare i pesi condivisi. Diventa così equivalente a una rete feed-forward profonda tanti layer quanti sono gli step temporali della sequenza.

Per aggiornare i pesi relativi ai primi istanti temporali sulla base di un errore calcolato in un istante successivo, la *Chain Rule* richiede di moltiplicare i gradienti lungo tutti i passi della sequenza. Tale operazione comporta la moltiplicazione ripetuta della matrice dei pesi ricorrenti (W_{hh}) e delle derivate della funzione di attivazione (generalmente tanh), dando origine a due possibili fenomeni:

- **Vanishing Gradient:** Se gli autovalori della matrice dei pesi sono minori di 1, o se le derivate della funzione di attivazione sono piccole, moltiplicare questi numeri per se stessi dozzine di volte fa tendere il risultato a zero;
- **Exploding Gradient:** Se i pesi sono maggiori di 1, la moltiplicazione a catena fa crescere il gradiente esponenzialmente verso l'infinito (causando errori NaN nei calcoli).

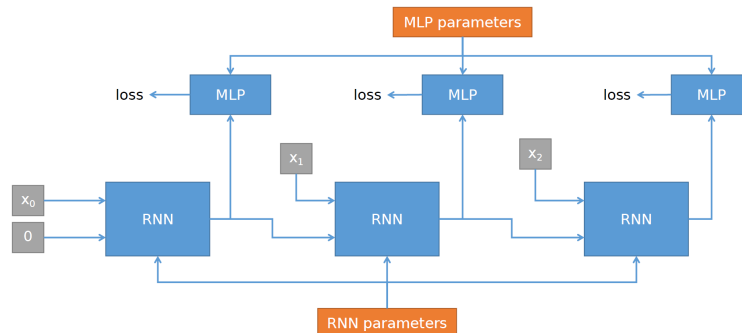
Entrambi i fenomeni diventano particolarmente rilevanti nelle sequenze di elevata lunghezza.

Recurrent Neural Networks: Computational Graph (end loss)



Nel **Seq2Task**, la rete riceve una sequenza di input (x_0, x_1, x_2, x_3) . Ad ogni step, la cella RNN aggiorna la propria memoria interna (lo stato nascosto), ma non produce alcun output immediato. Solo alla fine della sequenza, l'ultimissimo stato nascosto (che funge da "riassunto" aggregato dell'intera serie di dati) viene passato a un Classificatore o Regressore finale (l'MLP) per risolvere un task specifico, generando un singolo output. La Loss viene calcolata solo alla fine. Da notare che i parametri di RNN sono condivisi in tutti i blocchi RNN.

Recurrent Neural Networks: Computational Graph (step loss)



Nel **Seq2Seq**, la rete riceve la sequenza in ingresso ma, a differenza del caso precedente, genera un output (y_t) a ogni singolo istante temporale. Ad ogni step t , lo stato nascosto corrente viene inviato a un MLP per effettuare una predizione istantanea (y_0, y_1, y_2, y_3) . Di conseguenza, viene calcolata una Loss per ogni singolo step. L'errore totale utilizzato durante la retropropagazione (BPTT) sarà una funziona di aggregazione (la somma degli errori intermedi o la media). Da notare che i parametri di RNN sono condivisi in tutti i blocchi RNN, così come quelli di MLP sono condivisi in tutti i blocchi MLP.

```

1  import torch
2  import torch.nn as nn
3
4  class RNNModel(nn.Module):
5      def __init__(self, input_size, hidden_size, output_size):
6          self.hidden_size = hidden_size
7
8          self.rnn = nn.Sequential(
9              nn.Linear(input_size + hidden_size, hidden_size),
10             nn.LeakyReLU(),
11             nn.Linear(hidden_size, hidden_size),
12             nn.LeakyReLU(),

```



```

13         nn.Linear(hidden_size, hidden_size),
14     )
15
16     self.ff = nn.Sequential(
17         nn.Linear(hidden_size, output_size),
18         nn.Sigmoid()
19     )
20
21     def forward(self, x):
22         hidden_state = torch.zeros(x.shape[0], self.hidden_size, device=x.device)
23
24         for t in range(x.shape[1]):
25             input = torch.cat(x[:, t, :], hidden_state, dim = 1),
26             hidden_state = self.rnn(input)
27
28         return self.ff(hidden_state)

```

17.2 AutoEncoder

17.2.1 Domanda Standard

Descrivi in che modo gli AutoEncoder (AE) vengono utilizzati per apprendere codifiche efficienti in modo non supervisionato, concentrandoti sui ruoli dei componenti di encoder e decoder. Discuti come vengono addestrati gli AE fornendo lo pseudocodice e analizza i passaggi in avanti e all'indietro attraverso un semplice AE.

Gli **Autoencoder** sono architetture di **unsupervised learning** progettate per apprendere una rappresentazione compatta dei dati, anziché eseguire compiti di Classificazione o Regressione.

La loro struttura è composta da due reti neurali:

- **Encoder:** comprime il dato di input (ad esempio un'immagine) in una rappresentazione a dimensionalità ridotta.
- **Decoder:** utilizza questa rappresentazione per ricostruire il dato originale nel modo più fedele possibile.

L'uscita dell'Encoder prende il nome di **Latent Space** (*Spazio Latente*), ovvero una rappresentazione interna e compressa che conserva le caratteristiche più rilevanti dei dati, eliminando le informazioni ridondanti.

L'addestramento consiste nel minimizzare il **Reconstruction Error** (*Errore di Ricostruzione*), cioè la differenza tra l'input originale e quello ricostruito dal Decoder (ad esempio mediante la *Mean Absolute Error*). In questo modo la rete impara a codificare i dati nello spazio latente mantenendo le informazioni essenziali necessarie alla loro ricostruzione.

```

1  import torch
2  import torch.nn as nn
3  import torch.optim as optim
4  from torch.utils.data import Dataset, DataLoader
5
6  class AutoEncoder(nn.Module):
7      def __init__(self, input_size, hidden_size):
8          super().__init__()
9

```

```

10     self.encoder = nn.Sequential(
11         nn.Linear(input_size, 300),
12         nn.LeakyReLU(),
13         nn.Linear(300, 300),
14         nn.LeakyReLU(),
15         nn.Linear(300, hidden_size)
16     )
17
18     self.decoder = nn.Sequential(
19         nn.Linear(hidden_size, 300),
20         nn.LeakyReLU(),
21         nn.Linear(300, 300),
22         nn.LeakyReLU(),
23         nn.Linear(300, input_size)
24     )
25
26     def forward(self, x):
27         latent = self.encoder(x)
28         reconstruction = self.decoder(latent)
29         return reconstruction
30
31     dataloader = DataLoader(dataset , batch_size=32, shuffle=True)
32     model = AutoEncoder(10, 10)
33     cost = nn.L1Loss() # MAE
34     optimizer = optim.Adam(model.parameters(), lr=0.01)
35
36     for e in range(5):
37         for input, _ in dataloader:
38             optimizer.zero_grad()
39             output = model.forward(input)
40             loss = cost(output, input)
41             loss.backward()
42             optimizer.step()

```

17.3 VQ-VAE

17.3.1 Quantization Trick e .detach()

Spiega i principi di funzionamento dei Vector Quantized Variational Autoencoders (VQ-VAE) con particolare attenzione al processo di quantizzazione dell'embedding. Descrivi il ruolo del quantization trick $(q-x).detach() + x$

I **Vector Quantized Variational Autoencoder (VQ-VAE)** è un tipo di architettura di Neural Network che combina elementi dei Variational Autoencoder e di quantizzazione vettoriale per apprendere rappresentazioni discrete dei dati. Mentre i VAE standard mappano i dati in uno spazio latente continuo (generando la media e la varianza di una Gaussiana), i VQ-VAE forzano la rete a mappare i dati in uno spazio latente discreto e categorico.

L'architettura mantiene la struttura **Encoder-Decoder**, con l'Encoder che comprime l'input e il Decoder che cerca di ricostruire l'input dalla compressione. Tuttavia introduce anche un nuovo elemento: il **Codebook**, una lista di K vettori di dimensione D , appresi dalla rete e detti **embeddings**. Il Codebook serve a quantizzare l'uscita continua dell'Encoder.

Un VQ-VAE funziona così:

1. **Encoder:** Prende l'input (esempio: un'immagine) e lo trasforma in una rappresentazione vettoriale continua;
2. **Vector Quantization:** Il vettore continuo in uscita dall'Encoder viene confrontato con tutti i vettori presenti nel Codebook. L'algoritmo calcola la distanza (solitamente la distanza Euclidea tramite la funzione `cdist` in *PyTorch*) tra l'output dell'Encoder e i vettori del Codebook;
3. **Assegnazione (Argmin):** L'output dell'Encoder viene fisicamente sostituito dal vettore del Codebook che gli è più vicino (quello con la distanza minima);
4. **Decoder:** Il Decoder riceve questo vettore discreto pescato dal Codebook e lo usa per ricostruire una versione discreta dell'input originale.

Il passaggio 4 (l'uso della funzione *argmin* per trovare il vettore più vicino) non è un passaggio differenziabile e di conseguenza la backpropagation verrebbe bloccata. Per superare questo ostacolo si utilizza il **Quantization Trick**, che consiste nello scollegare dal grafo computazionale l'arco che calcola la differenza. L'operatore `detach()` scollega un tensore dal grafo computazionale e quindi per l'Autograd di PyTorch viene trattato come una costante, con gradiente nullo. Ciò comporta che:

- **Forward pass:** l'espressione $q - x.\text{detach}() + x$ è numericamente uguale a q , quindi il Decoder riceve il vettore quantizzato.
- **Backward pass:** il termine con `detach()` non contribuisce al gradiente, che viene quindi propagato attraverso x . In questo modo il gradiente del Decoder raggiunge direttamente l'Encoder, aggirando la non differenziabilità della quantizzazione. In pratica il gradiente proveniente dal decoder viene moltiplicato per 1.

Poiché l'Encoder e il Codebook sono stati scollegati durante la Backpropagation, i vettori del Codebook non verrebbero mai aggiornati dal gradiente. Per allineare Encoder e Codebook, il VQ-VAE introduce due Loss aggiuntive (oltre all'Errore di Ricostruzione standard):

- **Codebook Loss:** Sposta i vettori del dizionario verso gli output dell'Encoder;
- **Commitment Loss:** Evita che l'Encoder continui a saltare da un codice all'altro, forzandolo a "impegnarsi" a produrre vettori vicini ai codici del dizionario che ha scelto.

17.4 GAN

17.4.1 Training GAN

Descrivi che cos'è una Generative Adversarial Network (GAN), definendo i ruoli del generator e del discriminator. Spiega il processo di addestramento e fornisci lo pseudocodice del main training loop.

Una **Generative Adversarial Network (GAN)** è un modello di deep learning per la generazione di dati sintetici, tipicamente immagini, basato su un gioco a somma zero tra due reti neurali.

Il sistema è composto da due moduli in competizione:

- **Discriminatore:** Funziona come un classico Classificatore binario. È responsabile di distinguere i dati reali da quelli generati. Nel caso delle immagini, prende in input un'immagine e deve decidere se quell'immagine è "Reale" (proveniente dal dataset originale, output 1) oppure "Falsa" (creata dal generatore, output 0). Il suo unico obiettivo è non farsi ingannare;
- **Generatore:** È il "falsario". Non vede mai le immagini originali. È responsabile di generare nuovi dati a partire da un vettore di rumore casuale. Nel caso delle immagini, prende in input un vettore di rumore casuale (random noise z) e cerca di modellarlo per generare un'immagine sintetica. Il suo obiettivo è produrre immagini così realistiche da ingannare il Discriminatore, costringendolo a classificarle come "Reali" (output 1).

L'addestramento avviene in modo alternato, aggiornando un solo modello per volta mentre l'altro viene congelato.

L'addestramento del Discriminatore funziona così:

1. Si genera un batch di immagini false passando del rumore al Generatore;
2. Si prende un batch di immagini reali dal dataset;
3. Si passano entrambe al Discriminatore;
4. Si calcola la Loss tramite **Binary Cross Entropy**. Il Discriminatore viene penalizzato se non assegna 1 alle immagini reali e 0 a quelle false;
5. Si aggiornano solo i pesi del Discriminatore. Il Generatore non viene toccato.

L'addestramento del Generatore funziona così:

1. Si genera un nuovo batch di immagini false partendo da nuovo rumore z ;
2. Si passano queste immagini al Discriminatore;
3. Questa volta, il target viene impostato forzatamente a 1 (Reale). Se il Discriminatore restituisce un valore basso (esempio 0.1, capendo che è falsa), il Generatore riceve un errore enorme;
4. Si aggiornano solo i pesi del Generatore per spingerlo a produrre immagini che otterranno un punteggio più alto. Il Discriminatore non viene toccato.

Il processo è intrinsecamente instabile e richiede un equilibrio tra le due reti: un Discriminatore troppo potente può annullare i gradienti utili al Generatore, impedendone l'apprendimento.

```

1  import torch
2  import torch.nn.functional as F
3
4  def train_discriminator(optimizer, generator, discriminator, real_images):
5      optimizer.zero_grad()
6
7      batch_size = real_images.size(0)
8
9      z = torch.randn(batch_size, generator.latent_dim, device=real_images.device)
10     fake_images = generator(z)
11
12     fake_logits = discriminator(fake_images)[: , 0]
13     real_logits = discriminator(real_images)[: , 0]
14 
```

```

15     fake_labels = torch.zeros_like(fake_logits)
16     real_labels = torch.ones_like(real_logits)
17
18     fake_loss = F.binary_cross_entropy(fake_logits, fake_labels)
19     real_loss = F.binary_cross_entropy(real_logits, real_labels)
20
21     loss = (fake_loss + real_loss) / 2
22     loss.backward()
23     optimizer.step()
24
25     return fake_loss.item(), real_loss.item()
26
27 def train_generator(optimizer, generator, discriminator, real_images):
28     optimizer.zero_grad()
29
30     batch_size = real_images.size(0)
31
32     z = torch.randn(batch_size, generator.latent_dim, device=real_images.device)
33     fake_images = generator(z)
34
35     fake_logits = discriminator(fake_images)[: , 0]
36     real_labels = torch.ones_like(fake_logits)
37
38     loss = F.binary_cross_entropy(fake_logits, real_labels)
39     loss.backward()
40     optimizer.step()
41
42     return loss.item()

```

17.4.2 Domanda su ACGAN

Descrivi i principi di funzionamento delle Generative Adversarial Networks (GAN) e delle Auxiliary Classifier GAN (ACGAN). Spiega il processo di addestramento, evidenziando come viene stabilita e mantenuta la relazione avversaria tra il generator e il discriminator. Fornisci lo pseudocodice pertinente per illustrare il loop di addestramento delle GAN e discuti in che modo le ACGAN estendono l'architettura GAN.

Una **Generative Adversarial Network (GAN)** è un modello di deep learning per la generazione di dati sintetici, tipicamente immagini, basato su un gioco a somma zero tra due reti neurali. Il sistema è composto da due moduli in competizione:

- **Discriminatore:** rete binaria che stima se un campione proviene dal dataset reale o è stato generato, con l'obiettivo di massimizzare la corretta Classificazione.
- **Generatore:** rete che, a partire da un vettore di rumore z , produce campioni sintetici con l'obiettivo di ingannare il Discriminatore.

L'addestramento avviene in modo alternato, aggiornando un solo modello per volta mentre l'altro viene congelato. Il Discriminatore viene ottimizzato su batch misti di dati reali e sintetici tramite *Binary Cross Entropy*. Il Generatore, invece, viene aggiornato per massimizzare la probabilità che i campioni generati siano classificati come reali.

```

1  import torch
2  import torch.nn.functional as F
3
4  def train_discriminator(optimizer, generator, discriminator, real_images):
5      optimizer.zero_grad()
6
7      batch_size = real_images.size(0)
8
9      z = torch.randn(batch_size, generator.latent_dim, device=real_images.device)
10     fake_images = generator(z)
11
12     fake_logits = discriminator(fake_images)[: , 0]
13     real_logits = discriminator(real_images)[: , 0]
14
15     fake_labels = torch.zeros_like(fake_logits)
16     real_labels = torch.ones_like(real_logits)
17
18     fake_loss = F.binary_cross_entropy(fake_logits, fake_labels)
19     real_loss = F.binary_cross_entropy(real_logits, real_labels)
20
21     loss = (fake_loss + real_loss) / 2
22     loss.backward()
23     optimizer.step()
24
25     return fake_loss.item(), real_loss.item()
26
27
28 def train_generator(optimizer, generator, discriminator, real_images):
29     optimizer.zero_grad()
30
31     batch_size = real_images.size(0)
32
33     z = torch.randn(batch_size, generator.latent_dim, device=real_images.device)
34     fake_images = generator(z)
35
36     fake_logits = discriminator(fake_images)[: , 0]
37     real_labels = torch.ones_like(fake_logits)
38
39     loss = F.binary_cross_entropy(fake_logits, real_labels)
40     loss.backward()
41     optimizer.step()
42
43     return loss.item()

```

Una **Auxiliary Classifier Generative Adversarial Network (ACGAN)** è una variante delle GAN in cui il Discriminatore svolge un duplice compito: distinguere tra campioni reali e sintetici e, simultaneamente, predire la classe di appartenenza dei dati.

A differenza delle GAN standard, che generano campioni in modo non controllato, le ACGAN introducono un meccanismo di condizionamento che consente la generazione guidata rispetto a classi specifiche.

In questa architettura, il Discriminatore è tipicamente suddiviso in due componenti:

- **Validity head:** stima la probabilità che un campione sia reale o generato;
- **Classification head:** predice la classe semantica del campione in ingresso.